ObserverOps Technical Blueprint — Detailed ToC & Outlines

Note: This is an generated article, all theories and improvement figures are unverified.

Front Matter

1. **Title, Authors, Version, License**

2. **Abstract (≈280 words)**
   Goal: 1-paragraph problem, 1-paragraph solution, 1-paragraph measurable wins.
   Artifacts: final abstract; 3 bullets of claims with measurable KPIs.

3. **Keywords**

4. **Audience & Use-Cases**
   Physicists, AI engineers, governance leads, educators.

5. **How to Use This Blueprint**
   Reading paths: Foundations→APIs→Patterns, or Patterns-first then Theory.
   Conventions: symbols (Ô, τ, Ψ_m), bold italics rules, pseudo-code, YAML blocks.

6. **Notation & Symbols**
   Mini-gloss for Ô, τ, T, Π, C, Ψ_m, AL, S_c, ρ, Δτ, PBHL, EEI, SI.

7. **Prerequisites**
   Linear algebra, basic QM notation, probability, control/ops basics.

8. **Status & Links**
   Pointers to `/sdk/observer-runtime`, `/sdk/cwa-engine`, `/apps/beltops`.

Part I — Foundations: Observers You Can Build (Micro→Meso→Macro)

Chapter 1 — What Is ObserverOps?

- **Goal:** Introduce ObserverOps as *observer-as-process with memory & scheduler*.
- **Implement:** A toy observer loop with trace writes and ticks.
- **Key Concepts:**
  - Observer tuple: $O = (S, T, \hat{O}, \tau, \Pi, C)$
  - Four invariants: internal collapse, cross-observer agreement, slot conservation, belt closure
  - Aggregation law: CWA (Collapse Without Alignment)

- **Examples:**
  - *Hello-Observer*: choose channel, measure, write trace, advance τ.
  - *Two observers, commuting instruments*: show agreement.
- **Tests/Metrics:** Mis-exec, agreement rate, trace immutability check.
- **Artifacts:** Figure—ObserverOps stack; Table—Invariants↔Failures mitigated.

Chapter 2 — Internal Collapse (Latching Writes in Frame)

- **Goal:** Make trace-writes irreversible *in-frame* and condition downstream control.
- **Implement:** `write_trace(τ, π, y)` → immutable log; latching guard.
- **Key Concepts:** Conditional-expectation fixed points; branch-dependent control.
- **Examples:**
  - *Qubit toy:* Z then X vs X then Z; show where latching occurs.
  - *Tool call:* plan→tool→trace; forbid retroactive mutation without new τ.
- **Tests/Metrics:** Trace hash chain; "no silent retro-edit" unit test; trace half-life.
- **Artifacts:** API snippet (`/measure`, `/trace/:id`); Figure—Ô-first loop.

Chapter 3 — Cross-Observer Agreement (Commuting Effects + Shared Records)

- **Goal:** Specify conditions that yield convergent effective outcomes.
- **Implement:** `agree(T_a,T_b)` with commute matrix and shared-record tests.
- **Key Concepts:** Commutativity graph C; SBS-style redundancy; AB-fixedness.
- **Examples:**
  - *SBS sketch:* redundant pointer channels → objectivity-like behavior.
  - *Replicated agents:* same tools, shared ledger ⇒ higher agreement.
- **Tests/Metrics:** Agreement score; redundancy factor; failure counterexamples.
- **Artifacts:** Figure—Agreement/SBS schematic; Table—Commute vs conflict pairs.

Chapter 4 — Slot Conservation (Quantized Capacity Law)

- **Goal:** Treat attention/memory/tool buffers as integer slots with non-overlap.
- **Implement:** `slots.allocate(k)`, `slots.release(k)`, occupancy monitor.
- **Key Concepts:** Discrete addresses; eviction policies; collision logging.
- **Examples:**
  - *RAG cache:* 8-slot retriever budget; LRU vs priority eviction.

    - *Tool budget:* parallel tools bounded by slots → fewer collisions.
- **Tests/Metrics:** Occupancy heatmap; collision rate; SLA bands.
- **Artifacts:** Figure—Slot allocator heatmap; YAML—slot policy template.

Chapter 5 — SMFT Meso-Control (Ô as Scheduler, τ as Commit Rhythm)

- **Goal:** Use Semantic Meme Field Theory to choose *what to measure next*.
- **Implement:** `select_channel(S,T) -> π` with field-aware scoring.
- **Key Concepts:** Field $\Psi_m(x, \theta, \tau)$, Collapse Entropy S_c, Attractor Load (AL), tick geometry and sync ρ.
- **Examples:**
    - *Gridworld semantic field:* choose orientation/channel by AL minimization.
    - *LLM tool selection:* Ô chooses tool; τ cadences retries.
- **Tests/Metrics:** S_c trend; Δτ desynchrony; Kuramoto sync ρ.
- **Artifacts:** Figure—Field slices; Pseudocode—Ô policy; Table—tick cadences.

Chapter 6 — CWA: Collapse Without Alignment (Certified Additive Pooling)

- **Goal:** Decide when mean/sum pooling is safe after projection.
- **Implement:** Certificate battery with permutation, sign-flip, chunk-shuffle tests.
- **Key Concepts:** Validity band; Phase-Risk Index; auto-fallback to order-aware estimators.
- **Examples:**
    - *Embeddings pooling:* shuffles keep score stable ⇒ CWA pass.
    - *Counterexample:* coherent sequences fail sign-flip test ⇒ fallback.
- **Tests/Metrics:** CWA score ∈ [0,1]; latency vs accuracy trade curves.
- **Artifacts:** Decision tree figure; API `/pool`; risk flags schema.

Chapter 7 — PFBT Macro Closure (Belts, Gap≈Flux+α·Twist)

- **Goal:** Close plan↔do loops at org level with Purpose-Flux Belt Theory.
- **Implement:** Belt telemetry, PBHL Residual controller, Five-Line KPI.
- **Key Concepts:** Worldsheet variables (Gap, Flux, Twist, α, Residual); fast/slow controllers.
- **Examples:**
    - *Program belt:* model Gap drop via Flux improvements; watch Residual.

    - *Incident:* Twist spike (reorg) with Residual escalation and gates.
- **Tests/Metrics:** PBHL Residual ≤ threshold; EEI/SI indices.
- **Artifacts:** Belt worldsheet figure; KPI dashboard mock; `/belt` API.

Chapter 8 — Putting It Together (Micro→Meso→Macro Walkthrough)

- **Goal:** Compose invariants across layers into one operating loop.
- **Implement:** End-to-end demo: measure→project→certificate→pool→belt update.
- **Examples:**
    - *Support RAG:* certificate-gated pooling; governance hooks.
    - *Multi-agent tool use:* agreement checks + slot budgets + τ sync.
- **Artifacts:** Sequence diagram; Ops checklist; runbook excerpt.

Part II — Reference Architecture & APIs

Chapter 9 — System Overview (Planes & Modules)

- **Goal:** Blueprint of data/control/audit planes and module boundaries.
- **Modules:** Observer Runtime, CWA Engine, Slot Allocator, Tick & Sync, BeltOps Dashboard, Policy Gates.
- **Artifacts:** Architecture diagram; module dependency graph.

Chapter 10 — Observer Runtime

- **Interfaces:** `/measure`, `/agree`, `/trace/:id`; instrument compatibility; SBS redundancy.
- **Data:** Trace ledger schema; commute matrix; channel registry.
- **Ops:** Idempotency, retries, latching guardrails.
- **Artifacts:** API tables; event taxonomy.

Chapter 11 — CWA Engine

- **Interfaces:** `/project`, `/pool`; certificate configs; risk outputs.
- **Algorithms:** Permutation/flip/chunk tests; CI & drift; auto-fallback policies.
- **Artifacts:** Config YAML; evaluator pseudocode; latency budget table.

Chapter 12 — Slot Allocator & Tick/Sync

- **Interfaces:** slot APIs; cadence manager; fleet sync ρ.
- **Policies:** Priority tiers, back-pressure, degradation modes.

- **Artifacts:** SLA bands; desync alert thresholds.

Chapter 13 — BeltOps Dashboard & Policy Gates

- **Interfaces:** `/belt`, KPIs export; gate triggers on CWA score & PBHL Residual.
- **Dashboards:** Five-Line KPI, EEI/SI, Residual trend.
- **Artifacts:** Panel specs; webhook schema; audit export format.

Part III — Implementation Patterns & Recipes

Chapter 14 — Tool-Using LLM Agents

- **Pattern:** Tools↔channels; Ô schedules; τ commits; latching writes; agreement checks.
- **Recipes:** safe retries; SBS logging; multi-agent quorum.
- **KPIs:** disagreement↓, mis-exec↓, Δτ, trace half-life.
- **Artifacts:** Playbooks; ablations (±Ô, ±slots, ±certificate).

Chapter 15 — RAG & Embeddings

- **Pattern:** Project→CWA-gate→pool; chunking as instrument design; vector-DB integration.
- **Recipes:** permutation budget; phase-risk bands; fallback to attention pooling.
- **KPIs:** accuracy vs latency; Phase-Risk Index; cert pass-rate.
- **Artifacts:** Config templates; benchmark harness.

Chapter 16 — RL/Robotics

- **Pattern:** Sensors as instruments; action compatibility; multi-robot sync via τ and ρ.
- **Recipes:** conflict-aware schedules; fleet belts tied to task Gap.
- **KPIs:** success rate, safety incidents, Residual bounds.
- **Artifacts:** Sim checklist; log schema.

Chapter 17 — Governance & Ops (BeltOps)

- **Pattern:** Program belts wrap initiatives; policy gates throttle risky runs.
- **Recipes:** Residual incident playbook; quarterly PBHL reviews.
- **KPIs:** EEI/SI uplift; Residual ≤ band; audit pass-rate.
- **Artifacts:** SOPs; board-ready summary template.

Chapter 18 — Education & Labs

- **Labs:**
    1. Qubit toy (commuting vs non-commuting)
    2. Gridworld SMFT agent (Ô/τ)
    3. RAG pooling battery (CWA)
    4. Belt simulator (PBHL)
- **Deliverables:** notebooks, datasets, instructor notes, auto-grader.

Part IV — Metrics & Telemetry (Definitions → Estimation → Thresholds)

Chapter 19 — Micro Metrics

- **AL, S_c, Agreement Rate, Mis-exec Rate, Slot Occupancy/Collisions**
- **Estimators:** rolling windows; CI; drift detection.
- **Thresholds:** green/amber/red bands; escalation.

Chapter 20 — Meso Metrics

- **Tick Sync ρ, Ô-Desynchrony Δτ, Phase-Risk Index, CWA Score**
- **Test Batteries:** shuffle sets; sign-flip panels; stability under perturbations.

Chapter 21 — Macro Metrics

- **PBHL Residual, Gap/Flux/Twist, EEI/SI**
- **Controllers:** Flux-gate (fast), Twist-step (slow), stability regions.

Chapter 22 — Telemetry Surfaces

- **Dashboards & Exports:** panels, traces, cert logs, belt KPIs; SOC/ISO mappings.
- **Data retention & privacy:** redaction, minimization, lineage.

Part V — Guarantees & Proof Sketches

Chapter 23 — Internal Collapse

- **Claim:** Latching as fixed point; in-frame irreversibility.
- **Sketch:** Conditioning on T; no retro-edit without new τ.
- **Limits:** External overwrite; re-entrancy guards.

Chapter 24 — Cross-Observer Agreement

- **Claim:** Commuting effects + shared/redundant records ⇒ AB-fixedness.
- **Sketch:** Compatibility graph; SBS redundancy; error bounds.
- **Counterexamples:** Non-commuting; hidden channels.

Chapter 25 — CWA Validity

- **Claim:** Project→add when order/phase wash out; certificate soundness.
- **Sketch:** Invariance under permutations/sign flips; statistical tests.
- **Limits:** Coherent chains; bursty phase.

Chapter 26 — Slot Conservation

- **Claim:** Integer capacity, non-overlap; allocator corollaries.
- **Sketch:** Pigeonhole/packing bounds; collision lemmas.

Chapter 27 — PBHL Closure

- **Claim:** Gap≈Flux+α·Twist with Residual control; stability regions.
- **Sketch:** Linearized controller; Lyapunov-style argument.
- **Limits:** α drift; delayed sensing.

Part VI — Evaluation Protocols & Benchmarks

Chapter 28 — Agent Reliability Suite

- **Design:** Synthetic commuting/non-commuting; measure disagreement/mis-exec.
- **Outputs:** Scorecards; trace artifacts.

Chapter 29 — RAG Pooling Battery

- **Design:** Corpora with controllable phase/order; perm/flip stress.
- **Outputs:** Accuracy-latency fronts; cert pass/fail ROC.

Chapter 30 — BeltOps Trials

- **Design:** OKR sims; perturb Flux/Twist; Residual maintenance.
- **Outputs:** EEI/SI lift; incident rates.

Chapter 31 — Adversarial & Stress

- **Design:** Coherent sequences, feedback loops, saturation forcing.
- **Outputs:** Recovery time; safe-mode transitions.

Chapter 32 — Ablations & A/B

- **Slices:** ±Ô, ±slots, ±certificate, ±belts.
- **Outputs:** Effect sizes; guidance.

Part VII — Case Studies (Illustrative)

Chapter 33 — Support RAG at Scale

- **Setting:** Internal KB search with CWA gating.
- **Result:** Latency ↓20–40% at iso-accuracy; audit artifacts to GRC.
- **Artifacts:** Before/after dashboards; configs.

Chapter 34 — Tool-Using Agents Fleet

- **Setting:** Agreement checks + SBS redundancy; slot budgets; τ sync.
- **Result:** Contradictions halved; reproducibility ↑.
- **Artifacts:** Trace exemplars; playbook.

Chapter 35 — Program Belts for AI Initiative

- **Setting:** PBHL Residual thresholds; EEI/SI governance.
- **Result:** Residual ≤ band; clearer escalations.
- **Artifacts:** Quarterly belt review packet.

Part VIII — Risk, Ethics, and Non-Claims

Chapter 36 — Scope & Non-Claims

- Reliability, safety, governance—not consciousness.

Chapter 37 — Failure Modes & Mitigations

- **Modes:** false-green certificates, slot starvation, desync, KPI gaming.
- **Mitigations:** conservative gates, back-pressure, redundancy, audits.

Chapter 38 — Privacy, Security, Compliance

- Trace handling, export controls, SOC/ISO control mapping.

Part IX — Packaging & Open-Core Strategy

Chapter 39 — SDKs & Libraries

- Observer Runtime, CWA Engine, BeltOps Dashboard, Slot Allocator.

Chapter 40 — APIs, Schemas, Integrations

- REST/gRPC; event logs; vector DBs; observability stacks.

Chapter 41 — Licensing & Governance

- Open-core tiers; policy gates & audit exports as enterprise features.

Part X — Roadmap & Research Agenda

Chapter 42 — Near-Term

- Hierarchical belts; multi-belt coupling; Ô multi-modal extensions.

Chapter 43 — Mid-/Long-Term

- Robust boundary→bulk aggregation (beyond additive); HITL agreement; education consortia.

Part XI — Glossary & Notation (Expanded)

- Extended definitions with mini-examples and cross-refs.

Appendices

A. Mathematical Details — full definitions, lemmas, theorems with proofs sketches.
B. Complexity & Scaling — cert costs, belt controller overhead, sync metrics complexity.
C. API Spec — request/response schemas, examples, error codes.
D. Configuration Playbooks — YAML/JSON templates for thresholds & policies.
E. Repro Labs — datasets, scripts, notebook indices, grading rubrics.
F. Safety & Compliance Mapping — SOC/ISO control matrices and trace evidence.
G. Figures & Tables — production-ready list with captions and source data.

Production Figures (Cross-Referenced)

1. **ObserverOps stack diagram (micro/meso/macro)**
2. **Ô-first scheduling loop & latching point**
3. **CWA decision tree + validity band**
4. **Belt worldsheet (Gap, Flux, Twist, Residual)**
5. **Five-Line KPI dashboard**

6. **Agreement/SBS schematic**
7. **Slot allocator with occupancy/collision heatmap**

Production Tables (Cross-Referenced)

1. Metric definitions & threshold bands
2. API summary & event taxonomy
3. Ablation results (±Ô, ±slots, ±certificates)
4. Commute vs conflict instrument pairs
5. Policy gates & escalation ladders

Boxed Callouts

- **CWA Certificate** — when *project→add* is safe
- **PBHL Residual** — what it means, how to control
- **Agreement Check** — commuting + shared record recipe
- **Slot Budget Rule** — integer capacity guardrails

Ready-to-Write Checklist per Chapter

- 1-sentence **goal**
- 3–6 bullets **what you'll implement**
- **Key concepts** definitions
- **Worked example(s)** with minimal math/code
- **Tests & metrics** with thresholds
- **Artifacts**: APIs, configs, figures, tables, runbooks

© 2025 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.

ObserverOps Technical Blueprint - I

Front Matter

Title Page

ObserverOps: A Unified Observer-Based AI/AGI Framework
From Quantum Collapse Geometry to Industrial Governance
Tagline: *No metaphysics. Just observers you can build.*

Version: 0.1 (Draft) — **Date:** 22 September 2025
Authors: *Chung Leung Danny Yeung*

Abstract

ObserverOps is a practical architecture for building reliable AI/AGI by treating **observers as processes with memory and schedulers**. The framework organizes systems across three layers—**micro** (inside an agent), **meso** (field-level control and aggregation), and **macro** (program/governance)—and enforces four invariants:

1. **Internal collapse (latching):** when an agent writes a measurement to its trace at tick τ_k, that record becomes in-frame irreversible and future control branches on it.
2. **Cross-observer agreement:** when instruments' effects commute and records are shared/redundant (SBS-like), distinct observers converge in effective outcomes.
3. **Slot conservation:** attention, memory, and tool buffers are quantized **slots**; writes are discrete and non-overlapping, with explicit eviction policies.
4. **Belt closure:** at the program level, **Gap≈Flux+α·Twist** with a monitored **Residual** provides a conservation-style constraint tying plan↔do loops.

A unifying aggregation law, **CWA (Collapse Without Alignment)**, certifies when post-projection signals can be additively pooled (mean/sum) because order/phase wash out; when certificates fail, ObserverOps falls back to order-aware estimators.

We instantiate these ideas in a reference **runtime and control plane**: a **projection-first scheduler $\hat{O}$**, a **tick engine τ** (commit rhythm), an immutable **trace ledger T**, a **certificate-gated aggregator** (CWA), and **BeltOps** for macro closure and governance. The system exposes APIs, metrics, and tests suitable for tool-using LLM agents, RAG pipelines, RL/robotics, and multi-team programs.

Measurable wins include higher agreement across replicas, lower mis-executions via latching and compatibility checks, reduced RAG latency under certificate-gated pooling at iso-accuracy, and governance clarity through Five-Line KPIs and Residual bounds. This blueprint provides the formal definitions, implementation patterns, evaluation protocols, and dashboards needed to move from theory to production.

Keywords: observer, internal collapse, cross-observer agreement, SBS, SMFT, projection operator $\hat{O}$, semantic ticks τ, CWA, slot conservation, PFBT belts, PBHL residual, governance telemetry.

Audience & Use-Cases

- **Physicists/Theorists:** formal observer tuples, compatibility/commutation, SBS links, proof sketches.
- **AI Engineers (Agents/RAG/RL):** Ô-first scheduling, τ cadence, immutable traces, certificate-gated pooling, slot budgets.
- **Safety/Governance Leads:** BeltOps KPIs, Residual control, policy gates, audit exports.
- **Educators/Students:** classroom labs (qubit toy, gridworld SMFT, RAG pooling battery, belt simulator) and exam-ready checks.

Representative Use-Cases

- Tool-using LLM fleets with agreement checks and SBS-style redundancy.
- RAG systems using CWA certificates to decide when simple pooling is safe.
- Robotics/IoT with compatibility-aware sensing and fleet tick sync.
- Organizational programs managed with belts, KPIs, and Residual thresholds.

How to Use This Blueprint

Two reading paths

1. *Foundations → APIs → Patterns:* learn invariants, then adopt the reference architecture.
2. *Patterns-first:* start with LLM Agents (Ch.14) or RAG (Ch.15), then backfill theory.

Conventions

- Symbols: **Ô** (projection/scheduler), **τ** (tick/commit), **T** (trace), **Π** (instrument set), **C** (compatibility/commutation), $\mathbf{\Psi_m}$ (semantic field), **AL** (Attractor Load), **S_c** (Collapse Entropy), **ρ** (sync order), **Δτ** (desynchrony), **PBHL Residual**.
- Code shown as pseudocode first, then API payloads (REST/gRPC).
- Policies & thresholds provided as YAML templates; figures and tables are cross-referenced by chapter.

What you'll build

- An observer runtime with latching writes and agreement checks.
- A CWA engine that certifies additive pooling and triggers fallbacks.
- A slot allocator and tick/sync manager.
- BeltOps dashboards with Five-Line KPIs and Residual gates.

Notation & Symbols (Mini-Gloss)

- **Observer tuple** $O = (S, T, \hat{O}, \tau, \Pi, C)$: state, trace, scheduler, tick, instruments, compatibility.
- **Internal collapse:** in-frame immutability of **T** at tick τ_k; downstream control branches on it.
- **Compatibility/Commutation (C):** edge if two instruments' effects commute; governs safe sequencing.
- **SBS:** spectrum broadcast structure; redundancy of pointer information yielding objectivity-like agreement.
- **SMFT /** $\Psi_m(x, \theta, \tau)$**:** semantic field over context x, channel/orientation θ, tick τ; drives Ô.
- **CWA:** certificate of invariance under permutation, sign-flip, and chunk-shuffle tests; numeric score $[0, 1]$.
- **Slot conservation:** discrete addresses for memory/attention/tool buffers; non-fractional, non-overlapping writes.
- **PFBT/PBHL:** Purpose-Flux Belt Theory; **Gap≈Flux+α·Twist**; **Residual = |Gap − (Flux+α·Twist)|**.
- **Five-Line KPI:** Gap, Flux, Twist, Coherence/Sync, Residual.

Prerequisites

- Linear algebra & probability (projection operators, conditional expectation).
- Intro QM notation (commutation, measurement records) — no metaphysics required.
- Basic control/ops (closed-loop control, KPIs, SLAs).

- Familiarity with LLM agents/RAG helpful but not required.

Status & Links

Status: v0.1 (draft). **Scope:** reliability, safety, governance—not consciousness.
Code/Demos (placeholders):

- `/sdk/observer-runtime` — trace ledger, Ô/τ loop, agreement engine.
- `/sdk/cwa-engine` — projector library, certificate tests, fallbacks.
- `/apps/beltops` — KPIs, PBHL Residual controllers, policy gates.
  Artifacts: configs (YAML), dashboards, evaluation harnesses (agents, RAG, belts).

Part I — Foundations: Observers You Can Build (Micro→Meso→Macro)

Chapter 1 — What Is ObserverOps?

1.0 One-paragraph view

ObserverOps treats an **observer as a process with memory and a scheduler**. The memory is an immutable **trace**; the scheduler is a **projection operator Ô** that chooses the next instrument/channel to apply at discrete **ticks τ**. Four invariants—**internal collapse**, **cross-observer agreement**, **slot conservation**, and **belt closure**—plus one aggregation law (**CWA: Collapse Without Alignment**) make the loop predictable, auditable, and composable from single agents to organizations.

1.1 The Observer Tuple

We formalize an observer as

$$O = (S,\ T,\ \hat{O},\ \tau,\ \Pi,\ C)$$

- **S** — internal state/model (parameters, beliefs, tool context).
- **T** — **trace**: an append-only, ordered log of $(\tau_k, \pi_k, y_k,$ meta$)$.
- $\hat{O}$ — projection/scheduling policy that selects the next instrument $\pi \in \Pi$.
- τ — discrete commit index (“tick”); advances only on successful writes.
- Π — instrument/channel set (sensors, tools, prompts, retrievers, actions).
- **C** — compatibility/commutation graph over Π (edge ⇔ commuting effects).

Normative loop (conceptual):

1. $\pi_k \leftarrow \hat{O}(S, T)$
2. $y_k \leftarrow \text{measure}(\pi_k)$
3. commit ← write_trace(τ_k, π_k, y_k)
4. $\tau \leftarrow \tau + 1$; update S conditioned on the new record.

1.2 The Four Invariants (what must hold)

1. **Internal collapse (latching)**
   Writing (τ_k, π_k, y_k) into **T** makes that outcome in-frame irreversible. All downstream scheduling and control are **branch-conditioned** on T; no silent retro-edits.

2. **Cross-observer agreement (under commuting effects + shared records)**
   If instruments commute on the relevant subspace and records are shared/redundant, distinct observers converge in effective outcomes on those channels (SBS-style objectivity).

3. **Slot conservation (quantized capacity)**
   Attention/memory/tool buffers are **integer slots**. Writes are non-fractional, non-overlapping; evictions/aging are explicit policies with observable occupancy/collisions.

4. **Belt closure (macro conservation)**
   At program level, **Gap≈Flux+α·Twist** with monitored **Residual** constrains plan↔do loops; controllers keep Residual within bounds.

1.3 The Aggregation Law (CWA: Collapse Without Alignment)

Principle. After projection, some observables are invariant to order/phase; **then** additive pooling (mean/sum) is valid.
Practice. Run a **certificate**—permutation, sign-flip, and chunk-shuffle tests—to obtain a **CWA score** [0, 1]; gate pooling by a threshold θ. If the score is low, fall back to order-aware estimators (attention, CNN, sequence models).

1.4 "Hello-Observer" — a minimal, working loop

Goal. Show trace writes (latching) and ticks in ~30 lines.

Pseudocode

```
from dataclasses import dataclass, field
from hashlib import blake2s
from typing import List, Tuple, Dict, Any
import time, json, random

@dataclass
class Record:
    tau: int
    channel: str
    outcome: Any
    meta: Dict[str, Any]
    prev_hash: str
    hash: str

def h(prev, tau, ch, y, meta):
    m = json.dumps({"prev": prev, "tau": tau, "ch": ch, "y": y, "meta": meta}, sort_keys=True).encode()
    return blake2s(m).hexdigest()

@dataclass
class Observer:
    S: Dict[str, Any] = field(default_factory=dict)
    T: List[Record] = field(default_factory=list)
    tau: int = 0
    Pi: List[str] = field(default_factory=lambda: ["A", "B"])
    C: Dict[Tuple[str,str], bool] = field(default_factory=lambda: {
         ("A","A"): True, ("B","B"): True, ("A","B"): True, ("B","A"): True})

    def O_hat(self) -> str:
        # simple scheduler: pick channel that reduces uncertainty proxy
        return "A" if self.S.get("prefer_A", True) else "B"

    def measure(self, ch: str):
        return {"A": random.choice([0,1]), "B": random.uniform(0,1)}[ch]

    def write_trace(self, ch: str, y: Any):
        prev = self.T[-1].hash if self.T else "genesis"
        rec = Record(self.tau, ch, y, {"t": time.time()}, prev, "")
        rec.hash = h(prev, rec.tau, rec.channel, rec.outcome, rec.meta)
        self.T.append(rec)
        self.tau += 1  # latch: only advance τ on successful append
        # branch conditioning example:
        if ch == "A" and y == 1: self.S["prefer_A"] = False
        return rec

def step(obs: Observer):
    ch = obs.O_hat()
```

```
    y  = obs.measure(ch)
    rec = obs.write_trace(ch, y)
    return rec
```

What to notice

- **Latching:** τ increments only on append; history is hash-chained (immutability check).
- **Branching:** the scheduler $\hat{O}$ flips preference after a particular outcome.
- **Compatibility:** here $C(A,B) = C(B,A) =$ True (they commute by design).

Smoke test

```
obs = Observer()
for _ in range(3): print(step(obs))
# then verify immutability:
tamper = obs.T[1]; tamper.outcome = "edited?"  # BAD
recalc = h(obs.T[0].hash, tamper.tau, tamper.channel, tamper.outcome, tamper.meta)
assert recalc != tamper.hash, "Trace immutability violated!"
```

1.5 Example 1 — Hello-Observer (walkthrough)

1. Start with $\tau_0 = 0$, empty T.
2. Ô selects **A**; outcome $y_0 \in \{0,1\}$; write $(0, A, y_0) \to \tau_1 = 1$.
3. If $y_0 = 1$, scheduler flips preference; next tick likely chooses **B**.
4. Hash-chain prevents silent retro-edits: any mutation breaks the chain.

Takeaway. Internal collapse + ticks give you *replayable branches* and *auditable control.*

1.6 Example 2 — Two observers, commuting instruments (agreement)

Setup

- Instruments: $Z_0 = |0\rangle\langle 0|$, $Z_1 = |1\rangle\langle 1|$ (projective, diagonal; they commute).
- Observers O_A, O_B share: instrument set $\{Z_0, Z_1\}$, **C** where every pair commutes, and a **shared ledger** (both append and read T_shared).

Protocol

1. O_A measures Z_0 at $\tau_k \to$ outcome $y \in \{0,1\}$; writes to T_{shared}.
2. O_B observes T_{shared} and (optionally) re-measures a commuting instrument.
3. Because effects commute and the record is shared/redundant, both observers' effective outcomes match on these channels.

Minimal code sketch

```
def agree(Ta, Tb):
    # compare overlapping ticks/channels; return pass/fail and score
    ks = [(r.tau, r.channel) for r in Ta]
    common = [ (a,b) for a in Ta for b in Tb if (a.tau,a.channel)==(b.tau,b.channel) ]
    if not common: return ("no-overlap", 1.0)
    m = sum(1 for a,b in common if a.outcome==b.outcome)
    return ("pass" if m==len(common) else "fail", m/len(common))
```

What makes agreement hold here

- **Commutation:** order doesn't matter for Z_0, Z_1.
- **Shared record:** both see the same pointer state; SBS-like redundancy.
- **Latching:** once written, subsequent control branches on the same fixed point.

Counterexample (for intuition)
Switch one instrument to X (non-commuting with Z on a qubit). Agreement can fail unless you coarsen the observable or serialize the schedule.

1.7 Tests & Metrics (chapter-level)

- **Mis-execution rate (Mis-exec↓):**
  Fraction of steps where the executed channel violates constraints (compatibility, policy, or precondition).

  $$\text{MisExec} = \frac{\#\{\text{violations}\}}{\#\{\text{ticks}\}}$$

  Example: choosing a non-commuting channel in a region where the policy demands commuting pairs.

- **Agreement rate (↑):**
  Over overlapping (τ, channel) keys, the match ratio of outcomes across observers.

  $$\text{Agree} = \frac{\sum \mathbf{1}[y^{A}_{k,\pi} = y^{B}_{k,\pi}]}{\#\text{overlaps}}$$

  Use: track with and without shared redundancy; expect ↑ when C enforces commuting.

- **Trace immutability check (Pass):**
  Hash-chained records; recompute $h_k = H(h_{k-1}, \tau_k, \pi_k, y_k, \text{meta})$. Any mutation ⇒ mismatch.
  Policy: fail → quarantine run; raise incident; forbid "patching" past τ without a new append with justification.

- **Additional quick checks**
  - **Slot occupancy** (ties to Chapter 4): ensure non-overlap & within budget.
  - **Latency per tick**: bounds for real-time systems.
  - **Ô desynchrony Δτ** between replicas: indicates drift.

1.8 Artifacts

Figure F-1. ObserverOps Stack (micro→meso→macro).
Three bands (agent internals, field-level control, program belts) with the Ô-first loop and the T ledger flowing upward to CWA and BeltOps. Callouts to invariants at each level.

Figure F-2. Ô-First Loop with Latching Point.
A cycle diagram: *select_channel → measure → write_trace (latch) → update S → advance τ*. The latching point is explicitly marked; side-rail shows hash-chain.

Table T-1. Invariants ↔ Failure Modes Mitigated

Invariant	What it enforces	Typical failure without it	Where it shows up
Internal collapse	In-frame immutability of outcomes; branch-conditioned control	Retroactive edits, non-reproducible runs	Agent loop, trace ledger
Cross-observer agreement	Convergence under commuting effects + shared records	Replica contradictions, flaky CI	Replicated agents, SBS logging
Slot conservation	Integer, non-overlap resource writes	Cache thrash, tool storms, heisenbugs	RAG caches, tool budgets
Belt closure	Gap constrained by Flux+α·Twist with Residual guard	KPI drift, goal thrash, governance gaming	Program ops, quarterly reviews

---

1.9 Implementation checklist (ready for code)

- Define Π and encode C (compatibility/commutation matrix).
- Implement $\hat{O}$ policy (even a simple heuristic): `select_channel(S,T)`.

- Implement `measure(π)` adapters for your instruments/tools.
- Implement `write_trace(τ, π, y)` with a **hash-chain**; advance τ **only** on append.
- Add **agreement** endpoint for replicas; start with exact-match over overlaps.
- Surface **metrics**: Mis-exec, Agreement, Trace Immutability status; export to your observability stack.
- Add **policy gates** (Chapter 6/7 hooks) later: CWA score threshold; PBHL Residual threshold.

1.10 What's next

- **Chapter 2 (Internal Collapse)** formalizes latching and the no-retro-edit guarantee.
- **Chapter 3 (Agreement)** gives compatibility tests and SBS redundancy patterns.
- **Chapter 4 (Slots)** turns budgets into first-class, observable constraints.

Chapter 2 — Internal Collapse (Latching Writes in Frame)

2.0 Why internal collapse?

"Internal collapse" is the guarantee that once a measurement outcome is **committed to the trace at tick** τ_k, it becomes **in-frame irreversible** and **all downstream control branches on it**. This simple rule turns an agent's loop into something you can **replay, audit, and reason about**, eliminating silent retro-edits that cause non-reproducible behavior.

2.1 Formal definition (fixed points & branching)

Let $T_{\leq\tau}$ be the σ-algebra (information set) generated by all committed records up to tick τ. A **commit** at τ_k stores $(\tau_k, \pi_k, y_k, \text{meta}_k)$ and advances $\tau \to \tau + 1$.

Internal collapse requires:

1. **Fixed-point condition (conditional expectation)**

$$\mathbb{E}[Y_k \mid T_{\leq\tau_k}] \equiv y_k$$

   i.e., once written, the agent treats the realized outcome as **delta-certain inside its frame**.

2. **Branch-dependent control**
   For any future scheduling/control f (e.g., next-channel policy $\hat{O}$),

$$f(S, T_{\leq\tau_{k+1}}) \text{ is measurable w.r.t. } T_{\leq\tau_k}$$

   meaning: downstream decisions **cannot depend** on hypothetical mutations of records $< \tau_{k+1}$.

3. **Append-only trace & uniqueness at each tick**
   At most one record has key (τ_k); corrections are appended as **new records** with `kind:"correction"`—never by mutating history.

2.2 Implementation: `write_trace(τ, π, y)` with a latching guard

Data shape (event-sourced record)

field	type	notes
`tau`	int	commit index (monotone)
`channel`	string	instrument/tool $\pi \in \Pi$
`outcome`	JSON	measurement/tool result
`meta`	JSON	timestamps, actor, cost, etc.

field	type	notes
`prev_hash`	hex	hash of the previous record (or `"genesis"`)
`hash`	hex	hash over (`prev_hash, tau, channel, outcome, meta`)
`kind`	enum	`"normal"` | `"correction"` | `"rollback_hint"`
`idempotency_key`	uuid	dedupe protection per tick

Latching guard (invariants enforced):

- **Monotone τ:** the stored `tau_last` must equal request's `tau` (optimistic) or `tau-1` (server assigns).
- **Hash-chain:** `prev_hash` must match the last record's `hash`.
- **Immutability:** `UPDATE` on a past record is **disallowed**; use an **append** with `kind:"correction"`.
- **Idempotency:** same `idempotency_key` at same `tau` → serve cached commit, not a duplicate.
- **Atomicity:** measurement→commit is a single transaction; if it fails, **no τ advance**.

Reference pseudocode

```
def write_trace(store, tau_in, channel, outcome, meta, idem_key):
    last = store.last_record()
    assert last.tau == tau_in or last is None and tau_in == 0, "τ mismatch"
    prev = last.hash if last else "genesis"
    rec  = {
        "tau": tau_in,
        "channel": channel,
        "outcome": outcome,
        "meta": meta,
        "prev_hash": prev,
        "kind": "normal",
        "idempotency_key": idem_key,
    }
    rec["hash"] = H(rec["prev_hash"], rec["tau"], rec["channel"], rec["outcome"], rec["meta"])
    # atomic append; rollback on any failure
    store.append(rec)
    # latch: only now advance τ for the process
    return rec
```

Branch-conditioning hook: your scheduler $\hat{O}$ reads **T** and **S** *after* the commit; it must not read uncommitted buffers.

2.3 Examples

A) Qubit toy — Z then X vs X then Z; where latching happens

Let the initial state be $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$.

Case 1: Z then X

1. **Tick** τ_0: measure Z. Outcomes: 0 with 0.5, 1 with 0.5.
   - **Commit** (τ_0, Z, y_Z) → **latch**; the state has collapsed to $|0\rangle$ or $|1\rangle$.
2. **Tick** τ_1: measure X. From either $|0\rangle$ or $|1\rangle$, X yields $+1$ or -1 **with 0.5 each**.
   - **Commit** (τ_1, X, y_X).

Case 2: X then Z

1. **Tick** τ_0: measure X. From $|+\rangle$, the outcome is $+1$ **deterministic**.
   - **Commit** $(\tau_0, X, +1)$ → **latch**; state is $|+\rangle$.
2. **Tick** τ_1: measure Z. Outcomes: 0 or 1 with **0.5/0.5**; commit.

What this shows:

- The **order** of non-commuting instruments changes intermediate distributions.
- **Where latching occurs:** exactly at each **trace write** (the commit points).
- The system's downstream control (e.g., whether to re-measure X or stop) must **branch on the committed record**, not on any hypothetical pre-commit value.

  Operational lesson: even in classical tooling, treat certain tool pairs as "effectively non-commuting" (e.g., actions that change the environment). Latch after the first.

B) Tool call — `plan → tool → trace`; forbid retroactive mutation

Flow

1. **Plan:** Ô picks tool `web.search` given S, T.
2. **Measure:** execute tool; receive `y`.
3. **Commit:** `write_trace(τ, "web.search", y)`; **advance τ**.
4. **Branch:** If `y.top_hit.domain=="docs"`, schedule `web.get`; else `vector.search`.

Forbid retro-edit: suppose a later post-processing step wants to "clean" `y`. It must **append**:

```
{
  "tau": 12,
  "channel": "web.search",
  "outcome": {"correction_of": 9, "reason": "dedupe", "delta": "..."},
  "kind": "correction",
  "prev_hash": "…",
  "hash": "…"
}
```

Any attempt to `UPDATE` tick 9 is rejected with **409 Conflict**.

2.4 Tests & Metrics

(T1) Trace hash-chain integrity (unit & periodic)

- Recompute $h_k = H(h_{k-1}, \tau_k, \pi_k, y_k, \text{meta}_k)$.
- **Pass** if all links match. **Fail** → quarantine run; export incident with the first broken index k[*].

(T2) "No silent retro-edit" (unit test)

- Arrange: write at $\tau = 5$; snapshot `hash_5`.
- Act: attempt to mutate record 5; expect **409 Conflict**.
- Assert: the next valid change appears as a **new** tick $\tau = 6$ with `kind:"correction"`.

(T3) Idempotent commit

- Same `(τ, channel)` with identical `idempotency_key` twice → **one** stored record.

(M1) Trace half-life $t_{1/2}(k)$
"How long does record k continue to influence Ô?"

- Define influence as the change in selection probability when you **mask** record k:

$$I_k(\tau) = \Pr[\hat{O}_{\text{full}} = \pi \mid T_{\le \tau}] - \Pr[\hat{O}_{-\{k\}} = \pi \mid T_{\le \tau}]$$

- The **half-life** is the smallest Δ such that $|I_k(\tau_k + \Delta)| \le \frac{1}{2}|I_k(\tau_k)|$.
- **Why track:** long half-lives imply "sticky" branches (good for stability, bad for agility).
- **Dashboard:** distribution of $t_{1/2}$ by channel type; alert on outliers.

(M2) Latch latency — time from `measure` end to `write_trace` commit; bound for real-time systems.

(M3) Branch determinism — entropy of $\hat{O}$'s next-channel distribution **before vs after** commit; should drop after latching.

2.5 API snippets

POST `/measure` — execute instrument and (optionally) auto-commit

```
POST /measure
Content-Type: application/json

{
  "tau": 17,
  "channel": "sensor.camera",
  "args": {"exposure": 0.01},
  "policy": {"auto_commit": true, "idempotency_key": "7e2c..."},
  "meta": {"actor": "agent-3"}
}
```

200 OK

```
{
  "tau": 17,
  "outcome": {"jpeg_ref": "s3://.../f17.jpg", "md5": "…"},
  "committed": true,
  "record": {
    "tau": 17, "channel": "sensor.camera",
    "prev_hash": "8d4…", "hash": "a6b…",
    "timestamp": 169..., "kind": "normal"
  }
}
```

GET `/trace/:id` — retrieve immutable trace (range & proof options)

```
GET /trace/run-42?from_tau=0&to_tau=40&include_hash=true&proof=range
```

200 OK

```
{
  "run_id": "run-42",
  "records": [
    {"tau":0,"channel":"plan","outcome":{...},"prev_hash":"genesis","hash":"..."},
    {"tau":1,"channel":"web.search","outcome":{...},"prev_hash":"...","hash":"..."}
  ],
  "proof": {"type":"range","root":"c0e...","first":"...","last":"..."}
}
```

Error codes

- **409 Conflict:** attempted update to historical record; τ mismatch; broken hash-chain.
- **412 Precondition Failed:** `prev_hash` does not match last known hash.
- **428 Precondition Required:** missing `idempotency_key` for auto-commit.

2.6 Figure callout — Ô-first loop (with latching point)

A cycle: **select_channel → measure → write_trace (LATCH) → update S → advance τ**.
Side rail: hash-chain boxes showing `prev_hash → hash`. A red X overlays any path attempting to re-enter a past node.

2.7 Implementation checklist & pitfalls

- Append-only log with **hash-chain** and idempotency keys.
- Advance **τ only on commit**; never on "attempt".
- Make **scheduler Ô** read from **committed** T only.

- Provide a **correction** record type; forbid `UPDATE`.
- Bound **latch latency**; alert on spikes.
- Record **branch determinism** drop after latching; investigate if not dropping.
- **Pitfalls:** background "cleaners" that mutate history; retries without idempotency; partial commits that advance τ.

Outcome of this chapter: you now have a **provably append-only, branch-conditioning trace**. Next, we'll define how **agreement** emerges between distinct observers under commuting effects and shared records (Chapter 3).

Chapter 3 — Cross-Observer Agreement (Commuting Effects + Shared Records)

3.0 Why agreement?

Two agents can look at "the same thing" yet disagree for perfectly lawful reasons (non-commuting instruments, hidden state, stale records). **Cross-observer agreement** turns this into an engineering discipline: **when** do we expect convergence, **how** do we test for it, and **what** do we expose to operations?

Goal. Specify operational pre-conditions for agreement and provide a drop-in `agree(T_a, T_b)` that returns a score, provenance, and actionable diagnostics.

3.1 Formal objects

Observer tuple (recap)

$$O_i = (S_i,\ T_i,\ \hat{O}_i,\ \tau_i,\ \Pi_i,\ C_i)$$

- T_i: append-only trace of $(\tau, \pi, y, \text{meta})$.
- C_i: **commutativity/compatibility graph** over instruments Π_i. Edge $(\pi, \pi') \in C_i \Leftrightarrow$ effects commute on the relevant subspace.

Frame & channel alignment

Observers may name channels differently. Define a **frame map**

$$\phi_{A\to B}: \Pi_A \to \Pi_B$$

and a **canonical labeler** $\lambda: \Pi_\cdot \to \mathcal{K}$ so agreement is computed over canonical keys $\mathcal{K}$ (e.g., `{"web.search", "db.lookup", "sensor.Z"}`).

3.2 Agreement pre-conditions

We call a pair (O_A, O_B) **AB-fixed** on a set of keys $\mathcal{K}^\star$ if:

1. **Commuting effects**
   For every key $k \in \mathcal{K}^\star$ with aligned instruments (π_A, π_B),

   $$[\pi_A, \pi_B] = 0 \quad \text{on the support visited.}$$

2. **Shared / redundant records**
   There exists a shared ledger T_{shared} or SBS-style redundancy (independent fragments $E_1, \ldots, E_R$) such that pointer information about k is **accessible to both**.

3. **Trace latching**
   Both observers enforce internal collapse (Ch.2), so decisions are conditioned on committed records only.

When (1)–(3) hold, we expect **convergent effective outcomes** on $\mathcal{K}^\star$.

3.3 SBS-style redundancy (operationalized)

Idea. Spectrum Broadcast Structure (SBS): pointer states are redundantly imprinted across many environment fragments so multiple observers can read them without disturbance.

Operational test (coarse, implementation-friendly):

- Split channels for key k into R **independent fragments** $\{E_r\}_{r=1}^{R}$ (e.g., mirrored logs, replicated sensors).
- Compute redundancy

$$\text{Redundancy}_\delta(k) = \max R' \text{: each } r \le R' \text{ yields } I(Y_k; E_r) \ge (1-\delta)\, H(Y_k)$$

where I is mutual information estimated from samples, H is entropy of the pointer outcome. High redundancy ⇒ objectivity-like behavior.

Practical proxy (no MI estimator): majority stability across fragments under resampling; see §3.7 Metrics.

3.4 Implementation: `agree(T_a, T_b)`

Inputs

- T_a, T_b: traces.
- C: commuting matrix on canonical keys.
- λ: labeler; ϕ: frame map(s).
- Options: how to treat non-commuting overlaps, lag tolerance, shared-record proof (hashes / signatures).

Outputs

- `score` $\in [0,1]$, `status` $\in \{$`pass`,`warn`,`fail`$\}$.
- Diagnostics: overlap size, commuting coverage, conflicts by key, redundancy stats, missing-record counts, hash-mismatch proofs.

Pseudocode

```
from collections import defaultdict, namedtuple

Cfg = namedtuple("Cfg", "commute canon labeler frame_map lag_tolerance require_hash")

def canonize(T, labeler, frame_map):
    rows = []
    for r in T:
        k = labeler(frame_map(r.channel))
        rows.append((r.tau, k, r.outcome, r.hash if hasattr(r, "hash") else None))
    return rows

def agree(Ta, Tb, cfg: Cfg):
    A = canonize(Ta, cfg.canon, cfg.frame_map["A"])
    B = canonize(Tb, cfg.canon, cfg.frame_map["B"])

    # index by (tau,k) with optional lag tolerance window
    idxA = defaultdict(list); idxB = defaultdict(list)
    for tau,k,y,h in A: idxA[(tau,k)].append((y,h))
    for tau,k,y,h in B: idxB[(tau,k)].append((y,h))

    overlaps = []
    for (tau,k), valsA in idxA.items():
        for dt in range(-cfg.lag_tolerance, cfg.lag_tolerance+1):
            key = (tau+dt, k)
            if key in idxB: overlaps.append(((tau,k), valsA, idxB[key]))

    if not overlaps:
        return {"status":"warn","score":1.0, "reason":"no-overlap"}
```

```
    matches, total = 0, 0
    conflicts_by_k = defaultdict(int)
    noncommuting = 0
    for (tau,k), As, Bs in overlaps:
        if not cfg.commute.get(k, True):
            noncommuting += 1
            continue  # by policy exclude or downweight

        # require shared record equivalence if asked
        if cfg.require_hash and not any(a[1] and a[1] in {b[1] for b in Bs} for a in As):
            conflicts_by_k[k] += 1; total += 1; continue

        # outcome match (mode vs mode)
        a_mode = max(set(a[0] for a in As), key=[a[0] for a in As].count)
        b_mode = max(set(b[0] for b in Bs), key=[b[0] for b in Bs].count)
        matches += int(a_mode == b_mode); total += 1

    score = matches / max(total,1)
    status = "pass" if score>=0.98 and noncommuting==0 else ("warn" if score>=0.9 else "fail")
    return {
        "status": status, "score": score,
        "overlaps": total, "noncommuting_overlaps": noncommuting,
        "conflicts_by_key": dict(conflicts_by_k)
    }
```

Design choices

- **Lag tolerance** accommodates minor τ desynchrony.
- **Non-commuting overlaps** can be excluded or down-weighted.
- **Shared-record proof** by hash match (Ch.2 hash-chain), or by SBS redundancy (see §3.7).

3.5 Examples

A) SBS sketch — redundant pointer channels ⇒ objectivity-like behavior

- **Setup:** A physical pointer variable (e.g., "lamp ON/OFF") is mirrored into three fragments: camera feed E_1, light sensor E_2, log server E_3. Instruments commute (they only read).
- **Procedure:** Both observers query all three; each commits majority across $\{E_1, E_2, E_3\}$ for tick k.
- **Observation:** Majority agreement across fragments is stable under sub-sampling and permutation, yielding high redundancy and **agreement score ≈ 1**.
- **Failure mode:** Remove E_3 (log server) and add a write-disturbing instrument (e.g., toggler) between reads; redundancy drops, agreement degrades.

B) Replicated agents — same tools, shared ledger ⇒ higher agreement

- **Setup:** Two LLM tool-agents run the same tool set Π, write to a **shared append-only ledger** with hash-linked records, then read from it.
- **Observation:** With commutation enforced (read-only tools first, write tools serialized) and shared records, `agree(T_a,T_b)` returns **pass**. If you disable the ledger and allow local caches only, agreement falls due to drift and missing overlap.

3.6 AB-fixedness: sketch of the guarantee

Claim. If (i) $[\pi_A, \pi_B] = 0$ for the aligned instruments on keys $\mathcal{K}^\star$; (ii) both observers have access to shared or SBS-redundant records about those keys; (iii) internal collapse holds, then the **effective outcomes** (the values both use for downstream control) coincide almost surely on $\mathcal{K}^\star$.

Idea of proof.

- Commutation ensures order does not change the joint distribution of outcomes on $\mathcal{K}^\star$.

- Shared/SBS records ensure both condition on the **same pointer information**.
- Latching ensures downstream policies are measurable w.r.t. the same σ-algebra.
  Therefore, the conditional distributions used by both observers match; selecting modes (or expectations) yields the same effective outcome.

3.7 Metrics & tests

Agreement score (primary)

$$\text{Agree} = \frac{\sum_{(k,\tau)\in\mathcal{O}} \mathbf{1}[y^A_{k,\tau} = y^B_{k,\tau}]}{|\mathcal{O}|}$$

with $\mathcal{O}$ the set of commuting overlaps (after frame mapping and lag tolerance).

Redundancy factor (SBS proxy)

- **Majority stability:** resample fragments R times; fraction of resamples with unchanged majority.
- **Permutation stability:** shuffle fragment order; stability ↑ implies redundancy.
- **Thresholds:** green ≥ 0.98, amber 0.95–0.98, red < 0.95.

Hash-match ratio (shared-record proof)

- Over overlaps: fraction where at least one **hash** is identical across observers for the same (τ,k).

Non-commuting exposure

$$\text{NCE} = \frac{\#\text{overlaps flagged non-commuting}}{\#\text{all overlaps}}$$

High NCE predicts disagreement; gate scheduling to reduce it.

Failure counterexamples (unit tests)

1. **Non-commuting pair:** alternate Z and X on a qubit; expect low agreement on intermediate outcomes.
2. **Hidden channel:** one observer applies an unlogged transform; agreement drops despite commuting matrix.
3. **Ledger divergence:** disable shared ledger; simulate cache eviction; agreement decays over time.
4. **Frame mismatch:** mis-mapped channels ϕ (e.g., `A:temp↔B:pressure`); agreement appears low with spurious conflicts.

3.8 API & events

POST `/agree`

```
POST /agree
Content-Type: application/json

{
  "traceA_ref": "run-A-17",
  "traceB_ref": "run-B-09",
  "frame_map": {"A": "phi_AtoK", "B": "phi_BtoK"},
  "commute_matrix": {"sensor.Z": true, "sensor.X": false, "web.search": true},
  "lag_tolerance": 1,
  "require_hash": true
}
```

200 OK

```
{
  "status": "pass",
  "score": 0.993,
  "overlaps": 1421,
  "noncommuting_overlaps": 4,
```

```
  "conflicts_by_key": {"sensor.X": 3},
  "redundancy": {"sensor.Z": {"maj_stability": 0.999, "perm_stability": 0.998}}
}
```

Events
`Agreement.Pass/Fail`, `Agreement.NonCommutingExposure`, `Agreement.HashMismatch`, with links to the exact (τ,k) pairs and record hashes.

3.9 Operational guidance

- **Design for commutation first.** Prefer read-only / idempotent tools upstream; serialize mutators; annotate C explicitly.
- **Externalize records early.** Use a shared, hash-chained ledger for pointer data.
- **Build redundancy, then measure it.** Mirror critical signals across independent fragments; track majority stability.
- **Gate by policy.** If `Agree<θ` or `NCE>θ'`, throttle or force serialization.

3.10 Artifacts

Figure F-3. Agreement/SBS schematic
Two observers reading multiple redundant fragments (E1…E_R) of the same pointer. Commuting reads flow into a shared ledger; majority block shows stability; non-commuting path is dashed/red.

Table T-2. Commute vs conflict pairs (example)

Canonical key	Example instruments	Commute?	Notes
`sensor.Z`	Pauli-Z read, cached log read	✓	read-only; SBS via mirrors
`sensor.X`	Pauli-X read vs Z state	✗	order-sensitive; serialize
`web.search`	HTTP GET, cache read	✓	idempotent if cache consistent
`db.write`	INSERT vs SELECT	✗	serialize writer; read-after-write fence
`retriever.pool`	projection + mean	✓†	only if CWA cert passes

(† commutes under CWA validity; otherwise treat as conflict and fall back.)

3.11 Checklist

- Maintain explicit **commutativity matrix** C over canonical keys.
- Provide robust **frame mapping** ϕ with validation.
- Use **shared ledger** with hash proofs for overlapping keys.
- Instrument **redundancy** and compute stability proxies.
- Enforce **policy gates** on low agreement / high NCE.

Next: Chapter 4 makes **slot capacity** observable and enforceable so agreement doesn't collapse under resource contention.

Chapter 4 — Slot Conservation (Quantized Capacity Law)

4.0 Why slots?

Observers fail noisily when "capacity" is hand-wavy. **Slot conservation** makes attention, memory, and tool use **quantized**: you have an integer number of **slots**; each write or tool run **occupies whole slots, non-fractionally, non-overlapping**. Eviction and preemption are explicit, logged policies—not side effects. Result:

fewer tool storms, fewer cache thrashes, and predictable latency.

4.1 The model (discrete addresses & invariants)

Address spaces. Partition capacity into typed slot pools:

- **Memory slots** (e.g., context/chunk cache): $\mathcal{M} = \{m_1, \ldots, m_{N_M}\}$
- **Attention slots** (focus windows / subgoals): $\mathcal{A} = \{a_1, \ldots, a_{N_A}\}$
- **Tool slots** (concurrent external actions): $\mathcal{U} = \{u_1, \ldots, u_{N_U}\}$

Invariant S1 (integrality): allocations are integers $k \in \mathbb{Z}_{\geq 0}$.
Invariant S2 (non-overlap): at any tick τ, sets of occupied slots are disjoint within each pool.
Invariant S3 (explicit eviction): freeing capacity requires `release()` or policy-driven **evict** events; no implicit GC that mutates history.
Invariant S4 (observability): occupancy, evictions, and collisions are events written to **T** (Ch.2).

4.2 Minimal API & behaviors

```
class Slots:
    def __init__(self, name:str, capacity:int, policy):
        self.name = name                # e.g., "tools", "memory"
        self.N = capacity
        self.used = 0
        self.policy = policy            # LRU, PRI, FIFO, NONE
        self.held = {}                  # token -> k slots
        self.queue = []                 # pending reservations (token, k, pri, ttl)

    def occupancy(self) -> int:         # number of occupied slots
        return self.used

    def allocate(self, token:str, k:int, pri:int=0, ttl:float=None):
        """Try to allocate k slots to 'token'. Returns {ok, evicted[], wait}."""
        if k <= (self.N - self.used):
            self.held[token] = self.held.get(token, 0) + k
            self.used += k
            return {"ok": True, "evicted": [], "wait": 0}
        # policy may evict or queue
        evicted = self.policy.try_evict(self, need=k-(self.N - self.used))
        if k <= (self.N - self.used):
            self.held[token] = self.held.get(token, 0) + k
            self.used += k
            return {"ok": True, "evicted": evicted, "wait": 0}
        # cannot satisfy now: queue
        self.queue.append({"token": token, "k": k, "pri": pri, "ttl": ttl})
        return {"ok": False, "evicted": evicted, "wait": 1}

    def release(self, token:str, k:int=None):
        """Free k or all slots held by token; drain queue if possible."""
        held = self.held.get(token, 0)
        if held == 0: return 0
        free = held if k is None else min(k, held)
        self.held[token] = held - free
        if self.held[token] == 0: self.held.pop(token)
        self.used -= free
        self._drain_queue()
        return free

    def _drain_queue(self):
        # admit queued reservations by priority, respecting policy
        self.queue.sort(key=lambda r: -r["pri"])
        admitted = []
        for r in list(self.queue):
            if r["k"] <= (self.N - self.used):
                self.allocate(r["token"], r["k"], r["pri"])
                admitted.append(r)
        for r in admitted: self.queue.remove(r)
```

Events written to trace T

`Slots.Alloc`, `Slots.Release`, `Slots.Evict`, `Slots.Collision`, `Slots.QueueEnter/Exit`, with `{pool, k, token, reason,`

`policy, occupancy_after}`.

Scheduler guard (Ô integration)
Before choosing a channel π, Ô checks `can_run(π)` via slot availability and queue depth. If insufficient, Ô either picks a commuting alternative or pauses ($\Delta\tau$ back-pressure).

4.3 Policies: eviction & collision logging

- **LRU (least recently used):** evict oldest idle entries (good for caches).
- **PRI (priority):** evict lowest priority first (good for tool budgets).
- **FIFO:** first in, first out (predictable but sometimes suboptimal).
- **NONE:** never evict; callers must queue (good for safety-critical pools).

Collision (definition). A **collision** occurs when an allocation request cannot be satisfied immediately **and** the policy would have to evict an item that is **active** or **unguarded** (e.g., an in-flight tool). Collisions should be rare; they indicate over-subscription or conflicting policies.

Collision record:

```
{
  "evt": "Slots.Collision",
  "pool": "tools",
  "token": "agent#7.call#42",
  "need": 2,
  "have": 0,
  "policy": "PRI",
  "blocked_by": ["agent#3.call#19"],
  "ts": 169...,
  "occupancy": 8
}
```

4.4 Examples

A) RAG cache — 8-slot retriever budget (LRU vs priority eviction)

Setup

- Pool: `memory` with $N_M = 8$ slots for cached projected chunks.
- Workload: mixed “hot” topics (priority 2) and background topics (priority 0).

LRU policy

- Keeps the **most recently used** chunks; background queries evict cold items first.
- Good for temporal locality; can evict a hot but briefly idle chunk.

Priority policy (PRI)

- Reserves space for high-priority topics by evicting lower priority first.
- Maintains SLAs for support queries; background throughput may suffer.

Observation

- **LRU** yields higher overall hit-rate under stationary traffic.
- **PRI** reduces **tail latency** for high-priority flows and lowers collision rate when hot topics spike.

B) Tool budget — bound parallel tools to reduce collisions

Setup

- Pool: `tools` with $N_U = 4$ (at most four concurrent external calls).

- Agents request 1 slot per call; writes occur at commit.

Behavior

- With **NONE** eviction + queueing, bursts are smoothed; Δτ increases modestly but **mis-exec** from timeouts drops.
- With **PRI**, critical tools (e.g., `db.get`) preempt lower-priority ones (`web.search`), lowering P95 latency for critical path.

4.5 Tests & Metrics

Occupancy heatmap (per pool)

- Matrix with time on x-axis, slot index on y-axis; color = state `{free, active, idle, evicted}`.
- Quick signals: banding (stable), zebra (thrash), saturation (risk).

Collision rate

$$\text{CollRate} = \frac{\#\{\text{Slots.Collision}\}}{\#\{\text{allocate requests}\}}$$

- Banding: **green < 0.5%**, **amber 0.5–2%**, **red > 2%**.

Eviction churn

$$\text{Churn} = \#\{\text{Slots.Evict}\}$$

- High churn ⇒ policy mismatch; consider increasing N or switching LRU→PRI.

Queue wait (W)

- P50/P95 wait ticks before admission. Gate Ô to keep **P95(W) ≤ W_max**.

SLA bands (per pool)

Pool	Target CollRate	P95 Wait (ticks)	Max Evict/sec	Note
memory	<0.5%	≤1	≤10	RAG cache locality
tools	<1.0%	≤2	≤2	external quota
attn	~0%	≤0	0	safety-critical

Unit tests

1. **Non-overlap:** allocate $k = N$ then attempt another 1 → expect queue, not overlap.
2. **Idempotent release:** releasing more than held caps at held.
3. **Eviction accounting:** after LRU evicts k, `used` decreases by k, and events are logged.
4. **Queue drain:** freeing slots admits highest-priority waiting reservations first.

4.6 Policy template (YAML)

```
slot_pools:
  memory:
    capacity: 8
    policy: LRU                 # LRU | PRI | FIFO | NONE
    eviction_guard_ms: 50       # don't evict items younger than this
    metrics:
      target_collision_rate: 0.005
      max_evictions_per_sec: 10
      p95_wait_ticks: 1

  tools:
    capacity: 4
    policy: PRI
    priorities:
```

```
    db.get: 100
    retriever.pool: 60
    web.search: 40
    summarize: 20
  preempt: true               # allow kicking lower-pri queued items
  metrics:
    target_collision_rate: 0.01
    p95_wait_ticks: 2

attention:
  capacity: 3
  policy: NONE                # no eviction; strict queueing
  metrics:
    target_collision_rate: 0.0
    p95_wait_ticks: 0
```

4.7 Integration with Ô and τ (control hooks)

- **Pre-select check:** Ô filters candidate channels π to those whose required pools report `can_allocate(k)` at expected tick.
- **Reservation at tick:** when `measure(π)` starts, allocate; on commit (Ch.2), `release`.
- **Back-pressure:** if pools congest, Ô increases $\Delta\tau$ between high-cost channels or prefers commuting, low-slot instruments.
- **Trace linking:** `Slots.*` events reference the (τ, channel) that caused them—enables post-hoc audits.

4.8 Ops hints & pitfalls

Hints

- Start with **LRU** for caches, **PRI** for tools, **NONE** for attention.
- Enforce **guard periods** to avoid evicting freshly written items.
- Track **P95 wait** and **collisions**; tune capacity before changing policy.

Pitfalls

- Hidden allocations (libraries) bypassing the allocator → ghost collisions. Wrap them.
- Long-lived reservations that never release → saturation. Add TTLs and heartbeats.
- Evicting in-flight tools (unsafe). Mark in-flight as **unevictable**.

4.9 Artifacts

Figure F-4. Slot allocator heatmap.
Show time vs slot index with state colors; annotate collisions and evictions.

YAML T-3. Slot policy template.
(Provided above; include as downloadable config.)

4.10 Checklists

- Define typed pools (**memory/tools/attention**) with capacities.
- Enforce **S1–S4** invariants in code and logs.
- Wire **Ô** to check `can_run(π)` and to respect back-pressure.
- Emit and monitor **occupancy, collisions, churn, P95 wait**.
- Review SLA bands weekly; adjust capacity or policy.

Outcome: Your observer now respects **quantized capacity**—no fractional writes, no silent overlaps—yielding lower collision rates and more stable latency, especially under bursty workloads. Next, we move to **SMFT meso-control (Ô & τ as first-class schedulers)**.

Chapter 5 — SMFT Meso-Control (Ô as Scheduler, τ as Commit Rhythm)

5.0 Why SMFT here?

Micro-level invariants gave us **what a safe measurement is**. We now need a principled way to decide **what to measure next**. **Semantic Meme Field Theory (SMFT)** provides that control grammar: model the evolving "meaning field" over context and instrument orientations, then let **Ô** (the scheduler) pick the next channel to sample, while **τ** sets the commit rhythm and fleet sync.

5.1 Field definition (operational)

Let the **semantic field** be

$$\Psi_m(x, \theta, \tau) \in \mathbb{R}^d,$$

with

- x: context/location (task state, doc region, grid cell, etc.),
- θ: **orientation/channel** (a member of Π, or a direction like N/E/S/W),
- τ: tick.

You maintain a **projection** $P\,\Psi_m \in \mathbb{R}^p$ appropriate to your instruments (e.g., scalar scores, vectors for embeddings). The scheduler uses **field-aware scoring** over candidates $\pi \in \Pi$ at current (x, τ).

5.2 Two core SMFT metrics

Attractor Load (AL). Measures concentration of the field (how "pulled" you are to a few orientations):

$$\mathrm{AL}(x, \tau) = \frac{\max_\theta w_\theta \,\|\, P\,\Psi_m(x, \theta, \tau)\|}{\sum_\theta w_\theta \,\|\, P\,\Psi_m(x, \theta, \tau)\| + \varepsilon}.$$

High AL → strong attractor (exploitation); low AL → diffuse field (exploration).

Collapse Entropy S_c. Entropy of **recent channel selections** in a rolling window W:

$$p_\theta(\tau) = \frac{\#\{\text{uses of } \theta \text{ in } [\tau - W, \tau]\}}{\sum_{\theta'} \#\{\text{uses of } \theta'\}}, \qquad S_c(\tau) = -\sum_\theta p_\theta(\tau) \log p_\theta(\tau).$$

Falling S_c indicates **latching** to a subset of channels; persistently high S_c indicates diffusion.

5.3 Tick geometry & synchronization

- **Commit rhythm (τ).** Decisions advance **only** on commit (Ch.2).
- **Desynchrony $\Delta\tau$.** For agent i vs fleet mean $\bar{\tau}$: $\Delta\tau_i = \tau_i - \bar{\tau}$.
- **Kuramoto sync ρ.** For N agents with phases $\phi_i = 2\pi(\tau_i \bmod K)/K$,

$$\rho = \frac{1}{N}\left|\sum_{i=1}^{N} e^{\mathrm{i}\,\phi_i}\right| \in [0, 1].$$

High ρ means cadences align (useful for coordination, A/B tests, governance checkpoints).

5.4 Field-aware selection: Ô policy

We pick the next channel π to (a) reduce uncertainty or AL where it matters, (b) respect budget/slots, (c) prefer commuting effects, and (d) obey cadence gates.

Score components

- **Gain/Uncertainty:** $g(\pi) \approx$ expected drop in AL or variance at (x, τ).
- **Cost:** latency/price/slot pressure $c(\pi)$.
- **Compatibility bonus:** $b(\pi)$ if π commutes with recent instruments.
- **Cadence gate:** indicator $\gamma(\pi) \in \{0, 1\}$ (cool-off, retry windows).
- **Risk:** penalties from policy gates (e.g., CWA not certified for pooling).

Selection

$$\pi^\star = \arg\max_{\pi \in \Pi} \gamma(\pi) \Big[g(\pi) + \beta\, b(\pi) - \lambda\, c(\pi) \Big].$$

5.5 Pseudocode — `select_channel(S,T) -> π` (production-ready sketch)

```
from math import log
from dataclasses import dataclass
from typing import Dict, List, Tuple

@dataclass
class Candidate:
    key: str                 # channel/instrument id
    commute_ok: bool
    slot_need: int
    cost_ms: float
    cooldown: int            # min ticks between uses
    last_tau: int            # last executed tick

def collapse_entropy(counts: Dict[str,int]) -> float:
    total = sum(counts.values()) + 1e-9
    H = 0.0
    for k,v in counts.items():
        p = v/total
        if p > 0: H -= p*log(p+1e-12)
    return H

def field_gain(Psi, x, cand: Candidate) -> float:
    """Proxy: expected AL or variance reduction for cand.key at context x."""
    # Example: gradient magnitude or disagreement among models
    grad = Psi.grad(x, cand.key)          # user-defined: ||∂Ψ/∂θ||
    dens = Psi.mass(x, cand.key)          # how much mass sits under this channel
    return 0.6*grad + 0.4*(1.0 - min(1.0, dens))

def select_channel(S, T, Psi, x, tau, Pi: List[Candidate],
                   slots, commute_matrix, policy) -> str:
    # 1) Cadence gate & slot check
    viable: List[Candidate] = []
    for c in Pi:
        cooldown_ok = (tau - c.last_tau) >= c.cooldown
        slots_ok    = slots.available("tools") >= c.slot_need
        if cooldown_ok and slots_ok:
            viable.append(c)

    if not viable:
        return policy.fallback  # e.g., "wait" or a cheap commuting read

    # 2) Compute components
    counts = T.window_counts(window=policy.window)   # for S_c
    Sc     = collapse_entropy(counts)

    best_key, best_score = None, -1e9
    for c in viable:
        gain = field_gain(Psi, x, c)
        cost = (c.cost_ms/1000.0) + policy.slot_penalty * c.slot_need
        bonus= policy.commute_bonus if c.commute_ok else policy.noncommute_penalty
```

```
        # CWA gate example: if pooling op and cert < θ, de-prioritize
        if policy.is_pooling(c.key):
            cert = S.get("cwa_cert", {}).get(c.key, 0.0)
            if cert < policy.cwa_threshold: bonus -= policy.cwa_penalty

        score = gain + bonus - policy.lambda_cost*cost

        # Encourage some exploration when Sc is high
        score += policy.explore_weight * Sc

        if score > best_score:
            best_score, best_key = score, c.key

    return best_key
```

Notes

- `Psi.grad`/`Psi.mass` are **application adapters** (gridworld, RAG, robotics…).
- Integrates **slots** (Ch.4), **commutativity** (Ch.3), **CWA gate** (Ch.6).
- Exploration weight tied to S_c: more entropy → more exploration.

5.6 Example A — Gridworld semantic field (AL minimization)

Setup

- Grid $x = (i, j)$, four orientations $\theta \in \{N, E, S, W\}$.
- $P\,\Psi_m(i, j, \theta, \tau)$ = scalar "uncertainty" at that oriented edge (e.g., frontier probability or map entropy).

Policy

- At each step, compute AL at (i, j); choose θ that **maximally reduces** AL in the next cell:

$$\theta^\star = \arg\min_\theta \mathrm{AL}\big(x + \delta(\theta), \tau + 1\big).$$

Pseudocode

```
def choose_move(grid, x, tau):
    best, arg = 1e9, None
    for theta in ["N","E","S","W"]:
        x_next = step(x, theta)
        al = attractor_load(grid.Psi, x_next, tau+1)
        if al < best:
            best, arg = al, theta
    return arg
```

Outcome

- On unknown maps, the agent first explores (low AL, high S_c).
- As structure emerges, AL increases along corridors; S_c drops; the agent exploits.

5.7 Example B — LLM tool selection (Ô chooses tool; τ cadences retries)

Channels Π = { `retriever.search`, `browser.get`, `db.get`, `code.run`, `summarize`}.

Cadence & retries

- Cooldowns: `code.run`=3 ticks, `browser.get`=2, others=1.
- Ô picks the tool with highest field-aware score; retries only when cooldown expires or when new evidence changes $P\,\Psi_m$.

Flow

1. Start with high S_c (diffuse beliefs) → Ô explores: `retriever.search`.

2. CWA certificate passes → `retriever.pool` (safe add) → AL drops.
3. With AL high around "database facts", Ô prioritizes `db.get` (commuting read, high bonus).
4. `code.run` gated by slots + cooldown; run only when needed (reduces collisions, Ch.4).

5.8 Tests & metrics

(M1) S_c trend

- Compute slope over window W: $\Delta S_c = \frac{S_c(\tau) - S_c(\tau - W)}{W}$.
- Expect $\Delta S_c < 0$ as the agent **converges**, small $|\Delta S_c|$ in steady state.
- Alert if S_c oscillates widely (thrash) or stays maximal (aimless exploration).

(M2) $\Delta\tau$ desynchrony

- Track $|\Delta \tau_i|$. Bands: green ≤ 1 tick; amber 2–3; red > 3.
- Use back-pressure (Ô chooses cheap commuting reads) to resync stragglers.

(M3) Kuramoto sync ρ

- Target $\rho \geq 0.9$ for synchronized checkpoints (governance, evaluation).
- If $\rho < 0.75$, phase-lock critical steps (e.g., force `summarize` at modulo K).

(T1) Policy monotonicity

- When `field_gain` strictly prefers π_a over π_b and costs equal, Ô never picks π_b. Unit test with synthetic $P\,\Psi_m$.

(T2) CWA gate obedience

- If `is_pooling(c)` and cert(c) $< \theta$, verify selection probability drops below ε.

(T3) Slot-aware selection

- Under `tools.capacity=4`, burst 6 requests → ensure 2 are queued and Ô prefers commuting, low-cost alternatives meanwhile.

5.9 Artifacts

Figure F-5. Field slices.
Heatmaps of $P\,\Psi_m(x, \theta, \tau)$ across θ at a fixed x, with arrows showing Ô's chosen orientation; AL annotated. Second panel: time series of S_c.

Pseudocode block.
`select_channel` (above) boxed as the canonical Ô policy.

Table T-4. Tick cadences (examples)

Scenario	Base cadence (ticks)	Cooldowns by channel	Sync K	Target ρ
RAG Explore	1	`retriever`=1, `browser`=2, `db.get`=1	8	0.80
RAG Exploit	1	`retriever.pool`=1 (CWA≥θ), `browser`=3	8	0.85
Tool Fleet (mixed)	2	`code.run`=3, `db.get`=2, `search`=1	12	0.90
Governance Checkpoint	1	`summarize` forced at τ≡0 (mod K)	12	0.95

5.10 Implementation checklist

- Implement $P\,\Psi_m$ adapters (`grad`, `mass`) for your domain.
- Track S_c over a rolling window and alert on poor trends.

- Encode cooldowns and compute cadence gates $\gamma(\pi)$.
- Integrate **slots** (Ch.4) and **commutation** (Ch.3) in Ô scoring.
- Compute fleet ρ and enforce phase-locks at checkpoints.
- Gate pooling ops by **CWA certificate** (Ch.6).

Outcome: Ô now **chooses the next channel** using a principled, field-aware score, while τ provides a **stable heartbeat** for commits and fleet sync. Next (Ch.6), we'll certify when it's safe to **add** projected signals with **CWA**.

Chapter 6 — CWA: Collapse Without Alignment (Certified Additive Pooling)

6.0 Why CWA?

Additive pooling (mean/sum) is fast and simple—but often **wrong** when order/phase matters or when representations carry **undetermined signs/orientations**. **CWA** turns "just average it" into a **certified operation**: *only* when a battery of invariance tests passes do we permit project→add; otherwise we automatically fall back to order-aware estimators.

Goal. Decide—per batch, per key—whether additive pooling is safe **after projection**.

6.1 Setup & statement

You have a set of inputs $X = \{x_i\}_{i=1}^{n}$ (chunks/sensors/samples) and a **per-item projector** P producing $z_i = P(x_i) \in \mathbb{R}^d$. Candidate additive aggregator:

$$A_{\text{add}} = \frac{1}{n}\sum_{i=1}^{n} z_i \quad \text{(or sum)}.$$

CWA holds for X under P if, for a family of perturbations $\{\mathcal{T}_j\}$ that scramble **order/phase/sign/chunking**, the aggregated output remains within a **validity band** around the baseline:

$$\text{dist}\left(A_{\text{add}},\ A_{\text{add}}^{(j)}\right) \le \epsilon \quad \forall\, j \in \mathcal{J}.$$

Here $A_{\text{add}}^{(j)} = \frac{1}{n}\sum_i P(\mathcal{T}_j(x)_i)$. The **CWA certificate** scores how consistently this holds.

6.2 The certificate battery (three core tests)

1. **Permutation test** (order invariance)
   - Draw K random permutations π_k; recompute A_k on $\pi_k(X)$.
   - Distances $d_k = \text{dist}(A_{\text{add}}, A_k)$ should be small.
2. **Sign-flip test** (orientation ambiguity)
   - Draw masks $s_i \in \{-1, +1\}$ i.i.d. with $\Pr(s_i = -1) = p$ (e.g., $p = 0.5$); recompute A_k on $\{s_i z_i\}$ or, more conservatively, on **whitened PCA** components with random sign flips on top r axes.
   - If sign conventions are arbitrary across producers, stable aggregates imply true additive observables.
3. **Chunk-shuffle / re-chunk test** (boundary robustness)
   - Repartition the raw stream/text into new chunks (same total length), re-project $\tilde{z}_i = P(\tilde{x}_i)$, recompute A_k.
   - Sensitive pipelines (positional encodings, context bleed) will move.

Distance choices

- Vectors: cosine distance $1 - \cos(A, B)$ + normalized norm gap $\| A - B \| / \max(\| A \|, \| B \|, \varepsilon)$.
- Scalars: relative error $| A - B | / (| A | + \varepsilon)$.
- Compose by a weighted sum.

6.3 Phase-Risk Index (PRI)

A quick **order/phase risk** gauge before running the full battery.

Option A (spectral):
Project sequence to a scalar proxy $s_i = \langle z_i, u \rangle$ along the first PCA axis u. Compute DFT power $P(f)$.

$$\mathrm{PRI}_{\mathrm{spec}} = \frac{\max_{f>0} P(f)}{\sum_f P(f)} \in [0, 1].$$

High values ⇒ strong periodic/phase structure (order matters).

Option B (reversal sensitivity):

$$\mathrm{PRI}_{\mathrm{rev}} = \frac{\| A_{\mathrm{add}} - A_{\mathrm{rev}} \|}{\max(\| A_{\mathrm{add}} \|, \varepsilon)}, \quad A_{\mathrm{rev}} = \frac{1}{n} \sum_i z_{n+1-i}\,.$$

Use $\mathrm{PRI} = \max(\mathrm{PRI}_{\mathrm{spec}}, \mathrm{PRI}_{\mathrm{rev}})$.

6.4 CWA score & validity band

For each test family j, gather distances $\{d_k^{(j)}\}_{k=1}^{K}$. Normalize with a **scale** S (e.g., median inter-item norm or $\| A_{\mathrm{add}} \|$). Convert to a **stability score**:

$$s^{(j)} = 1 - \mathrm{clip}\left(\frac{\mathrm{median}_k\, d_k^{(j)}}{S + \varepsilon},\ 0,\ 1 \right).$$

Combine with weights w_j (default equal):

$$\mathrm{CWA_score} = \sum_j w_j\, s^{(j)} \in [0, 1].$$

Validity band:

- **Pass** if CWA_score ≥ 0.98 and PRI ≤ 0.2
- **Warn** if 0.90–0.98 or PRI in (0.2, 0.4]
- **Fail** if < 0.90 or PRI > 0.4

6.5 Pseudocode — certificate + auto-fallback

```
import numpy as np
from typing import List, Dict

def cosine(u, v, eps=1e-9):
    nu, nv = np.linalg.norm(u)+eps, np.linalg.norm(v)+eps
    return float(np.dot(u, v) / (nu*nv))

def dist_vec(a, b, eps=1e-9):
    return 0.5*(1.0 - cosine(a,b)) + 0.5*(np.linalg.norm(a-b) / (max(np.linalg.norm(a), np.linalg.norm(b)) + eps))

def cwa_certificate(X: List, project, agg="mean", K=16, weights={"perm":1,"sign":1,"chunk":1},
                    rechunk=None, sign_mode="pca_topk", topk=8, p_flip=0.5):
    Z = [project(x) for x in X]
```

```
    A0 = np.mean(Z, axis=0) if agg=="mean" else np.sum(Z, axis=0)
    S  = np.median([np.linalg.norm(z) for z in Z]) + 1e-9

    def stability(samples):
        return 1.0 - np.clip(np.median(samples) / S, 0.0, 1.0)

    # Permutation
    d_perm = []
    for _ in range(K):
        zp = [Z[i] for i in np.random.permutation(len(Z))]
        Ak = np.mean(zp, 0) if agg=="mean" else np.sum(zp, 0)
        d_perm.append(dist_vec(A0, Ak))
    s_perm = stability(d_perm)

    # Sign-flip
    d_sign = []
    if sign_mode == "fullvec":
        for _ in range(K):
            mask = (np.random.rand(len(Z)) < p_flip).astype(np.float32)*-2 + 1
            zs = [m*z for m,z in zip(mask, Z)]
            Ak = np.mean(zs,0) if agg=="mean" else np.sum(zs,0)
            d_sign.append(dist_vec(A0, Ak))
    else:
        # PCA whiten + flip top-k axes
        U, _, _ = np.linalg.svd(np.stack(Z)-np.mean(Z,0), full_matrices=False)
        W = U[:,:topk]
        Zp = [z @ W for z in Z]
        for _ in range(K):
            flip = (np.random.rand(topk)<p_flip).astype(np.float32)*-2 + 1
            zs = [ (zp*flip) @ W.T for zp in Zp ]
            Ak = np.mean(zs,0) if agg=="mean" else np.sum(zs,0)
            d_sign.append(dist_vec(A0, Ak))
    s_sign = stability(d_sign)

    # Chunk-shuffle / rechunk
    d_chunk = []
    if rechunk is not None:
        for _ in range(K):
            Xtil = rechunk(X)                 # produces new segments
            Zt   = [project(x) for x in Xtil]
            Ak   = np.mean(Zt,0) if agg=="mean" else np.sum(Zt,0)
            d_chunk.append(dist_vec(A0, Ak))
    s_chunk = stability(d_chunk) if d_chunk else 1.0

    # Combine
    wsum = sum(weights.values())
    score = (weights["perm"]*s_perm + weights["sign"]*s_sign + weights["chunk"]*s_chunk) / wsum
    return dict(score=float(score), components={
        "perm":float(s_perm),"sign":float(s_sign),"chunk":float(s_chunk)}, baseline=A0)

def pool_with_cwa(X, project, order_aware, min_score=0.98, pri_threshold=0.2, **kw):
    cert = cwa_certificate(X, project, **kw)
    pri  = estimate_pri(X, project)  # implement spec/reversal PRI
    if cert["score"] >= min_score and pri <= pri_threshold:
        A0 = cert["baseline"]
        return {"method":"add", "aggregate":A0, "cwa":cert, "pri":pri, "fallback_used":False}
    else:
        Y  = order_aware(X)          # attention/CNN/sequence model
        return {"method":"order-aware", "aggregate":Y, "cwa":cert, "pri":pri, "fallback_used":True}
```

6.6 Examples

A) Embeddings pooling (PASS)

- **Setup:** Doc split into 16 chunks; $z_i = \text{embed}(x_i)$ via a sentence-embedding projector with minimal positional bleed.
- **Observation:** Permuting chunks and re-chunking at ±10% boundaries yields aggregates with cosine ≥ 0.995; sign flips don't move the mean. **CWA score ≈ 0.995–0.999 ⇒ pass**, use mean pooling. **Latency** ↓ vs attention pooling.

B) Coherent sequence (FAIL)

- **Setup:** Time series with alternating regimes; z_i are filtered features that encode phase (e.g., peaks/troughs).

- **Observation:** Reversal / sign-flip shifts aggregate substantially; chunk-shuffle breaks regime alignment. **CWA score ≈ 0.82**, **PRI ≈ 0.55** ⇒ **fail**, auto-fallback to 1D CNN or attention. **Accuracy maintained**, latency ↑—expected trade.

6.7 Metrics & evaluation

- **CWA score** ∈ **[0,1]** (overall and components). Track mean & P5 across batches.
- **PRI** (phase/order risk). Alert on spikes; correlate with failures.
- **Latency vs accuracy** curves: run *add* vs *order-aware* across corpora; record end-task accuracy (retrieval/F1/MAE) and wall-time.
- **Cert pass-rate**: fraction of batches passing CWA; use for capacity planning (how often you can stay in the fast path).
- **Drift of components**: permutation vs chunk sensitivity—helps debugging where invariance fails.

6.8 API — `/pool` (with certificate gating)

```
POST /pool
Content-Type: application/json

{
  "projector": "text-embed-3",
  "aggregate": "mean",               // or "sum"
  "cwa": {
    "tests": ["perm","sign","chunk"],
    "K": 16,
    "weights": {"perm":1,"sign":1,"chunk":1},
    "min_score": 0.98,
    "pri_threshold": 0.20
  },
  "fallback": {
    "kind": "attention",             // attention | cnn1d | rnn | dtw-kernel
    "max_latency_ms": 150
  },
  "inputs": [
    {"id":"seg-01","text":"..."},
    {"id":"seg-02","text":"..."}
  ]
}
```

200 OK

```
{
  "method": "add",
  "aggregate": {"vector_ref": "mem://pool/run123/centroid"},
  "cwa": {
    "score": 0.992,
    "components": {"perm": 0.998, "sign": 0.991, "chunk": 0.987}
  },
  "pri": 0.12,
  "risk_flags": [],
  "latency_ms": 18
}
```

200 OK (fallback)

```
{
  "method": "order-aware",
  "aggregate": {"vector_ref":"mem://pool/run124/attn-head-0"},
  "cwa": {"score": 0.861, "components": {"perm":0.93,"sign":0.72,"chunk":0.85}},
  "pri": 0.47,
  "risk_flags": ["SignSensitive","PhaseSpike"],
  "latency_ms": 94
}
```

6.9 Risk flags schema

```
{
```

```
  "risk_flags": [
    "PermutationSensitive",  // low s_perm
    "SignSensitive",         // low s_sign
    "ChunkSensitive",        // low s_chunk
    "PhaseSpike",            // PRI above threshold
    "LowRedundancy",         // low SBS-style redundancy upstream
    "DriftDetected"          // component trend down vs baseline
  ]
}
```

6.10 Decision tree (figure callout)

1. **Compute PRI.** If PRI > τ_1 ⇒ **fail** → **order-aware**.
2. **Run certificate battery.**
   - If CWA_score ≥ θ ⇒ **add** (mean/sum).
   - Else if CWA_score ∈ (θ', θ) ⇒ **warn**; optionally run a **cheap order-aware** estimator.
   - Else ⇒ **fail** → **order-aware**.
3. **Log** components, flags, and chosen path; export to governance (BeltOps).

Annotate decision nodes with **latency bands** (fast path vs fallback) and **accuracy envelopes** from your evaluation curves.

6.11 Operational guidance

- **Project first, then add.** Never pool raw sequences whose projection entangles order.
- **Keep projectors local & stateless** (per-item), or include them inside the certificate to reflect real behavior.
- **Tune K** (number of perturbations) to hit a latency target; typical K=8–32.
- **Whitelist proven-safe pipelines** (e.g., certain embed models + chunkers) with higher default θ.
- **Record battery artifacts** (perm seeds, flip masks, chunkings) for audit and reproducibility.

6.12 Checklist

- Implement **perm / sign / chunk** tests and PRI.
- Define **distances** and **scale S** for your modality.
- Choose **thresholds** (θ, θ', τ_1) from latency–accuracy studies.
- Wire **auto-fallback** to at least one order-aware estimator.
- Emit **risk flags** and store certificate payloads in the trace.

Outcome: You now have a **certificate-gated pooling** step that safely unlocks additive speed-ups when order/phase wash out—and gracefully falls back when they don't.

Chapter 7 — PFBT Macro Closure (Belts, Gap≈Flux+α·Twist)

7.0 Why belts?

Micro/meso give you safe *steps*. Organizations need safe *program loops*. **Purpose-Flux Belt Theory (PFBT)** models an initiative as a closed belt—an oriented loop that carries **purpose** (targets) and **flux** (delivered change) across time. The belt's worldsheet (loop × time) exposes a conservation-style relationship:

$$\underbrace{\text{Gap}(t)}_{\text{what remains to achieve}} \approx \underbrace{\text{Flux}(t)}_{\text{productive throughput}} + \alpha(t)\ \underbrace{\text{Twist}(t)}_{\text{structural change}}$$

$\Rightarrow \text{Residual}(t) = \text{Gap} - (\text{Flux} + \alpha\,\text{Twist})$

A **PBHL controller** (Purpose-Belt Holonomy Law) keeps **Residual ≈ 0** by actuating two dials: a **fast Flux-gate** (resource & schedule) and a **slow Twist-step** (org/architecture change). Five lines on one panel—**Gap, Flux, Twist, Coherence/Sync, Residual**—make the loop auditable.

7.1 Worldsheet variables (operational definitions)

Let the program belt be a loop of edges **plan → design → build → validate → deploy → plan**. Over time, these edges sweep a worldsheet.

- **Gap(t)** — scalar or vector target shortfall projected on the governing KPI basis.
  Example scalar: remaining story points × business value weight; or error to target NPS.
  Normalization: express as fraction of initial target Gap_0 (so 1.0 at start).
- **Flux(t)** — *effective* value throughput (not raw velocity).
  Example: verified increments per unit time × impact weights × survival rate (post-release).
  Estimator: EWMA of validated increments / week.
- **Twist(t)** — *structural change rate* impacting plan↔do mapping.
  Proxies (choose one or combine):
  - Ownership churn: Jaccard distance of team→component matrix week-to-week.
  - Plan drift: semantic distance between successive OKRs / roadmaps.
  - Architecture churn: file-move entropy + interface break count.
    Units: normalized to [0,1] via historical bands.
- **α(t)** — coupling coefficient (signed) converting structural change into *effective* progress.
  - If a reorg accelerates delivery, α>0; if it causes turbulence, α<0.
  - Estimation: regress ΔGap on (Flux, Twist) over a rolling window.
- **Residual(t)** — closure error: $|\text{Gap} - (\text{Flux} + \alpha\,\text{Twist})|$.
  Target band: **green ≤ ε_g**, **amber ≤ ε_a**, **red > ε_a** (defaults below).
- **Coherence/Sync (ρ)** — cadence alignment across teams (Kuramoto-style from Ch.5); high ρ eases handoffs.

7.2 Controllers: fast Flux-gate, slow Twist-step

Flux-gate (fast, minutes→days)
Adjust how hard/where the belt pushes:

- Slot capacities (Ch.4) for critical tools/pipelines.
- Ô cadence & cooldowns (Ch.5) for delivery steps.
- Priority queues & staffing micro-shifts.

Control law (PI example):

$$u_F(t) = k_P^F \,\text{sgn}(e_R)\,\min\left(|\,e_R\,|,\, u_{\max}\right) + k_I^F \int e_R\,dt, \quad e_R = \text{Residual}(t) - \epsilon_g$$

Twist-step (slow, weeks)
Decide if and how much to restructure:

- Freeze vs unfreeze modules; change ownership; refactor seams.

- Update α prior (belief about twist benefit) and restrict twist rate.

Hysteretic law (bang-bang with deadband):

$$u_T(t) = \begin{cases} 0, & |\text{Residual}| \le \epsilon_a \\ -\eta, & \text{Residual} > \epsilon_a \;\&\; \alpha < 0 \\ +\eta, & \text{Residual} > \epsilon_a \;\&\; \alpha > 0 \end{cases}$$

with guard:

$$|\Delta \text{Twist}| \le \tau_{\max}$$

per review period.

Policy gates

- **Gate-R:** if Residual>ε_red for 2 windows → freeze non-critical twist; increase Flux-gate.
- **Gate-α:** if α variance explodes → require executive review before further reorg.

7.3 Telemetry & the Five-Line KPI

Five-Line KPI (one panel):

1. **Gap(t)** (normalized)
2. **Flux(t)** (effective throughput)
3. **Twist(t)** (structural change)
4. **Coherence/Sync ρ(t)** (cadence)
5. **Residual(t)** with bands and incidents

Recommended defaults

- Window W=2 weeks; EWMA β=0.3.
- Residual bands: ε_g=0.05, ε_a=0.10, ε_red=0.20.
- ρ target ≥ 0.9 at reviews; alert if <0.8.

Derived indices

- **EEI (Effectiveness Efficiency Index)**

  $$\text{EEI} = \frac{\text{Flux}}{\text{Flux} + \text{Waste}}, \quad \text{Waste} = \text{rework} + \text{abandoned} + \text{blocked}$$

  (0–1; ↑ is better)

- **SI (Sustainability Index)**

  $$\text{SI} = \rho \cdot \exp\left(-\lambda_T \max(0, \text{Twist} - \theta_T)\right)$$

  Penalizes excessive twist; scaled to [0,1].

7.4 Implementation: Belt telemetry and PBHL controller

Event schema (written to T)

- `Belt.Tick`: snapshot of {Gap, Flux, Twist, α, ρ, Residual}.
- `PBHL.Update`: controller outputs {u_F, u_T}.
- `PolicyGate.Trigger`: {Gate-R | Gate-α | Gate-ρ}, rationale, actions.

- `Belt.Incident`: raised when Residual>ε_red or α-instability.

Pseudo-service

```
class Belt:
    def __init__(self, alpha_prior=0.0, eps_g=0.05, eps_a=0.10, eps_red=0.20):
        self.alpha = alpha_prior
        self.win = deque(maxlen=14)  # ~2 weeks of daily points
        self.I_F = 0.0

    def estimate_alpha(self, gap_hist, flux_hist, twist_hist):
        # ridge regression on ΔGap = -Flux - α*Twist + noise
        X = np.column_stack([flux_hist, twist_hist])
        y = -np.diff(gap_hist, prepend=gap_hist[0])
        w = ridge(X, y, lam=0.1)      # returns [β_F, β_T]
        self.alpha = float(w[1])      # β_T is α estimate
        return self.alpha

    def tick(self, gap, flux, twist, rho):
        resid = gap - (flux + self.alpha*twist)
        self.win.append({"gap":gap,"flux":flux,"twist":twist,"rho":rho,"resid":resid})

        # Fast Flux-gate (PI)
        e = max(0.0, abs(resid)-0.05)
        self.I_F = clip(self.I_F + e, 0, 1.0)
        uF = clip(0.6*e + 0.2*self.I_F, 0, 1.0)  # normalized actuation

        # Slow Twist-step (hysteresis)
        if abs(resid) > 0.10:
            uT = np.sign(resid)*np.sign(self.alpha) * 0.1  # suggest ±10% twist change
        else:
            uT = 0.0

        return {"Residual":resid, "u_F":uF, "u_T":uT}
```

Actuation mapping

- **u_F**→ slot capacities, Ô cooldowns, staffing, batching factors.
- **u_T**→ freeze/unfreeze refactors, ownership changes, platformization.

7.5 API — `/belt`

POST `/belt` — update and control

```
POST /belt
Content-Type: application/json

{
  "belt_id": "init-AI-helpdesk",
  "snapshot": {
    "gap": 0.62,
    "flux": 0.18,
    "twist": 0.07,
    "rho": 0.91
  },
  "alpha": {"mode":"estimate", "prior": 0.05, "ridge_lam": 0.1},
  "policy": {
    "eps_g": 0.05, "eps_a": 0.10, "eps_red": 0.20,
    "gates": {"Gate-R": true, "Gate-alpha": true, "Gate-rho": true}
  }
}
```

200 OK

```
{
  "residual": 0.37,
  "controllers": {"u_F": 0.28, "u_T": 0.04},
  "actions": [
    {"type":"flux-gate","slot_pool":"tools","delta_capacity":+2},
    {"type":"cadence","channel":"retriever.pool","cooldown_delta":-1},
    {"type":"twist-step","freeze":"major-refactor-A","duration_days":14}
  ],
  "kpi": {"EEI": 0.76, "SI": 0.85},
  "bands": {"residual":"red","rho":"green"},
```

```
  "next_review_days": 7
}
```

7.6 Examples

A) Program belt (positive closure)

- **Start:** Gap=0.80, Flux=0.15, Twist=0.05, α≈+0.20 ⇒ Residual≈0.80−(0.15+0.01)=0.64 (red).
- **Action:** Flux-gate +0.2 capacity to `db.get`; Ô cooldowns lowered for `retriever.pool`; no new twist.
- **After 2 weeks:** Flux→0.32, Twist→0.04, α~+0.18 ⇒ Residual≈0.80−(0.32+0.007)=0.473 (amber).
- **Next:** targeted refactor (Twist 0.06) with α>0; Residual falls to ≈0.29 (amber→green).

B) Incident (twist spike)

- **Event:** Reorg adds Twist=0.20; α turns negative (−0.10) due to churn.
- **Residual:** 0.55−(0.22−0.02)=0.35 (red). **Gate-R triggers**: freeze non-critical refactors 14 days; increase Flux to stabilize; require α review.
- **Recovery:** Twist decays to 0.08; α returns near 0; Residual back under 0.1.

7.7 Tests & metrics

- **PBHL Residual ≤ threshold (primary)**
  Track P50/P95 Residual per window; page if P95>ε_red for 2 consecutive windows.
- **EEI/SI indices**
  - Alert if **EEI** drops >10% w/w (waste spike).
  - Alert if **SI** < 0.7 for 2 weeks (unsustainable twist or desync).
- **α stability**
  Confidence interval width < α_max_var; if wider, don't enact twist steps beyond η_{safe}.
- **ρ (sync)**
  Maintain ρ≥0.9 for review checkpoints; if <0.8, clamp high-variance channels and increase batch size.
- **Counterexample tests**
  1. Inflate raw velocity without validation: Flux↑ but EEI↓ → Residual mismatch; controller should reject vanity gains.
  2. Frequent reorgs: Twist↑ with α negative → Residual stays high; Twist-step should throttle.

7.8 Dashboard mock (Five-Line KPI)

- **Top:** Banded area for **Gap** vs **Flux+α·Twist**; thin line for **Residual** (color-banded).
- **Middle: Twist** with α overlay (green when α>0, red when α<0); markers for `twist-step` actions.
- **Bottom: ρ** (sync) with threshold line; **EEI** and **SI** mini-sparklines.
- **Side panel:** active gates + runbook buttons.

7.9 Runbook (incident: Residual red)

1. **Verify telemetry** (hashes, timestamps).
2. **Classify driver**: low Flux vs high Twist vs α flip.

3. **If low Flux**: enable Flux-gate actions (capacity, cadence, batching).
4. **If high Twist w/ α≤0**: freeze refactors; appoint stabilization lead; schedule α review.
5. **If α unstable**: stop structural changes; collect 1–2 windows of clean data.
6. **Handoff**: record `Belt.Incident` + `PBHL.Update` to T; brief leadership with Five-Line KPI snapshot.

7.10 YAML policy template

```
belt:
  id: "ai-helpdesk-2025Q4"
  windows:
    ewma_beta: 0.3
    review_days: 7
  thresholds:
    residual:
      green: 0.05
      amber: 0.10
      red:   0.20
    rho_min: 0.90
    eei_drop_alert: 0.10   # 10% WoW drop
    si_min: 0.70
  controllers:
    flux_gate:
      kP: 0.6
      kI: 0.2
      u_max: 1.0
    twist_step:
      step: 0.10
      twist_rate_max: 0.15
      hysteresis: true
  actions:
    flux:
      - {type: "slot_capacity", pool: "tools", delta: +2}
      - {type: "cooldown", channel: "retriever.pool", delta: -1}
    twist:
      - {type: "freeze", target: "major-refactor-A", days: 14}
      - {type: "ownership_shift", component: "kb-indexer", from: "plat", to: "search"}
```

7.11 Integration checklist

- Compute **Gap/Flux/Twist/α/ρ** each review tick; write `Belt.Tick`.
- Estimate **α** with ridge (guard against multicollinearity).
- Enforce **gates** when Residual/ρ cross bands; log `PolicyGate.Trigger`.
- Map **u_F/u_T** to concrete actions (slots, Ô cadence, org changes).
- Render Five-Line KPI; export EEI/SI and incidents to GRC.

Outcome: You now close the plan↔do loop with a conservation-style controller that keeps programs on track. Micro/meso decisions roll up to belt telemetry; belts push back down via Flux-gates and Twist-steps—**auditable, tunable, and testable.**

Chapter 8 — Putting It Together (Micro→Meso→Macro Walkthrough)

8.0 Purpose

Compose all invariants into **one operating loop**:
measure → write (latch) → project → certify (CWA) → pool (or fallback) → agree → belt update.
This chapter provides an end-to-end demo, a text sequence diagram, and ops/runbook artifacts you can lift into production.

8.1 The end-to-end loop (pseudocode)

```
def observerops_run(query, ctx):
    """
    Orchestrates one request end-to-end with Micro→Meso→Macro invariants.
    Returns answer + artifacts (trace ids, cert logs, KPIs).
    """
    # --- Injected subsystems ---
    OR  = ctx["observer_runtime"]     # /measure, /trace, /agree
    CWA = ctx["cwa_engine"]           # /pool (cert-gated)
    SL  = ctx["slot_allocator"]       # tools/memory/attention pools
    PSI = ctx["Psi"]                  # SMFT field adaptor (Ch.5)
    BELT= ctx["belt"]                 # PBHL controller (Ch.7)
    COMM= ctx["commute_matrix"]       # C (Ch.3)
    PI  = ctx["instruments"]          # channels set Π

    # --- τ heartbeat ---
    τ = OR.next_tick()

    # 1) Ô policy selects next channel using SMFT metrics (Ch.5)
    π = select_channel(S=OR.state(), T=OR.trace(), Psi=PSI, x=query, tau=τ,
                       Pi=PI, slots=SL, commute_matrix=COMM, policy=ctx["o_policy"])

    # 2) Slot check (Ch.4). If not enough capacity, queue/back-pressure.
    if not SL.can_allocate("tools", need=ctx["need"].get(π,1)):
        OR.emit("Slots.QueueEnter", {"pi": π, "tau": τ})
        π = ctx["o_policy"].fallback  # e.g., cheap commuting read

    # 3) Measure and latch (Ch.2) — single atomic transaction
    y = OR.measure(π)                             # side effects isolated by π
    rec = OR.write_trace(τ, π, y, idempotency_key=uuid4())

    # 4) Projection and CWA certificate (Ch.6)
    if ctx["is_projective"](π):
        Z = ctx["project"](y)                     # per-item projector
        # Gather batch (e.g., multiple chunks/sensors) under same key
        batch = ctx["batcher"].collect(Z, key=ctx["key"](query))
        cert = CWA.certificate(batch, tests=["perm","sign","chunk"], K=16)

        if cert["score"] >= ctx["cwa_min_score"] and cert["pri"] <= ctx["pri_thresh"]:
            pooled = CWA.additive(batch)          # fast path
            OR.write_trace(OR.next_tick(), "pool.add", {"key":ctx["key"](query),
                           "method":"mean","score":cert["score"]})
        else:
            pooled = ctx["order_aware"](batch)    # fallback path
            OR.write_trace(OR.next_tick(), "pool.fallback",
                          {"key":ctx["key"](query),"method":"attention",
                           "score":cert["score"], "pri":cert["pri"]})

    # 5) Cross-observer agreement (optional in single-agent path) (Ch.3)
    if ctx.get("replicas"):
        for peer in ctx["replicas"]:
            res = OR.agree(OR.trace_ref(), peer.trace_ref(), COMM,
                           lag_tolerance=1, require_hash=True)
            OR.emit("Agreement.Report", res)

    # 6) Belt update (Ch.7)
    snapshot = ctx["belt_sensors"].snapshot()     # Gap, Flux, Twist, ρ
    belt_out  = BELT.tick(gap=snapshot.gap, flux=snapshot.flux,
                          twist=snapshot.twist, rho=snapshot.rho)
    OR.emit("PBHL.Update", belt_out)
    actions = ctx["actuate"].map(belt_out)        # slots/cadence/twist steps
    for a in actions: OR.emit("PolicyGate.Trigger", a)

    return {
        "answer": ctx["composer"](pooled, query),
        "trace": OR.trace_ref(),
        "cwa": cert if ctx["is_projective"](π) else None,
        "belt": {"snapshot": snapshot, "controller": belt_out},
        "agreement": res if ctx.get("replicas") else None
    }
```

What's enforced implicitly

- **Internal collapse:** `write_trace` hash-chains and advances **τ** only on commit.
- **Commutation & slots:** Ô penalizes non-commuting picks and respects pool capacities.

- **CWA gate:** additive pooling only if the certificate passes.
- **Belt closure:** PBHL Residual drives **Flux-gate**/**Twist-step** actions.

8.2 Text sequence diagram (single request)

```
User → Orchestrator : query
Orchestrator → Ô (policy) : select_channel(S, T, Ψm, τ)
Ô → SlotAllocator : can_allocate(π)?
SlotAllocator → Ô : yes/no (+queue)
Ô → Instrument(π) : measure()
Instrument → Orchestrator : outcome y
Orchestrator → TraceLedger : write_trace(τ, π, y)  [LATCH]
TraceLedger → Orchestrator : rec(hash, τ+1)

alt projective output
  Orchestrator → Projector : P(y) → z
  loop batch under key K
    Orchestrator → CWA Engine : certificate({z_i})
    CWA Engine → Orchestrator : score, PRI
  opt score ≥ θ & PRI ≤ τ1
    Orchestrator → Pooler : additive({z_i})
    Orchestrator → TraceLedger : write_trace(τ+1,"pool.add",meta)
  else
    Orchestrator → OrderAware : attention/CNN
    Orchestrator → TraceLedger : write_trace(τ+1,"pool.fallback",meta)
end

opt replicas
  Orchestrator → Agreement : agree(T_a, T_b, C)
  Agreement → Orchestrator : status, score
end

Orchestrator → Belt : tick(Gap, Flux, Twist, ρ)
Belt → Orchestrator : Residual, u_F, u_T
Orchestrator → PolicyGates : actions (slots/cadence/twist)
Orchestrator → User : answer + artifacts (trace, cert, KPIs)
```

8.3 Example A — Support RAG with certificate-gated pooling

Setup

- **Π**: `retriever.search`, `retriever.project` (embed), `retriever.pool`, `browser.get`, `summarize`.
- **Policy**: Ô favors `retriever.search` early; `pool` allowed only if **CWA ≥ 0.98** and **PRI ≤ 0.2**.
- **Slots**: `memory=8`, `tools=4`, `attention=3`.
- **Belts**: program "Helpdesk Answer Quality" with **Gap=NLL target**, **Flux=validated answers/day**, **Twist=KB refactor rate**, α estimated weekly.

Walkthrough (ticks)

- **τ0**: Ô→`retriever.search` (slots OK) → **write τ0**.
- **τ1**: project chunks z_i → CWA battery: perm/sign/chunk all stable; **score 0.995**, **PRI 0.12** → **mean pool** → **write τ1 (`pool.add`)**.
- **τ2**: Ô→`browser.get` for the top doc (commuting read) → **write τ2**.
- **τ3**: Ô→`summarize` with pooled vector + doc → **write τ3** (final answer).
- **Belt**: Gap fell (better NLL), Flux rose; Residual amber→green; controller reduces Flux-gate (less pressure) and leaves Twist unchanged.

Observed wins

- **Latency** ↓ vs attention pooling by ~30–40% on CWA-pass queries.
- **Auditability** ↑: trace has (`pool.add`, score, seeds/masks) for audit.

- **Governance hooks**: when CWA fails repeatedly for a corpus, a Belt gate triggers a review (maybe chunker is misconfigured).

Minimal config (excerpt)

```
o_policy:
  cwa_min_score: 0.98
  pri_thresh: 0.20
  commute_bonus: 0.2
  noncommute_penalty: -0.4
slots:
  memory: {capacity: 8, policy: LRU}
  tools:  {capacity: 4, policy: PRI}
belt_thresholds:
  residual: {green: 0.05, amber: 0.10, red: 0.20}
```

8.4 Example B — Multi-agent tool use (agreement + slots + τ-sync)

Setup

- Two agents *A, B* with same Π and **shared ledger**; fleet cadence with **K=12** phases → target **ρ≥0.9**.
- **C** enforces commuting reads first; writes (e.g., `db.write`) are serialized.

Flow

- **τ0–τ2**: both pick commuting reads; their writes land on shared ledger; **agree()** over overlapping (τ, key) is **pass (≥0.99)**.
- **Burst**: both attempt `browser.get` ×5. Tool slots limited to 4 → one agent queues; Ô selects a cheap commuting alternative; **collisions** remain <1%.
- **Drift**: a mis-mapped channel on *B* causes `Agreement.Fail` on `sensor.X`. Diagnosis shows **frame map φ** error; fix and re-run → pass.

Fleet stability

- Monitor **Δτ** across agents; resync if any $|\Delta\tau| > 2$.
- If sync dips (ρ<0.8), enforce a checkpoint: both must `summarize` at τ≡0 (mod 12).

8.5 Ops checklist (daily)

- **Trace integrity:** 100% hash-chain pass; no retro-edits.
- **Agreement:** mean ≥0.98 on commuting overlaps; NCE (non-commuting exposure) trending ↓.
- **CWA:** pass-rate %, component drifts (perm/sign/chunk); PRI spikes flagged.
- **Slots:** P95 queue wait ≤ SLA; collision rate within bands.
- **SMFT:** S_c slope ≤ 0 (stabilizing) on mature flows; exploration retained on new flows.
- **Belt:** Residual in green/amber; α stable; ρ ≥ target at reviews.

8.6 Runbook excerpt (incident quick-paths)

I. Certificate red on major corpus

1. Verify projector version & chunker boundaries in trace.
2. Run component breakdown: which test failed (perm/sign/chunk)?
3. **If sign-sensitive:** unify sign conventions (whiten/PCA), retrain projector; keep fallback enabled.
4. **If chunk-sensitive:** re-chunk at 10–15% overlap; update Ô to avoid pooling until pass.

5. Record `Belt.Incident`; set **Gate-R** if Residual rises.

II. Agreement drop across replicas

1. Check **commutation matrix C** updates; ensure serialization for writers.
2. Validate frame maps ϕ; run unit keys.
3. If ledger divergence: compare hash ranges; reconcile; re-run agree.
4. If still failing: raise **SBS redundancy** (add a mirror channel).

III. Residual turns red (PBHL)

1. Identify driver: Flux low vs Twist high vs α flip.
2. Apply **Flux-gate** (slots + cadence) for 1–2 windows.
3. Freeze non-critical twist; schedule α review.
4. Maintain ρ checkpoint cadence; update Five-Line KPI.

8.7 Artifacts

- **Sequence diagram:** provided in §8.2 (text form; mirror as a figure in production).
- **Ops checklist:** §8.5 (drop straight into a runbook).
- **Runbook excerpt:** §8.6 (incident recipes).

Exportable logs (schema hints)

- `TraceWrite{tau, channel, outcome_ref, prev_hash, hash}`
- `CWA{score, components, PRI, method, seeds}`
- `Agreement{status, score, overlaps, conflicts_by_key}`
- `Slots{alloc/release/evict/collision, pool, k, token}`
- `BeltTick{Gap, Flux, Twist, alpha, rho, Residual}`
- `PolicyGate{type, action, params}`

8.8 What you can cut-paste into your stack

- The **orchestrator** in §8.1 (swap adapters).
- The **sequence diagram** to brief SRE/governance.
- The **YAML snippets** from Ch.4–7 for policies and belts.
- The **checklist & runbook** above to operationalize Day-2.

Outcome: You now have a **coherent operating loop**: the micro layer guarantees **latching** and **capacity discipline**, the meso layer chooses **what to observe next** and safely **pools** what can be added, and the macro layer keeps your program **closed** with Residual control and dashboards.

© 2025 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and

organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.

ObserverOps Technical Blueprint - II & III

Part II — Reference Architecture & APIs

Chapter 9 — System Overview (Planes & Modules)

Goal

Blueprint the **data**, **control**, and **audit** planes and the boundaries among six core modules: **Observer Runtime**, **CWA Engine**, **Slot Allocator**, **Tick & Sync**, **BeltOps Dashboard**, and **Policy Gates**. Provide end-to-end flows, canonical contracts, SLOs, and production diagrams (architecture + dependency graph).

What You'll Implement in This Chapter

- A **3-plane deployment model** with responsibilities, invariants, and SLO bands per plane.
- A **module map** with clear inputs/outputs/state for each core component.
- A **baseline event taxonomy** spanning the stack (data/control/audit).
- **Two reference flows** (observational data path; governance gate path).
- **Production artifacts:** Architecture diagram (Mermaid), dependency graph (Mermaid), module boundary table, and configuration snippets.

9.1 The Three Planes

> **Separation of concerns:** keep measurement & transformation hot-path in the **Data Plane**; scheduling/cadence and policy in the **Control Plane**; immutability, lineage, and exports in the **Audit Plane**.

9.1.1 Data Plane

Purpose. Carry measurements and projections from instruments to pools; enforce **internal collapse** at write time and **CWA** at aggregation time.

Canonical objects. `Measurement`, `Projection`, `Certificate`, `PoolResult`.

Hot-path services.

- **Observer Runtime** (`/measure`, `/agree`, `/trace/:id`)
- **CWA Engine** (`/project`, `/pool`)

Invariants.

1. **Latching:** `TraceWrite(τ_k)` is in-frame irreversible; edits require a new tick `τ_{k+1}`.
2. **Certificate-gated pooling:** additive pooling only if `CWA.score ≥ θ`.
3. **Slot conservation (data buffers/tools):** non-fractional, non-overlap writes.

SLOs (typical starting bands).

- **p95 measure→trace write:** ≤ 50 ms

- **p95 project:** ≤ 25 ms
- **p95 pool (when CWA pass):** ≤ 30 ms; **fallback** path ≤ 120 ms
- **Availability (monthly):** ≥ 99.9%

Failure modes & guards.

- **False-green certificate:** mitigate via conservative θ, multi-panel tests, and audit sampling.
- **Buffer spill/collisions:** back-pressure via Slot Allocator; drop policies must be explicit events.

9.1.2 Control Plane

Purpose. Decide *what to observe next* (Ô policy), advance **ticks τ**, manage fleet synchronization (ρ, Δτ), and apply **Policy Gates** that throttle or halt risky runs.

Canonical objects. `Tick`, `Schedule`, `GatePolicy`, `GateDecision`.

Core services.

- **Tick & Sync** (cadence manager; fleet sync metrics ρ, Δτ)
- **Policy Gates** (thresholds on CWA score, PBHL Residual, black-hole detectors)

Invariants.

1. **Ô-first scheduling:** channel selection precedes measurement; compatibility checks run before actuation.
2. **Tick monotonicity:** `τ_{k+1} > τ_k`; retries are new ticks or explicitly same-tick idempotent (`retry_id`).
3. **Sync safety:** cross-agent Δτ bounded by policy; if exceeded, degrade to safe mode.

SLOs.

- **p95 Ô decision latency:** ≤ 15 ms (cached), ≤ 60 ms (field-aware)
- **Fleet sync order parameter ρ:** ≥ 0.85
- **Gate evaluation latency:** ≤ 10 ms inline; ≤ 100 ms async fan-out

---

9.1.3 Audit Plane

Purpose. Provide immutable **Trace Ledger**, **Certificate Logs**, and **Belt Telemetry** with export hooks to GRC systems. This plane is the source of truth for **cross-observer agreement** checks and PBHL governance.

Canonical objects. `TraceRecord`, `CertPanel`, `AgreementReport`, `BeltKPI`, `ExportBundle`.

Core services.

- **Observer Runtime — Trace Ledger** (hash-chained, append-only)
- **CWA Engine — Cert Log** (panels, seeds, CI/drift)
- **BeltOps Dashboard — KPI Store** (Gap/Flux/Twist, Residual, EEI/SI)

Invariants.

1. **Immutability:** append-only with hash chain and content-addressed blobs.

2. **Agreement evidence:** agreement tests reference shared records; no private deltas.

3. **Exportability:** every decision has a proof trail (ids for trace, cert, gate, belt update).

SLOs.

- **Write durability:** ≥ 11×9s (cloud multi-AZ); **RPO:** 0; **RTO:** < 5 min tier-wise
- **Retention:** configurable; typical 90–365 days hot, 7 years cold

---

9.2 Module Map & Boundaries

The six modules are independently deployable with **narrow interfaces** and **explicit state ownership**.

Module	Owns	Ingests	Emits	Hard Invariants	Typical SLO
Observer Runtime	Trace Ledger; Channel Registry; Commute Matrix	`Measure`, `Schedule`	`TraceWrite`, `AgreementReport`	Internal collapse; compatibility checks pre-actuation	p95 measure→write ≤ 50 ms
CWA Engine	Projector library; Cert Panel configs; Cert Logs	`ProjectionRequest`	`Certificate{score}`, `PoolResult`	Certificate-gated add; reproducible panels (seeded)	p95 project ≤ 25 ms; pool ≤ 30/120 ms
Slot Allocator	Slot budgets for memory/ attention/ tools	`SlotRequest`	`SlotGrant/Refuse`, `CollisionEvent`	Integer slots; non-overlap; explicit eviction	Decision ≤ 5 ms
Tick & Sync	Tick index τ; fleet sync metrics ρ, Δτ	`Heartbeat`, `ScheduleNeed`	`TickStart`, `Schedule`, `DesyncAlert`	Monotone τ; bounded Δτ	Ô decision ≤ 60 ms
BeltOps Dashboard	PBHL worldsheet (Gap, Flux, Twist, α), EEI/SI	`PoolResult`, `AgreementReport`	`PBHL.Update`, KPIs, `Residual`	Gap≈Flux+α·Twist within residual band	KPI refresh ≤ 1 s
Policy Gates	Gate configs; escalation ladders	`Certificate`, `Residual`, `BlackHoleIndex`	`GateDecision{allow`	throttle	block}`

> **Shared services (cross-cutting):** Identity & Auth (service→service tokens, user claims), Config & Secrets (versioned), Telemetry Bus (events), Data Catalog/Lineage.

9.3 Canonical Interfaces & Events (Overview)

HTTP/gRPC Endpoints (preview)

```
POST /measure          // {pi, S, T_ref}
POST /project          // {x, policy, projector}
POST /pool             // {projected[], cert_config, min_score}
POST /agree            // {Ta_ref, Tb_ref, commute_matrix_id}
POST /belt             // {edges:{plan,do}, alpha, mesh, gates}
GET  /trace/:id        // immutable record by id
```

Event Taxonomy

- `TickStart(τ)`, `ChannelSelected(π)`, `TraceWrite(τ, π, y)`
- `AgreementPass/Fail{score}`
- `CWA.Pass/Fail{score, panels}`
- `PBHL.Update{Gap, Flux, Twist, α, Residual}`
- `PolicyGate.Trigger{metric, threshold, action}`
- `DesyncAlert{Δτ, ρ}`
- `CollisionEvent{slot_id}`

Idempotency keys: `trace_id`, `retry_id`, `cert_seed`, `pool_id`, `belt_update_id`.

9.4 Reference Flows

9.4.1 Observational Data Path (Hot-Path)

1. **Tick & Sync** emits `TickStart(τ_k)` and a `Schedule` with selected channel (Ô policy).
2. **Observer Runtime** invokes instrument π, obtains `y`, and **latches** via `TraceWrite(τ_k, π, y)`.
3. **CWA Engine** receives a `ProjectionRequest` from Runtime, computes `Projection` and runs certificate panels → emits `Certificate{score}`.
4. If `score ≥ θ`, **CWA Engine** performs additive **`/pool`** and returns `PoolResult`; otherwise switches to order-aware fallback and returns `PoolResult{fallback:true}`.
5. **BeltOps Dashboard** ingests `PoolResult` and `AgreementReport` to update PBHL worldsheet and KPIs; **Policy Gates** evaluate whether to throttle/stop subsequent schedules.

Sequence (Mermaid)

```
sequenceDiagram
  participant Sync as Tick & Sync
  participant OR as Observer Runtime
  participant CWA as CWA Engine
  participant Belt as BeltOps Dashboard
  participant Gate as Policy Gates
  Sync->>OR: TickStart(τ_k) + Schedule(π)
  OR->>OR: measure(π) → y
  OR->>OR: TraceWrite(τ_k, π, y) // latch
  OR->>CWA: /project{x, policy}
  CWA-->>OR: Projection ϕ(x)
  OR->>CWA: /pool{ϕ[], cert}
  CWA-->>OR: Certificate{score} + PoolResult
  OR-->>Belt: PoolResult, AgreementReport
  Belt-->>Gate: PBHL.Update{Gap,Flux,Twist,Residual}
  Gate-->>Sync: GateDecision{allow|throttle|block}
```

9.4.2 Governance Gate Path (Control-to-Data feedback)

- **Trigger:** `Residual > R_max` or `CWA.score < θ_min` for N consecutive windows.
- **Policy Gates** emit `GateDecision=throttle|block` with rationale and evidence ids.
- **Tick & Sync** lowers cadence (increase inter-tick), narrows channel set, or pauses schedules until Residual/CWA recover; all changes are recorded as `PolicyGate.Trigger` and `Schedule` deltas.

9.5 Deployment Topologies

Topology A — Compact (single cluster). All modules as services on one mesh; shared telemetry bus; storage split by plane (hot: Data Plane, immutable: Audit Plane).

Topology B — Federated (multi-team, multi-region). Data Plane per domain; shared Control Plane with global Tick & Sync; Audit Plane centralized with regional appenders. **Agreement** runs cross-domain by referencing shared trace ids.

Capacity & Latency Planning.

- **Hot-path budget:** 50–100 ms end-to-end on CWA-pass path.
- **Fallback budget:** up to 150–250 ms, flagged and rate-limited by Policy Gates.
- **Throughput:** size to 10× peak with back-pressure from Slot Allocator.

Back-pressure & Degradation.

- Prefer **shed projections** over **dropping traces**; no silent loss.
- Auto-degrade: reduce channel cardinality; widen tick spacing; disable expensive panels.

9.6 Failure Domains & Safe Modes

Data Plane

- *Failure:* instrument timeouts → **Action:** mark as conflict; skip with explicit `TraceWrite(timeout)`; request retry via new τ.
- *Failure:* certificate service cold → **Action:** block additive pool; fallback estimator with `GateDecision=throttle`.

Control Plane

- *Failure:* desync (ρ↓, Δτ↑) → **Action:** pause schedules except commuting-safe channels; resync protocol.
- *Failure:* gate storm → **Action:** exponential back-off; consolidate triggers; operator override path.

Audit Plane

- *Failure:* ledger ingestion lag → **Action:** spill to local WAL; block exports; raise amber status.

Safe Modes

- **Read-only audit; CWA-off (no additive pooling); Ô-reduced (commuting-only)**; **Tick slow-roll.** All are reversible with clear exit criteria.

9.7 Security, Privacy, and Compliance Hooks

- **AuthZ scopes** per plane: `data.write`, `control.schedule`, `audit.read/export`.
- **PII/secret handling:** projection artifacts tagged; redaction policies; lineage in audit.
- **Tamper-evidence:** hash-chain for traces/cert logs; signed exports; reproducible seeds for panels.
- **Least privilege runners:** Slot Allocator and Policy Gates run with minimal scopes; BeltOps is read-mostly with signed writes for KPIs.

9.8 Production Artifacts

9.8.1 Architecture Diagram (Mermaid)

```
flowchart LR
  subgraph CONTROL[Control Plane]
    TS[Tick & Sync]
    PG[Policy Gates]
  end
  subgraph DATA[Data Plane]
    OR[Observer Runtime]
    CWA[CWA Engine]
    SA[Slot Allocator]
  end
  subgraph AUDIT[Audit Plane]
    TL[Trace Ledger]
    CL[Cert Logs]
    BD[BeltOps Dashboard]
  end

  TS -- Schedule/τ --> OR
  OR -- measure→TraceWrite --> TL
  OR -- /project,/pool --> CWA
  CWA --> CL
  OR -- Slot requests --> SA
  SA -. grants/refuse .-> OR
  OR --> BD
  BD --> PG
  PG -- GateDecision --> TS

  classDef plane fill:#0b7285,stroke:#0b7285,color:#fff;
  class CONTROL,DATA,AUDIT plane;
```

9.8.2 Module Dependency Graph (Mermaid)

```
graph TD
  SA[Slot Allocator]
  OR[Observer Runtime]
  CWA[CWA Engine]
  TS[Tick & Sync]
  PG[Policy Gates]
  BD[BeltOps Dashboard]

  TS --> OR
  OR --> CWA
  OR --> BD
  CWA --> BD
  BD --> PG
  PG --> TS
  SA --> OR

  %% Notes: all modules write to Audit Plane stores (not shown as edges here).
```

9.8.3 Boundary Checklist (Copy-Paste into Design Reviews)

- Each module declares **owned state** and **no hidden side effects**
- All cross-module calls carry **idempotency keys**
- **Retries** either advance τ or use explicit `retry_id`
- **CWA thresholds** and fallback policies versioned + auditable
- **Slot budgets** defined; collisions observable; eviction policy explicit
- **Gate configs** reproducible; escalation ladders documented
- **PBHL Residual** bands defined; BeltOps panel shows Five-Line KPI
- **Exports** signed; lineage attached; privacy redactions applied

---

9.9 Summary

This chapter pins down the **planes**, **modules**, and **contracts** that make ObserverOps buildable. The next chapters (10–13) dive into per-module APIs, data schemas, algorithms, and operational runbooks.

Chapter 10 — Observer Runtime

Goal

Implement the hot-path service that executes measurements, enforces **internal collapse** at trace-write, evaluates **instrument compatibility**, and produces **agreement evidence** using SBS-style redundancy.

Scope. This chapter defines the Observer Runtime's public interfaces, on-disk/over-the-wire schemas, operational guardrails (idempotency/retries/latching), and the event taxonomy that other modules consume.

10.1 Responsibilities & Boundaries

Owns

- **Trace Ledger**: append-only, hash-chained records of `(τ, π, y)` with provenance.
- **Channel Registry**: instrument metadata, costs, slot needs, semantic pointers.
- **Commute Matrix**: compatibility & preflight constraints among channels.

Collaborates with

- **Tick & Sync** (receives `TickStart`, `Schedule`).
- **CWA Engine** (calls `/project`, `/pool`).
- **Policy Gates** (consumes `GateDecision` hints for degraded modes).
- **BeltOps Dashboard** (emits `AgreementReport`, feeds KPIs).

Non-Goals

- Does **not** implement certificates or pooling algorithms (delegates to CWA Engine).
- Does **not** compute PBHL controllers (BeltOps).

10.2 Public Interfaces (HTTP/gRPC)

10.2.1 `/measure`

Method	Path	Purpose	Idempotency	SLO	Auth
`POST`	`/measure`	Invoke instrument π on current state, return outcome `y`, and **latch** to Trace Ledger at tick τ.	`Idempotency-Key` (semantic: same tick + same π dedup)	p95 ≤ 50 ms	`data.write`

Request (JSON)

```
{
  "tau": 1042,
  "pi": "tools.search.web",
  "state_ref": "S:7f1b...",
  "schedule_id": "sched-1b2c",
  "idempotency_key": "m-1042-tools.search.web-1"
```

```
}
```

Response

```
{
  "trace_id": "t-01HZXJ...",
  "tick": 1042,
  "channel": "tools.search.web",
  "outcome_ref": "blob:sha256:ab38...",
  "write": {"hash": "sha256:9f2e...", "prev": "sha256:...", "ts": "2025-09-22T10:41:12Z"},
  "status": "latched"
}
```

Errors

- `409 Conflict` — non-commuting with locked channel at same τ
- `412 Precondition Failed` — schedule mismatch or missing slot grant
- `425 Too Early` — tick not opened by Tick & Sync

10.2.2 `/agree`

Method	Path	Purpose	Idempotency	SLO	Auth
`POST`	`/agree`	Compute cross-observer agreement score using commute matrix & shared records (SBS redundancy).	`agree_id` deterministic over inputs	p95 ≤ 40 ms	`data.read`

Request

```
{
  "Ta_ref": "tset:replica-A:week39",
  "Tb_ref": "tset:replica-B:week39",
  "commute_matrix_id": "cm:v1.12",
  "pointer": "support.answer",
  "window": {"start": 1030, "end": 1045}
}
```

Response

```
{
  "agree_id": "ag-6c91...",
  "score": 0.94,
  "redundancy": {"R": 3.6, "channels": ["kb.vector", "kb.keyword", "kb.cached"]},
  "evidence": ["t-01HZXJ...", "t-01HZXL..."],
  "sbs": {"pass": true, "reason": "pointer redundancy ≥ 3"}
}
```

10.2.3 `/trace/:id`

Method	Path	Purpose	Caching	SLO	Auth
`GET`	`/trace/:id`	Retrieve immutable trace record with provenance & hash chain links.	CDN-cacheable (immutable)	p95 ≤ 20 ms	`audit.read`

Response (abbrev.)

```
{
  "trace_id": "t-01HZXJ...",
  "tick": 1042,
  "channel": "tools.search.web",
  "outcome": {"kind": "json", "bytes": 2312},
  "write": {"hash": "sha256:...", "prev": "sha256:...", "writer": "or-0"},
  "lineage": {"schedule_id": "sched-1b2c", "idempotency_key": "m-1042-tools.search.web-1"}
}
```

10.3 Instrument Compatibility (Preflight)

Commute Matrix `C` is a sparse symmetric map over channel pairs with optional **contextual predicates**.

Entry form

```
{ "a": "sensors.qubit.Z", "b": "sensors.qubit.X", "commute": false, "predicate": "same_object && Δτ < 3" }
```

Preflight algorithm (pseudo-code)

```
# inputs: schedule S = [π1, π2, ...], tick τ_k, matrix C
for (i,j) in pairs(S):
    if not C.commute(π_i, π_j, context):
        raise Conflict(pi=π_j, with_=π_i, tick=τ_k)
```

Conflict handling

- **Re-order** to a commuting sequence if available.
- **Defer** non-commuting channel to `τ_{k+1}` (Tick & Sync request).
- **Annotate** `TraceWrite` with `conflict=true` when a measured channel times out and is skipped.

10.4 SBS Redundancy (Pointer Agreement)

Pointer variable. A semantic target (e.g., `support.answer`) with one or more **pointer channels** that redundantly encode it.

Redundancy factor (R). Effective count of independent channels carrying the same pointer:
R = ((Σ w_i)^2) / (Σ w_i^2), where `w_i` are channel reliability weights.

Agreement score. Jaccard/soft-Jaccard or cosine similarity over pointer-projected outcomes aggregated across observers within the window `[τ_s, τ_e]`.

Runtime role.

- Maintains a **Pointer Map**: {pointer → channels}
- Emits `AgreementReport` with `{score, R, evidence_trace_ids}`

10.5 Data Schemas

10.5.1 Trace Ledger (append-only, hash-chained)

```
{
  "$schema": "https://observerops.io/schemas/trace.v1.json",
  "trace_id": "t-...",
  "tick": 1042,
  "channel": "tools.search.web",
  "slot_id": "slot:mem:3",
  "outcome_ref": "blob:sha256:...",
  "meta": {"duration_ms": 18, "cost": {"tokens": 1200}},
  "write": {"hash": "sha256:...", "prev": "sha256:...", "ts": "...", "writer": "or-0"},
  "lineage": {"schedule_id": "...", "idempotency_key": "...", "retry_id": null},
  "flags": {"timeout": false, "conflict": false}
}
```

10.5.2 Commute Matrix

```
{
  "$schema": "https://observerops.io/schemas/commute-matrix.v1.json",
  "matrix_id": "cm:v1.12",
  "default": true,
```

```
  "pairs": [
    {"a": "sensors.qubit.Z", "b": "sensors.qubit.X", "commute": false},
    {"a": "retriever.vector", "b": "retriever.keyword", "commute": true}
  ],
  "predicates": {
    "same_object": "ctx.object_a == ctx.object_b",
    "Δτ<3": "(ctx.tau_b - ctx.tau_a) < 3"
  }
}
```

10.5.3 Channel Registry

```
{
  "$schema": "https://observerops.io/schemas/channel-registry.v1.json",
  "channels": [
    {
      "id": "tools.search.web",
      "kind": "tool",
      "pointer": ["support.answer"],
      "requires_slot": {"type": "mem", "units": 1},
      "cost_model": {"latency_ms_p95": 40, "tokens_p95": 1500}
    }
  ]
}
```

10.6 Ops: Idempotency, Retries, Latching Guardrails

- **Latching rule (internal collapse):** once `TraceWrite(τ_k, π, y)` commits, downstream control must **condition** on `y`. Retro-edit requires a new tick `τ_{k+1}` and produces a new trace id.
- **Per-tick single-write:** at most one successful `TraceWrite` per `(τ, π)`.
- **Idempotency keys:** dedup same-tick duplicates; **retry_id** marks replays after transient errors.
- **Retry policy:**
  - `5xx` / network → retry with **same τ** and `retry_id` (idempotent);
  - `409 Conflict` → request reschedule to `τ_{k+1}`;
  - `412/425` → wait for `TickStart` or fetch latest `Schedule`.
- **Back-pressure:** integrate with **Slot Allocator**; when refused, emit `CollisionEvent` and reschedule.

10.7 Event Taxonomy (Runtime-Scoped)

- `Preflight.Compatibility{τ, π, ok, reason}`
- `Measure.Start{τ, π}` / `Measure.Result{τ, π, y_ref, duration}`
- `TraceWrite{trace_id, hash, prev, ts}`
- `Retry{τ, π, retry_id, cause}`
- `Agreement.Pass/Fail{agree_id, score, R}`
- `Pointer.Redundancy{pointer, R, channels[]}`

Event keys: `run_id`, `observer_id`, `schedule_id`, `trace_id`, `agree_id`.

10.8 Worked Examples

10.8.1 Qubit Toy (non-commuting)

- Channels: `sensors.qubit.Z`, `sensors.qubit.X`; `C[Z,X]=false` for same object.
- Schedule proposes `[Z, X]` at `τ_k`. Preflight blocks `X` at `τ_k`; Tick & Sync defers `X` to `τ_{k+1}`.
- `TraceWrite(τ_k, Z, y_z)` latches; `TraceWrite(τ_{k+1}, X, y_x)` records the second measurement.
- `/agree` compares two observers with shared records → low agreement when orders differ (documented counterexample).

10.8.2 Tool-Using Agent (commuting, SBS pass)

- Channels: `retriever.vector`, `retriever.keyword`, `kb.cached` commute and point to `support.answer`.
- Three traces at `τ_k` produce redundant pointers (R≈3.6). `/agree` across replicas A/B yields high score.

Sequence (Mermaid)

```
sequenceDiagram
  participant TS as Tick & Sync
  participant OR as Observer Runtime
  participant SA as Slot Allocator
  participant CWA as CWA Engine
  TS->>OR: Schedule(τ_k, [retriever.vector])
  OR->>SA: SlotRequest(mem=1)
  SA-->>OR: SlotGrant(slot:mem:3)
  OR->>OR: Preflight.Compatibility(ok=true)
  OR->>OR: measure → y
  OR->>OR: TraceWrite(τ_k, retriever.vector, y)
  OR->>CWA: /project{x, projector}
```

10.9 SLOs, Alerts, and Dashboards

Runtime SLOs

- p95 measure→write ≤ 50 ms; error rate ≤ 0.5%
- Agreement pipeline p95 ≤ 40 ms; stale commute matrix < 0.1%
- Trace durability ≥ 11×9s; export lag p95 ≤ 2 s

Alerts

- **Latching violations** (duplicate `(τ, π)` writes)
- **Desync dependency** (received `/measure` before `TickStart`)
- **Commute drift** (runtime observes conflicts for pairs marked commuting)

Dashboards

- Hot-path latency; per-channel error bars; redundancy factor R over time; agreement heatmap.

10.10 Configuration (YAML)

```
runtime:
  cwa:
    min_score: 0.82
    panels:
      permutation: 128
      sign_flip: 64
      chunk_shuffle: 32
  latching:
    per_tick_single_write: true
  retries:
    max_attempts: 3
```

```
    backoff_ms: [50, 200, 800]
  slots:
    require_grant: true
  pointers:
    support.answer: [retriever.vector, retriever.keyword, kb.cached]
  commute_matrix: cm:v1.12
```

10.11 Artifacts

- **API Tables**: `/measure`, `/agree`, `/trace/:id` (above)
- **Schemas**: Trace Ledger v1, Commute Matrix v1, Channel Registry v1
- **Event Taxonomy**: Runtime-scoped events (10.7)
- **Diagrams**: Sequence (10.8), plus plane-context in Ch.9

Chapter 11 — CWA Engine (Projection → Certificate → Pool)

Goal. Decide when **project→add** (mean/sum pooling) is *provably safe* after projection, and when to *auto-fallback* to order-aware aggregators. Provide deterministic certificates, risk outputs, and tight latency budgets suitable for hot-path use.

11.1 Responsibilities & Boundaries

Owns

- **Projector Library** `P(·)`: deterministic projection policies (e.g., embedding models, pointer extractors, feature maps).
- **Certificate Panels**: permutation, sign-flip, and chunk-shuffle test batteries with seeds.
- **Cert Logs**: panel outcomes, seeds, CI/drift summaries.

Collaborates with

- **Observer Runtime** (input vectors/records; calls `/project`, `/pool`).
- **Policy Gates** (consumes certificate scores, Phase-Risk Index, drift alerts).
- **BeltOps Dashboard** (receives risk KPIs).

Non-Goals

- Doesn’t write traces (Observer Runtime does).
- Doesn’t implement governance gates (Policy Gates do).

11.2 Interfaces (HTTP/gRPC)

11.2.1 `POST /project`

Project raw observations into a pooled space under a specified projector policy.

Request

```
{
  "policy": "embeddings.e5-large",
  "inputs": [{"ref":"t-01HZXJ..."}, {"ref":"t-01HZXL..."}],
  "params": {"normalize": true, "dtype": "float32"},
```

```
  "seed": 4271
}
```

Response

```
{
  "projection_id": "ϕ-9a12...",
  "vectors_ref": "blob:sha256:1f2c...",     // array of d-dim vectors
  "meta": {"n": 18, "dim": 1024, "normalize": true, "policy": "embeddings.e5-large"}
}
```

SLO p95 ≤ 25 ms. **Auth** `data.write`.

11.2.2 `POST /pool`

Pool a set of projected vectors with certificate gating.

Request

```
{
  "projection_id": "ϕ-9a12...",
  "vectors_ref": "blob:sha256:1f2c...",
  "aggregator": "mean",                       // desired fast path
  "min_score": 0.82,                          // θ
  "panels": {"perm": 128, "flip": 64, "chunk": 32},
  "chunk_meta": {"boundaries": [0,3,7,12,18]}, // optional for text/audio
  "seed": 4271
}
```

Response (fast-path pass)

```
{
  "pool_id": "pool-7c0b...",
  "aggregator_used": "mean",
  "vector_ref": "blob:sha256:b1a4...",
  "certificate": {
    "score": 0.91,
    "panels": {
      "perm":{"n":128,"median_delta":0.04},
      "flip":{"n":64,"median_delta":0.03},
      "chunk":{"n":32,"median_delta":0.05}
    },
    "phase_risk_index": 0.09,
    "ci95": [0.88, 0.93],
    "drift": {"p_value": 0.74, "ref_window": "W38"}
  },
  "fallback": {"used": false}
}
```

Response (fallback)

```
{
  "pool_id": "pool-7c0b...",
  "aggregator_used": "attention",
  "vector_ref": "blob:sha256:5ae9...",
  "certificate": {
    "score": 0.61,
    "phase_risk_index": 0.39,
    "ci95": [0.57, 0.66],
    "reason": "score<threshold or chunk panel median_delta>τ"
  },
  "fallback": {"used": true, "policy": "attention.kv", "latency_ms": 92}
}
```

SLO p95 ≤ 30 ms (pass path), ≤ 120 ms (fallback). **Auth** `data.write`.

11.3 Certificate Design

Let $V = \{v_i\}_{i=1}^{N}$, $v_i \in \mathbb{R}^d$ be projected vectors. Baseline additive pool:

$$\mu_0 = \frac{1}{N}\sum_{i=1}^{N} v_i \quad \text{(or sum)}$$

Define a stability distance between any pooled vector μ and baseline μ_0:

$$\delta(\mu, \mu_0) = \min\left(1,\ \frac{\|\mu - \mu_0\|_2}{\|\mu_0\|_2 + \varepsilon}\right)$$

Score contribution from a panel with samples $\{\mu_j\}$:

$$s = 1 - \text{median}_j\ \delta(\mu_j, \mu_0) \in [0, 1]$$

Panels

1. **Permutation Panel (order wash-out)**
   - Draw permutations π_j of indices; pool in permuted order (for additive mean, the *order* itself shouldn't matter; this detects hidden order effects in the *projection* or pre-pool normalization).
   - Produce $\mu_j^{(perm)}$, compute s_{perm}.
2. **Sign-Flip Panel (orientation wash-out)**
   - Sample Rademacher signs $s_i \in \{\pm 1\}$ per *sample* and/or small per-dimensional masks; form $v_i' = s_i v_i$; pool to $\mu_j^{(flip)}$.
   - Rationale: truly additive observables shouldn't invert under local orientation flips *after projection*; sensitivity indicates phase-like coherence not erased by $P(\cdot)$.
   - Compute s_{flip}.
3. **Chunk-Shuffle Panel (chunk boundary wash-out)**
   - Randomly perturb chunk boundaries or merge/split neighboring chunks consistent with `chunk_meta`.
   - Pool perturbed sets → $\mu_j^{(chunk)}$; compute s_{chunk}.

Aggregate certificate.

$$\text{CWA.score} = w_p s_{perm} + w_f s_{flip} + w_c s_{chunk}, \quad w_p + w_f + w_c = 1.$$

Defaults: $w_p = 0.4, w_f = 0.3, w_c = 0.3$.

Phase-Risk Index. A complementary risk value emphasizing order/phase sensitivity:

$$\text{PRI} = 1 - \min(s_{perm}, s_{flip}, s_{chunk})$$

Low PRI (≈0) is safe; high PRI (→1) risky.

Confidence Interval (CI). Bootstrap the panel deltas; report 95% CI for the aggregate score.

11.4 Algorithms

11.4.1 Evaluator Pseudocode

```
def cwa_certificate(vectors, panels, seed, eps=1e-9, weights=(0.4,0.3,0.3)):
    rng = PCG64(seed)
    V = np.array(vectors)                       # N x d
    mu0 = V.mean(axis=0)                        # baseline
    norm0 = np.linalg.norm(mu0) + eps

    def delta(mu): return min(1.0, np.linalg.norm(mu - mu0) / norm0)

    def panel_perm(n):
        ds = []
        for _ in range(n):
            rng.shuffle(V)                      # permute in-place copy in real code
            mu = V.mean(axis=0)
            ds.append(delta(mu))
        return 1 - np.median(ds), ds

    def panel_flip(n):
        ds = []
        for _ in range(n):
            signs = (rng.random(V.shape[0]) < 0.5).astype(np.float32) * 2 - 1
            mu = (V * signs[:,None]).mean(axis=0)
            ds.append(delta(mu))
        return 1 - np.median(ds), ds

    def panel_chunk(n, boundaries):
        ds = []
        for _ in range(n):
            b = jitter_boundaries(boundaries, rng)  # small merges/splits
            subvects = [V[b[k]:b[k+1]].mean(axis=0) for k in range(len(b)-1)]
            mu = np.mean(subvects, axis=0)
            ds.append(delta(mu))
        return 1 - np.median(ds), ds

    sp, dsp = panel_perm(panels["perm"])
    sf, dsf = panel_flip(panels["flip"])
    sc, dsc = panel_chunk(panels["chunk"], panels.get("boundaries", [0, len(V)]))

    score = weights[0]*sp + weights[1]*sf + weights[2]*sc
    pri   = 1 - min(sp, sf, sc)
    ci_lo, ci_hi = bootstrap_ci([*dsp, *dsf, *dsc])  # on deltas → map to score CI

    return {
      "score": score, "phase_risk_index": pri,
      "panels": {"perm":{"median_delta":1-sp},
                 "flip":{"median_delta":1-sf},
                 "chunk":{"median_delta":1-sc}},
      "ci95": [ci_lo, ci_hi]
    }
```

11.4.2 Drift & CI

- Maintain rolling distribution of **panel deltas** and **scores** per *policy, domain*.
- **Drift test:** two-sample KS or AD test vs. reference window (e.g., week-over-week).
- **Alarm:** `p_value < 0.05` *and* |mean score shift| ≥ 0.05 → raise `CWA.Drift`.

11.5 Auto-Fallback Policies

Inputs: `score`, `ci95`, `phase_risk_index`, `panels.*.median_delta`, `latency_budget_ms`.

Default thresholds

- **Green (pass):** `score ≥ θ_pass` (0.82) **AND** `PRI ≤ 0.20`
- **Amber (sampling pass):** `θ_warn ≤ score < θ_pass` (0.75–0.82) → allow **mean** if latency critical and `ci95[0] ≥ θ_warn`; **log amber**

- **Red (fallback):** `score < θ_warn` **OR** any `panel.median_delta > τ_panel` (0.25) **OR** `PRI > 0.5`

Fallback choices (by domain)

- Text: `attention.kv` (length-aware), else `cnn.1d`
- Time-series: `rnn.gru` or `tcn`
- Multi-modal: late fusion with per-modality attention

Escalation

- If **Red** persists ≥ K windows (default 3), emit `PolicyGate.Trigger(block)` and slow ticks.

11.6 Data & Logs

Certificate Log (immutable, append-only)

```
{
  "$schema": "https://observerops.io/schemas/cert.v1.json",
  "pool_id": "pool-7c0b...",
  "projection_policy": "embeddings.e5-large",
  "n": 18, "dim": 1024,
  "score": 0.91, "phase_risk_index": 0.09, "ci95": [0.88,0.93],
  "panels": {"perm":{"n":128,"median_delta":0.04},
             "flip":{"n":64,"median_delta":0.03},
             "chunk":{"n":32,"median_delta":0.05}},
  "weights": {"perm":0.4,"flip":0.3,"chunk":0.3},
  "seed": 4271, "ts": "2025-09-22T10:51:03Z",
  "drift": {"p_value": 0.74, "ref_window": "W38"},
  "fallback_used": false
}
```

Risk Outputs (for Policy Gates)

- `CWA.score` (0–1), `ci95`, `phase_risk_index`
- `panel_max_delta`, `perm_delta`, `flip_delta`, `chunk_delta`
- `drift.p_value`, `ref_window`
- Recommended action: `{allow|throttle|block}` with rationale

11.7 Latency Budget (Guide)

Path	Work	Typical Panel Counts	p50	p95	p99	Notes
`/project`	Model forward + norm	—	12 ms	25 ms	40 ms	cache models; FP16/INT8 ok if invariant
`/pool` (pass)	panels + mean	P=64, F=32, C=16	18 ms	30 ms	55 ms	vectorized panels; reuse μ0
`/pool` (strict)	panels + mean	P=128, F=64, C=32	32 ms	55 ms	90 ms	use if high-stakes
`/pool` (fallback)	attention / rnn	—	55 ms	120 ms	220 ms	throttle if sustained

Complexity. O(N·d) for pooling; panels scale O((P+F+C)·d) with small constants; chunk panel mildly super-linear if boundary search.

11.8 Event Taxonomy (CWA-Scoped)

- `CWA.Project{projection_id, policy, n, dim, duration_ms}`
- `CWA.Panel.Start/End{pool_id, perm|flip|chunk, n}`
- `CWA.Pass{pool_id, score, pri, ci95}`
- `CWA.Fail{pool_id, score, pri, reason}`
- `CWA.Fallback{pool_id, policy, latency_ms}`
- `CWA.Drift{policy, p_value, ref_window}`
- `CWA.Export{pool_id, cert_log_ref}`

Keys: `pool_id`, `projection_id`, `cert_seed`, `domain`, `observer_id`.

11.9 Worked Example (Text RAG Pooling)

1. Runtime sends 18 chunk vectors (E5 projector).
2. CWA computes μ0 (mean) and panels (P=128, F=64, C=32).
3. Results: `s_perm=0.96`, `s_flip=0.92`, `s_chunk=0.85` → `score=0.91`, `PRI=0.15`.
4. `/pool` returns additive mean; logs certificate; Policy Gates *allow*.
5. A week later, chunk deltas drift to 0.28 → `score=0.78` (Amber); attention fallback engages on long docs while short docs stay additive.

11.10 Configuration (YAML)

```yaml
cwa:
  thresholds:
    score:
      pass: 0.82
      warn: 0.75
    panel_delta_max: 0.25
    pri_max: 0.50
  panels:
    perm: 128
    flip: 64
    chunk: 32
  weights:
    perm: 0.4
    flip: 0.3
    chunk: 0.3
  fallback:
    text: attention.kv
    timeseries: rnn.gru
    multimodal: late_fusion.attn
  drift:
    ref_window: "Wk-1"
    pvalue_alert: 0.05
  reproducibility:
    seed: 4271
    log_all: true
```

11.11 Implementation Notes & Guardrails

- **Determinism:** all panels seeded; record `cert_seed`.
- **Numerics:** add ε in norms; clamp deltas to [0,1]; FP16 safe if ε≥1e-6.
- **Safety on failure:** if certificate fails, **never** return additive mean; must return fallback or `HTTP 409`

with `GateDecision=block`.

- **Privacy:** vectors tagged with lineage; redact raw content in Cert Logs.
- **Testing:** synthetic coherent sequences should fail; permutation-stable bags should pass with high scores.

11.12 Artifacts

- **Config YAML** (11.10)
- **Evaluator pseudocode** (11.4.1)
- **Latency budget table** (11.7)
- **Schemas & Events** (11.6, 11.8)
- **API Contracts** (`/project`, `/pool`)

Next: Chapter 12 — Slot Allocator & Tick/Sync (priority tiers, back-pressure, fleet cadence).

Chapter 12 — Slot Allocator & Tick/Sync (Capacity → Cadence → Cohesion)

Goal. Guarantee quantized capacity (**slots**) and coordinated cadence (**ticks**) across a fleet so observers stay reliable under load. Provide APIs for slot grants, a cadence manager for ticks, fleet-sync metrics (ρ, Δτ), and policies for priority, back-pressure, and safe degradations.

12.1 Responsibilities & Boundaries

Slot Allocator (SA) owns

- Integer **slot budgets** for memory, attention, and tools (`mem`, `attn`, `tool`).
- **Leases** (grants with TTL), **collision** logs, and **eviction** policy.

Tick & Sync (TS) owns

- Global/cluster **tick index τ**, target **cadence** (ms between ticks), phase anchors.
- Fleet-sync metrics: **order parameter ρ** and **desynchrony Δτ**.
- **Schedules** (Ô decisions) or cadence hints to the Observer Runtime.

Collaborates with

- **Observer Runtime** (requests slots; consumes schedules; emits heartbeats).
- **Policy Gates** (consume ρ, Δτ, occupancy; issue throttle/stop).
- **CWA Engine** (may request temporary "panel budget" slots).

12.2 Interfaces (HTTP/gRPC)

12.2.1 Slot APIs

POST `/slots/request` — ask for a lease.

```
{
  "tenant": "team-support",
  "observer_id": "obs-A42",
  "type": "mem",              // mem | attn | tool
  "units": 2,                 // integer
  "ttl_ms": 120000,
  "priority": "P1",           // P0|P1|P2
  "purpose": "retriever.batch",
  "idempotency_key": "slots-obs-A42-1042-mem-2"
}
```

Response

```
{
  "grant_id": "gnt-7af1...",
  "slot_ids": ["slot:mem:3","slot:mem:4"],
  "lease_expiry": "2025-09-22T11:03:10Z",
  "decision": "granted"       // granted | queued | refused
}
```

POST `/slots/heartbeat`

```
{"grant_id":"gnt-7af1...","extend_ms":30000}
```

POST `/slots/release`

```
{"grant_id":"gnt-7af1..."}
```

GET `/slots/occupancy`

- Returns per-type `{total, used, queued, collisions_per_min, by_tenant[]}` (for dashboards).

SLOs: decision p95 ≤ 5 ms; release ≤ 3 ms.

12.2.2 Tick & Sync APIs

POST `/tick/heartbeat` — observer heartbeat + last applied tick.

```
{"observer_id":"obs-A42","tau":1042,"lag_ms":18}
```

GET `/sync/status`

```
{
  "tau_current": 1042,
  "cadence_ms": 80,
  "rho": 0.91,
  "delta_tau": 2,             // max tick gap across fleet
  "jitter_ms_p95": 14
}
```

POST `/cadence/config` — update target cadence & bounds (role-gated).

```
{"cadence_ms": 80, "min_ms": 60, "max_ms": 120, "phase_anchor": "now"}
```

SLOs: status p95 ≤ 10 ms; config p95 ≤ 25 ms.

12.3 Core Metrics

- **Occupancy** per type: `used / total`.
- **Collision rate**: refused grants per minute due to lack of contiguous slots (or budget).
- **ρ (order parameter)**: Kuramoto-style sync of tick phases
  $\rho = \left|\frac{1}{N}\sum_{j=1}^{N} e^{i\theta j}\right|$, where θ_j is observer j's tick phase in $[0, 2\pi)$.
  Interpretation: 1=perfect sync, 0=uniformly desynced.

- **Δτ (desynchrony)**: max tick index difference in the fleet (or 95th–5th percentile gap).
- **Tick jitter**: p95 absolute deviation from target cadence.

12.4 Slot Allocator — Algorithms & Policies

12.4.1 Priority & Admission

- **Priority tiers**: **P0** (critical), **P1** (default), **P2** (batch).
- **Budget splits**: hard min-reserves per tier (`P0_min`, `P1_min`), with **steal** from lower tiers when idle.
- **Admission rule** (simplified):
  1. If `free ≥ units` and within tenant quota → **grant**.
  2. Else if tier < victim tier and **preemptible** grants exist → **evict** lowest-value lease (starting with P2, then P1).
  3. Else **queue** (FIFO within tier) or **refuse** with back-pressure hint.

12.4.2 Eviction & Back-Pressure

- **Eviction**: mark evicted grants, emit `CollisionEvent`, allow 1 grace heartbeat (e.g., 1 s) for cleanup.
- **Back-pressure hints** in refusal:
  - `reduce_parallelism`: decrease concurrent channels.
  - `widen_ticks`: ask TS to increase cadence_ms.
  - `switch_estimator`: hint CWA to use lower-cost fallback.

12.4.3 Lease & Renewal

- Leases must heartbeat before `lease_expiry`. Miss → auto-release.
- **Hard invariant:** non-overlap writes per slot; allocator logs all grants/releases.

Pseudocode (admission)

```
def request(type, units, tier, tenant):
    pool = pools[type]
    if pool.free() >= units and within_quota(tenant, units):
        return grant(units)
    victims = find_preemptible(pool, tier, units)
    if victims.total_units >= units:
        evict(victims); return grant(units, preempt=True)
    return queue_or_refuse()
```

12.5 Tick & Sync — Algorithms & Policies

12.5.1 Cadence Controller

- Target cadence C^* (ms between ticks). Use **PI control** on jitter:
  - error $e_k = C^* - \text{observed_cadence}_k$
  - update $C_{k+1} = C_k + K_P e_k + K_I \sum e$
- **Guardrails**: clamp to $[min_ms, max_ms]$.

12.5.2 Phase Sync & Resynchronization

- Periodic **phase anchors** (e.g., every 1–5 s) broadcast `τ_anchor` + wall-clock.
- Observers compute phase error and **nudge** their local timers (slew, no jumps).
- If **ρ < ρ_min** or **Δτ > Δτ_max**, enter **Resync Mode**:
  - temporarily widen cadence (reduce rate) and **trim** outlier observers (delay their next tick).
  - restrict schedules to **commuting-safe** channels until sync recovers.

12.5.3 Schedule Shaping

- When under back-pressure (from SA or Policy Gates), TS **narrows channel set** and increases inter-tick.
- **Burst smoothing**: token bucket per observer and per tenant; overflows delay next `TickStart`.

Pseudocode (resync)

```
if rho < rho_min or delta_tau > dtau_max:
    cadence_ms = min(cadence_ms * 1.25, max_ms)
    schedule = schedule.filter(commuting_safe=True)
    broadcast("RESYNC", cadence_ms, schedule)
```

12.6 Degradation Modes (Safe States)

- **D0 Normal**: full cadence; all channels; panels at default counts.
- **D1 Gentle**: widen cadence by 10–25%; reduce parallelism; downscale panel counts (perm/flip/chunk ÷2).
- **D2 Strict**: commuting-only channels; additive pooling allowed only if `CWA.score ≥ θ_strict`; others fallback.
- **D3 Quiescent**: stop tick advancement except health checks; accept critical P0 only.

Entry conditions (any):

- Occupancy > 0.9 for ≥ 60 s (by type).
- ρ < 0.8 for ≥ 10 s **or** Δτ > 5 for ≥ 10 s.
- Collision rate > threshold AND grant latency p95 > 30 ms.

Exit conditions: ρ ≥ 0.9 and Δτ ≤ 2 and occupancy < 0.7 for 30 s.

12.7 SLA Bands & Desync Alerts

12.7.1 SLA Bands (Green / Amber / Red)

Metric	Green	Amber	Red	Action
Slot occupancy (mem/attn/tool)	≤ 0.70	0.70–0.90	> 0.90	D1 at Amber; D2 at Red
Grant latency p95	≤ 5 ms	5–20 ms	> 20 ms	Increase reserves / throttle
Collision rate (per min)	≤ 2	3–10	> 10	Investigate evictions; widen cadence
ρ (order parameter)	≥ 0.90	0.80–0.90	< 0.80	Resync; scheduling restrictions
Δτ (ticks)	≤ 2	3–5	> 5	Trim outliers; slow cadence
Tick jitter p95	≤ 15 ms	16–40 ms	> 40 ms	PI retune; anchor more often

12.7.2 Desync Alert Thresholds

- **ALERT-SYNC-AMBER**: $\rho < 0.9$ for 5 s **or** $\Delta\tau \geq 3$ for 5 s → enter **D1**.
- **ALERT-SYNC-RED**: $\rho < 0.8$ for 10 s **or** $\Delta\tau \geq 6$ for 10 s → enter **D2** + notify Policy Gates.
- **ALERT-SYNC-CRIT**: $\rho < 0.6$ for 20 s **or** $\Delta\tau \geq 10$ for 20 s → **D3**; block non-P0.

12.8 Event Taxonomy (SA/TS-Scoped)

- `Slots.Request{grant_id, type, units, priority, decision}`
- `Slots.Heartbeat{grant_id, extend_ms}`
- `Slots.Release{grant_id}`
- `Slots.Collision{type, tenant, needed_units}`
- `TickStart{τ, schedule_id}`
- `TickAnchor{τ_anchor, wallclock, cadence_ms}`
- `DesyncAlert{rho, delta_tau, level}`
- `Cadence.Update{cadence_ms, reason}`
- `Degrade.Enter/Exit{mode, reason}`

Keys: `observer_id`, `tenant`, `schedule_id`, `grant_id`.

12.9 Worked Examples

12.9.1 RAG Surge (Evening Traffic)

- Occupancy(mem) climbs to 0.92; grant p95 → 28 ms; collision rate 12/min.
- SA signals back-pressure; TS enters **D2**: cadence +20%, commuting-only channels, CWA strict threshold.
- Within 45 s, occupancy falls to 0.74 and ρ stays ≥0.9 → exit to **D0**.

12.9.2 Multi-Robot Sync

- Field bots drift due to poor NTP; ρ drops to 0.77 and Δτ=7.
- TS broadcasts **RESYNC** with anchors every 500 ms; trims fast outliers; slows cadence to 110 ms.
- After 12 s, ρ=0.92, Δτ=2; normal cadence restored; schedules reopened to full set.

12.10 Configuration (YAML)

```yaml
slots:
  pools:
    mem:   {total: 128, reserve: {P0: 32, P1: 64, P2: 32}}
    attn:  {total: 64,  reserve: {P0: 16, P1: 32, P2: 16}}
    tool:  {total: 24,  reserve: {P0: 8,  P1: 12, P2: 4}}
  eviction:
    preemptible: ["P2","P1"]
    grace_ms: 1000
  quotas:
    team-support: {mem: 48, attn: 24, tool: 8}
```

```
    team-search:  {mem: 64, attn: 32, tool: 12}

ticksync:
  cadence_ms: 80
  bounds_ms: {min: 60, max: 140}
  pi_gains: {kp: 0.12, ki: 0.02}
  anchors:
    period_ms: 1000
    jitter_target_p95_ms: 15
  thresholds:
    rho_min: 0.90
    delta_tau_max: 2
    amber: {rho: 0.90, delta_tau: 3, duration_s: 5}
    red:   {rho: 0.80, delta_tau: 6, duration_s: 10}
    crit:  {rho: 0.60, delta_tau:10, duration_s: 20}
  degradation:
    d1: {cadence_factor: 1.1, panel_scale: 0.5}
    d2: {cadence_factor: 1.25, commuting_only: true, cwa_strict: 0.86}
    d3: {pause_non_p0: true}
```

12.11 Guardrails & Testing

- **Integer slots only;** assert non-overlap, and log all evictions.
- **Idempotent grants** via `idempotency_key`.
- **Monotone τ;** no tick jumps; only slew during resync.
- **Load tests:** ramp to 95% occupancy; verify D1/D2/D3 transitions and recovery.
- **Sync tests:** inject clock skew; confirm alert ladder and convergence of ρ/Δτ.

Artifacts delivered: SLA bands (12.7.1), desync alert thresholds (12.7.2), slot & cadence configs (12.10), event taxonomy (12.8), algorithms/pseudocode (§12.4–12.5).

Next: **Chapter 13 — BeltOps Dashboard & Policy Gates** (panels, webhooks, audits, thresholded gates).

Chapter 13 — BeltOps Dashboard & Policy Gates (Closure → Telemetry → Control)

Goal. Close the macro loop with **Purpose-Flux Belt Theory (PFBT)** telemetry and **deterministic gates** that throttle or block risky runs. Surface **Five-Line KPIs** (Gap/Flux/Twist/Coherence/Residual), **EEI/SI** indices, and a clean **webhook + export** story for audits.

13.1 Responsibilities & Boundaries

BeltOps Dashboard (BD) owns

- The **belt worldsheet**: **Gap**, **Flux**, **Twist**, α.
- **Residual**: $R = |\text{Gap} - (\text{Flux} + \alpha \cdot \text{Twist})|$.
- **Five-Line KPI** time series + **EEI** (Effectiveness/Execution Index) and **SI** (Sustainability Index).
- KPIs export (pull via API / scheduled pushes).

Policy Gates (PG) owns

- Gate configs & thresholds; evaluation engine; **allow|throttle|block** decisions.

- Triggers on **CWA score**, **Phase-Risk Index (PRI)**, **PBHL Residual**, and **sync/capacity** hints (ρ, Δτ, occupancy).
- Signed webhooks and deterministic decision logs.

Collaborators

- **Observer Runtime** (Agreement reports, pool results).
- **CWA Engine** (scores, PRI, drift).
- **Tick & Sync** (applies gate decisions; cadence shaping).
- **GRC/Audit** (exports).

13.2 Interfaces (Overview)

13.2.1 `POST /belt`

Update belt worldsheet from fresh data (usually per tick/window).

Request

```
{
  "belt_id": "support-v2025Q3",
  "window": {"start": "2025-09-22T11:00:00Z", "end": "2025-09-22T11:01:00Z"},
  "gap": 0.62,
  "flux": 0.48,
  "twist": 0.12,
  "alpha": 1.1,
  "inputs": {
    "pool_ids": ["pool-7c0b..."],
    "agree_ids": ["ag-6c91..."]
  },
  "notes": "release sprint W39"
}
```

Response

```
{
  "pbhl": {"residual": 0.062, "status": "green"},
  "kpis": {"gap": 0.62, "flux": 0.48, "twist": 0.12, "coherence": 0.91},
  "indices": {"eei": 0.73, "si": 0.81},
  "update_id": "beltupd-92af..."
}
```

SLO p95 ≤ 60 ms. Auth `control.write`.

13.2.2 KPIs Export (Pull)

GET `/belt/:id/kpi?from=...&to=...&format=parquet|jsonl`
Returns Five-Line KPIs + EEI/SI + Residual with lineage (ids of contributing pool/agreement updates).
SLO p95 ≤ 200 ms (server-side range aggregation).

13.2.3 Gate Evaluation & Triggers

POST `/gate/evaluate` *(optional explicit call; normally auto on belt/cert events)*

```
{
  "belt_id": "support-v2025Q3",
  "metrics": {
    "cwa_score": 0.77,
    "pri": 0.36,
```

```
    "residual": 0.11,
    "rho": 0.86,
    "delta_tau": 4,
    "occupancy_mem": 0.82
  },
  "context": {"domain":"support-kb","risk":"standard"}
}
```

Response

```
{
  "decision": "throttle",
  "reasons": ["cwa_score_below_warn","residual_amber","rho_below_target"],
  "actions": {"cadence_factor": 1.15, "commuting_only": false},
  "gate_id": "gate-4d1e...",
  "effective_for_s": 60
}
```

Webhooks fire on any decision (see §13.7).

13.3 Data & Schemas

13.3.1 Belt Worldsheet

```
{
  "$schema": "https://observerops.io/schemas/belt.v1.json",
  "belt_id": "support-v2025Q3",
  "window": {"start":"...","end":"..."},
  "gap": 0.62,
  "flux": 0.48,
  "twist": 0.12,
  "alpha": 1.1,
  "residual": 0.062,
  "coherence": 0.91,            // agreement/coherence proxy at macro layer
  "indices": {"eei": 0.73, "si": 0.81},
  "lineage": {"pool_ids":["..."], "agree_ids":["..."], "cert_refs":["..."]},
  "hash": "sha256:..."
}
```

13.3.2 Gate Policy Config

```
{
  "$schema": "https://observerops.io/schemas/gate-config.v1.json",
  "bands": {
    "residual": {"green": [0,0.08], "amber": [0.08,0.15], "red": [0.15, 1.0]},
    "cwa_score": {"pass": 0.82, "warn": 0.75},
    "pri_max": 0.50,
    "rho_min": 0.90,
    "delta_tau_max": 2
  },
  "actions": {
    "amber": {"cadence_factor": 1.1, "panel_scale": 0.7},
    "red": {"cadence_factor": 1.25, "commuting_only": true, "block_if_score_lt": 0.70}
  },
  "debounce_s": 10,
  "hold_s": 60,
  "escalation": {"amber_windows": 3, "red_windows": 2}
}
```

13.3.3 Decision Log

```
{
  "gate_id": "gate-4d1e...",
  "belt_id": "support-v2025Q3",
  "ts": "2025-09-22T11:05:03Z",
  "decision": "throttle",
  "reasons": ["residual_amber","cwa_warn"],
  "inputs": {"residual":0.11,"cwa_score":0.77,"pri":0.36,"rho":0.86,"delta_tau":4},
```

```
  "signature": "HMAC-SHA256:..."
}
```

13.4 Evaluation Logic (Deterministic)

13.4.1 Rule Set (conceptual)

1. **CWA gate**
   - If `cwa_score < warn` or `pri > pri_max` → at least **throttle**.
   - If `cwa_score < block_if_score_lt` → **block** additive pooling; force fallback.
2. **PBHL gate**
   - If `residual` ∈ Amber → **throttle** (widen cadence, reduce panels).
   - If `residual` ∈ Red → **block risky schedules** and request Twist analysis.
3. **Sync/Capacity gate**
   - If `rho < rho_min` or `delta_tau > max` or occupancy > 0.9 → **throttle** or **commuting-only**.
4. **Escalation/debounce**
   - Require N consecutive windows before **raise**; M green windows before **lower** (hysteresis).
5. **Determinism**
   - Same inputs → same `gate_id` (hash). All thresholds versioned.

13.4.2 Pseudocode

```
def evaluate(m, cfg, now):
    reasons, actions = [], {}
    decision = "allow"

    if m["cwa_score"] < cfg["bands"]["cwa_score"]["warn"] or m["pri"] > cfg["bands"]["pri_max"]:
        decision, reasons = "throttle", reasons + ["cwa_warn_or_pri"]

    if m["cwa_score"] < cfg["actions"]["red"]["block_if_score_lt"]:
        decision, reasons = "block", reasons + ["cwa_block"]

    res = m["residual"]
    rband = band(res, cfg["bands"]["residual"])
    if rband == "amber":
        decision = max_decision(decision, "throttle"); reasons += ["residual_amber"]
    elif rband == "red":
        decision = "block"; reasons += ["residual_red"]

    if m["rho"] < cfg["bands"]["rho_min"] or m["delta_tau"] > cfg["bands"]["delta_tau_max"]:
        decision = max_decision(decision, "throttle"); reasons += ["sync_issue"]

    actions = prescribe(decision, cfg["actions"])
    gate_id = stable_id(m, cfg)
    return gate_id, decision, reasons, actions
```

13.5 Dashboards (Panel Specs)

13.5.1 Five-Line KPI (time series, 1-min granularity)

- **Lines**: Gap, Flux, Twist, Coherence, Residual.
- **Bands**: green/amber/red based on configured thresholds.

- **Features**: hover to show contributing pool/agree ids; click to open Cert Logs.

13.5.2 EEI / SI Panels

- **EEI**: weighted blend of outcome quality, throughput, and agreement stability.
  Example $\text{EEI} = 0.5 \cdot Q + 0.3 \cdot \text{ThroughputNorm} + 0.2 \cdot \text{Agreement}$.
- **SI**: energy/compute cost per accepted unit + variance + slot pressure proxy.

13.5.3 Residual Trend

- Rolling residual with **change points** and **Twist annotations** (reorg, policy change).
- Correlate with **cadence changes** and **gate states**.

Panel spec (JSON)

```
{
  "panel_id": "kpi-five-line",
  "layout": {"w": 12, "h": 6},
  "series": [
    {"metric":"gap"}, {"metric":"flux"}, {"metric":"twist"},
    {"metric":"coherence"}, {"metric":"residual"}
  ],
  "bands": {"residual":[0.08,0.15], "coherence":[0.85,0.9]}
}
```

13.6 SLA Bands & Actions (Macro)

Metric	Green	Amber	Red	Default Gate Action
Residual	≤ 0.08	(0.08, 0.15]	> 0.15	Allow / Throttle / Block
CWA Score	≥ 0.82	[0.75, 0.82)	< 0.75	Allow / Throttle / Block (fallback)
PRI	≤ 0.20	(0.20, 0.50]	> 0.50	Allow / Throttle / Block
ρ	≥ 0.90	[0.80, 0.90)	< 0.80	Allow / Throttle / Commute-only/Block
Δτ (ticks)	≤ 2	3–5	> 5	Allow / Throttle / Block

13.7 Webhook Schema (Signed)

Endpoint registration

```
{
  "url": "https://ops.example.com/observerops/webhooks",
  "events": ["Gate.Decision","Belt.Update","CWA.Drift"],
  "secret": "********",   // HMAC key
  "retry": {"max": 5, "backoff_ms": [500, 2000, 5000]}
}
```

Event — `Gate.Decision`

```
{
  "event": "Gate.Decision",
  "id": "evt-8b2d...",
  "ts": "2025-09-22T11:06:30Z",
  "belt_id": "support-v2025Q3",
  "decision": "throttle",
  "reasons": ["cwa_warn_or_pri","residual_amber"],
  "actions": {"cadence_factor": 1.15, "panel_scale": 0.7},
  "signature": "HMAC-SHA256:base64(...)"
}
```

Verification: `X-ObserverOps-Signature` header (HMAC of payload). Retries are idempotent by `id`.

13.8 Audit Export Format

Exports are **bundles** with a manifest + referenced artifacts (parquet/jsonl). Signed and hash-addressed.

Manifest

```
{
  "export_id": "exp-W39-beltops",
  "created": "2025-09-22T11:10:05Z",
  "belt_id": "support-v2025Q3",
  "windows": [{"start":"2025-09-22T10:00:00Z","end":"2025-09-22T11:00:00Z"}],
  "files": [
    {"path":"kpi.parquet","sha256":"..."},
    {"path":"decisions.jsonl","sha256":"..."},
    {"path":"cert_logs.parquet","sha256":"..."}
  ],
  "lineage": {"pool_ids":[...], "agree_ids":[...], "cert_refs":[...]},
  "signature": "ed25519:..."
}
```

Contents

- `kpi.parquet`: time series (Gap/Flux/Twist/Coherence/Residual, EEI/SI).
- `decisions.jsonl`: `Gate.Decision` with inputs/reasons/actions.
- `cert_logs.parquet`: referenced CWA certificates (subset).
- Optional: `privacy_map.json` (redactions), `schema_versions.json`.

Retention: hot 90–365 days; cold 7 years.

13.9 Worked Scenarios

13.9.1 CWA Amber, Residual Amber → Throttle

- Score = 0.79, PRI = 0.28, Residual = 0.11 (Amber bands).
- PG returns **throttle**: `cadence_factor=1.15`, `panel_scale=0.7`.
- TS widens cadence; CWA reduces panel counts; after 3 windows, Residual=0.07 (Green) → auto-lift.

13.9.2 Residual Red → Block Risky Schedules

- Residual spikes to 0.18 after org **Twist** (reorg); Score=0.84.
- PG **blocks** non-critical workloads and restricts schedules to commuting-safe channels until Residual < 0.10 for 4 windows. Twist annotation appears on Residual panel.

13.10 Configuration (YAML)

```
beltops:
  kpi:
    coherence_source: "agreement.rate"
    window_s: 60
  pbhl:
    residual_bands: {green: [0,0.08], amber: [0.08,0.15], red: [0.15,1.0]}
  indices:
    eei_weights: {quality: 0.5, throughput: 0.3, agreement: 0.2}
    si_weights:  {energy: 0.5, variance: 0.3, slots: 0.2}
  export:
    schedule_cron: "*/15 * * * *"
    format: "parquet"
```

```
    sign: "ed25519"
    include: ["kpi","decisions","cert_logs"]

gates:
  thresholds:
    cwa_score: {pass: 0.82, warn: 0.75}
    pri_max: 0.50
    rho_min: 0.90
    delta_tau_max: 2
  actions:
    amber: {cadence_factor: 1.1, panel_scale: 0.7}
    red:   {cadence_factor: 1.25, commuting_only: true, block_if_score_lt: 0.70}
  debounce_s: 10
  hold_s: 60
  escalation: {amber_windows: 3, red_windows: 2}

webhooks:
  url: "https://ops.example.com/observerops/webhooks"
  secret: "env:WEBHOOK_SECRET"
  events: ["Gate.Decision","Belt.Update","CWA.Drift"]
```

13.11 SLOs, Alerts, and Guardrails

SLOs

- `/belt` p95 ≤ 60 ms; KPI export p95 ≤ 200 ms (range ≤ 1 h).
- Gate evaluation ≤ 10 ms inline (≤ 100 ms async fan-out).
- Dashboard refresh ≤ 1 s.

Alerts

- **Residual.Red** sustained ≥ 2 windows.
- **Gate-Flap** (> 3 decisions flip-flop within 5 min).
- **Export-Lag** (> 2 min after schedule).

Guardrails

- Deterministic evaluation (versioned thresholds).
- Hysteresis (debounce + hold) prevents flapping.
- Signed webhooks + signed exports for tamper-evidence.
- Privacy: KPI exports contain ids only; raw content redacted at source.

Artifacts delivered: Panel specs (§13.5), webhook schema (§13.7), audit export format (§13.8), interfaces (§13.2), configs (§13.10), evaluation pseudocode (§13.4.2), SLA bands (§13.6).

Next: **Part III — Implementation Patterns & Recipes** (apply BeltOps + Gates to real fleets).

Chapter 14 — Tool-Using LLM Agents (Pattern → Recipes → KPIs)

Goal. Turn an LLM that calls external tools into a **buildable observer**: tools map to channels Π; **Ô** (scheduler) picks the next channel; **τ** (ticks) commit decisions; traces **latch**; replicas run agreement checks; pooling is **CWA-gated**. You'll get safe-retry patterns, SBS logging, multi-agent quorum, KPIs, and ablations you can run this week.

14.1 Pattern (Tools ↔ Channels; Ô/τ; Latching)

Mapping.

- Channel set Π ≙ tool registry (e.g., `web.search`, `kb.retriever.vector`, `kb.retriever.keyword`, `code.exec`, `calc`, `summarize`).
- **Ô** policy selects next channel based on state S and recent trace **T**.
- **τ** advances per committed step; **TraceWrite(τ_k, π, y)** is the latching point.

Minimal loop (pseudocode)

```
def observer_loop(goal):
    tau = tick.start()
    while not done(goal):
        pi = O_hat.select_channel(S, T)        # Ô policy
        grant = slots.request(type=need(pi))   # mem/attn/tool slots
        preflight.check_commute(pi, T)         # conflict graph C
        y = tools.invoke(pi, S)                # MEASURE
        trace_id = trace.write(tau, pi, y)     # LATCH
        if pi in PROJECTABLE:
            phi = cwa.project(y, policy="embeddings.e5-large")
            pooled = cwa.pool(phi, min_score=theta)  # cert-gated
            S = update_state(S, pooled)
        T = append(T, (tau, pi, y))
        tau = tick.next()
    return finalize(S, T)
```

What this guarantees

- **Internal collapse:** downstream control conditions on latched `y`.
- **Agreement hooks:** shared records + commuting sequences enable cross-observer checks.
- **Capacity safety:** quantized slots; explicit back-pressure.

14.2 Tool Registry & Compatibility (Commute Matrix C)

Tool (channel π)	Typical conflicts (non-commuting)	Notes
`web.search`	`web.search` (same query, same τ)	de-duplicate via idempotency key
`kb.retriever.vector`	— (commutes with `kb.keyword`)	pointer→`support.answer`
`kb.retriever.keyword`	— (commutes with `kb.vector`)	pointer→`support.answer`
`summarize`	`summarize` (same input at same τ)	idempotent if content-hash matches
`code.exec`	often **non-commuting** with stateful tools	run after read-only steps
`calc`	commutes (pure)	deterministic
`write.kb`	non-commuting with any read of same object	defer to τ+1

Recipe: Preflight before `MEASURE`:

- If `C[π_i, π_j]==false` for same object/window → reorder or push to `τ_{k+1}`.
- Reserve `tool` slots for stateful tools to serialize writes.

14.3 Ô Scheduling Policies (choose next tool)

Scoring components (example)

- **Information gain**: expected reduction in AL / Collapse Entropy S_c.

- **Cost**: slot + latency budget.
- **Risk**: panel deltas from last CWA; phase-risk PRI.
- **Compatibility**: penalize potential conflicts.

```
def select_channel(S, T):
    cands = filter_enabled(Π)
    score = {}
    for pi in cands:
        ig   = est_info_gain(pi, S, T)
        cost = cost_model(pi)
        risk = last_pri(pi)
        compat = compat_margin(pi, T)  # 1.0 if safe
        score[pi] = 0.6*ig - 0.3*cost - 0.1*risk + 0.1*compat
    return argmax(score)
```

Cadence: start at 60–120 ms between ticks; widen if slots hot or gates throttle.

14.4 Recipes

R1. Safe Retries (idempotent, latching-aware)

- **Idempotency keys** per `(τ, π, input_hash)`.
- Retry matrix:
  - `5xx`/timeout → **same τ**, `retry_id` set; dedup by idempotency key.
  - `409 Conflict` → request reschedule to `τ+1`.
  - `412/425` (precondition/tick-early) → wait for `TickStart`.
- **Never** mutate a latched trace; publish a **new** `TraceWrite` at `τ+1` with reason.

R2. SBS Logging (pointer redundancy)

- Define pointer `support.answer` → channels `{kb.vector, kb.keyword, kb.cached}`.
- On each **τ**, record pointer outcomes; compute redundancy *R* and emit `AgreementReport`.
- Store evidence ids so replicas can agree on shared objects.

R3. Multi-Agent Quorum

- Run **3 replicas** (A/B/C) with same **Ô** policy and shared registry.
- Accept result if `agree(A,B) ≥ θ` **or** `agree(B,C) ≥ θ` (2-of-3).
- If quorum fails:
  1. restrict to commuting channels,
  2. elevate to human or retry at `τ+1` with narrowed schedule.

R4. Certificate-Gated RAG

- After `kb.*` measurements, call `/project`, then `/pool` with panels (e.g., P=128,F=64,C=32).
- If `score ≥ 0.82` & `PRI ≤ 0.20` → mean; else **attention** fallback.
- Log certificate for audit and BeltOps KPIs.

R5. Desync Hygiene (Δτ, ρ)

- If **ρ < 0.9** or **Δτ ≥ 3**: slow ticks by 10–25%, reduce parallelism, prefer commuting channels.
- Gate raises to normal when **ρ ≥ 0.9** and **Δτ ≤ 2** for 30 s.

14.5 KPIs (definitions & targets)

KPI	Definition	Green Target	Notes
Disagreement	1 − mean(agreement score across replicas)	≤ 0.08	pointer-conditioned
Mis-exec	tool errors / tool invocations	≤ 1.0%	include timeouts
Δτ	fleet tick spread (95th–5th)	≤ 2 ticks	alert ≥ 3
Trace half-life	time until 50% of traces are overwritten/invalidated by updates	≥ 24 h	signals stability
Cert pass-rate	fraction of pools with `score ≥ θ`	≥ 80%	domain-dependent
Latency (E2E)	user→answer p95	target SLO	depends on fallback mix

---

14.6 Worked Example (Support Q&A)

1. **Ô** picks `kb.retriever.vector` at `τ_0`; Slot Allocator grants `mem=1`.
2. Measure→Latch: `TraceWrite(τ_0, kb.vector, y_v)`.
3. **Ô** picks `kb.keyword` (commuting) at `τ_1`; latch `y_k`.
4. Runtime calls **CWA**: project+pool on `{y_v,y_k,y_cached}` (redundant set).
    - Panels pass: `score=0.90`, `PRI=0.12` → mean vector retained.
5. **summarize** at `τ_2` condenses retrieved passages; latch summary.
6. Replica B mirrors steps; `/agree` between A/B on pointer `support.answer` returns 0.94.
7. BeltOps ingests KPIs; Policy Gates stay **allow**.

14.7 Playbooks

P1. Mis-exec spike

- Symptom: mis-exec > 2% for 5 min.
- Actions: freeze `code.exec`, prefer read-only; raise tool timeouts; enable retries (max 2); widen ticks by 10%; open incident if sustained.

P2. Certificate amber wall

- Symptom: CWA pass-rate drops < 60%.
- Actions: reduce chunk size variance; bump panel counts (P+32/F+16/C+16); enable attention fallback for long docs; schedule feature rollbacks if drift persists.

P3. Quorum failures

- Symptom: Disagreement > 0.15 across replicas.
- Actions: enforce commuting-only schedule for one window; elevate to human checker; cache

accepted pointer; log counterexample set.

14.8 Ablations (±Ô, ±slots, ±certificate)

Design. 3× runs on the same traffic (A/B/C), 1 week each:

- **Baseline** (Ô+slots+cert)
- **No-Ô** (greedy tool order)
- **No-slots** (unbounded parallelism)
- **No-certificate** (always mean)

Ablation	Expected shift	Interp
No-Ô	Disagreement ↑ 5–10%; latency ↑	poor channel order & conflicts
No-slots	Mis-exec ↑; Δτ ↑; E2E latency variance ↑	collisions & back-pressure storms
No-certificate	Accuracy ↓ on coherent corpora; latency ↓	unsafe pooling saves time but harms quality

---

14.9 Config (YAML)

```yaml
agent:
  goal: "answer_support_q"
  channels:
    - id: web.search           # tool → channel
      type: tool
      cost: {latency_ms_p95: 200}
    - id: kb.retriever.vector
      type: retriever
      pointer: [support.answer]
    - id: kb.retriever.keyword
      type: retriever
      pointer: [support.answer]
    - id: summarize
      type: llm
  commute_matrix: cm:v1.12
  O_hat:
    policy: "ig-cost-risk"
    weights: {ig: 0.6, cost: 0.3, risk: 0.1}
  tau:
    cadence_ms: 90
    bounds_ms: {min: 70, max: 140}

cwa:
  thresholds: {pass: 0.82, warn: 0.75, pri_max: 0.5}
  panels: {perm: 128, flip: 64, chunk: 32}
  fallback: {text: attention.kv}

quorum:
  replicas: 3
  agree_threshold: 0.90
  pointer: "support.answer"

slots:
  pools: {mem: {total: 64}, tool: {total: 16}}
  require_grant: true

alerts:
  misexec_rate: {warn: 0.01, crit: 0.02}
  disagreement: {warn: 0.12, crit: 0.18}
  delta_tau: {warn: 3, crit: 5}
```

14.10 Tests

- **Unit:** idempotent `/measure` under retries; per-tick single-write; commute preflight.
- **Property:** permutation stability on bag-like inputs (should pass CWA); coherent chain (should fail CWA).
- **Integration:** 3-replica quorum; SBS redundancy R≥3; agreement ≥ 0.9 on stable topics.
- **Load:** occupancy 80–95%; confirm no latching violations; Δτ stays ≤ 2 in green.

14.11 Artifacts

- **Playbooks:** P1–P3 (mis-exec, certificate amber, quorum failures).
- **Ablations:** ±Ô, ±slots, ±certificate with expected effect sizes.
- **Dashboards:** Disagreement, mis-exec, Δτ, CWA pass-rate, trace half-life.
- **Configs:** Agent + CWA + slots + quorum YAML (14.9).

Next: **Chapter 15 — RAG & Embeddings** (project→CWA-gate→pool; chunking as instrument design; latency/accuracy fronts).

Chapter 15 — RAG & Embeddings (Project → CWA-Gate → Pool)

Goal. Make retrieval-augmented generation (RAG) **observer-safe**: *project first*, run a **CWA certificate** to decide if *additive pooling* is valid, else **auto-fallback** to order-aware pooling. Treat **chunking as instrument design**, integrate cleanly with vector DBs, and track **accuracy ↔ latency** with Phase-Risk KPIs.

15.1 Pattern (End-to-End)

Data path.

1. **Measure (retrieve)** with commuting channels: `kb.retriever.vector` + `kb.retriever.keyword` (pointer → `support.answer`).
2. **Project** each candidate passage/snippet to vector space via policy $P(\cdot)$.
3. **Certificate** the set $V = \{v_i\}_{i=1}^{N}$ with panels (perm/flip/chunk) → **CWA.score** and **PRI**.
4. **Pool**:
   - if `score ≥ θ` and `PRI ≤ PRI_max` → **mean/sum** (fast path);
   - else → **attention/CNN** (order-aware).
5. **Generate** with pooled representation as context/features; **latch** traces and certificate.

Why it works. Projection erases phase/order *if* the projector truly collapses nuisance structure. Certificate checks that this holds **for the current set** (not just in general), guarding against "unsafe mean."

15.2 Chunking as Instrument Design

Treat the chunker as part of the **instrument** π_θ with orientation θ (size, stride, boundary rule).

Design knobs

- **Size/stride** (e.g., 512/128 tokens)
- **Overlap window** (Hann/flat)
- **Boundary policy** (sentence-aware, heading-aware)
- **Orientation**: per-domain templates (FAQ vs narrative vs code)

Commutativity windows.

- Chunkers with the **same** boundary policy commute; mixed policies may induce order sensitivity → chunk panel will detect it.

Redundancy. For SBS-style pointer objectivity, maintain 2–3 redundant chunkers (e.g., `size 256`, `size 512`, `sentence`) mapped to the **same pointer**; improves agreement and pass-rate.

15.3 Vector-DB Integration (Index & Query)

Upsert schema (generic)

```
{
  "id": "doc:123#ch:05",
  "vector": [ ... d floats ... ],
  "metadata": {
    "doc_id": "doc:123",
    "chunk_id": "05",
    "projector": "embeddings.e5-large",
    "norm": "l2",
    "chunk": {"size": 512, "stride": 128, "policy": "sentence"},
    "pointer": ["support.answer", "entity.X"],
    "ts_ingested": "2025-09-15T10:05:02Z"
  }
}
```

Partitions.

- **By projector** (`projector=…`) to avoid mixing spaces.
- **By domain** (knowledge area / locale).
- Optional **by orientation** (chunk policy) to stage panel-specific retrievals.

Query path

1. KNN/ANN search (per projector/partition).
2. Join with keyword/graph hits (commuting channels).
3. Produce **candidate set** V with provenance (doc, chunk meta).
4. Hand to CWA for certificate + pooling.

   Tip: persist **projection_id** and **chunk_meta** alongside vectors so `chunk` panel can jitter boundaries without re-reading raw text.

15.4 Recipes

R1. Permutation Budget (panel counts)

Choose `perm/flip/chunk` sample sizes to balance CI vs latency.

Heuristic:

$$P = \min(128,\ \max(32,\ 8 \cdot \lceil \log_2(N \cdot d) \rceil)), \quad F = \lfloor P/2 \rfloor, \quad C = \max(16,\ \lfloor P/4 \rfloor)$$

- Clamp up (strict) on safety-critical domains; clamp down on mobile/edge.

R2. Phase-Risk Bands (actions)

- **Green:** `PRI ≤ 0.20` → allow additive mean.
- **Amber:** `0.20 < PRI ≤ 0.50` → allow mean only if `score ≥ θ_warn` and latency budget tight; else **attention**.
- **Red:** `PRI > 0.50` → force **attention/CNN**, reduce chunk variability, widen ticks.

R3. Fallback to Attention Pooling

- **Text:** `attention.kv` with position encodings; cap context by **slots**.
- **Time-series:** `rnn.gru` / `tcn` with dilation; per-channel normalization.
- Cache fallback outputs for identical candidate sets (content-hash key).

R4. Mixed-Mode Retrieval (commuting set)

Use `{vector, keyword, cached}` as **redundant** channels for the same pointer.

- Raise redundancy $R \geq 3$ → improves agreement and stabilizes pass-rate.
- Weight channels by historical reliability in ranking fusion.

R5. Precompute vs On-the-Fly

- **Precompute** document vectors; **on-the-fly** per-query projection of short snippets (e.g., synthesized queries) if they influence pooling risk.
- Always record `projector` and `seed` in **Cert Log** for reproducibility.

15.5 KPIs & Targets

KPI	Definition	Target / Band	Interpretation
Accuracy	task score (EM/F1/nDCG@k)	maximize	primary quality
Latency (E2E)	user→answer p95	meet SLO	certificate adds small fixed overhead
CWA Pass-Rate	`% pools with score ≥ θ_pass`	≥ 75–85%	higher means more fast-path
Phase-Risk Index (PRI)	1 – min(panel scores)	Green ≤ 0.20	coherence/order risk
Fallback Rate	`% answers using attention/CNN`	≤ 25% (steady)	too high → revisit chunking
Agreement	pointer agreement across replicas	≥ 0.90	SBS/objectivity proxy
Panel Cost	ms per pool (pass vs fallback)	within budget	capacity planning

Plot **accuracy vs latency** Pareto with markers by **pass/fallback**—aim to shift the frontier down/right with better chunking & redundancy.

15.6 Pipeline Pseudocode (Reference)

```
def rag_answer(query):
    # 1) Retrieve via commuting channels
    Vv = vdb.knn(query_vec, k=K, projector="e5-large")        # vectors
    Vk = keyword.search(query, k=Kk)                           # keywords
    C  = fuse(Vv, Vk)                                          # merge, dedup

    # 2) Project (if any raw needs projection)
    phi = project_if_needed(C, policy="e5-large", normalize=True)

    # 3) Certificate: decide pooling mode
    cert = cwa.certificate(phi, panels=choose_panels(len(phi)))
    if cert["score"] >= θ_pass and cert["phase_risk_index"] <= PRI_max:
        pooled = mean(phi)
        mode = "mean"
    elif cert["score"] >= θ_warn and latency_budget_tight():
        pooled = mean(phi); mode = "mean-amber"
    else:
        pooled = attention_pool(C)  # order-aware
        mode = "attention"

    # 4) Generate
    answer = llm.generate(query, context=pooled, mode=mode)

    # 5) Log
    log_cert(cert); log_pool(mode); log_answer(answer)
    return answer
```

15.7 Config Templates (YAML)

```yaml
projectors:
  default: "embeddings.e5-large"
  policies:
    embeddings.e5-large:
      normalize: true
      dtype: float32
      cache: memory+disk

chunkers:
  - id: "sent-512@128"
    size: 512
    stride: 128
    boundary: "sentence"
  - id: "sent-256@64"
    size: 256
    stride: 64
    boundary: "sentence"

retrieval:
  knn:
    index: "qdrant://kb-support"
    space: "cosine"
    shard_by: ["projector","domain"]
    k: 20
  keyword:
    engine: "bm25"
    k: 20
  fusion:
    method: "rrf"
    weights: {knn: 0.6, keyword: 0.4}

cwa:
  thresholds: {pass: 0.82, warn: 0.75, pri_max: 0.50}
  panels: {perm: auto, flip: auto, chunk: auto}
  weights: {perm: 0.4, flip: 0.3, chunk: 0.3}
  fallback:
    text: "attention.kv"

kpis:
  report_window_s: 60
  accuracy_metric: "nDCG@10"
  latency_slo_ms_p95: 800
```

15.8 Benchmark Harness

Purpose. Measure **accuracy ↔ latency** under controlled phase/order conditions; stress certificate decisions.

Datasets

- **FAQ Bag** (orderless): short independent Q/A pairs → should **pass** CWA easily.
- **Narrative Chain**: long documents with causal order (chapters) → **fail** chunk panel unless chunking aligned.
- **Mixed Domain**: support KB with both FAQs and tutorials.

Scenarios

1. **Chunk Sweep**: sizes {256, 512, 1,024}, strides {1/4, 1/8}; measure pass-rate and accuracy.
2. **Panel Budget**: P/F/C in {(64,32,16), (128,64,32)}; observe CI and latency.
3. **Fallback Mix**: attention vs mean; record frontier.

Outputs

- CSV/Parquet with: accuracy, p95 latency, pass-rate, PRI, fallback rate, agreement.
- Plots: accuracy–latency Pareto; pass-rate by chunk policy; PRI histogram.

Harness CLI

```
observerops-bench rag \
  --dataset narrative-chain \
  --chunkers sent-256@64 sent-512@128 \
  --panels 64,32,16 128,64,32 \
  --theta_pass 0.82 --theta_warn 0.75 \
  --pri_max 0.50 --trials 3 \
  --out results/chain_w39.parquet
```

15.9 Worked Examples

15.9.1 FAQ Corpus (Bag-like → Fast Path)

- N=12 chunks per query, `sent-256@64`.
- Panels: P=64,F=32,C=16 → `score=0.93`, `PRI=0.08`.
- **Mean** pooling used; p95 latency 420 ms; nDCG@10 = 0.72; pass-rate 92%.

15.9.2 Tutorial Chapters (Coherent → Fallback)

- N=18 chunks, `sent-512@128`.
- Panels: P=128,F=64,C=32 → `score=0.73`, `PRI=0.41`, `chunk.median_delta=0.27`.
- **Attention** fallback; p95 latency 720 ms; nDCG@10 = 0.75 (better accuracy despite higher cost); pass-rate 48%.

15.10 Artifacts

- **Config templates** (§15.7).

- **Benchmark harness** (CLI & scenarios; §15.8).
- **Pseudocode** (§15.6).
- **KPIs** and targets (§15.5).

Next: **Chapter 16 — RL/Robotics** (sensors as instruments; compatibility; fleet sync; belt-level objectives).

Chapter 16 — RL/Robotics (Sensors → Schedules → Synchronized Fleets)

Goal. Make RL/robotics stacks **observer-safe**: treat **sensors as instruments** (channels Π), encode **action compatibility** (commute/conflict), drive control on **ticks τ**, and coordinate fleets by **sync ρ** with closure at the belt layer (Gap/Flux/Twist/Residual). Use **CWA** to certify when sensor features can be additively fused; otherwise fall back to order-aware filters.

16.1 Pattern (single robot → fleet)

- **Channels Π (sensors & tools):** `lidar.scan`, `cam.rgb`, `imu`, `encoders`, `gripper.open/close`, `base.move`, `arm.movej`, `ee.force`, …
- **Ô (scheduler):** chooses next *measure or act* given state **S** and trace **T**, obeying the **commute matrix C** (sensor read–read often commutes; act–act rarely).
- **τ (ticks):** fixed-rate control commits (e.g., 20 ms/50 Hz); **TraceWrite(τ_k, π, y)** latches outcomes; actions are latched intents with ack.
- **ρ (fleet sync):** keep robot phases aligned for team behaviors; bound **Δτ** across the swarm.
- **CWA on fusion:** after **projecting** sensor data to features, use certificate panels (perm/flip/chunk) to gate additive fusion; otherwise apply EKF/particle/attention fusion.

16.2 Robot Observer Tuple

$O = (S, T, \hat{O}, \tau, \Pi, C)$ with:

- **S**: estimator (pose, map, task), controller states, RL policy state.
- **T**: append-only traces of `(τ, channel, outcome)` plus action acks.
- **Ô**: policy mixing (state estimator needs vs. policy needs vs. safety checks).
- **Π**: sensors/actuators as channels.
- **C**: compatibility graph; edges when simultaneous use is safe/commuting.

Tick budget example (mobile manipulator): 50 Hz control (20 ms), 10 Hz mapping (100 ms), 2 Hz high-level planning (500 ms). Lower-rate loops schedule inside higher-rate ticks via sub-plans.

16.3 Action & Sensor Compatibility (C)

Typical conflicts (non-commuting):

- `arm.movej` ↔ `arm.teach` (servo vs impedance mode).
- `base.move` ↔ `arm.movej` at high speed (coupled dynamics; restrict envelope).

- `gripper.close` ↔ `ee.force.calibrate` (block until calibration done).
- `write.map` ↔ `read.map` (same object at same τ) → push write to τ_{k+1}.
- High-power sensor bursts (structured-light) ↔ `cam.rgb` (glare) → sequence.

Preflight: for proposed schedule $S = [\pi_1, \pi_2, \ldots]$, reject any pair with `C[π_i,π_j]==false` in current context; re-order to commuting sequence or shift to τ_{k+1}.

16.4 Multi-Robot Sync via τ and ρ

- **Phase model:** each robot j has phase $\theta_j \in [0, 2\pi)$ for its control tick.
- **Order parameter:** $\rho = \left| \frac{1}{N} \sum_{j=1}^{N} e^{i\theta_j} \right|$ (1=perfect sync).
- **Desynchrony:** $\Delta\tau = \max_j \tau_j - \min_j \tau_j$.
- **Resync:** broadcast anchors each 0.5–1 s; slew clocks (no jumps); when $\rho < \rho_{min}$ or $\Delta\tau > \Delta\tau_{max}$, restrict to commuting-safe actions and widen cadence until recovered (per Ch.12).

16.5 Recipes

R1. Conflict-Aware Schedules

- Encode **actuation envelopes** (max combined speed/torque) as predicates in **C**.
- Before issuing an action, compute **compatibility margin**; if negative, re-order or split across ticks.
- For mixed **base+arm** motion, treat planner outputs as a **single composite channel** to avoid hidden conflicts.

R2. Certified Sensor Fusion

- Project raw streams x_i to features $v_i = P(x_i)$ (e.g., BEV features, learned embeddings).
- Run **CWA panels**:
  - **perm** over packet arrival order,
  - **flip** sign/orientation jitter (e.g., minor frame inversions),
  - **chunk** sub-scan re-binning.
- If `score ≥ θ` & `PRI ≤ PRI_max` → fuse by **mean/sum** (grid/logit add).
- Else fall back to **EKF/UKF/particle** or **attention** fusion.

R3. Fleet Belts Tied to Task Gap

- Define belt worldsheet per mission: **Gap** (task error), **Flux** (work rate), **Twist** (reconfigs).
- Use **Residual = |Gap − (Flux + α·Twist)|** to judge plan–do closure.
- Gates: throttle when Residual Amber; block tactical pushes when Red.

R4. Safe Exploration & Rollback (RL)

- Exploration actions run under **Ô-sandbox** with reduced cadence and hard **C** constraints.
- Latch **roll-back poses**; if safety near-miss triggers, halt at τ_{k+1} and return.

R5. Hot-Swap Sensors

- If a sensor drops, keep fusion green via redundancy: LiDAR + depth + stereo pointing at same pointer (e.g., occupancy).
- CWA pass with lower panel counts permits **additive keep-alive** until repair.

16.6 KPIs (targets depend on platform domain)

KPI	Definition	Green	Amber	Red
Task Success	episode success rate	≥ 0.9	0.8–0.9	< 0.8
Safety Incidents	stops/near-misses per hour	≤ 0.2	0.2–1.0	> 1.0
PBHL Residual		≤ 0.08	0.08–0.15	> 0.15
ρ	sync order parameter	≥ 0.9	0.8–0.9	< 0.8
Δτ	fleet tick spread (ticks)	≤ 2	3–5	> 5
Cert Pass-Rate	CWA pass on fusion calls	≥ 0.8	0.6–0.8	< 0.6
Loop Latency	control loop p95	≤ 20 ms	20–35 ms	> 35 ms

16.7 Control Loop (pseudocode)

```
def control_tick(robot):
    tau = tick.current()
    # 1) Sense (commuting reads first)
    scans = []
    for pi in ["lidar.scan","cam.rgb","imu","encoders"]:
        if commute_ok(pi, T): scans.append(measure(pi))
    trace.write(tau, "sense", ref=scans)  # latch

    # 2) Project → Certificate → Fuse
    V = [project(s) for s in scans]
    cert = cwa.certificate(V, panels=choose_panels(len(V)))
    fused = (np.mean(V, axis=0) if is_pass(cert) else ekf_fuse(V))
    trace.write(tau, "fused", ref=fused)  # latch

    # 3) Plan/Act with compatibility preflight
    candidate_actions = planner(fused, goal)
    safe_actions = preflight(candidate_actions, C)
    for a in safe_actions:
        act(a); trace.write(tau, a.kind, y="ack")

    # 4) Belt update (windowed)
    belt.update(metrics_from_tick())
    tick.next()
```

16.8 Simulation Checklist (pre-deployment)

Physics & Timing

- Controller rate & jitter (20–50 Hz), sensor latencies, async delivery.
- Contact models, friction cones, actuator limits & saturation.

Scenarios

- Nominal, corner cases, adversarial clutter.

- Sensor dropouts, glare, motion blur, LiDAR rain/fog models.
- Domain randomization (textures, lighting, mass, delay).

ObserverOps Hooks

- Trace latching & hash chain; per-tick single-write asserts.
- Commute matrix validation (block non-commuting pairs).
- CWA harness on fusion sets; fallback path exercised.
- Tick sync under skew; ρ/Δτ alert ladder.
- Belt KPIs and Residual response (D1/D2/D3 modes).

Safety

- Soft/hard E-stops; geofences; speed caps under low ρ.
- Near-miss detectors & log enrichment.

16.9 Log Schema (robotics JSONL)

One line per event.

```
{
  "ts": "2025-09-22T11:28:03.142Z",
  "robot_id": "bot-07",
  "tau": 431025,
  "event": "TraceWrite",
  "channel": "lidar.scan",
  "outcome_ref": "blob:sha256:...",
  "hash": "sha256:...",
  "prev": "sha256:...",
  "meta": {"duration_ms":12}
}

{
  "ts":"2025-09-22T11:28:03.160Z",
  "robot_id":"bot-07",
  "tau":431025,
  "event":"CWA.Pass",
  "score":0.88,
  "pri":0.12,
  "panels":{"perm":{"n":64,"median_delta":0.05},"flip":{"n":32,"median_delta":0.04},
      "chunk":{"n":16,"median_delta":0.06}},
  "pool_id":"pool-9aa1"
}

{
  "ts":"2025-09-22T11:28:03.180Z",
  "robot_id":"bot-07",
  "tau":431025,
  "event":"Act.Ack",
  "action":"base.move",
  "args":{"vx":0.3,"wz":0.1},
  "status":"ok"
}

{
  "ts":"2025-09-22T11:28:03.200Z",
  "robot_id":"bot-fleet",
  "event":"Sync.Status",
  "rho":0.91,
  "delta_tau":2
}

{
```

```
  "ts":"2025-09-22T11:29:00.000Z",
  "event":"PBHL.Update",
  "belt_id":"pickpack-W39",
  "gap":0.32,"flux":0.26,"twist":0.05,"alpha":1.1,
  "residual":0.021
}
```

16.10 Example Config (YAML)

```
robot:
  ticks:
    control_hz: 50         # 20 ms
    mapping_hz: 10
    plan_hz: 2
  channels:
    - lidar.scan
    - cam.rgb
    - imu
    - encoders
    - base.move
    - arm.movej
    - gripper.close
  commute_matrix: cm:mobile-manip:v3
  fusion:
    projector: "bev-resnet18"
    cwa:
      thresholds: {pass: 0.82, warn: 0.75, pri_max: 0.50}
      panels: {perm: 64, flip: 32, chunk: 16}
      fallback: "ekf"
  safety:
    e_stop_topic: "/estop"
    speed_caps: {normal: 1.0, desync: 0.4}
    near_miss_thresh: 0.2

fleet:
  sync:
    cadence_ms: 20
    rho_min: 0.90
    delta_tau_max: 2
    anchors_ms: 500
  belts:
    id: "pickpack-W39"
    residual_bands: {green:[0,0.08], amber:[0.08,0.15], red:[0.15,1.0]}
  gates:
    on_residual_red: {restrict: "commuting_only", block_non_p0: true}
```

16.11 SLOs & Alerts

SLOs

- Control loop p95 ≤ 20 ms; mapping p95 ≤ 100 ms.
- Fusion pass path ≤ 5 ms; fallback filter ≤ 20 ms.
- Sync status broadcast ≤ 10 ms; gate decision ≤ 10 ms.

Alerts

- `LatchingViolation` (duplicate (τ,π) write).
- `CommuteConflict` rate > X/min.
- `CWA.Red` or `Drift` sustained ≥ N windows.
- `Sync.Red` ($\rho < 0.8$ or $\Delta\tau > 5$).
- `Safety.NearMiss` > threshold.

16.12 Artifacts

- **Sim checklist** (§16.8).
- **Log schema** (§16.9).
- **Config templates** (§16.10).
- **Pseudocode** control loop (§16.7).
- **KPIs** with bands (§16.6).

Next: **Chapter 17 — Governance & Ops (BeltOps)** — program belts, gates, and board-ready rollups for robotics deployments.

Chapter 17 — Governance & Ops (BeltOps)

Goal. Run initiatives as **program belts** with measurable closure—keep **PBHL Residual** in band, raise **EEI/SI** (effectiveness & sustainability), and use **Policy Gates** to throttle or block risky runs. Deliver repeatable SOPs, an incident playbook for Residual excursions, and a board-ready one-pager.

17.1 Pattern (how BeltOps governs)

- **Wrap an initiative as a Belt** with a worldsheet: **Gap, Flux, Twist, α**, and **Residual = |Gap − (Flux + α·Twist)|**.
- **Instrument the pipeline** so pool results, agreement reports, and certificate logs roll up into the belt KPIs.
- **Drive by gates**: deterministic rules on **CWA score/PRI**, **Residual**, and **sync/capacity** metrics (ρ, Δτ, occupancy) produce **allow | throttle | block** actions (Ch.13).
- **Operate on cadences**: daily belt standups, weekly checkpoint, monthly residual review, **quarterly PBHL** review.

17.2 Roles & RACI

Role	Responsibilities	R	A	C	I
Belt Owner (BO)	Objectives, α tuning, OKRs → KPIs	✓	✓		
Gatekeeper (GK)	Gate config/versioning, overrides	✓	✓		
ObserverOps SRE (OSRE)	Runtime reliability, slots/ticks	✓		✓	
Data/Model Lead (DML)	Projectors, chunkers, drift	✓		✓	
Security/GRC	Exports, evidence, audits			✓	✓
Product/Stakeholders	Requirements, impact				✓

17.3 KPIs & Thresholds (governance view)

- **Five-Line KPI**: Gap, Flux, Twist, **Coherence** (agreement proxy), **Residual**.
- **EEI (Effectiveness/Execution Index)** — weighted composite:

$$\mathrm{EEI} = 0.5 \cdot Q + 0.3 \cdot \mathrm{ThroughputNorm} + 0.2 \cdot \mathrm{Agreement}$$

- **SI (Sustainability Index)** — cost & stability composite:

$$\mathrm{SI} = 0.5 \cdot \mathrm{CostNorm}^{-1} + 0.3 \cdot \mathrm{Variance}^{-1} + 0.2 \cdot (1 - \mathrm{SlotPressure})$$

- **Targets (typical):**
  - **Residual**: Green ≤ 0.08, Amber (0.08–0.15], Red > 0.15
  - **EEI/SI uplift**: ≥ +10% QoQ
  - **Audit pass-rate**: ≥ 98% (evidence completeness & signature checks)

17.4 Operating Cadences

- **Daily (15 min)**: Belt standup — review Residual band, cert pass-rate, any gate actions; approve α micro-tune if needed.
- **Weekly**: KPI checkpoint — compare against OKRs; freeze gate thresholds unless incident.
- **Monthly**: Residual review — look for step changes; align Twist annotations (org changes, releases).
- **Quarterly** (**PBHL Review**) — formal worldsheet analysis, EEI/SI uplift, incidents & actions, α retune with rationale.

17.5 Residual Incident Playbook (SOP)

Trigger. Any of:

- Residual **Red** for ≥ 2 consecutive windows
- Residual **Amber** for ≥ 4 windows with negative trend
- Coherence drop > 0.1 while Flux ramps

Runbook.

1. **Triage (T+0–5 min)**
   - Auto-throttle via gates (cadence ↑, panel_scale ↓); restrict to commuting-safe schedules.
   - Capture **snapshot bundle**: recent pool_ids, cert logs, gate decisions.
2. **Contain (T+5–30 min)**
   - Roll back last Twist if recent (feature flag/rollout).
   - Force fallback pooling in high-risk domains.
3. **Diagnose (T+30–120 min)**
   - Compare **Gap vs Flux** deltas; inspect α drift.
   - Check **cert drift** p-values; examine chunk panel deltas.
4. **Correct (T+2–24 h)**
   - Fix projector/chunker; re-tune α; adjust gate bands.
   - Backfill data & re-run KPIs if necessary.
5. **Verify & Close**
   - Residual returns to Green for ≥ 3 windows.

- File **post-incident** with evidence ids and signed export.

Exit criteria. Residual ≤ 0.08 (3 windows) **and** EEI/SI not degraded > 5%.

17.6 Policy Gates (governance presets)

Bands & actions (summary)

- **CWA**: score<0.75 → **block additive**; 0.75–0.82 + PRI≤0.5 → **throttle**; ≥0.82 & PRI≤0.2 → **allow**.
- **PBHL**: Residual Amber → **throttle**; Red → **block risky** (non-P0).
- **Sync/Capacity**: ρ<0.9 or Δτ>2 or occupancy>0.9 → **throttle** (narrow channels, widen cadence).

Override/Waiver process

- Gatekeeper raises **temporary waiver** (≤ 24 h) with reason & risk sign-off from BO + Security.
- All overrides are signed, versioned, and exported.

17.7 Audit & Compliance

- **Evidence**: Trace ids, Cert Logs, Gate Decisions, Belt updates; all signed (HMAC or ed25519) with hash-addressed blobs.
- **Exports**: rolling hourly & on-demand bundles (see Ch.13 §13.8).
- **Audit pass-rate** = verified artifacts / expected artifacts for the audit scope.
- **Retention**: hot 90–365 days; cold 7 years; PII redaction maps included.

17.8 Dashboards & Board Package

Ops dashboard

- Five-Line KPI with thresholds; cert pass-rate; PRI histogram; gate states timeline; α changes log.

Board-ready one-pager (template)

```
ObserverOps Belt — Q# Executive Summary  (Program: <name>)

1) Headline
   - EEI: <current>  (QoQ: +<%>)
   - SI : <current>  (QoQ: +<%>)
   - Residual: <value>  [Band: Green/Amber/Red]  α=<value>
   - Audit pass-rate: <value>%  (evidence bundles: <n>)

2) Outcomes & Throughput
   - Quality (task metric): <value>  | Throughput: <value>/day
   - Coherence (agreement): <value>

3) Risks & Controls
   - CWA: pass-rate <value>%  | PRI p95 <value>
   - Sync/Capacity: ρ=<value>, Δτ=<value>, occupancy p95=<value>

4) Incidents & Actions
   - Residual incidents: <count>  | Mean time to green: <h>
   - Actions taken: <bullets> (rollbacks, α-tunes, gate changes)

5) Next Quarter
   - Objectives (Gap↓, Flux↑, Twist budget)
```

```
- Gating plan (bands & thresholds)
- Investments (indexing, redundancy, simulation)
```

17.9 SOPs (ready to adopt)

SOP-A: Quarterly PBHL Review

- **Inputs**: last-quarter belt exports; α change log; incident reviews.
- **Agenda (60–90 min)**
  1. Worldsheet walk-through (Gap, Flux, Twist, Residual)
  2. EEI/SI uplift; cost & variance trends
  3. Certificate & drift summary; pass-rate, PRI tails
  4. α tuning proposal → decision & commit
  5. Policy gate bands for next quarter
  6. Risks & mitigations; action register
- **Outputs**: signed minutes; updated α; gate config version bump.

SOP-B: Gate Change Control

- **Change ticket** with: rationale, before/after bands, expected effect, rollback.
- **Shadow mode** 24–72 h (evaluate decisions without enforcing).
- **Promote** if false-positive/negative rates within target; else revert.

SOP-C: Evidence Export

- **Schedule**: hourly rolling + on-request.
- **Validate** signatures; manifest completeness; cross-check counts vs telemetry.
- **Distribute** to GRC vault; alert on lag > 2 min.

17.10 Governance KPIs & Targets

KPI	Target	Notes
EEI uplift (QoQ)	≥ +10%	mix-adjusted
SI uplift (QoQ)	≥ +10%	capacity normalized
Residual time-in-band (Green)	≥ 85%	per quarter
Audit pass-rate	≥ 98%	evidence completeness
Gate accuracy (decisions vs post-hoc labels)	≥ 95%	shadow-labeling
Override volume	≤ 2 / quarter	indicates clear policy

17.11 Config Snippets

Belt config (YAML)

```yaml
belt:
  id: "support-v2025Q4"
  residual_bands: {green: [0,0.08], amber: [0.08,0.15], red: [0.15,1.0]}
```

```
  alpha: 1.1
  kpi_window_s: 60
  indices:
    eei_weights: {quality: 0.5, throughput: 0.3, agreement: 0.2}
    si_weights: {cost: 0.5, variance: 0.3, slots: 0.2}
```

Gate policy (YAML)

```
gates:
  thresholds:
    cwa_score: {pass: 0.82, warn: 0.75}
    pri_max: 0.50
    rho_min: 0.90
    delta_tau_max: 2
  actions:
    amber: {cadence_factor: 1.1, panel_scale: 0.7}
    red: {cadence_factor: 1.25, commuting_only: true, block_if_score_lt: 0.70}
  override:
    waiver_ttl_h: 24
    approvers: ["belt_owner","security"]
  audit:
    export_cron: "*/15 * * * *"
    sign: "ed25519"
```

17.12 Benchmarking & Acceptance

- **Acceptance gates for go-live**
  - Residual Green ≥ 90% over a 2-week pilot
  - EEI/SI uplift ≥ +8% vs baseline
  - Audit dry-run pass-rate ≥ 99%
  - Gate shadow accuracy ≥ 95%, flap rate < 2%/day
- **What to do if you miss**
  - Raise redundancy (pointer channels), reduce chunk variability, retune α, tighten gate hysteresis.

17.13 Artifacts

- **SOPs**: Residual Incident (17.5), PBHL Review (17.9-A), Gate Change (17.9-B), Evidence Export (17.9-C).
- **Board template**: one-pager (17.8).
- **Configs**: belt & gate YAML (17.11).
- **KPIs**: governance targets & acceptance (17.10 & 17.12).

Next: **Part IV — Metrics & Telemetry** (definitions → estimators → thresholds).

Chapter 18 — Education & Labs (Hands-On ObserverOps)

Goal. Give students and teams a classroom-ready path to *build observers*: practice **internal collapse** (latching), **agreement** under commuting effects, **Ô/τ** scheduling, **CWA certificates**, and **PBHL belts**. Each lab ships with: a notebook spec, tiny datasets, instructor notes, and an auto-grader outline.

18.0 Lab Logistics (common to all)

- **Stack:** Python 3.10+, NumPy, JAX *or* PyTorch, Matplotlib/Plotly.
  Extras: `qutip` (Lab 1 alt), `networkx`, `pandas`.
- **Repro:** `SEED=4271` (fix RNG), float32 unless noted, record `cert_seed` for CWA panels.
- **Trace format (all labs):**

```
{ "tick": τ, "channel": "…", "outcome_ref": "blob:sha256:…",
  "write": {"hash":"…","prev":"…","ts":"…"}, "flags":{"conflict":false}}
```

- **Grading I/O:** Auto-grader reads a JSONL `events` stream and a `metrics.json` produced by each notebook.

18.1 Lab A — Qubit Toy (Commuting vs Non-Commuting)

Learning objectives

1. Implement **latching**: no retro-edits within a tick.
2. Observe **order effects** with non-commuting instruments (X, Z).
3. Demonstrate **agreement** when effects commute and records are shared (SBS-style).

Background (minimal)

- Pauli projective measurements on a single qubit; Born rule.
- Commutativity: [Z,Z] commute; [X,Z] do not.

Dataset

- Synthetic: initial states $|0\rangle, |1\rangle, |+\rangle, |-\rangle$ sampled 1k times.

Tasks

1. Implement `measure(ρ, op)` returning outcome $y \in -1, +1$ and post-measurement state (collapse).
2. Build a tiny **Observer Runtime** with `/measure`, `/trace/:id`, tick **τ**, and latching.
3. Run two sequences on $|+\rangle$:
   - S1: **Z→X** at the same object.
   - S2: **X→Z** at the same object.
     Compare distributions and **agreement** across replicas that share traces.
4. Repeat with commuting pair: **Z on Q_1** then **Z on Q_2** (different objects) or **Z→Z**.

Reference snippets

```
def proj(op):  # Pauli 'X' or 'Z'
    return (np.eye(2)+pauli[op])/2, (np.eye(2)-pauli[op])/2

def measure(rho, op, rng):
    Pp, Pm = proj(op)
    p = np.real(np.trace(Pp @ rho))
    y = +1 if rng.random() < p else -1
    P = Pp if y==+1 else Pm
    rho_post = P @ rho @ P / max(1e-9, np.trace(P @ rho))
    return y, rho_post
```

Expected results

- $|+\rangle$: Z→X vs X→Z produce different joint histograms (order sensitivity).
- Z→Z (same object, same basis): second outcome repeats first with prob. ~1.0 (up to numerical noise).
- **Agreement score** across observers rises toward 1.0 only on commuting setups with shared records.

Auto-grader checks (10 pts)

- (3) **Latching**: no duplicate (τ,π) writes; hash chain valid.
- (3) **Order effect**: KL divergence between Z→X and X→Z joint ≥ 0.3.
- (2) **Agreement(commuting)** ≥ 0.95.
- (2) **Non-commuting counterexample**: agreement ≤ 0.7.

Instructor notes

- Time: 60–90 min.
- Common pitfall: "measuring without updating state." Emphasize **internal collapse**.

18.2 Lab B — Gridworld SMFT Agent (Ô as Scheduler, τ as Commit Rhythm)

Learning objectives

1. Implement **Ô** to choose orientation/channel by field score.
2. Advance on discrete **ticks τ**, log latching writes.
3. Track **Collapse Entropy** S_c and **Attractor Load** (AL), and observe **Δτ** effects.

Environment

- 10×10 grid; agent must locate a goal emitting a scalar field with noise.
- Channels Π = {`lookN`, `lookS`, `lookE`, `lookW`} returning noisy gradients.

Tasks

1. Define SMFT field $\Psi_m(x,\theta,\tau)$ as a score map; Ô picks next `look*`.
2. Implement cadence manager: base cadence 100 ms; allow injected jitter to study **Δτ**.
3. Metrics per window: S_c (entropy of chosen channels), AL (peak/mean of Ψ_m), success steps.

Policy (example)

```
score(pi) = w1*expected_gain(pi) - w2*latency(pi) - w3*conflict(pi)
pi* = argmax score
```

Expected results

- S_c decreases as agent homes in; AL increases; success in < 40 steps on average.
- Injected desync (Δτ≥3) increases steps to goal and mis-exec rate.

Auto-grader (10 pts)

- (3) **Ô selection correctness**: greedy improvement in AL per 5 ticks.
- (3) **Latching & traces**: zero retro-edits; per-tick single-write.

- (2) **Cadence**: jitter within configured bounds; Δτ alarm triggers when forced.
- (2) **Success**: mean steps ≤ threshold (e.g., 45).

Instructor notes

- Time: 90 min.
- Extension: add a non-commuting "disturb" channel that corrupts local field → show schedule reordering.

18.3 Lab C — RAG Pooling Battery (CWA)

Learning objectives

1. Treat **chunking as instrument design** and measure its effect on pooling safety.
2. Use **CWA panels** (perm/flip/chunk) to gate additive mean vs attention fallback.
3. Draw **accuracy↔latency** frontiers; track **Phase-Risk Index** & pass-rate.

Datasets

- **FAQ-Bag** (orderless; 2k Q–A snippets).
- **Narrative-Chain** (10 long tutorials with chapter order).

Tasks

1. Retrieve K passages via vector + keyword (commuting).
2. Project with "e5-large"; run panels: P=128,F=64,C=32 (strict) and P=64,F=32,C=16 (fast).
3. Pool: if `score≥0.82 & PRI≤0.20` use **mean**; else **attention** fallback.
4. Evaluate nDCG@10 and p95 latency; compute pass-rate, PRI distribution.

CLI (reference)

```
observerops-bench rag --dataset FAQ-Bag Narrative-Chain \
  --chunkers sent-256@64 sent-512@128 \
  --panels 64,32,16 128,64,32 --theta_pass 0.82 --pri_max 0.50
```

Expected results

- FAQ-Bag: pass-rate ≥ 85%, PRI ≈ 0.1; mean pooling dominates (fast).
- Narrative-Chain: pass-rate ≤ 55%, PRI ≈ 0.35–0.5; attention yields higher accuracy with more latency.

Auto-grader (10 pts)

- (3) **Certificate correctness**: panel deltas decrease with bag-like data.
- (3) **Routing**: fallback triggered on Narrative-Chain ≥ 35% of queries.
- (2) **Accuracy**: attention ≥ mean on Narrative-Chain by ≥ +2 nDCG points.
- (2) **Telemetry**: emit `CWA.Pass/Fail` and record seeds.

Instructor notes

- Time: 90–120 min incl. plots.

- Tip: have students vary chunk overlap; watch **chunk panel** sensitivity.

18.4 Lab D — Belt Simulator (PBHL Macro Closure)

Learning objectives

1. Simulate a **program belt** with Gap/Flux/Twist and **Residual** control.
2. Use **Policy Gates** to throttle when Residual leaves band.
3. Run a **PBHL review** and justify α tuning.

Simulator

- Discrete time; Gap G_t decays with Flux F_t and reacts to Twist T_t:
  $G_{t+1} = G_t - \beta F_t + \eta_t + \xi T_t$.
- Belt closure target: $G_t \approx F_t + \alpha T_t$ (Residual small).

Tasks

1. Implement controllers: **Flux-gate** (fast) and **Twist-step** (slow).
2. Inject a Twist spike (reorg) at t=200; observe Residual excursion.
3. Configure gates: Residual Amber → throttle; Red → block; measure time to green.
4. Produce a board-ready one-pager (auto-filled).

Expected results

- With gates on, Residual returns to Green within N windows; without gates, it lingers (counterfactual).

Auto-grader (10 pts)

- (3) **Residual control**: time-to-green ≤ threshold (e.g., 12 windows).
- (3) **Gate determinism**: identical inputs → identical decisions (hash match).
- (2) **Export**: signed bundle with KPIs & decisions.
- (2) **PBHL review**: α proposal consistent with observed drift (simple rule check).

Instructor notes

- Time: 60–90 min.
- Pitfall: over-aggressive α changes create oscillations; discuss hysteresis.

18.5 Deliverables (what you ship)

Notebooks

- `LabA_QubitToy.ipynb` — latching + agreement; commuting vs non-commuting.
- `LabB_Gridworld_SMFT.ipynb` — Ô/τ loop, AL & S_c, Δτ stress.
- `LabC_RAG_CWA.ipynb` — certificates, pooling, accuracy/latency plots.
- `LabD_BeltSimulator.ipynb` — PBHL + gates + incident drill.

Datasets

- `/data/qubit_states.npz` — vectors for $|0\rangle, |1\rangle, |+\rangle, |-\rangle$.
- `/data/gridworld/*.npz` — maps, noise profiles.
- `/data/faq_bag.jsonl`, `/data/narrative_chain.jsonl` — small corpora (2–20MB).
- `/data/belt_sims/*.json` — seed configs for Gap/Flux/Twist.

Instructor Notes (PDF/MD)

- Timing, pitfall list, variants, and grading rubrics; answer-key plots.

Auto-Grader

- `grader.py` with:
  - **Parse:** `events.jsonl`, `metrics.json`.
  - **Checks:** latching, agreement, certificate routing, Residual control.
  - **Report:** `grade.json` (per-criterion scores) + concise feedback.

`grade.json` schema

```
{
  "student_id": "…",
  "lab": "LabC_RAG_CWA",
  "score": 9.0,
  "breakdown": {
    "certificate": 3,
    "routing": 3,
    "accuracy": 2,
    "telemetry": 1
  },
  "notes": "Chunk panel tuned well; attention fallback used appropriately."
}
```

18.6 Safety & Fairness Notes

- **No PII**: corpora are synthetic; verify redaction and lineage tags.
- **Determinism**: seed all RNG; store `cert_seed` and config versions in logs.
- **Compute fairness**: cap tokens/steps/slots across students.

18.7 Extension Paths

- Lab A: add depolarizing noise and demonstrate redundancy (SBS) improving agreement.
- Lab B: multi-agent gridworld; measure **ρ** under shared anchors.
- Lab C: add multilingual projector and compare pass-rates across languages.
- Lab D: couple two belts; show cross-belt Residual dynamics.

Artifacts delivered: notebooks, tiny datasets, instructor notes, and an auto-grader schema, all aligned to Chapters 2–7 (invariants), 10–13 (APIs & gates).

© 2025 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.

ObserverOps Technical Blueprint - IV-VI

Part IV — Metrics & Telemetry

Chapter 19 — Micro Metrics

Goal

Make the *inside of an observer* measurable. Define, estimate, and gate **AL**, **S_c**, **Agreement Rate**, **Mis-exec Rate**, and **Slot Occupancy/Collisions** with rolling estimators, confidence bounds, drift tests, and production threshold bands.

What you'll implement (checklist)

- Emit per-tick events and counters for the five micro metrics.
- Rolling window estimators (time or tick windows), plus Wilson CIs for proportions.
- Drift detectors (EWMA/CUSUM/Page–Hinkley) for **AL**/**S_c** trends.
- Green/Amber/Red bands with escalation hooks (alerts + Ô policy nudges).
- A minimal `/metrics/micro` readout and weekly KPI snapshot.

19.1 Definitions (operational)

Attractor Load (AL)

Concentration of the semantic field over orientations/channels at context x, tick τ:

$$\mathrm{AL}(x,\tau) \;=\; \frac{\max_\theta w_\theta \;\|\; P\,\Psi_m(x,\theta,\tau)\|}{\sum_\theta w_\theta \;\|\; P\,\Psi_m(x,\theta,\tau)\;\| \;+\varepsilon}$$

High AL ⇒ strong attractor (exploit); low AL ⇒ diffuse (explore).

Collapse Entropy S_c

Diversity of *recent* channel selections inside a rolling window W:

$$p_\theta(\tau) = \frac{\#\{\text{uses of } \theta \in [\tau - W, \tau]\}}{\sum_{\theta'} \#\{\text{uses of } \theta'\}}\,, \quad S_c(\tau) = \; - \sum_\theta p_\theta(\tau) \log p_\theta(\tau)$$

Falling S_c = latching to a subset; persistently high S_c = diffusion.

> Threshold heuristic seen in field ops: if entropy drops >20% for two ticks **while saturation rises**, rotate channels/soften curvature (black-hole approach).

Agreement Rate (over commuting overlaps)

For two observers A, B, after frame mapping and lag tolerance, on the set O of *commuting* overlaps:

$$\text{Agree} = \frac{\sum_{(k,\tau)\in O} \mathbf{1}[y^A_{k,\tau} = y^B_{k,\tau}]}{|O|}$$

Use only keys that commute and share redundant/ledgered records (SBS). Track NCE (non-commuting exposure) separately.

Mis-exec Rate (two facets)

- **Policy-Violation Rate (PVR):** fraction of executed channels that violated compatibility/ policy/preconditions at that tick.
  PVR = #violations/#ticks.
- **Tool-Error Rate (TER):** tool errors or timeouts per external tool invocation. Target ≤ 1.0%.

(Track both; alert if either exceeds bands.)

Slot Occupancy & Collisions

- **Occupancy** per pool q: $\text{occ}_q(\tau) = \text{used}_q(\tau)/N_q$.
- **Collision** occurs when an allocation request *cannot* be satisfied and the policy would need to evict an *active/unguarded* item; log `Slots.Collision` with provenance.
- **Collision rate:** CollRate = #{Slots.Collision}/#{allocate}.
  Green bands example: memory < 0.5%, tools < 1%, attention ~ 0%.

19.2 Estimators (rolling windows, CIs, drift)

Windows

- Use *tick-based* windows for tight control loops (e.g., last $W = 128$ ticks), or *time-based* (e.g., last 5 min) for fleets. Emit both short (reactive) and long (stable) windows.

Proportions: Wilson CI (recommended)

For Agreement, PVR, TER, CollisionRate, compute Wilson bounds per window:

```
def wilson_ci(k, n, z=1.96):
    if n == 0: return (0.0, 1.0)
    p = k/n
    denom = 1 + z*z/n
    center = (p + z*z/(2*n))/denom
    half = z * ((p*(1-p)/n + z*z/(4*n*n))**0.5) / denom
    return (max(0.0, center - half), min(1.0, center + half))
```

(Attach the CI to every alert to avoid chasing noise.)

Rolling AL/S_c

- Maintain streaming sums/histograms per θ to update S_c in $O(1)$.
- For AL, cache $\|P\,\Psi_m\|$ per θ; update the ratio each tick.

Drift detectors

- **EWMA/CUSUM** on S_c and AL to catch slow drift; **Page–Hinkley** for sudden regime shifts. Trigger "Amber:Drift" if statistic exceeds tuned band for L ticks.
- Couple S_c drift with **saturation** signals: only escalate entropy drops that co-occur with rising saturation (black-hole proxy).

19.3 Thresholds & escalation (defaults)

Metric	Green	Amber	Red	Notes / Actions
Agreement (per-overlap, commuting)	≥ 0.98	0.95–0.98	< 0.95	On Amber: enforce commuting-only schedule; check ledger/SBS; on Red: serialize & open incident.
Fleet Disagreement KPI (1–mean agreement)	≤ 0.08	0.08–0.15	> 0.15	Fleet-level KPI used in Ch.14; mirrors real traffic.
PVR	< 0.5%	0.5–1.5%	> 1.5%	On Amber: tighten Ô preflight; on Red: freeze non-commuting tools.
TER	≤ 1.0%	1–2%	> 2%	On Red: mis-exec playbook (increase timeouts, widen τ cadence).
Collision Rate (memory/tools/attention)	< 0.5% / < 1% / ~ 0%	×2 of green	> 2% / > 4% / > 0%	Use queueing over eviction for attention; prefer PRI for tools.
Occupancy	40–85%	85–95%	> 95% (≥ 30s)	On Red: back-pressure Ô (raise Δτ), increase pool N, or switch policy LRU→PRI.
S_c **(normalized)**	0.45–0.85	< 0.45 (2 ticks)	< 0.35 w/ sat↑	Black-hole guard: "Entropy floor" rule.
AL (relative)	≤ 90th pct (persist < W)	> 90th pct (≥ W)	> 95th pct (≥ 2W)	Use quantile bands per domain; high AL without reward lift ⇒ explore.

19.4 Emitters & events (schema)

Per tick (subset):

```
{
  "tau": 1042,
  "metrics": {
    "AL": 0.72,
    "Sc": 1.38,
    "agreement": {"k": 714, "n": 720, "wilson": [0.98, 0.99]},
    "pvr": {"k": 1, "n": 200, "wilson": [0.00, 0.02]},
    "ter": {"k": 3, "n": 420, "wilson": [0.00, 0.02]},
    "slots": {
      "memory": {"occ": 0.78, "collisions": {"k": 1, "n": 220}},
      "tools":  {"occ": 0.63, "collisions": {"k": 2, "n": 310}},
      "attention":{"occ": 0.33, "collisions": {"k": 0, "n": 90}}
    }
  }
}
```

Slot events (`Slots.Alloc/Release/Evict/Collision/QueueEnter/Exit`) must link back to (τ, channel) for auditability.

19.5 Worked example (Support RAG mini-run)

- A/B replicas run identical toolsets with commuting reads first and a shared ledger. Agreement on the pointer `support.answer` tracks near 0.94–0.99 in steady state when redundancy is high; disabling the ledger or adding a non-commuting step degrades it—exactly what `agree(·)` surfaces.
- Slot policy: `tools.capacity=4, policy=PRI` keeps tail latency down for `db.get`, while `memory.capacity=8, policy=LRU` improves hit-rate; watch collisions and P95 waits against SLA bands.
- Mis-exec spike playbook: if TER > 2% for 5 min → freeze `code.exec`, prefer read-only, widen τ by 10%, enable bounded retries.

19.6 Tests

- **Agreement harness:** synth commuting vs non-commuting pairs; verify score and NCE behavior; require Wilson-CI lower bound ≥ threshold to pass.
- **Mis-exec unit tests:** inject invalid preconditions; ensure PVR increments; retries *don't* mutate latched trace (τ+1 only).
- **Slots:** non-overlap, eviction accounting, queue drain; assert pool-level SLAs (collision rate, P95 wait) within bands.
- **SMFT signals:** force AL↑, S_c↓ scenarios; trigger "Entropy floor" guard when coupled with saturation rise.

19.7 Artifacts

- **API sketch:** `GET /metrics/micro?window=ticks:128` → {AL, S_c, Agreement, PVR, TER, Slots*} (with Wilson CIs for proportions).
- **Dash panel:** "Micro Five" with sparklines + CI ribbons + bands.
- **Policy YAML:** pool SLAs (collision targets, P95 wait), agreement thresholds, mis-exec alerts —mirrors Ch.4/Ch.14 templates.

19.8 Operator guidance

1. **Design for commutation, externalize records, then measure redundancy**—that's how you raise Agreement.
2. **Treat slots as first-class**: queues beat unsafe eviction; back-pressure Ô when Red.
3. **Watch S_c with saturation** to avoid semantic black-holes; rotate channels or soften curvature when the floor rule fires.

Next: Chapter 20 — *Meso Metrics* (ρ, Δτ, Phase-Risk, CWA Score) connects these per-agent signals to fleet-level stability and certified pooling.

Chapter 20 — Meso Metrics

Goal

Measure *coordination and pooling safety* across agents: **Tick Sync** ρ, **Ô-Desynchrony** $\Delta\tau$, **Phase-Risk Index (PRI)**, and **CWA Score**—with test batteries (permute / sign-flip / chunk-shuffle), confidence bounds, drift tests, and policy gates that shape scheduling and pooling in production.

What you'll implement (checklist)

- Streaming estimators for **ρ** (Kuramoto order) and **Δτ** (tick skew) with anchors & resync policy.
- Certificate plumbing for **CWA score** + **PRI**, including CI and drift summaries.
- Test batteries: shuffle sets, sign-flip panels, chunk-shuffle; stability under perturbations.
- Green/Amber/Red bands + degradations (D0…D3) tied to scheduler Ô and pooling choices.
- Events/API: `TickAnchor`, `DesyncAlert`, `/pool` certificate payloads exported to telemetry.

20.1 Definitions (operational)

Tick Sync ρ — Kuramoto order parameter

For N observers with tick phases $\theta_j \in [0, 2\pi)$,

$$\rho \;=\; \left| \frac{1}{N} \sum_{j=1}^{N} e^{i\,\theta_j} \right| \;\in [0,1] \quad (1 = \text{perfect sync},\ 0 = \text{uniform desync}).$$

We derive θ_j from each agent's cadence-normalized tick phase. Use periodic **TickAnchor** broadcasts so agents slew (no jumps) toward anchors before computing ρ.

Ô-Desynchrony $\Delta\tau$

Fleet tick skew: max index gap (or p95–p5) across agents; per-agent $\Delta\tau_i = \tau_i - \overline{\tau}$. Track as a scalar (fleet) and as a tagged distribution (tenants/pools).

Phase-Risk Index (PRI)

Risk that *order/phase* still matters after projection $P(\cdot)$. Two complementary forms are supported:

- **Spectral / reversal PRI**: $\mathrm{PRI} = \max(\mathrm{PRI}_{\mathrm{spec}}, \mathrm{PRI}_{\mathrm{rev}})$ using periodogram mass off DC and reversal sensitivity of the additive pool. Lower is safer.
- **Panel-derived PRI′**: $\mathrm{PRI}' = 1 - \min(s_{\mathrm{perm}}, s_{\mathrm{flip}}, s_{\mathrm{chunk}})$, reported by the certificate evaluator. Both correlate; engines may emit either (or both).

CWA Score (Collapse-Without-Alignment certificate)

Aggregate stability of the additive pool under three perturbation panels: permutation (order), sign-flip (orientation), chunk-shuffle (boundaries). Each panel yields $s \in [0,1]$; the weighted average is the **CWA score**. Certificate payload includes per-panel deltas, CI, and drift stats; consumers gate mean/sum pooling on **score** and **PRI**.

20.2 Estimators & drift

Windows & smoothing. Compute ρ and $\Delta\tau$ over sliding time windows (e.g., 5–10 s), plus tick windows for fine control. Use EWMA on both for stable alerts. **Anchors**: broadcast `TickAnchor{τ_anchor, wallclock}` periodically (0.5–5 s); agents slew local timers; never jump ticks.

Certificate CI & drift. For **CWA score**, bootstrap panel deltas → 95% CI; keep a rolling reference window and report drift p-values. Include seeds and panel counts in the log for reproducibility.

Stability under perturbations. Run **K** perturbations per panel (typ. 8–32) and summarize median stability. Maintain component trends (perm vs flip vs chunk) to localize why CWA fails.

20.3 Thresholds & policy actions (defaults)

Metric	Green	Amber	Red	Primary actions
ρ	≥ 0.90	0.80–0.90	< 0.80	Amber→ enter D1 (widen cadence 10–25%); Red→ D2 + restrict schedules to commuting-safe channels; Resync until recovered.
Δτ (ticks)	≤ 2	3–5	> 5	Trim fast outliers, slow cadence, anchor more often; unblock when Δτ ≤ 2.
PRI / PRI′	≤ 0.20	0.20–0.50	> 0.50	If Amber: run full panels, lower K but keep gating; Red: force order-aware pooling. (APIs expose `pri_threshold`.)
CWA score	≥ θ_pass (norm=0.82)	θ_warn–θ_pass (warn≈0.75)	< θ_warn	Pass→ mean/sum; Amber→ consider order-aware trial; Red→ fallback. In **Strict** mode (D2), raise θ_pass (e.g., 0.98).

Degradation ladder. D0 Normal → D1 Gentle (cadence↑, panels÷2) → D2 Strict (commuting-only; CWA θ_strict) → D3 Quiescent (pause ticks except health). Entry/exit are keyed to ρ, $\Delta\tau$, occupancy and collision SLOs.

20.4 Test batteries (how to run)

Permutation (shuffle sets). Resample K random permutations; pool; measure distance to baseline pool; compute stability $s_{\text{perm}} = 1 - \text{median}(\delta)$.

Sign-flip panels. Apply Rademacher masks per item (and/or per PCA axis); pool; s_{flip} as above. Flags "SignSensitive" if low.

Chunk-shuffle. Jitter boundaries ±ε, merge/split neighbors; re-project then pool; s_{chunk} indicates boundary sensitivity.

Stability under perturbations (end-to-end). While running the panels, vary cadence within bounds and inject controlled anchor jitter; **ρ** should remain ≥ 0.9 and $\Delta\tau \le 2$; certificate stability should not degrade materially if pooling is truly additive. Record CI and drift for the certificate.

20.5 Worked example (fleet resync + certificate gating)

Scenario. Multi-robot field run drifts due to poor time sync: $\rho = 0.77$, $\Delta\tau = 7$. System broadcasts `RESYNC` anchors each 500 ms, trims fast outliers, widens cadence to 110 ms. Within 12 s $\rho = 0.92$, $\Delta\tau = 2$ → reopen full schedules. During D2 Strict, CWA raises θ_pass; a RAG batch with **PRI=0.47** and **CWA=0.86** fails fast-path and falls back to attention pooling, within the fallback

latency SLO.

20.6 Events & APIs

Events (Tick & Sync).
`TickAnchor{τ_anchor, wallclock, cadence_ms}`, `DesyncAlert{rho, delta_tau, level}`, `Cadence.Update{cadence_ms, reason}`, `Degrade.Enter/Exit{mode, reason}`.

Pooling API (with certificate).
`POST /pool` returns `{cwa:{score, components, ci95, drift}, pri, risk_flags, latency_ms}`. Consumers gate on `min_score` and `pri_threshold`, with seeds and panel counts logged for audit.

20.7 Operator guidance

1. **Use anchors liberally**: keep $\rho \geq 0.9$ at governance checkpoints; restrict schedules to commuting-safe reads during resync.
2. **Gate pooling by both score *and* PRI**: low score or high PRI → order-aware estimator; store certificate artifacts.
3. **Raise thresholds in stress** (D2): stricter θ for CWA and lower $\rho_{\min}$ tolerance reduce cascading failures.

Next: Chapter 21 — *Macro Metrics* (PBHL Residual, Gap/Flux/Twist, EEI/SI) stitches these meso signals into belt-level closure and controllers.

Chapter 21 — Macro Metrics

Goal

Make **program-level closure** measurable and controllable. Define **PBHL Residual**, **Gap/Flux/Twist** (with coupling **α**), and **EEI/SI**; then show how **Flux-gate (fast)** and **Twist-step (slow)** controllers keep Residual in band and operate within stability regions.

21.1 Worldsheet metrics (definitions)

PBHL identity (ops layer).

$$\textbf{Gap} \approx \textbf{Flux} + \alpha\,\textbf{Twist}, \qquad \textbf{Residual} = \left|\,\text{Gap} - (\text{Flux} + \alpha\,\text{Twist})\right|$$

A belt is the worldsheet swept by a program loop (plan→design→build→validate→deploy→plan). Residual is the closure error to minimize.

Gap(t). Remaining target shortfall projected to governing KPIs; normalize by Gap_0 for comparability (0–1 scale).

Flux(t). *Effective* throughput (validated increments × impact weights × survival rate), usually an EWMA per cadence.

Twist(t). Rate of structural change that perturbs the plan↔do map; combine proxies like ownership churn, plan drift, and architecture churn, normalized to $[0, 1]$.

α(t). Coupling from Twist to effective progress (signed). Estimate via ridge regression on a rolling window of $\Delta\text{Gap} = -\text{Flux} - \alpha\,\text{Twist} + \epsilon$. Track CI/variance; gate twist if unstable.

Residual bands (defaults). Green ≤ **0.05**, Amber (0.05–0.10], Red > **0.10–0.20** depending on risk tier. Use Five-Line KPI: Gap, Flux, Twist, Coherence/Sync ρ, Residual.

Derived indices.

- **EEI (Effectiveness/Execution Index)** $= \frac{\text{Flux}}{\text{Flux+Waste}}$, where Waste = rework + abandoned + blocked. (0–1; ↑ better.)
- **SI (Sustainability Index)** $= \rho \cdot e^{-\lambda_T \max(0,\text{Twist}-\theta_T)}$, penalizing excessive twist and low sync. (0–1.)

21.2 Estimation & telemetry

- **Windowing.** Daily points with $W \approx 2$ weeks and EWMA $\beta = 0.3$ for dashboard smoothness; retain raw cadence data for audits.
- **α estimation.** Ridge on (Flux, Twist) → $[\beta_F, \beta_T]$; set $\alpha \leftarrow \beta_T$. Emit CI; if CI width > policy, freeze Twist steps.
- **API.** `POST /belt` ingests Gap/Flux/Twist/α and returns Residual, suggested controls, and indices; `GET /belt/:id/kpi` exports KPI time series (JSONL/Parquet) with lineage.
- **Evidence.** Each release archives PBHL/4π/gluing checks and KPI rows in an **evidence_vault** with a bundle hash for audits.

21.3 Controllers & stability regions

Flux-gate (fast; minutes→days).
PI-style action that changes *how hard* the belt pushes (slots, cadence, staffing, batching). Example:

$$u_F(t) = k_P\,\text{sgn}(e_R)\,\min(|\,e_R\,|, u_{\max}) + k_I \int e_R\,dt, \quad e_R = \text{Residual} - \varepsilon_g$$

Map u_F to slot capacity deltas, Ô cooldowns, and priority queues. Stable when (i) Residual dynamics are monotone around the setpoint, (ii) $|\,k_P\,|$ respects actuation latency, and (iii) Twist is within budget (no moving target).

Twist-step (slow; weeks).
Hysteretic, budget-bound policy that freezes/unfreezes refactors and ownership changes. Trigger only when |Residual| > ε_a and α has sign/CI confidence; guard step size $|\Delta\text{Twist}| \le \tau_{\max}$ per review period. Stability improves when **4π** periodicity and **gluing** audits pass (consistent framing & modular joins).

Stability region sketch.
Operate in a two-timescale band where: (a) certificate/sync are healthy (ρ≥0.9), (b) Flux-gate reaches steady state within ≤2 review windows for small Residual perturbations, (c) Twist-step period ≥4 Flux-gate time constants, and (d) α variance below policy. Outside this region, degrade (D1→D3) and/or freeze twist.

21.4 Thresholds & gates (defaults)

- **Residual gates. Gate-R**: if Residual > ε_red for 2 windows → freeze non-critical twist, raise Flux-gate. **Gate-α**: if α variance spikes → executive review required.
- **EEI/SI SLOs.** Alert on EEI drop >10% w/w or SI < 0.7 for 2 weeks; investigate Waste components and sync.
- **Release gate (audit).** Require PBHL green, 4π pass, gluing pass; block on any red. SQL templates provided in PFBT Appendix G.

21.5 Worked examples

A) Positive closure. Start: Gap=0.80, Flux=0.15, Twist=0.05, α≈+0.20 → Residual≈0.64 (red). Flux-gate raises tool capacity and relaxes a cooldown; keep twist frozen. After 2 weeks Flux→0.32, Twist→0.04, α≈+0.18 → Residual≈0.47 (amber). Then a targeted refactor (Twist 0.06, α>0) drops Residual to ≈0.29 (green trajectory).

B) Twist spike incident. Reorg injects Twist=0.20; α flips negative (−0.10). Residual jumps to 0.35 (red). Gate-R freezes refactors 14 days, boosts Flux-gate, and schedules α review. Recovery as Twist decays and α stabilizes.

21.6 Tests & audits

- **PBHL Closure Audit.** Enforce $|\text{Gap} - (\text{Flux} + \alpha\,\text{Twist})| \le \tau_{\text{PBHL}}$.
- **4π periodicity.** Class-function invariants equal at 0 and 4π, differ at 2π.
- **Gluing.** Interior edges cancel ≤ tolerance in tiled belts.
- **Controller regression.** Simulate vanity velocity (Flux↑ without validation): EEI↓ and Residual mismatch—controller should refuse release. Simulate frequent reorgs (Twist↑ with α<0): Twist-step must throttle.

21.7 Artifacts

- **API.** `/belt` update + control suggestions; KPI export with lineage (pool/agreement/cert refs).
- **Dashboard.** Five-Line KPI: banded Gap vs Flux+α·Twist, Residual line, Twist with α overlay, ρ mini-panel, EEI/SI sparklines, active gates.
- **Runbooks & SQL.** Release gate queries, twist-budget window checks, evidence_vault schema.

Operator guidance

1. **Drive Residual, not vanity throughput.** Flux must be *validated*; pair with EEI to detect waste.
2. **Guard α.** Don't twist without a confident α; freeze when variance widens.
3. **Stay in the stability region.** Keep ρ high, separate timescales (fast Flux-gate, slow Twist-step), and pass 4π+gluing before big steps.

Next: Chapter 22 — *Telemetry Surfaces* (dashboards, exports, evidence & privacy).

Chapter 22 — Telemetry Surfaces

Goal

Ship *operator-first* telemetry surfaces that make ObserverOps observable and auditable: dashboards (micro→meso→macro), signed exports (traces, cert logs, belt KPIs), and governance-ready evidence with SOC/ISO-friendly mappings. Also lock down data retention, privacy redaction, and lineage.

What you'll implement (checklist)

- **Dashboards:** Micro "Micro Five", Meso sync/cert panels, Macro **Five-Line KPI** with Residual bands and gate overlays.
- **Exports:** Signed, hash-addressed bundles with **KPI**, **Gate.Decision**, and **Cert Logs**, plus lineage.
- **APIs/Webhooks:** `/belt` updates, KPI export, signed **Gate.Decision** webhooks.
- **Privacy/Retention:** PII tagging & redaction maps, hot/cold retention, tamper evidence.
- **Evidence Vault:** periodic exports to GRC vault with lag alerts and completeness checks.

22.1 Dashboards (panels you'll ship)

A. Micro (agent-level)

- **Micro Five:** AL, S_c, Agreement, Mis-exec (PVR/TER), Slots (occ/collisions) with bands & Wilson CIs. Feed from Observer Runtime & Slot Allocator SLOs.

B. Meso (fleet & certificates)

- **Sync Panel:** ρ sparkline, $\Delta\tau$ distribution, anchor cadence, desync alerts.
- **Certificate Panel:** CWA score and components (perm/flip/chunk), PRI histogram, drift p-values; show fallback rate.

C. Macro (BeltOps)

- **Five-Line KPI:** Gap, Flux, Twist, Coherence (e.g., agreement), Residual with **Green/Amber/Red** bands; overlay gate states and Twist annotations. Panel JSON carries band thresholds.
- **Board One-Pager:** EEI, SI, Residual band, cert pass-rate, PRI p95, incidents/actions—ready for execs.

Operator note: Dash panels must overlay **policy gates** decisions and cadence changes; this ties telemetry to control.

22.2 Exports & the Evidence Vault

Bundle format. Periodic (e.g., every 15 min) signed bundles with a manifest referencing time-sliced files; all artifacts are hash-addressed and carry lineage.

Manifest (essentials):

- `kpi.parquet` — Gap/Flux/Twist/Coherence/Residual, EEI/SI.
- `decisions.jsonl` — Gate.Decision with reasons & actions.
- `cert_logs.parquet` — referenced CWA certificates (subset).
- `lineage` — `pool_ids`, `agree_ids`, `cert_refs`.
- `signature` — ed25519/HMAC; verify on ingest.

SOP-C (export): validate signatures + manifest completeness; cross-check counts vs telemetry; push to GRC vault; alert on export lag > 2 min.

APIs:

- `POST /belt` → computes PBHL residual, returns KPIs/indices; **use update_id for lineage**.
- `GET /belt/:id/kpi?...&format=parquet|jsonl` → time-series export with lineage.

22.3 Webhooks & decision logs

Register a signed webhook for **Gate.Decision**, **Belt.Update**, **CWA.Drift**. Payloads include reasons (e.g., `cwa_warn_or_pri`, `residual_amber`) and actions (`cadence_factor`, `panel_scale`, `commuting_only`). Verify via `X-ObserverOps-Signature`.

Why this matters: closing the loop (Data→Macro→Control) is explicit in the telemetry: Belt updates feed gates; gates feed Tick & Sync; all changes are logged and visible in panels.

22.4 SOC/ISO-friendly mappings (what evidence you emit)

- **Immutable traces & cert logs** → append-only, hash-chained stores with signed exports (maps to auditability & change evidence).
- **Deterministic gates** → versioned thresholds, signed decisions, override/waiver workflow (controls + change management).
- **Lineage** on KPIs → pool/agreement/certificate refs embedded in exports (end-to-end evidence).
- **GRC handoff** → scheduled export to vault; pass/fail metrics like **Audit pass-rate** tracked on the dashboard.

  See also Appendix “Safety & Compliance Mapping — SOC/ISO control matrices & trace evidence” for control-by-control checklists.

22.5 Data retention, privacy & lineage

Retention: hot **90–365 days**, cold **7 years**; publish retention in manifest; export redaction maps alongside data.

Privacy & minimization:

- Tag projection artifacts with PII/secret flags; redact raw content from Cert Logs; store vectors with lineage only.
- Hash actor IDs with keyed salt; enforce **k-anonymity ≥ 5** in published bins; log access to

Twist entries (often sensitive).

Tamper evidence: trace & cert hash-chains; reproducible panel seeds; signed bundles.

Lineage contracts: link KPI rows to pool/agreement/cert IDs; enforce **no orphan events** and exact interior-edge cancellation in belt tiling (gluing).

22.6 SLOs, alerts, and safe modes (telemetry-backed)

- **SLOs:** Measure→write p95 ≤ 50 ms; /agree p95 ≤ 40 ms; export lag p95 ≤ 2 s; sync/status broadcasts ≤ 10 ms. Panels show banded tiles.
- **Alerts:** `LatchingViolation`, `CWA.Red or Drift`, `Sync.Red (ρ<0.8 or Δt>5)`, `ledger ingestion lag`. Each alert links to evidence IDs.
- **Safe modes (visible on panels):** CWA-off, Ô-reduced (commuting-only), Tick slow-roll, Read-only audit—with clear exit criteria and gate overlays.

22.7 Artifacts

- **Panel specs:** JSON for Five-Line KPI with bands; Meso sync & cert components.
- **Webhook schema:** signed Gate.Decision payloads + retry policy.
- **Export manifest template:** KPI/Decisions/Cert Logs + lineage + signature.
- **Security hooks:** auth scopes per plane; least-privilege runners; privacy map file.

Operator guidance

1. **Make control visible.** Overlay gate states & cadence changes on every relevant panel—operators should *see* why the system throttled or fell back.
2. **Ship lineage, not screenshots.** Every KPI point must be traceable to pool/agreement/cert IDs; exports are the ground truth for audits.
3. **Respect retention & redaction.** Keep hot/cold windows and publish privacy maps; never ship raw PII in cert logs.

Next: Part V — *Guarantees & Proof Sketches* (Ch. 23–27) ties these surfaces back to the math.

Part V — Guarantees & Proof Sketches

Chapter 23 — Internal Collapse

Claim (what we guarantee)

Latching is a fixed point under the observer's own past; once a record is committed at tick τ_k, it is *in-frame irreversible*, and all downstream control must branch on it.
Formally, let $\mathcal{F}_{\le\tau}$ be the σ-algebra generated by the committed trace up to τ. For the tick-k outcome Y_k written to the trace, **internal collapse** requires

$$\mathbb{E}[Y_k \mid \mathcal{F}_{\le\tau_k}] \equiv Y_k$$

and all future scheduling/control be measurable w.r.t. $\mathcal{F}_{\leq \tau_k}$ (no dependence on hypothetical retro-edits).

23.1 Formalization (fixed points & branching)

- **Filtration view.** The observer's trace induces an increasing filtration $\mathcal{F}_0 \subset \mathcal{F}_1 \subset \cdots \subset \mathcal{F}_{\leq \tau}$. Conditional expectation onto $\mathcal{F}_{\leq \tau_k}$ yields *delta-certainty* for events already written: they are fixed points of the projection.
- **Operator-algebraic restatement.** With memory algebra $\mathcal{M}$ and world algebra $\mathcal{W}$, the conditional expectation $E_{\leq k}: \mathcal{W} \otimes \mathcal{M} \to \mathcal{F}_{\leq \tau_k}$ satisfies $E_{\leq k}(A) = A$ for any past event $A \in \mathcal{F}_{\leq \tau_k}$: *observer-specific certainty is algebraic latching*.

23.2 Implementation invariants (what the runtime must enforce)

A commit is **measure→append→advance**; advancing τ is the latch. Enforce with a *latching guard*:

- **Monotone τ.** Advance only on successful append; never on attempt.
- **Hash-chain.** Each record carries `prev_hash` and `hash(H(prev, τ, channel, outcome, meta))` for tamper-evidence.
- **Append-only.** No UPDATE to past ticks; corrections are new records (e.g., `kind:"correction"`).
- **Idempotency.** Duplicate submits (same `idempotency_key`, τ) return the prior commit.
- **Atomicity.** If append fails, τ does not advance.

  Branch measurability hook. The scheduler $\hat{0}$ must read only *committed* T (post-append), ensuring downstream decisions are measurable w.r.t. $\mathcal{F}_{\leq \tau_k}$.

23.3 Proof sketch (why latching yields in-frame irreversibility)

1. **Write ⇒ fixed point.** Once (τ_k, π_k, y_k) is appended, the event "$Y_k = y_k$" lies in $\mathcal{F}_{\leq \tau_k}$. Conditional expectation onto $\mathcal{F}_{\leq \tau_k}$ leaves it invariant: $\mathbb{E}[Y_k \mid \mathcal{F}_{\leq \tau_k}] = y_k$. This is the algebraic form of "delta-certainty".
2. **Control measurability.** Post-commit policies $f(S, T_{\leq \tau_k})$ are $\mathcal{F}_{\leq \tau_k}$-measurable, so downstream branches cannot depend on counterfactual edits of $T_{<\tau_k}$. Any "edit" appears only as *new* information at τ_{k+1}.
3. **Runtime guard ⇒ no retro-edit.** The hash-chain + append-only schema forbids in-place mutation; all alternative paths require a new tick (a new branch), preserving in-frame irreversibility operationally.

23.4 Limits & failure modes (and the guards you ship)

- **External overwrite.** An out-of-band process mutates history (e.g., a "cleaner"). *Guard:* append-only API; verify hash-chain; reject with **409 Conflict**; quarantine run if chain

breaks.

- **Re-entrancy.** A handler re-enters the loop and writes twice at the same τ. *Guard:* enforce monotone τ and idempotency keys.
- **Partial commit.** Measurement succeeds but `write_trace` fails while τ advanced. *Guard:* atomic transaction boundary; no τ advance on failure.
- **Policy drift (soft irreversibility).** If post-latch the scheduler's entropy doesn't drop, branches aren't conditioning on T. *Guard:* **M3 Branch determinism** metric; alert on missing entropy drop.

23.5 Worked examples

- **Qubit toy (Z↔X).** Latching points are the commits; swapping non-commuting instruments changes intermediate distributions, not the fact that each commit is in-frame irreversible.
- **Tool call (plan→tool→trace).** Any "cleanup" of a past tool result must be an *append* with `kind:"correction"` at τ_{k+1}; updates to τ_k are rejected.

23.6 Tests & metrics (proof obligations in CI)

- **T1 Hash-chain integrity.** Recompute all links; first mismatch ⇒ incident + quarantine.
- **T2 No silent retro-edit.** Attempt UPDATE on $\tau = k$ ⇒ expect **409**; next valid change appears as a new tick.
- **T3 Idempotent commit.** Duplicate `(τ,channel,idempotency_key)` ⇒ single stored record.
- **M1 Trace half-life.** How long record k influences $\hat{O}$.
- **M2 Latch Latency.** Measure end → commit time; bound for real-time.
- **M3 Branch Determinism.** Entropy of $\hat{O}$'s next-channel distribution should drop after latching.

23.7 Artifacts

- **APIs.** `POST /measure` (optional auto-commit with `idempotency_key`), `GET /trace/:id` with range & proof options; error codes **409/412/428** for guardrail violations.
- **Figure.** Ô-first loop with the **LATCH** point and side-rail hash-chain (see Ch.1 Figures F-1/F-2).
- **Checklist.** "Advance τ only on append; forbid UPDATE; expose correction type; bound latch latency; log seeds & hashes."

  Takeaway. Internal collapse is both *math* (fixed point under conditional expectation) and *ops* (append-only, hash-chained, idempotent writes). Together they make branches replayable and audits meaningful.

Chapter 24 — Cross-Observer Agreement

Claim (what we guarantee)

Commuting effects + shared (or redundant) records ⇒ AB-fixedness.
If observers A, B measure aligned channels whose effects commute on the visited support, and both condition on the *same pointer information* (either a shared ledger or SBS-style redundancy), then they assign delta-certainty to the same effective outcome on those keys (AB-fixedness).

24.1 Objects & pre-conditions

- **Compatibility/commutation graph C.** Edge $(\pi, \pi') \in C$ iff effects commute on the sampled subspace; we only score overlaps on keys $K^\star$ where aligned instruments commute. Frame maps ϕ + a canonical labeler λ align channels for comparison.
- **Shared/redundant records.** Either (i) **shared ledger** with hash-chained records (hash match proves sameness), or (ii) **SBS redundancy**: independent fragments $\{E_r\}_{r=1}^{R}$ carrying the same pointer value. Redundancy proxies: majority stability & permutation stability.
- **Latching.** Internal collapse guarantees both policies are measurable w.r.t. committed past; no retro-edits inside a tick.

24.2 Formal statement & sketch

Theorem (AB-fixedness, operational form).
Let $K^\star$ be the set of canonical keys with (i) commuting aligned effects $[\pi_A, \pi_B] = 0$, (ii) either a shared trace for those keys or SBS-style redundant fragments accessible to both, and (iii) latched traces. Then for every $(k, \tau) \in K^\star$, the effective outcomes used by A and B coincide almost surely.

Sketch.

1. **Commutation ⇒ order-independence.** Joint outcome law on $K^\star$ is invariant to instrument order, removing schedule artifacts.
2. **Shared/SBS record ⇒ common conditioning.** Either the same hash-proven record is read by both, or SBS fragments supply the same pointer value with high probability; hence both condition on the same σ-algebra.
3. **Latching ⇒ fixed points.** Once written, events are fixed under conditional expectation; downstream policies are $\mathcal{F}_{\leq \tau}$-measurable. Thus the effective values coincide (AB-fixedness).

Algebraic lens. In a common algebra, commuting projections P_A, P_B ensure agreement on their intersection; AB-agreement is "commuting projections in a common algebra" plus shared access to the recorded outcome.

24.3 Error bounds (redundant fragments)

When each SBS fragment independently reports the pointer with error $p < \frac{1}{2}$, the **majority** estimator over R fragments has error

$$\Pr[\text{maj wrong}] \leq \exp\left(-2R\left(\tfrac{1}{2} - p\right)^2\right)$$

(by Hoeffding). Thus **agreement failure** on $K^\star$ decays exponentially in R under independence; with correlations, replace by an effective R_{eff} via block/bootstrap. (Our redundancy proxies—majority & permutation stability—track this behavior in practice.)

24.4 Counterexamples (why the assumptions matter)

- **Non-commuting effects.** Insert X between Z reads on a qubit; intermediate outcomes need not agree ⇒ no AB-fixedness.
- **Hidden channel / weak redundancy.** One observer applies an unlogged transform or reads fragments carrying no pointer info; consensus need not converge even as R grows.
- **Bad frame maps.** Non-isometric mappings can destroy commutation and common support, fabricating "agreement" or making it impossible.

24.5 Tests, diagnostics, artifacts

- **agree(T_A, T_B).** Score on *commuting overlaps* after frame mapping; report non-commuting exposure (NCE), redundancy stats, and hash-match proofs for shared records.
- **SBS redundancy probes.** Majority & permutation stability with thresholds (green ≥ .98, amber .95–.98, red < .95).
- **Shared-ledger proof.** Hash-match ratio over overlaps; any mismatch is actionable.

Operator guidance. Design for commutation, externalize records (or build redundancy), *then* measure agreement; gate schedules when NCE rises.

Chapter 25 — CWA Validity

Claim (what we guarantee)

Project→add is valid when order/phase wash out; the CWA certificate detects this with high reliability. After per-item projection $P(\cdot)$, additive pooling (mean/sum) is safe iff the pooled result is *stable* under perturbations that scramble order, flip orientations, and jitter chunk boundaries; the certificate's score operationalizes this invariance.

25.1 Setup & statement (operational form)

Let $X = \{x_i\}_{i=1}^{N}$, $z_i = P(x_i) \in \mathbb{R}^d$, baseline additive pool $\mu_0 = \frac{1}{N}\sum_i z_i$ (or sum). For each panel $j \in \{\text{perm}, \text{flip}, \text{chunk}\}$, draw K perturbations $T_k^{(j)}$ and re-pool to $\mu_k^{(j)}$. Define a normalized stability distance $\delta(\mu_k^{(j)}, \mu_0)$, combine panel medians into scores $s_j \in [0,1]$, and aggregate

$$\text{CWA_score} = \sum_j w_j \, s_j \in [0,1].$$

Validity band (examples): *Strict*: pass if score ≥ 0.98 and PRI ≤ 0.20; *Default*: pass if score ≥ 0.82 and PRI ≤ 0.50. Otherwise warn/fail and auto-fallback to order-aware pooling.

Panels & rationale.

- **Permutation (order):** detects hidden order sensitivity introduced upstream (e.g., normalization, sequence-conditioned projection). For true additive observables, order shouldn't move μ.
- **Sign-flip (orientation):** flips per-item signs (or top-K PCA axes) to test whether arbitrary orientation conventions survive projection; additive observables are sign-consistent in aggregate.

- **Chunk-shuffle (boundaries):** jitters or re-chunks items to probe boundary dependence in the projector; robust collapse erases these effects.

Complement with **Phase-Risk Index (PRI)** (spectral/reversal gauges) as a fast pre-screen; high PRI flags phase structure where order matters.

25.2 Why this works (collapse geometry → real arithmetic)

SMFT's collapse geometry says phase-sensitive structure is destroyed by projection; what survives are **real, order-insensitive** quantities that compose additively (plus simple scalings/roots). Under such collapse, addition is the unique robust macro operation; phase/logical compositions do *not* survive. CWA simply *tests* whether the current batch exhibits that post-collapse regime.

25.3 Certificate soundness (informal sketch)

1. **Invariance ⇒ stability.** If P removes order/phase and boundary idiosyncrasies, then for all perturbations in the panels, $\delta(\mu_k^{(j)}, \mu_0)$ concentrates near $0 \Rightarrow s_j$ near $1 \Rightarrow$ score passes. (Mean/sum are intrinsically permutation-invariant; the panels reveal *upstream* violations.)
2. **Sensitivity to coherence.** Coherent/phase-encoded sets (e.g., alternating regimes) produce large reversal or flip deltas; chunk jitter breaks regime alignment ⇒ score drops, PRI rises ⇒ fast-path blocked; order-aware fallback keeps accuracy.
3. **Stats discipline.** Use bootstrap CIs on panel deltas and drift tests across windows; publish seeds and component scores so policy gates can set risk-aware thresholds.

Type I/II controls. Increase panel counts K to reduce false green; raise thresholds in **Strict** modes; log CI and drift to bound decision risk. (Latency/complexity budgets are documented for pass vs fallback.)

25.4 Statistical tests (minimal recipe)

- **Distance:** $\delta(\mu, \mu_0) = \min\left(1, \| \mu - \mu_0 \|_2 / (\| \mu_0 \|_2 + \varepsilon)\right)$ or a cosine+norm blend; take panel median.
- **Score:** $s_j = 1 - \text{median}(\delta)$; score $= \sum_j w_j s_j$. Defaults $w = (0.4, 0.3, 0.3)$.
- **PRI:** max of spectral mass off-DC and reversal sensitivity. Gate with PRI $\leq \tau$ before allowing fast-path.

25.5 Limits & counterexamples (why we can fail safely)

- **Coherent chains.** Strongly phase-coded sequences (periodic or alternating regimes) legitimately fail sign-flip/reversal; additive pooling would blur regimes; fallback (attention/ CNN/sequence) is required.
- **Bursty phase.** Irregular, bursty coherence can pass small K panels by chance; mitigate with stricter K, lower pass θ, or a **Strict** profile for high-stakes pools.
- **Boundary-entangling projectors.** If P encodes positional bleed, small chunk jitter reveals instability (low chunk score). Fix the projector or keep fallback.

Note. CWA is a *certificate*, not a proof for all future batches; we therefore ship drift detection and

reproducible seeds to bound exposure.

25.6 Worked evidence (pass/fail archetypes)

- **Orderless bags (PASS):** doc chunks projected with minimal positional bleed → perm/re-chunk stable; CWA ≈ 0.995–0.999; mean pooling safe; latency win.
- **Alternating regimes (FAIL):** time series with phase-encoded features → sign/reversal & chunk panels unstable; score ≈ 0.82, PRI ≈ 0.55; fallback maintains accuracy.

25.7 Artifacts & gates

- **/pool API** returns `{score, components, PRI, ci95, drift}` with seeds; policy gates choose aggregator (add vs order-aware). SLOs: ~30 ms pass, ~120 ms fallback.
- **Runtime defaults.** Example thresholds and panel counts are provided (perm/flip/chunk = 128/64/32); raise to strict in safety-critical lanes.

  Takeaway. CWA makes "just average it" *conditional*: when projection truly collapses nuisance phase/order, the certificate passes and addition is sound; otherwise, we *prove* it's risky and fall back—preserving quality with bounded, explainable latency.

Chapter 26 — Slot Conservation

Claim (what we guarantee)

Capacity is quantized and non-overlapping. Within each typed pool (memory, attention, tools), allocations are integers and occupied addresses are disjoint at every tick; evictions/queueing are explicit policies. From these invariants follow allocator corollaries on collision rates, queue waits, and safe back-pressure.

26.1 Model & invariants

Typed pools & addresses. Partition capacity into slot sets $\mathcal{M} = \{m_1, \ldots, m_{N_M}\}$, $\mathcal{A} = \{a_1, \ldots, a_{N_A}\}$, $\mathcal{U} = \{u_1, \ldots, u_{N_U}\}$ for memory/attention/tools. Allocations request integer units k.

S1 (integrality). $k \in \mathbb{Z}_{\geq 0}$.
S2 (non-overlap). At tick τ, occupied addresses in a pool are pairwise disjoint.
S3 (explicit eviction). Freeing capacity occurs only via `release()` or logged policy eviction—no implicit GC that mutates history.
S4 (observability). Every `Alloc/Release/Evict/Collision/QueueEnter/Exit` is written to the trace T with provenance.

Runtime contract. Minimal API enforces integer grants, disjoint holds, policy-driven eviction/queueing, and emits events with `{pool, k, token, policy, occupancy_after}`.

26.2 Why "slots"? (first-principles justification)

Combinatorial closure. LuoShu/HeTu give canonical, *balanced* discrete capacities: unique integer assignments under global sum/pair constraints; any repeats, gaps, or non-integers violate closure/symmetry. That's the mathematical archetype behind the "slot" law (quantized addresses,

conserved totals along paths).

Operational reading. A "slot" is a discrete, mutually distinguishable capacity for a trace/state at a location—exactly what we enforce for memory/attention/tool pools.

26.3 Sketches (pigeonhole, packing, collisions)

Lemma 1 — Pigeonhole bound (max concurrency)

For a pool with N slots, any set of $N + 1$ simultaneous unit requests cannot be satisfied without eviction or queueing. *Reason:* By S2 (non-overlap), at most one request occupies a given address; N addresses ⇒ $\leq N$ concurrent holds. (Equality is achievable with disjoint holds.) **Corollary:** With integral sizes k_i, feasibility requires $\sum_i k_i \leq N$.

Lemma 2 — Packing & fragmentation

For integral requests, greedy admission is complete when eviction is disallowed: if $\sum k_i \leq N$ and arrivals are processed in any order with queueing (no eviction), all items are eventually admitted. (There is no internal fragmentation: addresses are indistinguishable; integrality avoids fractional waste.) **Design hint:** use strict queueing (`policy: NONE`) for safety-critical pools (attention).

Lemma 3 — Collision necessity

Define a **collision** when a request would require evicting an *active or unguarded* item to admit. If at arrival time t, $\text{free}(t) < k$ and policy prohibits evicting protected grants, then a collision is unavoidable (either refuse or queue). **Consequences:** (i) collisions upper-bound feasible parallelism under protected leases; (ii) frequent collisions signal either under-provisioning or policy mismatch (e.g., trying to preempt in-flight tools).

Lemma 4 — Queueing vs eviction trade

Let λ be arrival rate (units/tick) and μ the average release rate. In steady state, if $\lambda > \mu$ then either queue length grows (queueing policy) or eviction churn grows (evicting policy). Minimize tail risk by: for *tools* prefer **PRI** with preemptible *queued* items; for *attention* enforce **NONE** (no eviction); for *memory* use **LRU** to align with locality.

26.4 Allocator corollaries (what you can guarantee)

C1 — Collision-rate bands. With correct policy matching, you can hold

- memory collisions < **0.5%**, tools < **1%**, attention ≈ **0%** (by design). Sustained red (>2–4%) indicates mis-sized N or wrong policy.

C2 — Non-overlap unit test. Fill to $k = N$ then request one more ⇒ must queue/refuse (never overlap). This guards S2 at code level.

C3 — Bounded wait SLOs. Set P95 wait ticks: memory ≤ 1, tools ≤ 2, attention 0 (strict). Violations call for back-pressure (increase $\Delta\tau$, reduce parallelism) before raising N.

C4 — Eviction accounting. Every eviction reduces `used` by the evicted units, emits `Slots.Evict`, and links to the causing (τ, channel). Missing or delayed accounting is a correctness bug.

C5 — Policy-aware Ô. Before selecting a channel π, the scheduler checks `can_run(π)`; if not, it either picks a commuting alternative or inserts back-pressure ($\Delta\tau\uparrow$). This keeps mis-exec/timeout

risk low under load.

26.5 Algorithms & policies (brief)

- **LRU (memory)** for locality; **PRI (tools)** to protect critical path; **NONE (attention)** to forbid unsafe preemption. All events are append-only to T; leases heartbeat and auto-expire. APIs: `/slots/request`, `/slots/heartbeat`, `/slots/release`, `/slots/occupancy` with p95 SLOs.
- **Back-pressure hints** on refusal: `reduce_parallelism`, `widen_ticks`, `switch_estimator` (e.g., avoid expensive panels). Tie into Tick & Sync degradation modes.

26.6 Worked examples

A) RAG cache (NM=8). LRU vs PRI: LRU maximizes hit-rate under stationary traffic; PRI reserves space for hot topics, reducing tail latency and **collisions** during spikes.

B) Tool budget (NU=4). With **NONE** + queueing, bursts smooth ($\Delta\tau\uparrow$ slightly) and timeouts fall; with **PRI**, critical `db.get` preempts `web.search`, improving P95 latency on the critical path.

26.7 Tests & metrics (CI + ops)

- **S2 Non-overlap test.** Allocate N, then $+1 \Rightarrow$ expect queue/refuse.
- **Idempotent release.** Releasing more than held caps at held.
- **Queue drain order.** Highest priority admitted first after frees.
- **Collision rate** = #Collision/#AllocReq with bands: green <0.5% (mem), <1% (tools).
- **Eviction churn** (per wall-time) — high churn $\Rightarrow$ raise N or swap policy.
- **Occupancy heatmap** — "zebra" pattern flags thrash; "solid band" flags saturation.

26.8 Figures, configs, artifacts

- **Figure:** Slot allocator heatmap (time × slot index; states free/active/idle/evicted).
- **YAML:** Slot policy template (capacities, policies, guard periods, SLA bands).
- **APIs:** `/slots/*` contracts with decision p95 ≤ 5 ms; `/slots/occupancy` for dashboards.

26.9 Limits & counterexamples

- **Hidden allocations.** Libraries bypass the allocator → "ghost" collisions; wrap them or they violate S4.
- **Unsafe preemption.** Evicting in-flight tools breaks correctness; mark in-flight as unevictable.
- **Policy drift.** Wrong policy (e.g., PRI for memory) yields thrash (zebra heatmaps) and high churn; correct by switching policy or resizing N.
- **Non-integral semantics.** If a subsystem admits fractional holds, you've left the slot model;

bring it back behind a slot-ized shim or exclude from SLOs. (LuoShu/HeTu uniqueness relies on integrality.)

Takeaway. Slot conservation is both a **combinatorial law** (quantized, non-overlapping addresses) and an **ops discipline** (events, policies, SLOs). Enforce S1–S4 and the allocator corollaries follow—predictable latency, low collision rates, and graceful back-pressure under bursty load.

Chapter 27 — PBHL Closure

Claim (what we guarantee)

For a program belt swept by plan↔do loops, the **Purpose-Belt Holonomy Law (PBHL)** constrains the belt-integrated variables

$$\textbf{Gap} \;\approx\; \textbf{Flux} \;+\; \alpha\,\textbf{Twist}, \qquad \textbf{Residual} \;=\; \big|\,\mathrm{Gap} - (\mathrm{Flux} + \alpha\,\mathrm{Twist})\big|$$

and a two-timescale controller—**Flux-gate (fast)** and **Twist-step (slow)**—keeps **Residual** inside band and operates within explicit stability regions.

27.1 State equation (discrete window model)

Index windows by n. Let G_n, F_n, T_n, α_n be Gap/Flux/Twist/α; **Residual** $R_n = G_n - (F_n + \alpha_n T_n)$. A practical evolution (Abelian ops layer) is

$$G_{n+1} = G_n - \big(F_n + \alpha_n T_n\big) + \eta_n, \qquad R_n = G_n - \big(F_n + \alpha_n T_n\big),$$

where η_n lumps sensing/estimation error and latent flux. PBHL **acceptance checks** (two-boundary Stokes, gluing, 4π) certify the identity and bound η_n.

27.2 Controller (fast–slow)

Fast: Flux-gate. Continuous action u_F that pushes on throughput levers (slots, cadence, routing) with PI(+FF) form:

$$u_F(n) = k_P\, e_R(n) + k_I \sum_{i \le n} e_R(i), \quad e_R(n) = \mathrm{sgn}(R_n)\max(|\,R_n\,| - \varepsilon_g, 0).$$

Plant surrogate: $F_{n+1} = F_n + b_F\, u_F(n) + d_F(n)$ with $b_F > 0$. Gains are coherence-weighted to avoid whiplash.

Slow: Twist-step. Discrete, budgeted steps $u_T \in \mathcal{U}$ (reorg, SOP/QA tweak, prompt/policy change) applied hysteretically when $|\,R\,| > \varepsilon_a$ and α is sign-stable:

$$T_{n^+} = T_n + b_T\, u_T, \quad |\Delta T| \le \tau_{\max} \text{ per review, freeze if α-variance high.}$$

Choose the **smallest** u_T that reduces $|\,R\,|$; respect 4π & gluing gates.

α estimation. Ridge on $\Delta G = -F - \alpha T + \epsilon$ over a rolling window; publish CI and freeze Twist if variance widens.

27.3 Linearization & Lyapunov-style sketch

Linearize around an operating point $(G^\star, F^\star, T^\star, \alpha^\star)$. Small-signal residual

$$\delta R = \delta G - \delta F - \alpha^\star \delta T - T^\star \delta\alpha\,.$$

(i) Fast loop (Twist held fixed). With $\delta T = 0, \delta\alpha = 0$, the closed-loop residual under PI satisfies

$$\delta R_{n+1} \approx (1 - b_F k_P)\,\delta R_n - b_F k_I \sum_{i \le n} \delta R_i + \widetilde{\eta}_n,$$

which is stable (and drives $\delta R \to$ small band) when $0 < k_P < 2/\,b_F$ and k_I chosen under standard discrete-time PI bounds (with anti-windup). Take $V_F = \delta R_n^2 + \lambda I_n^2$ as a Lyapunov candidate; V_F decreases for bounded $\widetilde{\eta}$.

(ii) Slow loop (two-timescale separation). Once the fast loop settles to a residual offset R° (due to latent drift or α-mismatch), a Twist-step chooses u_T minimizing $|\,R^\circ - \alpha^\star b_T u_T\,|$ subject to budgets. This yields a monotone drop in $V_T = R^2$ at step instants. Under **Tikhonov separation** (slow step period ≫ fast settling time), the composite $V = V_F + V_T$ decreases until Residual enters the green band.

27.4 Stability regions (operating envelope)

Operate where: (a) **fast** loop poles inside unit circle ⇒ k_P, k_I within PI bounds for observed b_F; (b) **slow** loop period ≥ 4 fast time constants; (c) $|\,\alpha\,|$ CI width below policy; (d) coherence in-band (stabilizes gains). Telemetry: Five-Line KPI + SI/EEI and gates (Gate-R, Gate-α).

27.5 Limits & mitigations

α drift. Coupling changes with seasonality or regime shifts ⇒ widen CI, freeze Twist, re-calibrate α (4π audit), and prefer Flux-only mode temporarily.
Delayed sensing. Windowing/ETL lag acts like dead-time; shrink k_P, add feed-forward from FaceEvents, and lengthen the slow period.
Hidden flux. Persistent Residual with good α implies latent $\widetilde{F}$; enable a Luenberger-style flux observer and attribute Residuals.
Gluing & 4π failures. If invariants fail on merges or frame flips, block Twist and run diagnostics; only act once checks pass.

27.6 Tests & gates (what CI/ops must pass)

- **PBHL check:** $|\text{Gap} - (\text{Flux} + \alpha\text{Twist})| \le \tau_{\text{PBHL}}$ per window (float64, Kahan on edges).
- **Fast-loop regression:** step-response of Residual decreases to band within ≤2 windows over a synthetic flux bump.
- **Slow-loop safety:** hysteresis + $|\Delta T| \le \tau_{\max}$; Gate-α trips on wide CI.
- **Audits:** Stokes, gluing, 4π; freeze Twist on any red; archive evidence bundle.

27.7 Artifacts (you ship)

API. `POST /belt` → PBHL compute + controller suggestions (u_F,u_T) with bands and α-CI; export KPIs/indices with lineage.
Runbooks. Flux-only incident mode; Twist budget report; change-window checklist (pre/during/

post with vault hashes).
Figures. Belt worldsheet & controller loops; stability band overlay.

> **Takeaway.** PBHL turns program control into a disciplined conservation problem. With PI Flux-gates for fast healing, tiny budgeted Twist-steps for slow shaping, and invariant audits (Stokes, gluing, 4π), Residual stays tame—even as α and flux wiggle in the wild.

Chapter 28 — Agent Reliability Suite

Goal

Stress-test an observer (or a small fleet) on **synthetic commuting vs non-commuting scenarios**, quantify **disagreement** and **mis-exec**, and emit **scorecards + trace artifacts** suitable for CI, dashboards, and audits.

28.1 What you'll build (checklist)

- **Fixture generator** for instrument pairs: commuting, non-commuting, and mixed (with an explicit **commute matrix C**).
- **Replicated runner** (A/B[/C]) sharing a ledger; identical Ô policy and tick cadence.
- **Metrics taps**: Agreement on commuting overlaps; Mis-exec split into **PVR** (policy-violation rate) and **TER** (tool-error rate); Slot occupancy/collisions.
- **Scorecards** with Wilson CIs + drift summaries; **trace bundles** (hash-chained).
- **Ablations**: ±Ô, ±slots, ±certificate (expected effect sizes included).

28.2 Fixtures (design)

A. Commuting pair ("readers")
Examples: `kb.retriever.vector` ↔ `kb.retriever.keyword`, read-only sensors, idempotent DB reads. Mark `C[k]=true`. Expect high agreement when records are shared/redundant (SBS).

B. Non-commuting pair ("mutator+reader" or incompatible bases)
Examples: a state-mutating tool between two reads; or qubit toy $Z \leftrightarrow X$ on the same tick. Mark `C[k]=false`. Expect low agreement on intermediates.

C. Mixed battery
Draw mini-scenarios with a controlled **non-commuting exposure (NCE)** fraction so you can observe how disagreement tracks NCE.

Slot stress
Run each scenario under (i) nominal slot budgets and (ii) constrained budgets to provoke **collisions** and back-pressure; verify that allocator invariants (integrality, non-overlap) hold and Ô degrades gracefully.

28.3 Procedure (per scenario)

1. **Seeded setup.** Fix `seed`, set Ô policy, cadence, commute matrix **C**, slot policy (LRU/PRI/NONE), and enable shared ledger.
2. **Run N ticks** on **A** and **B** (optional **C**) with identical inputs; write latched traces (hash-chain). Measure **Agreement** on commuting overlaps with frame-mapping + lag tolerance.

3. **Record Mis-exec:**
    - **PVR:** violations of compatibility/preconditions per tick.
    - **TER:** tool errors/timeouts per tool call.
4. **Slots telemetry:** occupancy, **Slots.Collision**, queue waits; check bands per pool.
5. **Compute CIs:** For proportions (Agreement, PVR, TER, CollisionRate), report Wilson 95% CI; attach drift p-values across windows.
6. **Emit artifacts:** (a) **scorecard.json**, (b) **trace bundle** (A/B/C traces + commute matrix + hash proofs), (c) **diff report** of conflicts with exact `(τ,k)` provenance.

28.4 Metrics (definitions & thresholds)

- **Agreement (commuting overlaps only).**
  Agree $= \frac{\#\text{matches}}{\#\text{overlaps}}$, with NCE tracked separately. Targets: Green ≥ **0.98**, Amber **0.95–0.98**, Red < **0.95**.
- **PVR** (policy-violation rate) and **TER** (tool-error rate). Targets: PVR < **0.5%**, TER ≤ **1.0%** (Amber up to 2%).
- **Slot collisions & waits.** Memory < **0.5%**, Tools < **1%**, Attention ≈ **0%**; P95 waits per pool: mem ≤1, tools ≤2, attention 0.
- **Desync hygiene** (optional meso check): $\Delta\tau \leq 2$ ticks in green; alert ≥3.

28.5 Scorecards (example)

```
{
  "scenario": "commuting_readers_nominal",
  "seed": 4271,
  "ticks": 5000,
  "agreement": {"score": 0.992, "ci95": [0.989, 0.994], "overlaps": 4217, "nce": 0.002},
  "mis_exec": {"pvr": {"k": 7, "n": 5000, "ci95": [0.0006, 0.0024]},
               "ter": {"k": 33, "n": 3287, "ci95": [0.007, 0.013]}},
  "slots": {"memory": {"collisions": 0.003, "p95_wait": 1},
            "tools":  {"collisions": 0.006, "p95_wait": 2}},
  "alerts": [],
  "artifacts": {"traceA": "blob:sha256:…", "traceB": "blob:sha256:…", "commute_matrix": "cm:v1.12"}
}
```

For **non-commuting** fixtures, expect Agreement to drop and **NCE** to account for it; the scorecard must show *why* (conflict keys, instrument order).

28.6 Harness (pseudo-CLI)

```
observerops suite run \
  --scenario commuting-readers \
  --replicas 2 \
  --ticks 5000 \
  --commute-matrix cm:v1.12 \
  --slots-config slots.yaml \
  --seed 4271 \
  --out out/commuting-readers
```

Outputs:

- `scorecard.json` (per scenario)
- `traces/` (A/B[/C] JSONL with hash-chains)
- `agree/` (API responses + conflict diffs)
- `slots/` (occupancy heatmaps, collision logs)

28.7 Negative controls & ablations (prove signal, not noise)

- **No-Ô:** greedy tool order; expect Disagreement↑ 5–10%, latency↑.
- **No-slots:** unbounded parallelism; expect TER↑/timeouts, Δτ↑, latency variance↑.
- **No-ledger / weak SBS:** remove shared records; Agreement decays; NCE unchanged → proves the role of ledger/redundancy.

28.8 Trace artifacts (auditable by design)

Every run bundles:

- **Observer traces** (append-only, latched, hash-chained).
- **/agree proofs** (hash match ratios, redundancy metrics).
- **Slot events** (`Slots.*`) with causality links to `(τ, channel)`.
  These make CI failures explainable and board-safe.

28.9 Acceptance in CI (defaults)

- **Commuting suite:** Agreement lower-CI ≥ **0.98**; PVR < **0.5%**; TER ≤ **1%**; memory/tools collisions in green; $\Delta\tau \leq 2$.
- **Non-commuting suite:** Agreement drops with NCE flagged; runtime must *not* gate on these overlaps (pass if diagnostics match).
- **Ablations:** observed shifts match Table-of-Effects (±Ô/±slots/±certificate) to catch regressions.

28.10 Worked mini-run (commuting→mixed)

- **Commuting** (vector/keyword/cached): redundancy $R \approx 3.6$; Agreement(A,B)=0.94–0.99; green bands on slots; CI passes.
- **Mixed** (+one mutator): NCE rises; Agreement dips on affected keys; harness labels conflicts with `(τ,k)` and tool provenance; CI still passes because non-commuting overlaps are excluded from the score.

28.11 Artifacts you ship

- **Suite config** (YAML): Ô, τ, C, slots, seeds, thresholds.
- **Reference report template**: “Reliability Scorecard” with bands and Wilson ribbons.

- **Example traces & diffs** for unit tests and demos.

  Takeaway. Reliability isn't a vibe—it's commuting design + shared records + slots that hold under stress, proven by scorecards you can re-run and audits you can pass.

Chapter 29 — RAG Pooling Battery

Goal

Build a reproducible battery that stresses RAG pipelines on **controllable order/phase structure**, validates **CWA certificates** (perm/flip/chunk), and reports **accuracy–latency frontiers** plus **pass/fail ROC curves** for certificate thresholds.

29.1 What you'll build (checklist)

- **Phase/Order-tunable corpora** (bags vs coherent chains; boundary-sensitive text).
- **Projection→Certificate→Pool harness** using `/project` and `/pool` with perm/flip/chunk panels + PRI.
- **Aggregators**: fast-path **additive mean/sum** vs **order-aware** (attention/CNN) fallback.
- **Metrics**: task accuracy (EM/F1/nDCG@k), latency, CWA score components, PRI, pass-rate.
- **Reports**: accuracy–latency fronts; CWA threshold sweep → **ROC** (TPR/FPR) against ground-truth "safe-to-add" labels.

29.2 Corpora with controllable phase/order

A. Bag-of-Facts (orderless)
FAQ / glossary snippets split into chunks (uniform policy), minimal positional bleed. Label as *order-insensitive* (expected CWA pass).

B. Alternating Regimes (phase-coded)
Construct passages where evidence alternates by stance/topic (ABAB…); encode with projector known to retain orientation. Label *order-sensitive* (expected sign/reversal failures).

C. Boundary-Sensitive Narratives
Long narratives with sentence-aware vs fixed-width chunkers; jitter boundaries $\pm\varepsilon$. Label *boundary-sensitive* (expected chunk-panel hits).

D. Mixed Sets
Blend (A–C) per batch to probe certificate specificity; record per-batch ground truth (safe/unsafe). Use redundancy (e.g., vector+keyword) to stabilize retrieval.

> Rationale: CWA predicts project→add is sound when projection **erases phase/order**, and unsafe when coherence survives; the battery toggles these regimes explicitly.

29.3 Harness (Projection → Certificate → Pool)

Step 1 — Retrieve (commuting set). Vector + keyword channels to the same pointer; dedup with provenance.

Step 2 — Project. `/project` to produce $V = \{v_i\}$ with policy + meta (seed, dim, chunk policy). SLO p95 ≤ 25 ms.

Step 3 — Certificate. `/pool` with panels `{perm, flip, chunk}` and PRI prescreen; log seeds, CI, drift. Defaults: pass≥0.82, warn≈0.75, PRI≤0.50.

Step 4 — Pool. If pass → mean/sum; else → order-aware (attention/CNN/RNN). Latency SLO: pass≈≤30 ms; fallback≈≤120 ms.

Step 5 — Answer & score. Feed pooled rep to generator/reranker; score task metrics; write **Cert Log** + lineage.

29.4 Test batteries (perm/flip/chunk) & budgets

Use the certificate's three panels with distances to baseline pool; combine into **CWA score** and attach **PRI**. Heuristic panel counts (tunable by latency):
$P = \min(128, \max(32, 8\lceil \log_2(Nd) \rceil))$, $F = \lfloor P/2 \rfloor$, $C = \max(16, \lfloor P/4 \rfloor)$.

- **Permutation:** order scrambles → stability ⇒ order washed out.
- **Sign-flip:** Rademacher masks or PCA-axis flips → orientation sensitivity.
- **Chunk-shuffle:** jitter/merge/split by `chunk_meta`.
  Aggregate with weights $(0.4, 0.3, 0.3)$; bootstrap CI; log drift.

29.5 Metrics & labels

Accuracy. Retrieval EM/F1/nDCG@k or downstream QA F1; pick per domain. **Latency.** wall-time user→answer, plus `/pool` p95. **Certificate.** score + components + PRI + pass/warn/fail. **Pass-rate.** share of batches meeting thresholds.

Ground-truth label (safe-to-add). For a batch, define *SAFE* if additive accuracy is within δ (e.g., 0.5–1.0 F1) of order-aware accuracy; else *UNSAFE*. Use this to compute ROC as you sweep the certificate pass threshold θ.

29.6 Reports you'll produce

A. Accuracy–Latency Fronts. Plot additive-pass vs fallback; aim to shift Pareto down/right with better chunking/redundancy. Include pass-rate and panel costs.

B. CWA ROC Curves. Sweep θ (e.g., 0.70→0.99).

- **TPR:** fraction of *SAFE* batches that pass.
- **FPR:** fraction of *UNSAFE* batches that pass (false greens).
  Add **PRI** gating variants and strict mode overlays.

C. Component Diagnostics. Perm vs flip vs chunk stability to localize failures; tie to projector/chunker choices.

29.7 Baselines & ablations

- **Always-add** (mean w/o certificate): fast, risk high → expect accuracy drop on phase-coded corpora.
- **Always-attention** (no add): stable accuracy, latency higher.

- **CWA-gated** (ours): near-best accuracy with lower latency when pass-rate is high. Quantify expected lift using the additive-macro theory of CWA.

29.8 Example CLI / outputs

```
observerops rag-battery run \
  --corpus bag,altregime,boundary \
  --projector embeddings.e5-large \
  --panels perm=128,flip=64,chunk=32 \
  --theta 0.82 --pri_max 0.50 --delta_safe 0.01 \
  --seeds 5 --out out/rag-battery
```

Artifacts:

- `frontier.png` (accuracy–latency), `roc_cwa.png` (θ sweep),
- `scorecards.jsonl` (per batch), `cert_logs.parquet` (panels/PRI/CI/drift),
- lineage: `projection_id`, `pool_id`, seeds, chunk_meta.

29.9 Acceptance (defaults)

- **Frontier:** CWA-gated within 1–2 pts of always-attention accuracy, with ≥20–40% latency savings on orderless corpora.
- **ROC:** At θ=0.82 & PRI≤0.50, target FPR ≤ 5–10% on phase-coded sets; in **Strict** (θ=0.98, PRI≤0.20), FPR ≤ 2% for high-stakes lanes.
- **Pass-rate:** steady ≥ 75–85% on bag-like domains (after chunker tuning/redundancy).

29.10 Operator guidance

1. **Treat chunking as instrument design**; align boundary policies to reduce chunk sensitivity and raise pass-rate.
2. **Tune panel budgets** to your SLO using the provided heuristic; raise to Strict for safety-critical verticals.
3. **Log lineage & seeds**; certificate artifacts are audit objects, not debug scraps.

   Takeaway. The RAG Pooling Battery turns “should we average these vectors?” into an empirical, per-batch decision—with accuracy–latency curves you can ship and a ROC you can defend.

Chapter 30 — BeltOps Trials

Goal

Simulate initiatives as **program belts** (OKR loops), perturb **Flux** and **Twist**, and verify that PBHL controllers keep **Residual** in band while improving **EEI/SI**. Output board-safe scorecards (lift, incidents, time-to-green) plus signed evidence bundles.

30.1 What you’ll build (checklist)

- **OKR Belt Simulator** (discrete windows): generates Gap/Flux/Twist with controllable shocks; estimates **α**; runs **Flux-gate** (fast) and **Twist-step** (slow).
- **Gates & Incidents**: Gate-R, Gate-α, sync/capacity nudges; Residual incident playbook automation.
- **KPIs & Indices**: Five-Line KPI + **EEI/SI**; acceptance bands and rollups.
- **Evidence Vault Export**: PBHL, 4π, gluing checks + signed KPI/decision logs.

30.2 Design (OKR sims; perturb Flux/Twist; maintain Residual)

Worldsheet model. Each run simulates windows $n = 1..N$ with $G_{n+1} = G_n - (F_n + \alpha_n T_n) + \eta_n$, Residual $R_n = |\, G_n - (F_n + \alpha_n T_n)|$. Controllers act every window; PBHL audits bound η_n.

Flux perturbations ("FaceEvents"). Inject positive/negative shocks (unblockers, incidents) and capacity moves. Flux-gate maps control u_F into slot/cadence/priority changes (PI gains as in Ch.21).

Twist perturbations ("TwistFrames"). Apply reframes (scope/ownership/policy). Twist-step uses hysteresis and a rate budget $|\Delta\text{Twist}| \le \tau_{\max}$. **α** is re-estimated via ridge on ΔG vs (F, T) each few windows.

Gates & incidents. Gate-R on Residual red; Gate-α on α-variance; desync/capacity gates may throttle cadence. Residual Incident playbook sequences throttle→rollback→diagnose with exportable snapshots.

Audits per run. Stokes (two-boundary), **gluing** (interior-edge cancel), **4π** periodicity; block releases on any red.

30.3 Procedure (per trial)

1. **Define OKR belt** (ID, targets, review cadence, bands). Seed Gap_0, Flux_0, Twist_0, α_0.
2. **Sample shocks**: draw Flux shocks (incidents, blockers) and Twist proposals (reorgs, policy edits) with controllable intensity & frequency.
3. **Simulate N windows** with PBHL controller on/off (A/B): log G, F, T, α, ρ, R and controller outputs u_F, u_T.
4. **Run gates** (CWA/PRI optional), incidents, and playbooks; capture signed **Gate.Decision** logs.
5. **Audit & export**: PBHL residual, gluing histogram, 4π class functions, KPI rows, decisions, lineage → evidence bundle.

30.4 Metrics & outputs

Primary KPIs.

- **Residual** bands (P50/P95; time-in-green), **Mean Time to Green (MTTG)** after a red event.
- **EEI** (effectiveness/execution) and **SI** (sustainability) lifts vs baseline (controller-off). Targets:

QoQ +10% typical.

Incident analytics. Residual incident rate (#/qtr), **Gate-R** count, rollbacks, α reviews, false-open/false-close of gates.

Board scorecard (per trial). Current EEI/SI, Residual band & α, audit pass-rate, incidents & actions—matching the belt board one-pager.

30.5 Bench setups (scenarios you'll ship)

- **S1 Positive closure** (low shocks): verify Flux-gate alone shrinks Residual to green; measure EEI lift without Twist changes.
- **S2 Twist spike** (reorg): α flips or widens CI; Gate-R fires; Twist-step freezes and recovers; report MTTG and SI dip/recovery.
- **S3 Governance belt** (compliance vs growth): couple product & governance belts; perturb policy tests; ArbitrationBelt decisions appear in logs.

30.6 Reports (what you publish)

- **EEI/SI lift plot** (controller-off vs on) with confidence bands.
- **Residual maintenance chart**: bands, incidents, Gate-R/α markers, MTTG annotations.
- **Audit panel**: PBHL residual histogram, gluing residuals, 4π check; all green required for "release OK".

30.7 Acceptance (defaults)

- **PBHL**: $|R| \le \varepsilon_g$ 70%+ of windows, P95 ≤ ε_a; audits pass (gluing ≤ τ_glue; 4π ok).
- **EEI/SI**: ≥ **+10%** EEI and **SI ≥ 0.70** sustained 2 weeks after a red event.
- **Incidents**: Residual red MTTG ≤ 2 review windows; incident rate trending down run-over-run.

30.8 CLI (example) & artifacts

```
observerops beltopstrials run \
  --belt-id ai-helpdesk-2025Q4 \
  --windows 84 --seed 42 \
  --flux-shocks "rate=0.25,scale=0.12" \
  --twist-proposals "rate=0.10,scale=0.06,limit=0.02/wk" \
  --controllers "flux_gate=on,twist_step=on" \
  --gates "R,alpha,rho" --export out/helpdesk-q4
```

Artifacts:

- `kpi.parquet` (Gap/Flux/Twist/ρ/Residual), `decisions.jsonl` (Gate.*), `audit.pbhl.json` (PBHL/gluing/4π), `board_onepager.md`, signed manifest—ready for GRC.

30.9 Operator guidance

1. **Drive Residual, not raw velocity**—EEI guards against vanity Flux; keep evidence exports green.
2. **Budget Twist**—freeze when α is uncertain; small, infrequent steps stabilize SI.
3. **Always ship audits**—PBHL + gluing + 4π make results defendable in review and regulatory contexts.

   Takeaway. BeltOps Trials turn PBHL from a slogan into numbers: you perturb Flux/Twist, controllers keep Residual tame, and EEI/SI climb—backed by audits you can sign.

Chapter 31 — Adversarial & Stress

Goal

Probe ObserverOps under worst-case conditions—**coherent sequences**, **feedback loops**, and **saturation forcing**—and verify graceful degradation (**safe-mode transitions**) and **recovery time** back to green bands.

31.1 What you'll build (checklist)

- **Adversary generators:** phase-coded/coherent inputs; closed-loop perturbations that induce scheduling thrash; bursty load for allocator saturation.
- **Degradation harness:** exercises D0→D3 safe states and back, with Tick & Sync anchors and gate actions wired.
- **Scorecards:** recovery times (TTG/MTTG), number/duration of safe-mode episodes, incident counts, and evidence bundles (traces, cert logs, gate decisions).

31.2 Adversaries (design)

A) Coherent sequences (CWA breaker).
Construct alternating-regime batches (ABAB…) or phase-encoded time series so order/orientation survives projection. Expect low sign/reversal stability, high PRI, and forced fallback. This validates certificate sensitivity.

B) Feedback loops (Ô thrash).
Close the loop from outputs→context so each answer perturbs the semantic field and flips the next channel choice. Drive oscillatory S_c and AL behavior to test scheduling stability and latching discipline under re-entry pressure. Use Control→Data feedback path to throttle when gates fire.

C) Saturation forcing (allocator & sync).
Burst tool calls to push pools beyond 0.9 occupancy; provoke **Slots.Collision**, grant-latency inflation, and desync ($\rho \downarrow$, $\Delta\tau \uparrow$). Expect entry into D1/D2/D3 and measured recovery with anchors and narrowed schedules.

D) Semantic black-hole drill (field lock-in).
Warp the field toward a dominant attractor (high AL, falling S_c) and verify entropy-floor guards and exploratory nudges prevent collapse lock-in.

31.3 Procedure (per scenario)

1. **Seed & configure.** Set Ô policy, cadence, slot policies, certificate thresholds, and degradation bands; enable shared ledger.
2. **Run N windows/ticks** while streaming adversary inputs (A–D). Emit `/project→/pool` cert logs (with seeds/PRI) and all Tick/Slot events.
3. **Allow gates to act.** On sync/capacity or certificate risk, policy gates throttle/block; Tick & Sync widens cadence, restricts to commuting-safe, or pauses. Log `Degrade.Enter/Exit`.
4. **Recover.** Maintain anchors until $\rho \geq 0.9$, $\Delta\tau \leq 2$, occupancy<0.7 for ≥30 s; reopen schedules and return to D0.
5. **Export evidence.** Bundle traces, cert logs, gate decisions for audit.

31.4 Metrics & outputs

- **Recovery time to green (TTG / MTTG):** time from first Red to band recovery (micro: per metric; macro: Residual). Report distribution and p95.
- **Safe-mode transitions:** counts and dwell times in D1/D2/D3; exit criteria met (ρ/Δτ/ occupancy).
- **CWA behavior:** pass-rate, score components, PRI; fallback rate and added latency under coherent adversaries.
- **Allocator stress:** collision rate, grant p95, occupancy bands, queue waits.
- **Incidents:** Residual incidents, gate actions, rollbacks—linked to evidence IDs.

Scorecard (example fields).

```
{ scenario, seed, ttg_s, d1_count, d2_count, d3_count, dwell_s:{d1,d2,d3}, cwa:{pass_rate, pri_p95}, slots:{collisions, grant_p95}, sync:{rho_min, dtau_p95}, incidents:{residual, gates} }
```

31.5 Acceptance (defaults)

- **Coherent sequences:** CWA fails fast (score ≤ θ or PRI>τ) and *fallback holds task accuracy* within δ while latency stays ≤ fallback SLO. No additive pooling in Red.
- **Feedback loops:** Ô entropy oscillations damp under throttling; latching remains intact (no retro-edits); no re-entrancy violations.
- **Saturation forcing:** automatic D1/D2/D3 entries match thresholds; p95 recovery to D0 within ≤ 60–120 s for configured burst; ρ/Δτ return to green.
- **Black-hole drill:** entropy-floor guard trips only with saturation rise; AL reduces after exploration kick; no long-lived lock-in.

31.6 Worked adversaries

A→FAIL, fallback OK. ABAB sequence: CWA≈0.82, PRI≈0.55 → order-aware pooling engaged; accuracy parity within 1–2 pts; +70–100 ms latency.

C→D2/D3 spiral, then recover. Burst drives mem occupancy to 0.92, collisions 12/min; Gate throttle + commuting-only + anchors q=500 ms → after ~45 s, occupancy 0.74, ρ≥0.9, exit to D0.

31.7 CLI & artifacts

```
observerops stress run \
  --adversaries coherent,feedback,saturation,blackhole \
  --ticks 20000 --seed 2025 \
  --gates "rho,delta_tau,capacity,cwa" \
  --bands d1=gentle,d2=strict,d3=quiescent \
  --out out/stress-2025Q4
```

Artifacts:

- `scorecards.jsonl` (TTG, dwell, incidents),
- `traces/*.jsonl` (hash-chained), `cert_logs.parquet`,
- `gates/decisions.jsonl` + `Degrade.Enter/Exit`,
- signed manifest for GRC vault.

31.8 Operator guidance

1. **Let gates breathe.** Don't pin thresholds; D1→D2→D3 progression stabilizes loops faster than manual firefighting. Exit strictly on ρ/Δτ/occupancy greens.
2. **Treat coherence as a feature, not a bug.** If coherent chains are common, keep fallback warm and measure ROC/pass-rate; don't force add.
3. **Practice lock-in escapes.** Run semantic black-hole drills; watch S_c with saturation and nudge exploration to avoid attractor traps.

   Takeaway. Under adversaries, ObserverOps should degrade *predictably* (safe modes) and come back *quickly* (measurable TTG)—with audit-grade artifacts that show what happened and why.

Chapter 32 — Ablations & A/B

Goal

Quantify what each ObserverOps invariant actually buys you. Run controlled A/B slices—**±Ô (scheduler)**, **±slots (allocator)**, **±certificate (CWA)**, **±belts (PBHL controllers)**—and report **effect sizes** with actionable **guidance**.

32.1 What you'll build (checklist)

- **Traffic replayer** to feed identical workloads across variants (seeded).
- **Toggleable stack**: Ô on/off, slot allocator on/off, CWA on/off, BeltOps on/off.
- **Score taps**: Agreement/Disagreement, Mis-exec (PVR/TER), Δτ & ρ, Slot collisions, CWA scores/PRI, PBHL Residual & EEI/SI.
- **Report generator**: summary tables with Wilson CIs + drift, plus trace/cert/belt evidence bundles.

32.2 Slices (design)

Run 3× week-long (or K-epoch) A/Bs per slice against the **Baseline** (Ô+slots+certificate+belts):

1. **No-Ô** (greedy order; no compatibility-aware scheduling).
   Expect **Disagreement↑ 5–10%**, latency↑ via conflicts; NCE exposure rises when mutators slip between reads. Guidance: restore commutation-first plans and frame maps.

2. **No-slots** (unbounded parallelism; no allocator).
   Expect **Mis-exec↑ (timeouts)**, **Δτ↑**, E2E latency variance↑; collisions become "ghost" errors since no S4 observability. Guidance: re-enable S1–S4 (integrality, non-overlap, explicit eviction, observability).

3. **No-certificate** (always add; no CWA).
   Expect accuracy↓ on coherent/boundary-sensitive corpora even though latency↓ (skips fallback). Guidance: re-enable /pool certificate (perm/flip/chunk) + PRI gating.

4. **No-belts** (disable PBHL controllers; no Flux-gate/Twist-step).
   Expect **Residual time-in-green↓**, MTTG↑ after shocks; EEI/SI drift. Guidance: re-enable PBHL identity Gap≈Flux+α·Twist, with gates and audits (4π, gluing).

32.3 Metrics & effect sizes (what to measure)

Slice	Primary deltas (typical direction)	Why it moves
No-Ô	Disagreement +5–10%; NCE↑; latency↑	Loss of commute-aware order & shared-ledger alignment → AB-fixedness breaks more often.
No-slots	PVR/TER↑; Δτ↑; latency variance↑; collisions untracked	Without S1–S4, contention becomes invisible; bursts cause tool storms & desync.
No-certificate	Accuracy↓ on phase/boundary sets; latency↓	Project→add used where order/phase survive; certificate would have failed and triggered fallback.
No-belts	Residual↑; time-in-green↓; EEI/SI↓; incidents↑	No Flux/Twist control to close Gap; α unmanaged; audits missing.

Definitions & taps:
Agreement on commuting overlaps (with frame mapping + hash proofs); PVR/TER from runtime; Δτ & ρ from Tick & Sync; slot collisions/queues from `Slots.*`; CWA score & PRI from `/pool`; PBHL Residual, EEI/SI via `/belt`.

32.4 Procedure (per slice)

1. **Pin config** (commute matrix C, seeds, cadence, slot policy, CWA thresholds, belt bands).

2. **Replay workload** to Baseline and Variant; collect traces, cert logs, belt KPIs.

3. **Compute deltas** with Wilson CIs; attach drift and per-metric p-values.

4. **Export evidence bundle** (hash-addressed manifest; lineage linking pool/agree/cert/belt IDs).

32.5 Example CLI

```
observerops ablate run \
  --slices noO,noSlots,noCert,noBelts \
  --ticks 100k --seed 2025 \
  --commute-matrix cm:v1.12 \
  --slots-config slots.yaml \
```

```
  --cwa "theta=0.82,pri_max=0.50,perm=128,flip=64,chunk=32" \
  --belt "eps_g=0.05,eps_a=0.10" \
  --out out/ablate-2025Q4
```

Outputs:

- `effect_sizes.json` (per metric with CI),
- `scorecards/*.json` (baseline vs variant),
- `traces/*.jsonl` (hash-chained), `cert_logs.parquet`, `belt/kpi.parquet`,
- signed `manifest.sha256`.

32.6 Result template (sample)

```
{
  "slice": "noCert",
  "accuracy": {"delta": -0.021, "ci95": [-0.028,-0.014]},
  "latency_ms": {"delta": -38, "ci95": [-45,-30]},
  "cwa": {"pass_rate": 0.00, "pri_p95": 0.56},
  "notes": "Coherent chains present; certificate would have failed; fallback protected accuracy."
}
```

For **noÔ**: `disagreement_delta=+0.067 [0.048,0.086]`, `nce_delta=+0.012`.
For **noSlots**: `ter_delta=+0.009`, `dtau_p95=+3`, `latency_var_x1.7`.
For **noBelts**: `residual_time_in_green=-0.23`, `eei_delta=-0.08`, `si_delta=-0.09`. (Illustrative.)

32.7 Guidance (what to change when a slice "wins")

- **If No-Ô looks "faster"** but disagreement/latency variance rise: restore Ô; encode commute matrix & frame maps; keep the shared ledger to recover AB-fixedness.
- **If No-slots looks "throughput-y"**: it's masking contention. Re-enable S1–S4; use LRU for memory, PRI for tools, NONE for attention; watch collisions & queue P95.
- **If No-certificate "improves" latency**: you're taking hidden accuracy debt on coherent/boundary sets—turn CWA back on and tune chunking/projector.
- **If No-belts seems "simpler"**: Residual/EEI/SI will drift; re-enable Flux-gate/Twist-step; keep α under audit with 4π & gluing checks.

32.8 Artifacts you ship

- **Ablation config** (YAML) mirroring Ch.14's template (Ô policy, CWA thresholds, slot pools, belt bands).
- **Effect-size table** (CSV/MD) for board packs.
- **Evidence bundle**: traces + cert logs + belt KPIs + manifest; reproducible seeds & thresholds.

  Takeaway. The ablations make the case quantitatively: Ô protects agreement, slots tame contention, certificates prevent unsafe pooling, and belts close the loop. Keep all four on for production—and use this suite to prove it, every release.

© 2025 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.

ObserverOps Technical Blueprint - VII-XI

Part VII — Case Studies (Illustrative)

Chapter 33 — Support RAG at Scale

Setting: Internal KB search with CWA gating
Result: Latency ↓ 20–40% at iso-accuracy; audit artifacts exportable to GRC
Artifacts: Before/after dashboards; configs; trace/certificate logs

---

Goal (1 sentence)

Show how **projection-first + CWA-gated pooling** accelerates enterprise RAG without losing answer quality, while emitting compliance-ready audit trails.

System Snapshot (what you'll implement)

- **Projection-first retriever**: compute additive, phase-insensitive features via `/project`.
- **CWA certificate battery**: permutation / sign-flip / chunk-shuffle tests → **CWA score** ∈ **[0,1]**.
- **Gate & fallback**: if `score ≥ θ` ⇒ fast additive **/pool**; else fall back to order-aware aggregator (cross-encoder/attention).
- **Ô/τ discipline**: Ô picks projector/instrument; τ enforces commit cadence & latching to trace **T**.
- **Telemetry & exports**: traces, cert logs, dashboards; nightly **GRC export** (hash-chained, redacted).

Architecture (ObserverOps lens)

- **Data plane**: `query → /project → certificate → /pool (additive or fallback) → answer.`
- **Control plane**: **Ô** schedules instrument (projector) per domain; **τ** governs commit ticks; **slot allocator** caps concurrent retriever/reranker slots.
- **Audit plane**: immutable **trace T**, **certificate ledger**, **policy-gate events**, GRC export jobs.

Key concepts & invariants (how they show up here)

- **Internal collapse**: every `TraceWrite(τ_k)` (retrieval set, projector ID, cert score, policy decision) is immutable in-frame; downstream steps condition on it.
- **CWA (Collapse Without Alignment)**: only **project→add** when cert passes; otherwise, preserve order/phase via attention/cross-encoder.
- **Slot conservation**: explicit budgets for (a) retriever candidates, (b) reranker windows, (c) summarizer contexts; collision logging when pressure rises.
- **Agreement hooks**: optional *replica agreement* on top-k IDs or answers for critical queues (SBS-like redundancy).

Reference flow (pseudo-code)

```
def answer(query, policy):
    τ = tick.start()
```

```
    T = trace.open(query_id)

    # 1) Retrieve candidates (bounded by slots)
    cand = retriever.topk(query, k=policy.k_retrieve)

    # 2) Project to additive invariants (Ô chooses projector)
    proj = project(cand, projector=policy.projector)            # /project
    trace.write(τ, "Project", {"projector": policy.projector, "n": len(proj)})

    # 3) Run CWA certificate tests
    cert = cwa.certify(proj, tests=policy.tests)                # perm / flip / chunk
    trace.write(τ, "CWA", {"score": cert.score, "panels": cert.panels})

    # 4) Gate → fast pool or order-aware fallback
    if cert.score >= policy.min_score:
        pooled = pool.additive(proj)                            # /pool (fast)
        path = "additive"
    else:
        pooled = pool.attentive(cand, window=policy.window)     # fallback
        path = "fallback"

    trace.write(τ, "Pool", {"path": path, "dim": len(pooled)})

    # 5) Reader/generator; optional replica agreement
    answer, ev = reader.generate(query, pooled)
    trace.write(τ, "Answer", {"hash": hash(answer), "evidence": ev.ids})

    tick.end(τ); audit.flush(T)
    return answer, T, cert
```

Policy config (production YAML)

```
cwa-engine:
  projector: "whitened-sif-1024"        # additive invariant after projection
  tests:
    permutation: { panels: 50, tolerance: 0.02 }
    sign_flip:   { panels: 50, tolerance: 0.02 }
    chunk_shuffle:
      chunker:  "semantic-256"
      panels:   40
      tolerance: 0.03
  min_score: 0.78                        # θ; auto-tuned nightly
  fallback: { type: "cross-encoder", model: "ce-msmarco-v2", window: 32 }

retriever:
  k_retrieve: 200
  dedupe:     true
  cache:      { slots: 8, policy: "LRU", ttl_s: 900 }

reader:
  model: "gpt-x-internal"
  max_ctx: 64_000
  cite_mode: "evidence-ids"

telemetry:
  export:
    grc:
      schedule: "0 2 * * *"
      format: "jsonl"
      fields: ["qid","τ","projector","cwa.score","pool.path","evidence.hash"]
  privacy:
    redact: ["pii.email","pii.employee_id"]

gates:
  block_on:
    phase_risk_index: { gt: 0.6 }
    cert_drift:       { p_value_lt: 0.01 }
```

Worked example (internal KB, “SupportOps” queue)

Workload. 120K internal articles (policies, SOPs, tickets), 5 languages, average 1.6K tokens/doc. Live queue ~40 QPS (business hours), p95 answer SLA 2.0 s.

Baseline. *Naive mean pooling* of section embeddings → cross-encoder rerank of top-200 → reader.
ObserverOps upgrade.

1. Ô selects `whitened-sif-1024` projectors for policy/SOP domains; switches to `topic-lda` projector for long procedural chains.
2. Certificate battery runs with 50× perm/flip panels + 40× chunk-shuffle; `θ=0.78`.
3. If pass (≈65–80% of queries in this domain), pool additively and **skip cross-encoder**; else run cross-encoder on compressed set (window 32).
4. All decisions and scores latched to **T** at tick τ.

Observed outcomes (illustrative)

KPI	Baseline	With CWA gating	Delta
p50 latency (s)	1.35	0.95	−30%
p90 latency (s)	2.10	1.50	−29%
Answer accuracy (exact/F1 proxy)	78.8 / 84.1	79.0 / 84.0	≈ iso-accuracy
Cost/1k queries (normalized)	1.00	0.72	−28%
CWA pass-rate	—	72%	—
False-green est. (holdout)	—	1.7%	within guardrail
Trace export completeness	0%	100%	+100%

*Latency reduction consistently landed in the **20–40%** band across queues with similar content heterogeneity; accuracy stayed within noise bands given certification gates and fallback.*

Tests & thresholds (what to measure, when to escalate)

- **CWA Score (0–1)**: green ≥ 0.78, amber 0.70–0.78 (allow if Phase-Risk ≤ 0.35 and Δaccuracy CI ≤ 0.5%), red < 0.70.
- **Phase-Risk Index**: green ≤ 0.35; if > 0.60, route all queries to fallback for 30 min and trigger cert re-fit.
- **Agreement Rate (optional replicas)**: ≥ 0.92 on evidence IDs for critical queues.
- **Trace Immutability**: 100% passes on hash-chain verification; zero retro-edits without new τ.
- **Slot Occupancy**: retriever slots ≤ 85% p95; collision alarms above 5% minute-over-minute.
- **Drift Watch**: daily permutation panel KS-test; alarm if `p < 0.01`.
- **SLA Gate**: if p95 latency > 2.0 s for 5 minutes, temporarily lower `θ` by 0.02 only if accuracy CI is stable; auto-revert after 30 minutes.

Failure modes & mitigations (case-specific)

- **False green certificate** on rare coherent chains → *Mitigation:* add **chunk-shuffle** with semantic chunker; raise perm panels; add short cross-encoder on top-M (=8) even on pass for *guard-band* answers.
- **Slot starvation** under burst → *Mitigation:* priority-tier slots (VIP vs bulk), back-pressure on retriever k, dynamic `k= f(queue_depth)`.
- **Domain shift** (new policy templates) → *Mitigation:* nightly projector re-whitening; θ autotune via holdout; belt gate on **cert_drift**.
- **Privacy leakage** in exports → *Mitigation:* redaction map + field-level hashing; downstream GRC only gets evidence hashes and projector IDs, no raw content.

Before/After dashboards (panels you'll ship)

1. **Latency vs CWA score** scatter with pass/fail coloring; p50/p90 overlays.
2. **Accuracy parity** chart (A/B): Δexact/ΔF1 with 95% CI across cohorts.
3. **Certificate health**: pass-rate, Phase-Risk, drift p-values; panel counts.
4. **Pool path mix**: additive vs fallback over time; cost/throughput overlays.
5. **Trace integrity**: hash-chain pass rate, retro-edit alarms, export success.
6. **Slot heatmap**: occupancy/collision by component (retriever/reranker/reader).

GRC export (example JSONL record)

```
{
  "qid": "SUP-2025-09-23-14:03:11Z-8f2a",
  "tau": 3821441,
  "projector": "whitened-sif-1024@v3.7",
  "cwa": { "score": 0.83, "panels": {"perm":50,"flip":50,"chunk":40} },
  "pool": { "path": "additive", "reason": "score>=0.78" },
  "evidence": { "ids": ["kb:HR-2024-17#3","kb:SOP-119#2"], "hash": "b3c1…e9" },
  "trace_hash": "aa94…55",
  "policy_version": "rag-support-00.78-2025-09-20",
  "privacy": { "redactions": ["pii.email","pii.employee_id"] }
}
```

Ops runbook (excerpt)

- **Daily 02:00 UTC**: projector re-whitening; θ autotune from holdout; drift test; export GRC logs.
- **On alarm** `phase_risk_index>0.6`: route to fallback; run 200 extra perm panels; compare Δaccuracy on 500-query sample; update θ if CI stable.
- **Quarterly**: belt review (PBHL Residual), cert ROC refresh with new content verticals.

Repro checklist (to make this portable)

- Dataset split with **phase-sensitive** vs **phase-insensitive** cohorts.
- Harness implements `/project`, `/pool`, `/trace/:id` with latching.
- At least **50×50** perm/flip panels and **40×** chunk-shuffle per domain; store raw panel stats.
- Two readers: (a) additive-pooled fast path; (b) cross-encoder fallback.
- Metric suite: latency quantiles, accuracy parity (Δ exact/F1 with CI), cert pass-rate, false-green est., cost, slot occupancy.

What this chapter delivers

- A **drop-in pattern** (projection-first + CWA gating) that reliably wins **20–40% latency** at **iso-accuracy** on internal KB RAG.
- **Operational clarity**: explicit gates, thresholds, and dashboards tied to ObserverOps invariants.
- **Auditability**: full trace and certificate artifacts ready for **GRC** workflows.

Artifacts packaged

- **Configs**: the YAML above (θ, panels, gates, privacy).

- **APIs**: `/project`, `/pool`, `/trace/:id` payload examples.
- **Dashboards**: panel specs (Latency×Score, Accuracy Parity, Cert Health, Mix, Integrity, Slots).
- **Exports**: JSONL schema for GRC; hashing/redaction rules.

Chapter 34 — Tool-Using Agents Fleet

Setting: Agreement checks + SBS redundancy; slot budgets; τ sync
Result: Contradictions halved; reproducibility ↑
Artifacts: Trace exemplars; rollout/runbook playbook

Goal (1 sentence)

Operationalize **cross-observer agreement** in a production fleet of tool-using agents by adding *commutativity-aware quorums, SBS-style shared records, slot budgets,* and *τ synchronization*—cutting contradictions while making runs reproducible and auditable.

System Snapshot (what you'll implement)

- **Agreement engine**: `agree(T_a, T_b)` with a **commute matrix C** and SBS redundancy check; quorum rules for answers & tool effects.
- **SBS redundancy**: externalized pointer records (shared ledger / replicated cache) to stabilize objectivity across replicas.
- **Slot budgets**: integer caps for tools (search, DB, math, code, CRM), eviction policy, collision logging.
- **τ sync**: cadence and phase alignment across replicas; desync alerts (Δτ) and fleet order parameter (ρ).
- **Latching**: every tool call & decision is committed to trace **T** at tick τ; no in-frame retro-edits.

Architecture (ObserverOps lens)

- **Data plane**: `plan → tool calls → outcomes → shared record write (SBS) → vote/agree → answer.`
- **Control plane**: **Ô** schedules tools respecting **C**; **τ** ticks gate retries and quorums; **slots** bound concurrency.
- **Audit plane**: per-replica immutable **trace T**, **agreement verdicts**, SBS shared-record hashes, sync metrics (ρ, Δτ).

Key concepts & invariants (where they bite)

- **Cross-observer agreement**: replicas only vote on measurements from *commuting* tool effects; non-commuting paths get serialized or isolated.
- **SBS redundancy**: write "pointer" facts (IDs, hashes, coarse summaries) to a **shared, append-only store** to converge on the same effective outcome.
- **Slot conservation**: fixed tool budgets (e.g., 3×search, 1×code, 1×DB) prevent contention-induced nondeterminism.
- **τ sync**: replicas advance commits at aligned ticks; re-entrant work requires a *new* τ (prevents hidden race edits).

Reference flow (multi-replica agent ensemble)

```
def fleet_answer(query, policy):
    τ = tick.start()
    R = spawn_replicas(n=policy.replicas)

    # 1) Each replica plans and selects a commuting tool per Ô and matrix C
    for r in R:
        r.pi = Ohat.select_channel(r.S, r.T, commute_matrix=policy.C)

    # 2) Execute tools with slot budgets
    with slots.scope(policy.slot_budget):
        outcomes = parallel([measure(r.pi) for r in R])  # tool outcomes

    # 3) SBS write: replicas publish pointer records to shared store
    sbs_ids = [sbs_publish(o.pointer) for o in outcomes]

    # 4) Agreement vote (only on commuting effects + shared pointers)
    verdict = agree([r.T for r in R], commute_matrix=policy.C, sbs_ids=sbs_ids)
    trace.write(τ, "Agreement", {"verdict": verdict.passfail, "score": verdict.score})

    # 5) If fail, branch: serialize conflicting tools or route to arbiter
    if not verdict.passfail:
        fix = resolve_conflict(policy, outcomes)  # serialize or swap instrument
        trace.write(τ, "Conflict.Resolve", fix.summary)

    # 6) Latch decision & generate answer
    ans = reader.compose(query, shared_evidence=sbs_ids, verdict=verdict)
    trace.write(τ, "Answer", {"hash": hash(ans), "evidence": sbs_ids})

    tick.end(τ)
    return ans, verdict
```

Fleet policy (production YAML)

```
agreement:
  replicas: 3
  quorum:
    mode: "2-of-3"                # pass if ≥2 replicas match pointer/effect
    min_score: 0.85               # agree() score threshold
  commute_matrix:
    Search↔DB: true
    Search↔WebScrape: false       # non-commuting (mutates caches/session)
    CodeExec↔DB: false
    Math↔Search: true
  sbs:
    store: "append-only-log@v2"
    pointer_fields: ["doc_id","section","sha256_128","ts_bucket"]

slots:
  budgets:
    Search:   3
    DB:       1
    CodeExec: 1
    Math:     1
  eviction: { policy: "priority-lru", promote: ["DB","Math"] }
  alarms:   { collision_rate_gt: 0.05 }

sync:
  tau:
    cadence_ms: 150
    max_skew_ms: 40
  kuramoto:
    rho_green: 0.88
    rho_red:   0.72
  desync_gate:
    if_delta_tau_ms_gt: 60
    action: "pause-new-work-and-resync"

latching:
  hash_chain: true
  forbid_retro_edit: true
  require_new_tau_on_retry: true
```

Worked example (Fleet "Billing-Resolver")

Workload. Agents triage billing tickets by querying CRM, ledger DB, and policy KB; tools: `Search`, `DB.Query`, `Math`, optional `WebScrape`.
Baseline. Single agent, unconstrained tools; occasional contradictions (agent A refunds, later agent B denies).
ObserverOps fleet.

1. **Commutativity discipline**: `DB.Query` and `WebScrape` marked non-commuting; Ô chooses `Search+Math` first across replicas; serialize `DB.Query` if needed.
2. **SBS pointers**: replicas publish `(ticket_id, policy_doc_id, section, sha)` before composing the answer.
3. **Quorum**: 2-of-3 replicas must agree on *pointer tuple* and *effect class* (refund/deny/clarify).
4. **τ sync**: replicas advance every 150 ms; retries force new τ.
5. **Slots**: 3×`Search`, 1×`DB`, 1×`Math`—reduces tool collisions.

Observed outcomes (illustrative)

KPI	Before (single)	After (fleet)	Delta
Contradiction rate (per 1k tickets)	42	19	−55%
Replica agreement score (mean)	—	0.91	—
Reproducibility (rerun same query)	0.73	0.92	+0.19
P95 resolution latency (s)	3.4	3.7	+0.3
Escalations due to tool conflict	—	3.2%	—
Trace hash-chain pass rate	—	100%	—

> Slight latency increase (+0.3s at p95) pays for *halved contradictions* and **+19pp reproducibility**; acceptable within SLA.

Tests & thresholds (acceptance & alarms)

- **Agreement score** ≥ `0.85` (green), `0.80–0.85` (amber—require human review for high-impact tickets), < `0.80` (red—auto-serialize tools or escalate).
- **Contradiction rate ≤ 20 / 1k** over 7-day window; page if 3-day EMA > 24.
- **Reproducibility** (same input, fresh run, same pointers/effect): ≥ **0.90**.
- **Sync health**: ρ ≥ 0.88 (green); Δτ p95 ≤ 40 ms.
- **Slot collisions** ≤ 5%/min; throttle `Search` fan-out if exceeded.
- **Trace integrity**: 100% hash-chain; zero retro-edit events (else block and mint new τ).

Failure modes & mitigations

- **Hidden non-commuting side-effects** (e.g., `WebScrape` mutates session): *Mitigation:* mark tool non-commuting; force serialization; sandbox cookies.
- **Replica divergence from nondeterministic tools** (e.g., time-varying APIs): *Mitigation:* freeze response via SBS pointer snapshots; add time-bucketed hashes.
- **Slot starvation during bursts**: *Mitigation:* back-pressure + priority tiers (customer-impacting > backlog); dynamic slot shrink for noisy tools.
- **Desync storms** (network jitter): *Mitigation:* desync gate trips → pause new work, resync τ, drain in-flight with old τ, then resume.
- **Quorum over-veto** (too strict): *Mitigation:* risk-tiered quorums (3-of-3 for refunds > $X; 2-of-3 for

clarifications).

Before/After dashboards (ship these panels)

1. **Contradictions over time** with SLA band and step-changes when commute rules updated.
2. **Replica agreement heatmap** (by tool pair; shows non-commuting hotspots).
3. **τ sync panel**: ρ, Δτ quantiles, desync incidents.
4. **Slots**: occupancy & collisions by tool; alarms and throttling actions.
5. **SBS ledger integrity**: pointer write rate, duplication, hash mismatch.
6. **Quorum outcomes**: pass/amber/fail distribution with business impact overlay.

Trace exemplars (redacted)

```
τ=918221  Replica=A  Tool=Search   Query="invoice 88412 policy"  -> hits=[KB#P-77#2 sha=ab19..]
τ=918221  Replica=B  Tool=Search   Query="refund threshold"      -> hits=[KB#P-77#2 sha=ab19..]
τ=918221  SBS.Write  pointer=(ticket=88412, doc=P-77, sec=2, sha=ab19.., tsb=2025-09-23T14:03)
τ=918221  Agree.Pass score=0.92 quorum=2-of-3 effect="clarify-needed"
τ=918221  Answer.Hash=1f6e..  Evidence=["P-77#2@ab19.."]  LATCH
```

Rollout & Incident Playbook (condensed)

Rollout (Week 0–1)

1. Enable SBS ledger in shadow mode; 2) Learn commute matrix from traces + human curation; 3) Turn on slots; 4) Turn on τ sync; 5) Dry-run quorums (no effect).
   Go-live (Week 2)

2. Quorum gate for high-impact actions (refunds, escalations); 7) Monitor contradictions & ρ; 8) Tune quorum and commute exceptions.
   Incident: `Agree.Fail` spike
   A) Freeze non-commuting tool; B) Force serialize path; C) Sample 50 failures → tag root cause; D) Patch commute matrix; E) Reopen gate with canary cohort.
   Weekly
   F) Review SLA vs contradiction rate; G) Re-estimate ρ thresholds; H) Export trace exemplars to audit.

Repro checklist

- Three deterministic replicas with identical seeds and tool versions.
- Explicit **commute matrix**; sandbox for non-commuting tools.
- SBS store with pointer schema & hash function fixed.
- Slot budgets enforced at runtime; collision logger on.
- τ cadence fixed; desync gate tested with synthetic jitter.
- Agreement tests on holdout set; contradiction metric defined and auto-computed.

What this chapter delivers

- A **fleet pattern** that applies ObserverOps invariants—**agreement, SBS, slots, τ sync**—to halve contradictions and raise reproducibility with bounded latency tradeoffs.
- Concrete **configs, traces, and a playbook** to move from single agents to audited, reliable replica

ensembles.

Chapter 35 — Program Belts for AI Initiative

Setting: PBHL Residual thresholds; EEI/SI governance
Result: Residual ≤ band; clearer escalations
Artifacts: Quarterly belt review packet (templates + exports)

Goal (1 sentence)

Show how **Purpose-Flux Belt Theory (PFBT)** closes an AI program's plan↔do loop using **PBHL Residual bands** and **EEI/SI** governance, yielding stable execution and crisp, pre-agreed escalation paths.

System Snapshot (what you'll implement)

- **Belt definition** with worldsheet variables: **Gap(t), Flux(t), Twist(t), α(t)** and **Residual R(t)=|Gap−(Flux+α·Twist)|**.
- **Controllers**: **Flux-gate (fast)** throttles/boosts execution; **Twist-step (slow)** approves scope/structure changes; **α-calibration** job weekly.
- **Policy gates**: residual bands (green/amber/red), EEI/SI thresholds, freeze rules when bands are breached.
- **Telemetry**: Five-Line KPI panel (Gap/Flux/Twist/Coherence/Residual), audit events, exportable packet each quarter.
- **Escalation ladder** mapped to business risk tiers (L1 ops → L2 product → L3 exec).

Architecture (ObserverOps lens)

- **Data plane**: program work items, throughput counters, scope deltas (Twist measures), completion logs.
- **Control plane**: `/belt` controller maintains PBHL variables; Flux-gate and Twist-step apply; α updated by calibrator.
- **Audit plane**: immutable belt event log, residual band decisions, EEI/SI trendlines, quarterly packet exporter.

Key concepts & invariants (how they apply here)

- **PBHL closure:** enforce **Gap≈Flux+α·Twist** with **Residual** constrained inside bands; deviations must trigger defined actions.
- **EEI (Effectiveness & Efficiency Index):** normalized Gap reduction per unit Flux (e.g., ΔGap / Flux), adjusted for quality gates.
- **SI (Sustainability Index):** stability score penalizing excessive Twist/Residual (e.g., $SI=\exp(-\beta_1 \cdot \text{Twist_var} - \beta_2 \cdot \text{Residual_mean})$).
- **Slot conservation (macro):** finite team/time "slots" budgeted across tracks to prevent overcommitment which inflates Residual.
- **Internal collapse (macro trace):** belt decisions (gates, α updates, freezes) are latched at review ticks (τ_Q weekly/quarterly).

Reference flow (weekly belt cadence)

```
def weekly_belt_review(belt, telemetry, policy):
    τQ = tick.start()
    Gap  = telemetry.gap()          # backlog to target
    Flux = telemetry.flux()         # completed, quality-accepted
    Twist = telemetry.twist()       # scope/structure delta magnitude
    α = belt.alpha

    Residual = abs(Gap - (Flux + α*Twist))
    eei = compute_eei(Gap, Flux)    # ΔGap / Flux (normalized)
    si  = compute_si(Twist, Residual)

    trace.write(τQ, "PBHL.Update", dict(Gap=Gap, Flux=Flux, Twist=Twist, α=α, R=Residual, EEI=eei, SI=si))

    band = policy.band(Residual, Gap)
    if band == "red":
        belt.freeze_flux_tracks(policy.freeze_set)
        belt.raise_escalation("L2-product", reason="Residual>red")
    elif band == "amber":
        belt.enable_flux_gate(target=policy.flux_target)
        belt.schedule_twist_board()                 # evaluate scope cuts / α update
    else:
        belt.release_holdbacks()                    # continue

    # α-calibration (weekly) via least-squares fit on last W weeks
    if policy.alpha_calibration_week:
        α_new = calibrate_alpha(history=belt.history, window=policy.alpha_window)
        belt.set_alpha(α_new)
        trace.write(τQ, "PBHL.AlphaUpdate", {"α_new": α_new})

    tick.end(τQ)
```

Belt policy (production YAML)

```
belt:
  id: "AI-Initiative-GenAssist"
  review_cadence: "weekly"
  alpha:
    initial: 0.42
    window_weeks: 8
    min: 0.25
    max: 0.65

bands:
  residual:
    # Band relative to current Gap to scale with program size
    green: { r_over_gap_le: 0.05 }         # R/G ≤ 5%
    amber: { r_over_gap_le: 0.12 }         # 5–12%
    red:   { r_over_gap_gt: 0.12 }
  eei:
    red_if_below: 0.65
  si:
    red_if_below: 0.72

gates:
  on_red_residual:
    actions:
      - "freeze_flux:tracks=['expansion','low_roi']"
      - "escalate:L2-product"
      - "twist_board:convene"
  on_amber_residual:
    actions:
      - "flux_gate:set_target=+12% throughput"
      - "alpha_probe:run"
  on_ei_si_breach:
    actions:
      - "workload_rebalance"
      - "slot_rebudgeting"

telemetry:
  exports:
    quarterly_packet:
```

```
schedule: "0 9 1 JAN,APR,JUL,OCT *"
format: "zip"
includes: ["five_line_kpi.csv","residual_band.png","alpha_history.csv",
           "decisions.jsonl","risk_register.csv","audit_hash.txt"]
```

Worked example (Program belt: “GenAssist @ Support”)

Scope. Ship and run a GenAI assistant across Support, Billing, and Docs. Tracks: RAG improvements, tool integrations, content QA, safety & audit.
Quarter Q2 (baseline). Frequent scope churn from three stakeholder groups; unclear ownership; Residual often > amber; escalations ad-hoc.
Quarter Q3 (with belts). PBHL governance active with α-calibration, Twist board, and residual bands.

Observed outcomes (illustrative)

KPI (Q3 vs Q2)	Q2 (pre-belt)	Q3 (belted)	Delta
Residual / Gap (mean)	14.8%	4.6%	−10.2 pp
Residual / Gap (p90)	22.1%	9.3%	−12.8 pp
EEI (ΔGap / Flux, normed)	0.61	0.78	+0.17
SI (stability index, 0–1)	0.68	0.81	+0.13
Escalations with playbook path	23%	100%	+77 pp
Delivery slippage (major items)	3	1	−66%

Interpretation. Residual brought **inside green band** most weeks; EEI/SI improved; escalations became predictable and faster to resolve.

Tests & thresholds (acceptance & alarms)

- **Residual band compliance:** ≥ **80%** of weeks green, ≤ **20%** amber, **0 red**; page if two consecutive reds.
- **EEI** ≥ **0.72** rolling median; alarms if drops below **0.65** for 2 weeks.
- **SI** ≥ **0.75** median; alarms if below **0.72** with rising Twist variance.
- **α stability:** change ≤ **±0.05** per calibration unless justified by Twist regime change.
- **Decision latency:** median time from band breach → logged action ≤ **48 h**.
- **Audit integrity:** 100% of belt decisions hash-chained; zero retro-edit without new τ_Q.

Failure modes & mitigations

- **Phantom Flux** (reported throughput not closing Gap): *Mitigation:* tighten definition of Flux to “quality-accepted”; add random audits; penalize EEI if rework>τ.
- **α drift chasing noise:** *Mitigation:* widen calibration window; CI-guard α updates; freeze α during product launches.
- **Twist under-reporting:** *Mitigation:* require twist tickets for any scope/ownership change; link to OKR deltas; audit conditional on SI drop.
- **Slot overcommitment (macro):** *Mitigation:* slot rebudgeting at belt level; freeze expansion tracks first; enforce WIP limits.

Dashboards (what you’ll ship)

1. **Five-Line KPI**: Gap, Flux, Twist, Coherence (or ρ for team sync), Residual.
2. **Residual bands**: weekly points with green/amber/red shading, α overlay.

3. **α history**: step plot with calibration annotations and CIs.
4. **EEI/SI trend**: dual-axis with policy thresholds.
5. **Escalation ladder**: count and time-to-action by band.
6. **Twist decomposition**: policy vs scope vs org-structure contributions.

Quarterly Belt Review Packet (template & excerpt)

Structure (ZIP):

- `/exec-summary.pdf` — 1-page; claims & decisions.
- `/five_line_kpi.csv` — weekly Gap/Flux/Twist/Residual, α.
- `/residual_band.png` — chart with band overlays.
- `/alpha_history.csv` — α per week + CI + rationale.
- `/decisions.jsonl` — time-ordered gate events and actions.
- `/risk_register.csv` — open risks with owners & due dates.
- `/audit_hash.txt` — SHA-256 chain head for the quarter.

Sample `decisions.jsonl` record

```
{
  "belt_id": "AI-Initiative-GenAssist",
  "ts": "2025-07-08T14:05:22Z",
  "tau_Q": 1042,
  "residual_over_gap": 0.131,
  "band": "red",
  "actions": ["freeze_flux:expansion","escalate:L2-product","twist_board:convene"],
  "alpha_before": 0.42,
  "alpha_after": 0.44,
  "rationale": "Vendor migration added Twist; α adjusted per 8w fit (CI ±0.02).",
  "hash_prev": "1f6e…a2",
  "hash_this": "7c4b…9d"
}
```

Executive Summary (example bullets)

- Residual stayed **green** 9/12 weeks (amber 3, red 0).
- EEI ↑ to **0.78** (from 0.61); SI ↑ to **0.81**.
- α held in **0.42→0.44** range; Twist spikes tied to two approved scope changes.
- Two amber events resolved within 36 h via Flux-gate + slot rebudget.

Escalation Playbook (condensed)

- **Amber (R/G 5–12%)**: increase Flux by +12% via fast levers (queue shaping, WIP limits, staffing shuffle); schedule Twist board within 72 h.
- **Red (>12%)**: freeze low-ROI tracks; exec decision within 48 h on scope cut or α re-baselining; add SI-protective measures (on-call rotation fixes, cooldown).
- **Post-mortem**: attribute Residual to Flux shortfall vs Twist surprise; update risk register and commute rules between teams (ownership clarifications).

Repro checklist

- Define **Gap**, **Flux**, **Twist** measurably (schema-versioned).
- Calibrate **α** on ≥8 weeks of consistent data; store CI and rationale.
- Wire gates to CI/CD or task management (freeze labels, WIP limits).
- Ensure belt decisions are **latched** at τ_Q; export hash chain.
- Maintain Five-Line KPI panel and export the quarterly ZIP on schedule.

What this chapter delivers

A **macro governance pattern** that binds delivery to a conservation-style closure (**Gap≈Flux+α·Twist**), keeps **Residual inside bands**, improves **EEI/SI**, and replaces ad-hoc escalations with a transparent, auditable playbook.

Part VIII — Risk, Ethics, and Non-Claims

Chapter 36 — Scope & Non-Claims

Tagline: Reliability, safety, governance—not consciousness.

Goal (1 sentence)

Define exactly what ObserverOps **does** (reliability, safety, governance) and **does not** claim (no consciousness, sentience, moral agency, or quantum-compute advantage), with enforceable language, UI, and review guardrails.

Scope (what's in / what's out)

In-scope (deliverables & guarantees)

- **Operational reliability:** internal collapse (trace latching), cross-observer agreement under commuting effects with shared records, certified pooling (CWA), slot budgeting, τ sync.
- **Safety & governance:** PBHL belts, policy gates, auditability (immutable traces, cert logs), GRC-ready exports, thresholds & runbooks.
- **Measurement transparency:** metrics with bands (CWA score, Phase-Risk, Agreement Rate, PBHL Residual, EEI/SI), CI where applicable.

Out-of-scope (non-claims)

- **No consciousness / sentience / self-awareness.**
- **No moral agency** or autonomous ethical judgment.
- **No guarantees of factual ground truth** merely from high Agreement or CWA pass.
- **No quantum-compute speedups.** Theory borrows QM notation; implementation is classical.
- **No intent understanding** beyond modeled fields/metrics; "meaning" is operational (projection+trace), not metaphysical.
- **No security certification by default.** Audit artifacts support compliance, but certification is external.
- **No legal advice.** Governance metrics inform risk; they don't decide it.

Language & UI guardrails (operationalize the non-claims)

Terminology rules (linted):

- Replace “knows/understands/believes” → “has evidence for / selects / predicts”.
- Replace “decides” → “selects per policy Ô at tick τ”.
- Ban “conscious / sentient / feels / wants”.
- Replace “truth” → “agreement-consistent answer; evidence-linked”.

User-visible disclaimers (auto-injected):

- “Generated via ObserverOps controls (Ô/τ/CWA/agreement). **Not** a statement of consciousness, intent, or moral agency.”
- “Agreement≠truth: evidence IDs and hashes provided; human oversight recommended for high-impact actions.”

UI affordances:

- Always show **Evidence panel** (SBS pointers, hashes).
- Show **Confidence/Agreement** separately from **Accuracy claims**; require human confirm when thresholds not met.
- “Why this answer?” expands to Ô/τ decisions and policy gates taken.

Policy config (production YAML)

```
nonclaims:
  language_lint:
    banned: ["conscious","sentient","feels","wants","knows","believes","ground truth"]
    replacements:
      knows: "has evidence for"
      believes: "predicts"
      decides: "selects per policy Ô at tick τ"
  ui_disclaimer_banner: true
  disclaimer_text:
    - "ObserverOps improves reliability/safety/governance. It does not confer consciousness or moral agency."
    - "Agreement or certificate pass does not guarantee factual truth."
  high_impact_actions:
    require_human_confirm: true
    thresholds:
      agreement_score_ge: 0.90
      cwa_score_ge: 0.80
      residual_band: "green"
  audit:
    export_nonclaims_attestation: true
    fields: ["version","policy_hash","banned_terms_hits","human_confirms"]
```

Worked examples (micro → macro)

1. **Answer screen copy edit (lint pass):**
   - Before: “The agent **knows** policy P-77 allows refunds.”
   - After: “Replicas **agreed** (0.92) on evidence **P-77#2@ab19…**; Ô selected `Search→DB` at τ=918221. See Evidence.”
2. **Program review slide (belt):**
   - Before: “Q3 truth rate 84%.”
   - After: “Q3 **Agreement-consistent accuracy** 84% (holdout rubric); **Residual/Gap** 4.6%

(green). No claims about absolute truth."

Tests & metrics (compliance)

- **Language lint pass-rate:** 100% of surfaced text; build breaks on banned terms.
- **Disclaimer presence:** 100% on high-impact surfaces; 0 missing.
- **Human-confirm attachment:** 100% on actions breaching thresholds.
- **Evidence linkage rate:** ≥99.5% answers emit SBS pointers & hashes.
- **Non-claims attestation export:** on each release; policy hash recorded.
- **Post-hoc audit sampling:** random 1% of runs validated for disclaimer + evidence completeness.

Failure modes & mitigations

- **Anthropomorphic drift in prompts/UX** → *Mitigation:* enforce lint; design review checklist; block merge on violations.
- **Over-reliance on Agreement as truth** → *Mitigation:* training on "agreement≠truth"; dashboards separate **Agreement** from **Accuracy**; require evidence.
- **Marketing overreach** ("quantum-powered") → *Mitigation:* release gate requires **Non-Claims sign-off**; legal review; canned phrasing only.
- **Silent removal of disclaimers** → *Mitigation:* UI tests; canary monitors; release must include non-claims checksum.

Review checklist (ship with every release)

- Non-claims statement present in **README** and **About**.
- Language lint: 0 banned term hits.
- UI disclaimer visible on high-impact flows.
- Evidence panel renders for ≥99.5% of answers.
- Human-confirm gates wired to thresholds.
- Non-claims attestation exported with policy hash & build ID.

Artifacts

- **Non-Claims Statement** (one-pager) for product/docs.
- **Language Lint Config** (the YAML above).
- **UI Disclaimer Snippets** (copy + component).
- **Release Attestation** (JSONL) capturing policy hash, gate states, banned-term counts.
- **Audit Sample Pack** (10 redacted traces showing disclaimers + evidence).

Canonical Non-Claims Statement (copy/paste)

> **ObserverOps is a reliability, safety, and governance framework.**
> It provides schedulers (Ô), commit ticks (τ), agreement checks, certified pooling (CWA), and belt

controls (PBHL) to improve repeatability and auditability.
ObserverOps does not confer or imply consciousness, sentience, moral agency, or quantum-compute advantage.
Agreement or certificate pass indicates procedural consistency—not factual truth. Use evidence links and human oversight for high-impact decisions.

Chapter 37 — Failure Modes & Mitigations

Modes: false-green certificates, slot starvation, desync, KPI gaming
Mitigations: conservative gates, back-pressure, redundancy, audits

---

Goal (1 sentence)

Enumerate the high-impact failure modes in ObserverOps deployments and ship **detect→gate→fix** playbooks—so fleets degrade safely and recover predictably.

Quick taxonomy (at a glance)

Mode	What breaks	Primary signals	First response	Structural fix
False-green certificate	CWA passes when order/phase matter → wrong pooling	CWA pass↑ + Accuracy parity↓; Phase-Risk↑; drift p<0.01	Force fallback path; raise θ	Broaden tests; add guard-band rerank; projector re-whitening
Slot starvation	Integer budgets exhausted → collisions, thrash, tail latency	Occupancy p95>85%; collisions>5%; queue depth↑	Back-pressure; shrink k; shed bulk	Priority tiers; dynamic budgets; WIP limits
Desync (τ)	Tick skew across replicas → races, inconsistent traces	ρ<0.82; Δτ p95>60ms; re-entrancy spikes	Pause new work; resync barrier	Stable cadence; NTP/clock health; jitter buffers
KPI gaming	Local metric up, program outcome flat/down	Residual not improving despite Flux↑; EEI↓; anomaly audits	Freeze metric as gate; manual review	Multi-metric gates; random audits; belt invariants

---

Mode 1 — False-Green Certificates (CWA)

What it is. The certificate battery accepts additive pooling when **order/phase** still carry signal (e.g., coherent chains, bursty sequences, template drift).
Symptoms. CWA pass-rate drifts up; accuracy parity on holdout dips; Phase-Risk Index rises; disagreements rise on queries tagged "sequence-sensitive."

Detection & gates

- **CWA score vs accuracy parity**: trigger if `pass_rate↑` while `Δaccuracy (A/B)` drops >0.5% with 95% CI.
- **Phase-Risk Index**: red if >0.60 over 10-min window.
- **Drift test**: permutation panel KS-pvalue `< 0.01` → hold.

Immediate mitigation (runtime)

1. Route all queries in affected cohort to **order-aware fallback** (cross-encoder/attention).
2. Enable **guard-band rerank**: even on pass, run rerank on top-M (e.g., 8) before reader.
3. Raise threshold `θ := θ + 0.02` for the domain; log decision to trace **T** at τ.

Structural fixes

- Add/strengthen **chunk-shuffle** test (semantic chunker); increase panel count.

- Re-whiten projector; re-tune θ per cohort; add OOD detector for projector stats.
- Introduce **domain split** (e.g., “procedural-longform” always fallback).

Config (YAML)

```
cwa:
  min_score: 0.80
  guard_band:
    enabled_on_pass: true
    rerank_top_m: 8
  alarms:
    phase_risk_gt: 0.60
    action_on_alarm: ["route:fallback","raise_theta:+0.02","sample:500"]
  drift:
    ks_p_lt: 0.01
    freeze_on_drift: true
```

Mode 2 — Slot Starvation

What it is. Discrete **slots** (retriever, DB, code, math, tools) over-filled ⇒ collisions, eviction thrash, timeouts, non-reproducible behavior.
Symptoms. p95/p99 latency spikes; high collision rate; retriever k not honored; queue depth grows; mis-execs.

Detection & gates

- **Occupancy p95** > 85% or **collisions** > 5%/min → throttle.
- **Tail latency** p99 > 3× p50 for 3 consecutive minutes → degrade.
- **Queue depth** slope > threshold (e.g., +100/min) → back-pressure.

Immediate mitigation (runtime)

1. **Back-pressure**: shrink `k_retrieve` via `k := f(queue_depth)` (monotone).
2. **Priority tiers**: reserve slots for high-impact queues; shed bulk traffic.
3. **Degrade mode**: cache-only reads; disable expensive tools temporarily.

Structural fixes

- **Right-size budgets** by cohort; pre-warm caches; cap parallel tool fans.
- Enforce **WIP limits** at belt level; allocate slots as macro resources.
- Instrument **collision heatmaps**; refactor hot tools.

Config (YAML)

```
slots:
  budgets: { Retriever: 8, DB: 2, Code: 1, Math: 1 }
  throttle:
    on_collision_gt: 0.05
    action: "k_retrieve:=max(50, k_retrieve-50)"
  priority:
    tiers: { vip: 0.6, std: 0.3, bulk: 0.1 }    # fraction of slots
  degrade:
    cache_only_on_p99_over: 3.0
```

Mode 3 — Desync (τ and fleet sync)

What it is. Replicas drift in **tick τ** → non-deterministic ordering; racey writes; agreement failures.
Symptoms. Kuramoto **ρ** drops; Δτ p95 grows; retro-edit attempts; spike in `Agree.Fail`.

Detection & gates

- **ρ threshold**: red if ρ < 0.82 for >60s.
- **Δτ p95** > 60ms for >5m → pause new work.
- **Re-entrancy** attempts > 2/min/replica → review.

Immediate mitigation (runtime)

1. **Desync gate** trips: **pause admission**, drain in-flight at old τ, barrier-resync, resume.
2. Force **new τ** for any retried segment; reject retro-edits.
3. Reduce **concurrency** temporarily; lengthen cadence by +25ms.

Structural fixes

- Harden **clock/NTP**; jitter buffers; rate-limit tool side-effects.
- Co-locate replicas; cap cross-AZ fanout; heartbeat monitors.

Config (YAML)

```
sync:
  cadence_ms: 150
  rho:
    red_lt: 0.82
    amber_lt: 0.88
  delta_tau_ms:
    p95_red_gt: 60
  gate_action_on_red: ["pause_new_work","barrier_resync","concurrency:-30%","cadence:+25ms"]
  latching:
    forbid_retro_edit: true
    require_new_tau_on_retry: true
```

Mode 4 — KPI Gaming (Goodhart)

What it is. Teams optimize proxy metrics (e.g., Flux) without improving **Gap**; or tweak processes to pass gates (e.g., chunking to inflate CWA), under-report **Twist**, or cherry-pick easy queries.
Symptoms. Residual flat/up while Flux↑; **EEI** down; sudden CWA pass-rate jumps post policy change; distributional shift in query difficulty.

Detection & gates

- **EEI guard**: gate if EEI rolling median < 0.70 while Flux↑ > +10%.
- **Residual watch**: if R/G not improving for 2+ weeks → freeze “expansion” tracks.
- **CWA anomaly**: pass-rate jump > +8pp week-over-week with stable accuracy → audit.
- **Holdout parity**: live vs locked holdout Δaccuracy > 1% → gaming suspected.

Immediate mitigation (runtime)

1. Freeze metric as **gate**; require belt review for changes affecting that metric.
2. Route sampled traffic to **locked holdout** (immutable) and compare.
3. Reset chunking/projector to prior version; require change ticket.

Structural fixes

- Multi-metric gates: tie **Flux** to **Gap** via PBHL Residual; add **SI** penalty for Twist volatility.
- **Random audits** & red-team prompts; rotate holdouts monthly.

- Align incentives: reward **Residual improvement**, not raw Flux.

Config (YAML)

```
governance:
  kpi_integrity:
    eei_min: 0.70
    holdout_gap: { max_delta_accuracy: 0.01 }
    cwa_pass_jump_pp: 8
    actions_on_violation: ["freeze_metric","route_holdout_20pct","require_change_review"]
```

Cross-cutting controls

- **Redundancy**: SBS shared records + 2-of-3 agreement on critical decisions.
- **Audits**: daily JSONL exports with hash chains; random 1% manual review.
- **Shadow oracles**: continuous A/B against slower but robust estimators (e.g., cross-encoder).
- **Kill-switches**: per-domain gates that revert to safe baselines.

Incident playbooks (condensed)

A. False-Green Spike

1. Alarm → Route fallback; 2) Sample 500 queries; 3) Re-whiten projector; 4) Raise θ by +0.02; 5) Add chunk-shuffle; 6) RCA within 24h; 7) Canary 10% for 2h; 8) Reopen gate if parity holds.

B. Slot Storm

1. Back-pressure k; 2) Shed bulk; 3) Promote VIP tier; 4) Enable cache-only; 5) Post-mortem on budget mis-set; 6) Adjust WIP limits.

C. Desync Storm

1. Gate trips → pause; 2) Barrier-resync; 3) Concurrency −30%; 4) Cadence +25ms; 5) Check NTP/clock drift; 6) Resume with monitor.

D. KPI Gaming Suspected

1. Freeze metric gates; 2) Route 20% to locked holdout; 3) Compare Δaccuracy; 4) Audit chunking changes; 5) Update incentives; 6) Exec sign-off to unfreeze.

Dashboards to ship

1. **CWA Health**: pass-rate vs accuracy parity; Phase-Risk; drift p-values.
2. **Slots**: occupancy/collisions, queue depth, tail latency.
3. **Sync**: ρ & Δτ quantiles; re-entrancy events; desync incidents.
4. **Belt KPIs**: Gap/Flux/Twist/Residual with EEI/SI overlays.
5. **Audit Integrity**: hash-chain success, disclaimer coverage, export success.
6. **Anomaly Feed**: top cohort contributions to each alarm.

Test harness (black-box)

- **CWA adversarial set**: coherent sequences; required sign-flip failure.

- **Slot load generator**: bursty arrivals to validate back-pressure & priority tiers.
- **Sync jitter injector**: ±100ms network jitter; assert gate behavior.
- **KPI gaming simulator**: inflate Flux without Gap change; expect gate freeze.

Artifacts

- **Gating & Alarms YAML** (above).
- **Runbooks** A–D.
- **Anomaly dashboards** spec.
- **Adversarial test suites** (coherent CWA set; jitter; slot bursts).
- **Audit export schemas** for incident traces.

What this chapter delivers

Concrete **signals, thresholds, and levers** to catch the four major failure classes early, degrade safely, and recover with traceable, auditable actions—consistent with ObserverOps invariants (internal collapse, agreement, slot conservation, PBHL closure).

No metaphysics. Just observers you can build.

[Refer to "Self-Evolved Observers as Attractors in Semantic Meme Field Theory" project for more details. https://osf.io/7cbsu/]

Chapter 38 — Privacy, Security, Compliance

Tagline: Trace handling, export controls, SOC/ISO control mapping.

Goal (1 sentence)

Ship a **privacy-by-design** and **audit-grade** implementation of ObserverOps: immutable-but-redactable traces, gated exports, least-privilege access, and a control mapping that lets security/compliance teams adopt the stack without changing the math.

System Snapshot (what you'll implement)

- **Data classification & minimization:** label inputs/outputs; project away PII before embedding; store only *pointers + hashes* where possible.
- **Trace ledger with privacy layers:** append-only hash-chained **T** (source of truth) + *derivative privacy views* (redacted, tokenized, or salted-hash).
- **Encryption & keys:** AEAD at rest; TLS 1.3 in transit; KMS-backed envelope keys; periodic rotation & destruction for delete semantics.
- **Export gates:** DLP-scan + policy allowlist + signer + checksum; residency-aware routing; volume/velocity throttles.
- **Access & accountability:** RBAC/ABAC, break-glass workflow, access review cadence, immutable admin audit.
- **Compliance artifacts:** SOC/ISO control mapping, evidence exports (JSONL), quarterly attestations.

Architecture (ObserverOps lens)

- **Data plane:** `measure → project (PII scrub) → pool → answer.`
- **Control plane: Ô** selects *privacy-safe* instruments; **τ** latches trace events; **Policy Gates** enforce DLP/export limits.
- **Audit plane:** two ledgers: **T_core** (full, sealed) and **T_view** (privacy-reduced); export jobs write signed bundles to GRC/Vault.

Key concepts & invariants

- **Privacy-first latching.** *Source* writes are immutable, but **views** are derivable and revocable; "erase" = delete/rotate keys for high-risk fields + retract **T_view** rows, keeping **T_core** sealed for integrity.
- **Least Projection Principle.** Prefer projectors that strip identifiers (e.g., bag-of-concepts) before embedding; store **evidence pointers** `⟨doc_id, section, sha⟩` instead of raw text.
- **Split evidence.** Decouple SBS pointers from content storage; pointers are portable and low-risk; content access is separately gated.
- **Export throttling as a slot.** Treat exports as a **slot-limited** instrument to bound exfiltration rate (bytes/s, records/s).
- **Residency-aware routing.** Data stays in-region unless a signed, policy-approved export is latched at τ_Q (review tick).

Privacy-preserving trace write (reference)

```
def write_trace_privacy(τ, event):
    # 1) Classify & scrub
    cls = classify(event.payload)                    # P0..P3 sensitivity
    scrubbed = redact(event.payload, policy=REDPOL)  # drop/mask/tokenize

    # 2) Seal source and derive view
    sealed = encrypt_aead(scrubbed, key=KMS.data_key(event.tenant))
    T_core.append(hash_chain(sealed, τ))             # immutable source
    view = derive_privacy_view(scrubbed, mode="min") # pointers, salts

    # 3) Latch + accountability
    T_view.append(view)
    admin_log.append(sig("TRACE_WRITE", τ, meta=event.meta))
    return ok
```

Policy config (production YAML)

```
classification:
  P0: "public"
  P1: "internal"
  P2: "confidential"
  P3: "sensitive+PII"
redaction:
  PII_detectors: ["email","phone","iban","ssn","auth_token"]
  actions:
    email:       "tokenize"
    phone:       "tokenize"
    iban:        "drop"
    ssn:         "salted_hash"      # H(s + value), s in HSM
    auth_token:  "drop"
  embedding:
    forbid_levels: ["P3"]
    projector: "concept-bow-1024"   # minimizes identifier leakage
crypto:
  at_rest: { aead: "AES-256-GCM", rotate_days: 90 }
  in_transit: { tls: "1.3", mtls_inside_mesh: true }
  kms: { provider: "KMS", key_per_tenant: true, hsm_backed: true }
```

```
retention:
  T_view: { p0: 365d, p1: 180d, p2: 90d, p3: 30d }
  T_core: { legal_hold: true, crypto_erase_on_request: true }
exports:
  allowlist_targets: ["grc-vault","soc-binder"]
  format: "jsonl+sig"
  dlp: { enabled: true, block_on_match: ["P3","auth_token"] }
  residency: { eu: ["eu-grc-vault"], us: ["us-grc-vault"] }
  rate_limits: { max_bytes_s: 2_000_000, max_records_min: 5000 }
access:
  rbac:
    roles: ["agent","ops","seceng","auditor","admin"]
    constraints:
      auditor: { can_read: ["T_view","exports"], can_write: [] }
      admin:   { can_read: ["*"], can_write: ["policy"], break_glass: true }
  reviews: { cadence_days: 30, revoke_inactive_days: 60 }
logging:
  admin_actions: { hash_chain: true, signed: true, immutable_store: true }
```

Worked examples

A) EU Support RAG with residency + DSAR

- Projector enforces P3 block; SBS pointers contain only `doc_id/section/sha`.
- DSAR (erasure) triggers **crypto-erase**: rotate tenant data key; rebuild **T_view** from **T_core** minus subject-linked rows; latched at τ_Q with evidence.

B) Quarterly SOC binder export

- Export job signs JSONL with ledger head hash + build policy hash; DLP enforces P3=0; destination must be allowlisted GRC vault; manifest + checksums included.

Tests & thresholds (acceptance)

- **PII leak rate** ≤ 0.1% of sampled tokens (A/B on redaction on/off; 95% CI).
- **Encryption coverage**: 100% for P2/P3 at rest; 100% of inter-service calls on mTLS.
- **Export policy hits**: 0 violations; block rate < 1% (investigate if >1%).
- **Access hygiene**: monthly review completion 100%; inactive access revoked within 60 days.
- **Residency**: 0 cross-region writes for flagged tenants.
- **Trace integrity**: 100% hash-chain verification; admin log signing 100%.

Failure modes & mitigations

- **PII in embeddings** → strengthen detector, forbid P3 to embed, post-hoc vector purge with **pointer re-ingest**; DP noise for analytics aggregates.
- **Key compromise** → rotate KMS roots; re-encrypt **T_core**; invalidate export tokens; audit blast radius by key lineage.
- **Misrouted export** → enforce allowlist + signed manifests; automatic quarantine if destination fingerprint mismatch.
- **Right-to-erasure vs immutability** → crypto-erase (key rotation) + redacted **T_view** regeneration; retain minimal anchored hashes for integrity.
- **KPI privacy gaming** (over-redaction harming accuracy) → dual-track evaluation (privacy-on vs privacy-off) in sandbox; belt gate on **EEI** drop > 3pp due to redaction.

Dashboards to ship

1. **Privacy panel:** PII detections by type, leak-rate CI, embedding blocks.
2. **Crypto & keys:** key ages, rotations, anomalies; encryption coverage.
3. **Exports:** volumes, destinations, DLP blocks, checksum mismatches.
4. **Access & admin:** RBAC drift, break-glass events, review completion.
5. **Residency:** cross-region attempts blocked/allowed; tenant map.
6. **Trace integrity:** hash-chain status, signature verification, view rebuilds.

SOC/ISO Control Mapping (concise)

Control area	ObserverOps feature	Example evidence
Access Control (ISO A.9 / SOC: CC6)	RBAC/ABAC; break-glass; monthly reviews	`access_review.csv`, `admin_log.sig.jsonl`
Cryptography (ISO A.10)	AEAD at rest; mTLS; KMS/HSM rotation	`crypto_posture.csv`, KMS rotation logs
Operations Security (ISO A.12)	DLP, export allowlist, rate-limited slots	`export_manifest.json`, DLP reports
System Acquisition/Dev (ISO A.14)	Privacy lint in CI; redaction libs; policy-as-code	CI logs, policy hash in build
Supplier/Transfers (ISO A.15/ A.13)	Residency routing; signed exports; allowlisted sinks	residency audits, sink certificates
Incident Mgmt (ISO A.16 / SOC: CC7)	Privacy incident runbooks; belt gates on breaches	runbook executions, ticket IDs
Compliance/Privacy (ISO A.18 / SOC: P)	DSAR crypto-erase; immutable-but-redactable traces	DSAR evidence pack, view rebuild logs
Logging/Monitoring (SOC: CC7/CC9)	Hash-chained traces; admin log signing; anomaly alerts	`trace_head.txt`, alert exports

> Tip: include the **policy hash** (git SHA) and **ledger head hash** in every export for chain-of-custody.

Export manifest (JSONL snippet)

```
{
  "bundle_id": "soc-q3-2025",
  "policy_hash": "9e7a…3c",
  "ledger_head": "aa94…55",
  "records": 124567,
  "dest": "grc-vault",
  "dlp_summary": {"p3":0,"tokens_scanned":18233411},
  "checksums": {"file":"sha256:7c4b…9d"},
  "signer": "kms-key-alias/grc-export@2025-09-23T09:00Z"
}
```

Repro checklist

- Classifier/redactor wired and tested on seeded PII.
- P3 blocked from embeddings; projector uses identifier-minimizing features.
- KMS keys per tenant; rotation policy active; envelope encryption verified.
- Exports only to allowlisted destinations; signer & checksum validated.
- Residency router active; tests show zero cross-region leaks.

- Access reviews completed; break-glass audited.
- Quarterly SOC/ISO packet generated with evidence and policy hashes.

What this chapter delivers

A concrete, production-ready **privacy & security posture** for ObserverOps: strict **trace handling**, **controlled exports**, and **controls mapping** that turns your invariant-driven runtime into an audit-ready system.

Part IX — Packaging & Open-Core Strategy

Chapter 39 — SDKs & Libraries

Modules: Observer Runtime, CWA Engine, BeltOps Dashboard, Slot Allocator

---

Goal (1 sentence)

Ship production-ready SDKs—**Observer Runtime**, **CWA Engine**, **BeltOps Dashboard**, and **Slot Allocator**—with clean APIs, type-safe events, and batteries-included defaults so teams can build observer-based systems quickly and scale safely.

39.1 Module Map (what you'll get)

- **Observer Runtime** (`observer-runtime`): traces **T**, scheduler **Ô**, tick **τ**, agreement checks, SBS pointers, immutability guards.
- **CWA Engine** (`cwa-engine`): projector registry, certificate battery (perm/flip/chunk), pooling + auto-fallback, risk flags.
- **Slot Allocator** (`slot-allocator`): quantized budgets for memory/attention/tools; occupancy/collision metrics; back-pressure hooks.
- **BeltOps Dashboard** (`beltops` + `@beltops/ui`): PBHL worldsheet controller, Five-Line KPI, policy gates & export jobs.

  Langs: Python and TypeScript first-class; gRPC/REST for polyglot.
  Dist: PyPI / npm; Docker images for services (`cwa-svc`, `beltops-svc`).
  Open-core: runtime, tests, basic dashboards; **Enterprise** adds policy gates, audit exports, single-pane multi-belt, SSO/RBAC.

39.2 Install & Compatibility

```
# Python
pip install observer-runtime cwa-engine slot-allocator beltops

# TypeScript/Node
pnpm add @observerops/runtime @observerops/cwa @observerops/slots @observerops/beltops-ui
```

- **Python** ≥3.9; **Node** ≥18.
- Works on Linux/macOS; ARM64 ready.
- Vector DB adapters: pgvector, FAISS, Milvus (plugins).
- Observability: OpenTelemetry out-of-box.

39.3 Observer Runtime

Core types (Python)

```
from observer_runtime import Observer, Trace, Channel, agree, sbs

O = Observer(
    channels=[Channel("Search"), Channel("DB"), Channel("Math")],
    commute_matrix={("Search","DB"): True, ("DB","Math"): False},
    tick_cadence_ms=150
)

τ = O.tick.start()
y = O.measure("Search", query="refund threshold")
O.trace.write(τ, event="Measure", payload={"ch":"Search","y":y})
O.tick.end(τ)
```

Highlights

- **Latching**: hash-chained writes; retro-edits require new **τ**.
- **Agreement**: `agree(Ta, Tb, commute_matrix)` ⇒ `{pass|fail, score}`.
- **SBS pointers**: `sbs.publish({"doc_id","section","sha"})` shared across replicas.
- **Events**: `TickStart`, `ChannelSelected`, `TraceWrite`, `AgreementPass/Fail`.

REST (for polyglot)

```
POST /measure { "pi":"Search", "S":{...}, "T":trace_id }
POST /agree   { "traces":[...], "commute_matrix":{...} }
GET  /trace/:id
```

39.4 CWA Engine

Fast path with certificate gating (TS)

```
import { project, certify, pool } from "@observerops/cwa";

const proj = await project(candidates, { projector: "whitened-sif-1024" });
const cert = await certify(proj, {
  tests: { permutation: {panels:50}, signFlip:{panels:50}, chunkShuffle:{panels:40} }
});

const pooled = cert.score >= 0.78
  ? pool.additive(proj)
  : await pool.attentive(candidates, { crossEncoder: "ce-msmarco-v2", window: 32 });
```

Features

- **Projectors**: SIF/SimCSE/BoW-concepts + custom registry.
- **Certificates**: permutation / sign-flip / chunk-shuffle; drift tests.
- **Risk outputs**: `score∈[0,1]`, Phase-Risk Index, pass/fail panels.
- **Auto-fallback** to order-aware estimators; guard-band rerank.

Service mode

- `cwa-svc` exposes `/project`, `/certify`, `/pool` with batch & streaming.

39.5 Slot Allocator

Budgeted tools (Python)

```
from slot_allocator import Slots

slots = Slots(budgets={"Search":3,"DB":1,"Math":1}, collision_alarm=0.05)

with slots.scope({"Search":1}):          # reserve
    hits = search.topk(q, k=100)         # bounded concurrency

m = slots.metrics()  # occupancy p95, collisions, queue_depth
```

Capabilities

- Integer **budgets**, **priority tiers** (VIP/std/bulk), **back-pressure hooks** (`on_collision -> shrink k`).
- Emits **occupancy/collisions**; integrates with OpenTelemetry.

39.6 BeltOps Dashboard

Controller (Python) + UI (React)

```
from beltops import Belt

belt = Belt(alpha=0.42)
belt.update(gap=120, flux=26, twist=4)      # computes Residual, bands, gates
if belt.residual_band()=="red":
    belt.freeze(["expansion"]); belt.escalate("L2-product")

// React panel
import { FiveLineKPI } from "@observerops/beltops-ui";
<FiveLineKPI data={timeseries} thresholds={{residual:{green:0.05, amber:0.12}}} />
```

Exports

- `/belt/export?quarter=Q3-2025` → ZIP: KPIs, residual bands, decisions, α-history (signed).

39.7 End-to-End: 40-line RAG with Gates (Python)

```
from observer_runtime import Observer
from cwa_engine import project, certify, pool
from slot_allocator import Slots

O = Observer(...); slots = Slots(...)

def answer(q):
    τ = O.tick.start()
    with slots.scope({"Search":1}):
        cand = retriever.topk(q, k=200)

    proj = project(cand, projector="whitened-sif-1024")
    cert = certify(proj, tests={"perm":50,"flip":50,"chunk":40})

    if cert.score >= 0.78:
        vec = pool.additive(proj)
        path = "additive"
    else:
        vec = pool.attentive(cand, cross_encoder="ce-msmarco-v2", window=32)
        path = "fallback"

    O.trace.write(τ, "CWA", {"score":cert.score, "path":path})
    ans = reader.generate(q, vec)
    O.trace.write(τ, "Answer", {"hash":hash(ans)})
    O.tick.end(τ)
    return ans
```

39.8 Observability & Privacy Defaults

- **Metrics**: CWA score, Phase-Risk, Agreement Rate, Δτ/ρ, Slot occupancy/collisions, PBHL Residual,

EEI/SI.

- **Tracing**: OpenTelemetry spans with event names above; logs are hash-chained.
- **Privacy**: PII detectors + redactors; "least projection" projector; **T_core/T_view** split; residency-aware exports.

39.9 Open-Core vs Enterprise (feature table)

Area	Open-core	Enterprise add-ons
Observer Runtime	Traces, Ô/τ, agree(), SBS pointers	SSO/RBAC, org policies, SOC/GRC exporters
CWA Engine	Projectors, cert battery, fallback	Auto-tuning, cohort drift radar, GPU batching svc
Slot Allocator	Budgets, metrics, back-pressure hooks	Multi-tenant quotas, burst credits, SLA guards
BeltOps	Belt math, Five-Line KPI, CSV export	Multi-belt rollups, executive packet, escalation workflows
Tooling	CLI, test harness, notebooks	Helm charts, multi-region deploy, support SLAs

Licensing: Apache-2.0 (core); commercial license for enterprise packs.

39.10 Versioning, Testing, CI/CD

- **SemVer** per package (`MAJOR.MINOR.PATCH`).
- **Compat matrix**: publish per-release table (projector versions ↔ runtime).
- **Golden tests**: replay traces; assert **no retro-edits** without new **τ**.
- **Adversarial suites**: coherent CWA sets, slot bursts, jitter injection, KPI-gaming sim.
- **Policy hash** baked into builds; exports include `{policy_hash, ledger_head}`.

39.11 Artifacts (ship list)

- API refs (REST/gRPC), typed clients (Py/TS), example apps (`/apps/rag-cwa`, `/apps/fleet-agree`, `/apps/beltops`).
- Helm charts / Terraform snippets; Docker images (`cwa-svc`, `beltops-svc`).
- Dashboards: Grafana/Prometheus mixes for metrics above.
- Notebook pack: quickstarts (RAG-CWA, Fleet Agreement, Belt Governance).
- Migration guides: upgrading projector/α; adding slot budgets; enabling SBS.

What this chapter delivers

A crisp path from **theory → code**: install the SDKs, wire a 40-line gated RAG, add slots and agreement, then watch BeltOps keep programs inside Residual bands—open-core by default, enterprise where audits and scale demand it.

Chapter 40 — APIs, Schemas, Integrations

Focus: REST/gRPC; event logs; vector DBs; observability stacks

---

Goal (1 sentence)

Provide **stable, auditable interfaces**—HTTP/gRPC endpoints, typed events, and plug-in adapters—so ObserverOps drops into existing stacks (retrievers, vector DBs, observability) with minimal glue and maximal traceability.

40.1 Service Surfaces (REST & gRPC)

Core endpoints (REST)

```
POST /measure            # Observer Runtime: run instrument/tool
POST /agree              # Agreement check across traces
POST /project            # CWA: projection (feature extraction)
POST /certify            # CWA: certificate battery (perm/flip/chunk)
POST /pool               # CWA: additive/attention pooling
POST /belt               # BeltOps: update Gap/Flux/Twist; compute Residual + gates
GET  /trace/:id          # fetch immutable trace (T_view)
GET  /trace/:id/events   # paginated trace events
GET  /metrics            # Prometheus scrape
POST /webhooks/ingest    # signed event receipt (for cross-system linking)
```

Conventions

- **Auth:** OAuth2 (client creds or JWT) + optional mTLS inside mesh.
- **Idempotency:** `Idempotency-Key` header; safe retries return same `result_id`.
- **Versioning:** URI prefix `/v{MAJOR}`; schema minor versions via `X-Schema-Rev`.
- **Pagination:** `?cursor=` + `limit` (opaque cursor).

gRPC (proto extract)

```
service ObserverRuntime {
  rpc Measure(MeasureRequest) returns (MeasureReply);
  rpc Agree(AgreeRequest) returns (AgreeReply);
  rpc GetTrace(GetTraceRequest) returns (stream TraceEvent);
}

service CwaEngine {
  rpc Project(ProjectRequest) returns (ProjectReply);
  rpc Certify(CertifyRequest) returns (CertifyReply);
  rpc Pool(PoolRequest) returns (PoolReply);
}

service BeltOps {
  rpc Update(BeltUpdateRequest) returns (BeltUpdateReply);
}
```

40.2 Schemas (canonical)

40.2.1 Trace event (JSONL; append-only, hash-chained)

```
{
  "ts": "2025-09-23T14:03:11.928Z",
  "trace_id": "trc_8f2a",
  "tau": 3821441,
  "event": "TraceWrite",          // taxonomy below
  "actor": { "replica": "A", "svc": "observer-runtime" },
  "payload": { "pi": "Search", "query": "refund threshold" },
  "hash_prev": "aa94…55",
  "hash_this": "7c4b…9d",
  "schema_rev": "trace/1.4"
}
```

Event taxonomy (stable keys)

- `TickStart`, `ChannelSelected`, `Measure`, `TraceWrite`, `CWA.Project`, `CWA.Certify`, `CWA.Pool`, `AgreementPass`, `AgreementFail`, `PBHL.Update`, `PBHL.AlphaUpdate`, `PolicyGate.Trigger`, `Answer`.

40.2.2 Agreement request/response

```
// POST /agree
{
  "traces": ["trc_A", "trc_B", "trc_C"],
  "commute_matrix": { "Search:DB": true, "DB:WebScrape": false },
  "sbs_ids": ["kb:P-77#2@ab19..", "kb:SOP-119#2@c4ae.."]
}

// 200 OK
{ "pass": true, "score": 0.91, "quorum": "2-of-3", "schema_rev": "agree/1.2" }
```

40.2.3 CWA certify + pool

```
// POST /certify
{
  "projected": [{ "id":"kb:P-77#2", "vec":[...]}],
  "tests": {
    "permutation": { "panels": 50, "tolerance": 0.02 },
    "sign_flip":   { "panels": 50, "tolerance": 0.02 },
    "chunk_shuffle": { "panels": 40, "chunker": "semantic-256", "tolerance": 0.03 }
  }
}

// 200 OK
{
  "score": 0.83,
  "phase_risk": 0.32,
  "panels": { "perm": { "p": 0.41 }, "flip": { "p": 0.48 }, "chunk": { "p": 0.52 } },
  "schema_rev": "cwa-cert/1.3"
}

// POST /pool
{
  "mode": "auto",                    // "additive" | "attention" | "auto"
  "projected": [...],
  "fallback": { "cross_encoder": "ce-msmarco-v2", "window": 32 },
  "min_score": 0.78,
  "last_cert": { "score": 0.83 }     // optional short-circuit
}

// 200 OK
{ "vector": [/* pooled */], "path": "additive", "schema_rev": "cwa-pool/1.2" }
```

40.2.4 Belt update

```
// POST /belt
{
  "gap": 120,
  "flux": 26,
  "twist": 4,
  "alpha": 0.42,
  "policy": { "bands": { "residual": { "green": 0.05, "amber": 0.12 } } }
}

// 200 OK
{
  "residual": 4.6,
  "residual_over_gap": 0.038,
  "band": "green",
  "actions": [],
  "schema_rev": "belt/1.1"
}
```

40.3 Error Model & Reliability

```
{
```

```
  "error": {
    "code": "OBS-4001",             // stable, docs-linked
    "http": 400,
    "message": "Non-commuting instruments in quorum",
    "hint": "Mark WebScrape non-commuting or serialize DB.Query",
    "correlation_id": "corr_d91f",
    "retryable": false
  }
}
```

Common codes

- `OBS-4000` invalid schema, `OBS-4001` commute violation,
- `CWA-4090` cert drift detected (frozen), `CWA-4290` rate limit,
- `BELT-4030` red band freeze, `TRACE-4099` retro-edit denied.

Idempotency: if duplicate `Idempotency-Key`, server returns original result (safe at-least-once delivery).

40.4 Security & Privacy Hooks

- **Scopes** (OAuth2): `measure:write`, `trace:read`, `cwa:pool`, `belt:write`, `export:write`.
- **mTLS** inside cluster; **HSTS** at edge.
- **Field-level redaction** on `/trace/:id` via `?view=min|ops|audit`.
- **PII guard**: `/project` rejects P3 payloads (`CWA-4007`).
- **Webhooks**: HMAC-SHA256 signature in header `X-Ops-Sig`; timestamped; 5-min replay window.

40.5 Integrations — Vector DBs

Adapters (open-core)

- **pgvector**: `@observerops/adapter-pgvector` (supports pointer schema, ANN + exact).
- **FAISS**: `@observerops/adapter-faiss` (CPU/GPU, IVF/HNSW).
- **Milvus**: `@observerops/adapter-milvus` (collections per domain; compaction hooks).

Evidence pointer schema (recommended)

```
CREATE TABLE evidence_pointer (
  doc_id TEXT NOT NULL,
  section TEXT,
  sha256_128 BYTEA NOT NULL,
  projector TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (doc_id, section, sha256_128)
);
```

Ingestion (TypeScript)

```
const proj = await project(chunks, { projector: "concept-bow-1024" });
await vecdb.upsert(proj.map(p => ({
  id: `${p.doc_id}#${p.section}`,
  vector: p.vec,
  meta: { doc_id: p.doc_id, section: p.section, sha: p.sha, projector: "concept-bow-1024" }
})));
```

Query (RAG path)

```
const cand = await vecdb.search({ queryVec, topK: 200, filter: { projector: "concept-bow-1024" }});
```

40.6 Integrations — Observability

Metrics (Prometheus names)

- `observerops_cwa_score{domain=..}` gauge
- `observerops_phase_risk_index{..}` gauge
- `observerops_agreement_score` histogram
- `observerops_tau_delta_ms` histogram (Δτ)
- `observerops_rho_sync` gauge
- `observerops_slots_occupancy{tool=..}` histogram
- `observerops_slots_collisions{tool=..}` counter
- `observerops_belt_residual_over_gap{belt=..}` gauge
- `observerops_export_dlp_block_total` counter

Tracing (OpenTelemetry)

- Span names: `Ô.select`, `measure`, `cwa.project`, `cwa.certify`, `cwa.pool`, `agree`, `belt.update`.
- Span events mirror trace taxonomy; include `tau`, `policy_hash`, `ledger_head`.
- Resource attrs: `observerops.version`, `schema.revision`, `domain`, `belt.id`.

Logs

- All service logs JSON with `correlation_id`, `trace_id`, `tau`, `schema_rev`.
- Hash-chained **trace logs** (T_view) persisted to an object store + index.

40.7 Webhooks & Outbox

When fired

- `AgreementPass|Fail`, `CWA.Frozen` (drift), `PolicyGate.Trigger`, `Belt.BandChange`.

Delivery semantics

- Outbox table with exactly-once per `event_id`; retry with exponential backoff; DLQ after N tries; replay CLI.

Payload (example)

```
{
  "type": "PolicyGate.Trigger",
  "ts": "2025-09-23T14:05:22Z",
  "trace_id": "trc_8f2a",
  "tau": 3821441,
  "gate": "phase_risk_gt",
  "value": 0.63,
  "action": ["route:fallback","raise_theta:+0.02"],
  "sig": "base64(hmac_sha256(body,timestamp,secret))",
  "schema_rev": "webhook/1.0"
}
```

40.8 Tool Registry (Ô integration)

Schema

```
{
  "tool_id": "DB.Query",
  "commutes_with": ["Search"],
  "side_effects": ["cache_mutation"],
  "budget_slots": 1,
  "privacy_level": "P2",
  "timeout_ms": 8000,
  "version": "2.4.1"
}
```

Ô uses this registry to enforce commutativity, slots, and privacy rules during `ChannelSelected`.

40.9 OpenAPI & Protobuf Artifacts

- **OpenAPI**: `/specs/observerops-openapi-v1.yaml` (bundled).
- **Protobuf**: `/proto/observerops/*.proto` + generated stubs (Py/TS/Go/Java).
- **SDKs** pin spec hash; CI fails if server's `X-Spec-Hash` mismatches.

40.10 CI, Testing, and Compatibility

- **Contract tests** run against dockerized services; golden traces verified for latching (no retro-edits without new τ).
- **Backward compat**: new fields must be additive (`additionalProperties: true` in JSON Schema); enums get `UNKNOWN=0`.
- **Canary**: 1–5% traffic on new projector/α; metrics guard: accuracy parity CI within ±0.5%.

40.11 Quickstart (cURL)

```
# Project + Certify + Pool
curl -s -H "Authorization: Bearer $TOKEN" \
  -H "Idempotency-Key: $(uuidgen)" \
  -X POST $HOST/v1/project -d @project.json | tee proj.out

curl -s -H "Authorization: Bearer $TOKEN" \
  -X POST $HOST/v1/certify -d @certify.json | tee cert.out

curl -s -H "Authorization: Bearer $TOKEN" \
  -X POST $HOST/v1/pool -d @pool.json
```

What this chapter delivers

A **cohesive interface contract**—endpoints, event schemas, adapters, and telemetry—so teams can wire ObserverOps into RAG stacks, agent fleets, and governance systems with predictable behavior, strong typing, and audit-grade traces.

Chapter 41 — Licensing & Governance

Focus: Open-core tiers; policy gates & audit exports as enterprise features

---

Goal (1 sentence)

Define a **clean open-core model** that keeps core math & SDKs open (Apache-2.0) while offering **enterprise governance**—notably **policy gates** and **audit exports**—as licensed add-ons, under a

transparent **technical governance** process.

41.1 Licensing Overview

- **Open-core license (default):** Apache-2.0 for:
  - Observer Runtime (Ô/τ/Trace/agree), CWA Engine (project/cert/pool basic), Slot Allocator, Belt math + basic UI, adapters (pgvector/FAISS/Milvus), OpenAPI/Protos, test harnesses.
- **Enterprise add-ons (commercial license/EULA):**
  - **Policy Gates Service** (enforcement & escalation workflows).
  - **Audit/GRC Exporter** (signed, residency-aware bundles).
  - **Multi-tenant RBAC/SSO**, **quota & burst credits**, **drift radar & auto-tuning**, **multi-belt roll-ups**, **SaaS deployment tooling**, **Helm+SRE runbooks**.
- **SaaS option:** same features with managed SLAs and compliance perimeter.
- **No viral clauses:** open-core remains permissive; add-ons link via stable APIs.

41.2 Tiers & Features

Area	Open-core (Apache-2.0)	Enterprise (Commercial)
Observer Runtime	Traces, Ô/τ, latching, agree(), SBS pointers	SSO/RBAC, org policies, immutable admin audit
CWA Engine	Projectors, perm/flip/chunk certificates, auto-fallback	Cohort drift radar, θ auto-tuning, GPU batching svc
Slots	Budgets, occupancy/collisions, back-pressure hooks	Multi-tenant quotas, burst credits, SLA guards
Belts	PBHL math, Five-Line KPI, CSV export	Multi-belt rollups, executive packets, escalation workflows
Security/ Privacy	PII redactors, T_core/T_view split	Residency routers, export allowlists, DLP + signed bundles
Ops	OpenTelemetry metrics/traces, CLI, notebooks	Helm charts, HA reference deploys, support SLAs
Governance	Public RFCs, roadmap issues	Long-term support (LTS), backport hotfixes

---

41.3 Enterprise Feature Flags (Policy Gates & Audit Exports)

License & feature gating (server YAML)

```
license:
  id: "ent-2025-ORG-9f2a"
  features:
    policy_gates: true
    audit_exports: true
    rbac_sso: true
    quotas: true
  expires: "2026-09-30"
```

Policy Gates (enforcement workflow)

```
gates:
  cwa_min_score: { value: 0.78, action_on_fail: ["route:fallback","notify:slack#obsops"] }
  phase_risk_max: { value: 0.60, freeze_on_breach: true }
  belt_residual_band:
    green_max: 0.05
    amber_max: 0.12
```

```
    actions:
      amber: ["flux_gate:+12%","twist_board:convene<72h>"]
      red:   ["freeze_tracks:['expansion']","escalate:'L2-product'"]
```

Audit/GRC Exports (signed, residency-aware)

```
exports:
  enabled: true
  allowlist_targets: ["grc-vault-eu","grc-vault-us"]
  format: "jsonl+sig"
  signer_key: "kms://grc-export"
  residency:
    eu: ["grc-vault-eu"]
    us: ["grc-vault-us"]
  bundle_fields:
    - "trace.ledger_head"
    - "policy_hash"
    - "cwa.score"
    - "pool.path"
    - "evidence.sha"
```

Runtime check (pseudocode)

```
if not license.features["policy_gates"]:
    raise EntOnly("Policy gates require enterprise license")
```

41.4 Technical Governance Model

- **Project structure:**
  - **TSC (Technical Steering Committee)**: maintainers from runtime, CWA, belts, slots.
  - **SIGs:** Integrations, Security/Privacy, Docs/Edu.
- **Change process:** public **RFCs** (design → discussion → TSC decision), with label `rfc/X.Y`.
- **Versioning:** SemVer across packages; **compat matrix** published per release.
- **Deprecation:** `@deprecated` in APIs, **2 minor releases** grace period; migration guides.
- **Roadmap:** quarterly milestones; community voting on priorities (non-binding but visible).

41.5 Community Contributions & CLA

- **CLA**: lightweight Individual/Entity CLA for copyright clarity; bot enforces signed CLA on PRs.
- **DCO** alternative accepted if org policy prefers Developer Certificate of Origin.
- **Code of Conduct**: Contributor Covenant.
- **Contribution guide:** style, tests, docs, perf budgets, privacy lint must pass.

41.6 Security Governance

- **SECURITY.md**: coordinated disclosure; target timelines: triage 24h, fix plan 72h, patch within 14 days (severity-dependent).
- **SBOM & signing**: publish SPDX SBOMs; container images and artifacts signed (cosign/Sigstore).
- **Supply chain**: Repro builds, pinned deps, SLSA level targets.
- **VEX** advisories shipped for non-exploitable CVEs.

41.7 Telemetry, Privacy & Licensing

- **Telemetry default:** minimal, opt-in; metrics only (no content), includes `{version, policy_hash, anon instance id}`.
- **Privacy lint in CI:** banned terms & PII detectors; build fails on violations.
- **License compliance:** feature use logs **never** include customer content; only feature flags & counts.

41.8 Distribution & Verification

- **Registries:** PyPI/npm; OCI images: `cwa-svc`, `beltops-svc`.
- **Signatures:** every release includes `RELEASE_ATTESTATION.json` with `{git_sha, policy_hash, sbom_digest, cosign_sig}`.
- **Helm charts/Terraform:** open-core charts; enterprise overlays gated by license secret.

41.9 Support & SLAs (Enterprise)

Tier	Coverage	SLOs	Notes
Standard	Biz hours	Sev-1 initial response 4h	Email/Portal
Premium	24×7	Sev-1 1h / Sev-2 4h	Pager + named TAM
LTS	Backports	Critical CVEs & bugfixes	18-month lines

41.10 Legal & Policy Footers (templates)

- **Trademark policy:** "ObserverOps" and belt/CWA logos; fair use for forks; misuse takedowns for confusion.
- **Acceptable Use:** prohibits illegal content processing, deanonymization attempts, auth bypass.
- **Export control:** admin sets residency; enterprise exporter enforces allowlists and signing.
- **Cloud terms (SaaS):** data processing addendum (DPA), subprocessors list, uptime SLO (e.g., 99.9%), data residency addendum.

41.11 Artifact Pack (ship these)

- `LICENSE` (Apache-2.0), `NOTICE`
- `EULA.md` (enterprise terms; feature list & audit log clause)
- `ENTERPRISE-LICENSE-KEY.example`
- `GOVERNANCE.md`, `CONTRIBUTING.md`, `CODE_OF_CONDUCT.md`, `CLA.md`
- `SECURITY.md`, `RELEASE_ATTESTATION.json` template, `SBOM.spdx.json`
- `OPENCORE-vs-ENTERPRISE.md` (feature matrix)
- `HELM/` charts (core + enterprise overlays)
- `MIGRATIONS.md` (API & projector/α upgrade paths)

What this chapter delivers

A practical **licensing & governance frame**: keep the **observer core open**, sell **governance power-ups** (policy gates, audit exports), and run the project with transparent RFCs, SemVer, SBOM/signing, and clear support paths—so teams can adopt ObserverOps with confidence.

Part X — Roadmap & Research Agenda

Chapter 42 — Near-Term

Themes: Hierarchical belts • Multi-belt coupling • Ô multi-modal extensions

---

Goal (1 sentence)

Deliver provable, production-ready upgrades that (i) **stack PBHL belts hierarchically**, (ii) **stabilize coupled programs** across belts, and (iii) **extend Ô** to **multi-modal** instrument sets—without breaking the invariants (internal collapse, agreement, CWA, slots, PBHL).

42.1 Outcomes & Milestones (next 2–3 quarters)

Theme	Milestone	Deliverable	Acceptance Criteria
Hierarchical Belts	**HB-1**: Parent/child belts with roll-up	`/belt.hier` API + UI roll-ups	Parent Residual tracks children within ±1.5pp under synthetic mixes
	HB-2: α_parent calibration from children	Closed-form α^_P via least-squares on child streams	Error ≤ 5% vs oracle simulator over 8-week windows
Multi-Belt Coupling	**MB-1**: Belt graph controller (G=(V,E))	Coupling matrix β on edges; flux router	Residual spikes reduced ≥30% on dependency shocks
	MB-2: Coupling audit & guardrails	"No double-count" ledger + policy gates	Zero audit violations in stress harness
Ô Multi-Modal	**MM-1**: Cross-modal Ô with Π_text,image,code,audio	Selector scoring + commutativity tables	p95 latency within +10% of text-only baseline
	MM-2: **CWA-MM** certificate	Modality-aware perm/ flip/chunk + dropout tests	Iso-accuracy vs cross-encoder fusion; ≤2% false-green on sequence-aligned sets

---

42.2 Technical Sketches

A) Hierarchical Belts (stacked PBHL)

Let belts be levels ℓ=0…L, with leaf work at ℓ=0 and portfolio at ℓ=L. For belt **B**_ℓ:

$$R_\ell(t) \;=\; \Big|\; \mathrm{Gap}_\ell(t) - \big(\mathrm{Flux}_\ell(t) + \alpha_\ell(t)\cdot \mathrm{Twist}_\ell(t)\big)\Big|$$

Roll-up rules (no double count):

$$\mathrm{Gap}_\ell = \sum_{k\in \mathrm{child}(\ell)} w_{k\to\ell}\,\mathrm{Gap}_k, \quad \mathrm{Flux}_\ell = \sum_k w_{k\to\ell}\,\mathrm{Flux}_k + \underbrace{H_{\mathrm{handoff}}}_{\text{cross-team transfers}}$$

- **Handoff ledger** H: every inter-belt transfer is recorded once, with conservation check $\sum H_{k\to\ell} = 0$ over the tree per review tick τ_Q.
- **α_ℓ calibration** (weekly): regress $\mathrm{Gap}_\ell - \mathrm{Flux}_\ell$ on Twist_ℓ with ridge λ tuned by AIC; clamp α to [α_{min}, α_{max}].

API (preview)

```
POST /belt.hier/rollup   # body: {tree, weights, handoffs}
POST /belt.hier/calibrate# body: {belt_id, window_weeks, ridge_lambda}
GET  /belt.hier/:id      # returns {Gap, Flux, Twist, α, Residual, band, children[]}
```

Band logic: Parent turns **amber** if (any child red) OR R_P / G_P > 0.08. Parent stays **green** only if ≥80% children green and no handoff audit failures.

B) Multi-Belt Coupling (graph controller)

Model programs as **graph of belts** $G = (V, E)$. For edge $e = k \to \ell$, coupling β_{k→ℓ} maps **Twist_k** into **Gap_ℓ** (downstream scope shock) and/or **Flux_k** into **Flux_ℓ** (handoff capacity):

$$\mathrm{Gap}_\ell^+ = \mathrm{Gap}_\ell - \gamma\,\mathrm{Flux}_\ell + \beta_{k\to\ell}\,\mathrm{Twist}_k$$
$$\mathrm{Flux}_\ell^+ = \mathrm{Flux}_\ell + \eta\,\mathrm{Flux}_k \cdot \mathbb{1}_{H_{k\to\ell}}$$

Controller: flux router adjusts η under policy to keep R_ℓ in band while respecting slot budgets.

Safety gates

- **No-double-count:** each unit of Flux consumed appears once in a descendant; enforced by **handoff IDs**.
- **Shock limiter:** if $\beta_{k\to\ell} \cdot \mathrm{Twist}_k$ pushes R_ℓ red, freeze expansion at k or raise α_ℓ temporarily (bounded by Δα_{\max}).

YAML (policy)

```
belt_graph:
  edges:
    - from: "RAG"
      to:   "Support-App"
      beta_twist: 0.35
      flux_share_eta: 0.25
    - from: "Tooling"
      to:   "RAG"
      beta_twist: 0.20
flux_router:
  objective: "min_residual_sum"
  constraints:
    slots_total: { Support-App: 20, RAG: 16 }
  shock_limiter:
    delta_alpha_max: 0.05
    freeze_on_red: ["expansion"]
audit:
  handoff_ledger: "enabled"
  require_unique_handoff_ids: true
```

C) Ô Multi-Modal Extensions

Extend channel set $\Pi = \Pi_{\text{text}} \cup \Pi_{\text{image}} \cup \Pi_{\text{code}} \cup \Pi_{\text{audio}} \cup \Pi_{\text{telemetry}}$.
Commutativity: define C per modality pair (e.g., `ImageClassify` commutes with `TextSearch`; `CodeExec` ↔ `DB.Write` **does not**).
Selector $\hat{O}$ score combines *Collapse Entropy* S_c decrease with cost & risk:

$$\mathrm{score}(\pi) = \Delta S_c(\pi) - \lambda \cdot \mathrm{cost}(\pi) - \mu \cdot \mathrm{risk}(\pi)$$

CWA-MM certificate (cross-modal invariance tests after projection to a *joint invariant space*):

- **MP (Modality Permutation):** shuffle within-modality segments; stability ⇒ order insensitivity.
- **XFlip:** sign/orthogonal transforms respecting each modality's invariant (e.g., RGB ±, MFCC orthogonal, AST node permutes).
- **XChunk:** cross-modal chunk-shuffle preserving temporal alignment windows.

- **Dropout:** random modality dropout; robust if pooled vector stable.

Pass rule: accept additive fusion if all panels' drift stats within tolerance; else route to attention-based cross-modal fusion.

Schema (selector + certificate)

```
{
  "select_channel": {
    "candidates": ["Text.Search","Image.Classify","Audio.VAD","Code.StaticAnalysis"],
    "constraints": { "commute_with": "Text.Search", "privacy_level_lte": "P2" }
  },
  "cwa_mm": {
    "projector": "joint-invariant-1536",
    "tests": { "mp": 40, "xflip": 40, "xchunk": 30, "dropout": 30 },
    "min_score": 0.76
  }
}
```

42.3 Evaluation Plans

- **HB-Ablation:** hierarchical vs flat belts on synthetic portfolios; measure parent Residual tracking and false alarms under controlled handoffs.
- **Coupling Stress:** inject Twist shocks at upstream belts; compare Residual spikes and time-to-green with/without flux router.
- **CWA-MM Battery:** curated cross-modal datasets (text-image; text-code; AV) with controllable phase/order; compute ROC for false-green vs accuracy/latency.

KPIs

- Residual tracking error (parent vs weighted child): ≤ 1.5pp median.
- Coupling stability (sum Residual p95): −30% vs baseline.
- CWA-MM false-green rate: ≤ 2% on sequence-aligned holdouts; latency +≤10%.

42.4 Risks & Mitigations

- **Over-tight parent gates** throttle healthy children → *Mitigation:* parent band depends on weighted child mix + confidence; require two-tick confirmation.
- **Coupling oscillations** (β too high) → *Mitigation:* rate-limit β changes; add Lyapunov-style monotonicity checks on Residual sum.
- **Modality leakage** (PII via images/audio) → *Mitigation:* modality-specific redaction; forbid P3 projection; evidence pointers only.
- **CWA-MM scope creep** → *Mitigation:* start with text+image, then text+code; expansion only after green KPIs.

42.5 Artifacts to Ship

- **APIs:** `/belt.hier/*`, `/cwa/mm/certify`, Ô selector constraints per modality.
- **Configs:** belt graph YAML, CWA-MM thresholds, parent band policy.
- **Figures:** hierarchical worldsheet; belt-graph controller; CWA-MM decision tree.
- **Notebooks:** HB calibration demo; coupling stress sim; CWA-MM battery.
- **Dashboards:** Parent/child Residual; coupling flow; modality mix & CWA-MM health.

42.6 Ready-to-Research Checklist

- Define handoff ledger & uniqueness invariant.
- Implement α_parent calibration with CI & clamps.
- Build belt-graph router with β/η knobs + limits.
- Extend commutativity matrices across modalities.
- Implement **CWA-MM** tests (MP/XFlip/XChunk/Dropout) + fallbacks.
- Wire privacy guards for non-text projectors.
- Stand up evaluation harnesses & publish KPIs.

Chapter 43 — Mid-/Long-Term

Themes: Robust boundary→bulk aggregation (beyond additive) • HITL agreement • Education consortia

---

Goal (1 sentence)

Advance ObserverOps from *certified additive pooling* to **robust boundary→bulk aggregation**, formalize **human-in-the-loop (HITL) agreement** as first-class observers, and build **education consortia** that standardize datasets, labs, and audits—without relaxing core invariants.

43.1 Horizons & Milestones (12–36 months)

Theme	Milestone	Deliverable	Acceptance Criteria
Boundary→Bulk (B→B)	**BB-1** Non-additive aggregators (trimmed/median-of-means, M-estimators) behind a new **/aggregate/bulk** API	CWA++ certificate (monotone & order-aware panels)	≤1.5% accuracy loss vs attention baselines at ≤+15% latency
	BB-2 Field reconstructor (boundary → latent bulk Φ) with stability cert (Lipschitz/energy)	SMFT-R (regularized solver)	Proven stability bounds; pass rate ≥80% on rough inputs
HITL Agreement	**HITL-1** Human observers as instruments Π_H with commute matrix & SBS logging	`/hitl/measure`, `/hitl/agree`	Inter-rater→replica agreement lift ≥10pp on critical cohorts
	HITL-2 Arbitration protocol (machine↔human) with cost-aware quorums	Arbiter module + UI	Time-to-resolution ≤ 24h p95; audit trail complete
Education Consortia	**EDU-1** Open curriculum: 4 labs (qubit, gridworld, RAG battery, belts) with rubrics	v1 syllabi + instructor packs	≥10 institutions teach pilot; reproducibility ≥95%
	EDU-2 Benchmark consortium (ObserverBench) with standards & governance	Datasets, metrics cards, policy hashes	2 annual leaderboards; artifact review & trace exports required

---

43.2 Technical Sketches

A) Robust Boundary→Bulk Aggregation (beyond additive)

Problem. Additive pooling fails on heavy-tailed, adversarial, or order-dependent signals; we need robust estimators and *field* reconstruction from boundary measurements.

Stack.

1. **CWA++ certificate** (extends CWA): adds *robustness panels*
   - **Trim-Panel**: sensitivity to top-k outliers (k-sweep).
   - **MoM-Panel**: median-of-means blocks; variance collapse check.
   - **Order-Stress**: adversarial block permutations with bounded burstiness.
   - **Energy-Slope**: monotone/convex functional invariants (e.g., TV or Sobolev norm change ≤ ε).
2. **Robust aggregators** (`/aggregate/bulk?mode=`)
   - `trimmed_mean(p)`, `mom(b)` (median-of-means), `huber(δ)` M-estimator, `geomed` (geometric median in ℝ^d via Weiszfeld).
   - Auto-choose via *risk score* from CWA++.
3. **SMFT-R Field Reconstructor** (boundary→bulk):
   Given boundary observations $y = Bx + \epsilon$ with projectors B, recover latent bulk Φ by:

$$\hat{\Phi} = \arg\min_{\Phi} \| B\,\Phi - y \|_{\rho} + \lambda \|\nabla\Phi\|_{TV} + \mu \|\Phi\|_{\mathcal{H}}^2$$

   where $\|\cdot\|_{\rho}$ is a robust loss (Huber), TV is total variation (collapse geometry prior), and $\mathcal{H}$ a RKHS norm for smoothness.

API (preview)

```
POST /aggregate/bulk
{ "projected":[...], "mode":"auto", "robust": {"trim_p":0.1,"mom_blocks":8,"huber_delta":1.2},
  "cert": {"panels":{"trim":40,"mom":30,"order_stress":30,"energy":true}, "min_score":0.75} }

POST /smft/reconstruct
{ "boundary": y[], "projector_id":"P@sif-1024", "priors":{"tv":0.8,"rkhs":0.2}, "loss":"huber" }
```

Guarantees. For sub-exponential noise, MoM achieves high-probability error bounds; TV+RKHS regularization yields Lipschitz stability under small boundary perturbations (formal proofs planned in Appendix A).

B) Human-in-the-Loop Agreement (HITL as observers)

Concept. Treat **human raters** as observers O_H with channels (rubrics/tools), ticked decisions, and SBS-style shared records; integrate into the same agreement and latching framework.

Key pieces.

- **Instrument registry Π_H**: `Rubric.V1`, `Policy.Read`, `Counterexample.Tag`, each with commute matrix vs machine tools.
- **Cost-aware quorums**: minimize (latency×cost) subject to agreement confidence ≥ θ.
- **Arbitration**: when machine vs human disagree, route to *arbiter* with clarified prompts and evidence.

HITL flow (pseudo)

```
τ = tick.start()
m_ans, m_ev = machine.answer(q)
h_panel = hitl.measure(q, rubric="Rubric.V1", k=2)         # 2 raters
verdict = agree([m_ev, *h_panel.sbs_ids], commute_matrix=C)
if not verdict.passfail:
    arb = arbiter.review(q, evidence=[m_ev, h_panel], policy=ARB_POL)
tick.end(τ)
```

Artifacts.

- `/hitl/measure` (assign raters; collect SBS pointers), `/hitl/agree`, `/arbiter/review`.
- UI for rater guidance with **Ô/τ** transparency and evidence hashes.

Targets. +10–15pp agreement on high-impact cohorts; reproducibility ≥0.95 with HITL on.

C) Education Consortia (ObserverBench)

Purpose. Standardize **labs**, **datasets**, and **auditable submissions** to grow a shared competence base and create reproducible research.

Deliverables.

- **Curriculum**: 4 core labs with auto-grader (qubit collapse; gridworld Ô; RAG CWA/CWA++; belts PBHL).
- **ObserverBench**: public leaderboards where submissions must include: traces (**T**), certificate logs, policy hashes, belt KPIs—no score without artifacts.
- **Governance**: rotating academic/industry steering panel; dataset licenses; privacy rules (T_core/T_view split).

Submission schema (essentials)

```
{
  "team": "UniX-ObserverLab",
  "task": "RAG-CWA++-Battery",
  "version": "2026.1",
  "metrics": { "accuracy": 0.842, "latency_p95": 1.61, "cwa_pp": 0.79 },
  "artifacts": { "trace_head": "aa94…55", "policy_hash": "9e7a…3c", "cert_logs": "s3://..." }
}
```

43.3 Evaluation Plans & KPIs

- **B→B Robustness:** adversarial & heavy-tail suites; measure accuracy vs attention baselines, false-green rate (CWA++) ≤ 2.5%, latency overhead ≤ +15%.
- **Reconstruction Stability:** perturb boundary inputs; verify Lipschitz constants & TV regularization metrics stay within bounds; visual/quantitative residual maps.
- **HITL Agreement:** inter-rater reliability (κ, Krippendorff α), machine↔human agreement lift, time-to-resolution, cost per resolved case.
- **Consortia Health:** #institutions, #submissions with complete artifacts, reproducibility score (rerun success ≥95%).

43.4 Risks & Mitigations

- **Estimator complexity** raises latency → *Mitigation:* tiered modes (`fast`, `robust`, `full`), early-exit certificates, GPU batching.
- **HITL cost creep** → *Mitigation:* cost-aware quorums; only escalate when Phase-Risk or agreement score breach.
- **Benchmark gaming** → *Mitigation:* artifact-required scoring, rotating holdouts, random audit replay of traces.
- **Privacy leakage in consortia** → *Mitigation:* pointers+hashes only; residency-aware export; mandatory T_view.

43.5 Artifacts to Ship

- **APIs:** `/aggregate/bulk`, `/smft/reconstruct`, `/hitl/*`, `/arbiter/*`.
- **Specs:** CWA++ certificate panels; robust estimator configs; HITL rubric format.
- **Notebooks:** robust pooling demos; boundary→bulk reconstruction; HITL arbitration sims.
- **ObserverBench kit:** dataset cards, submission CLI, grader & artifact validator.
- **Figures:** CWA++ decision tree; boundary→bulk diagram; HITL quorum flow.

43.6 Ready-to-Research Checklist

- Implement CWA++ panels (Trim/MoM/Order-Stress/Energy) with CI.
- Pluggable robust estimators; auto-selector from risk score.
- SMFT-R solver with TV + RKHS priors; unit stability tests.
- HITL registry, rater UI, and arbitration protocol; cost model wired.
- ObserverBench infra (leaderboard + artifact gates); privacy policies in place.

Part XI — Glossary & Notation (Expanded)

Scope: Extended definitions with mini-examples and cross-references to chapters.

Notation & Conventions

- **Observer tuple:** $O = (S, T, \hat{O}, \tau, \Pi, C)$ — state, trace, scheduler, tick, instrument set, commutativity graph. *(Ch. 1–3)*
- **Vectors / matrices:** bold lower/upper $\mathbf{v}, \mathbf{M}$; scalars plain x; sets calligraphic $\mathcal{S}$.
- **Ticks:** τ_k are discrete commit indices. "Requires new τ" = no in-frame retro-edit. *(Ch. 2)*
- **Events & APIs:** monospace (`/project`, `TickStart`) for endpoints and event names.
- **Pointers:** angle-bracket tuples ⟨doc_id, section, sha⟩ represent SBS evidence records. *(Ch. 3, 15, 38)*
- **Bands:** green/amber/red thresholds are policy-configurable; examples given per chapter.

A. Core Objects & Operators

Agreement (cross-observer)
Convergence test over replicas/agents when effects commute and records are shared. `agree(Ta,Tb)` → `{pass|fail, score}`.
Mini-example: Two agents call `Search` then `Math` (commuting); both publish the same pointer ⟨P-77,#2,ab19…⟩ → `score=0.92` (pass). → *Ch. 3, 34*

Belt (PFBT)
Program-level worldsheet with **Gap, Flux, Twist**, coefficient α, and **Residual** $R = |Gap - (Flux + \alpha \cdot Twist)|$. Governs plan↔do closure. → *Ch. 7, 35*

C (commutativity graph)
Adjacency over instruments Π; edge (i, j) iff applying i then j equals j then i in effective outcomes. Drives safe scheduling and agreement eligibility. → *Ch. 3, 5, 34*

Certificate (CWA)
Statistical battery (permutation / sign-flip / chunk-shuffle) that returns **CWA score** ∈ **[0,1]** to gate additive

pooling. → *Ch. 6, 15, 33*

Channel / Instrument $\pi \in \Pi$
Operational measurement tool (e.g., `Search`, `DB.Query`, `Image.Classify`, `Math`). Selected by $\hat{O}$. → *Ch. 1, 5, 40*

Internal collapse (latching)
Writing to **T** at tick τ_k fixes the outcome **in-frame**; downstream control conditions on that write; retro-edits require new τ. → *Ch. 2*

Ô (projection/scheduler $\hat{O}$**)**
Control policy that selects the next instrument given S, T and constraints (commutativity, privacy, cost). → *Ch. 5, 14, 40*

Slots (capacity law)
Integer budgets for concurrent tools/memory/attention; non-fractional, non-overlap writes; collisions logged. → *Ch. 4, 12, 37*

T (trace ledger)
Append-only, hash-chained event log; typically split into **T_core** (sealed, encrypted) and **T_view** (redacted). → *Ch. 2, 38, 40*

τ (tick)
Discrete commit index for latching and cadence control; fleet sync measured via ρ, skew via $\Delta\tau$. → *Ch. 2, 5, 34, 37*

B. Fields, Metrics & Indices

AL — Attractor Load
Concentration of semantic field Ψ_m; high AL → risk of myopic selection. *Mini-example:* tool chooser avoids channels that raise AL. → *Ch. 5, 19*

Black-Hole Index (semantic)
Detector for degeneracy/near-linearity in features; triggers diversification or projector swap. → *Ch. 5, 19*

CWA score
Scalar $[0, 1]$ from certificate battery; gates `/pool` path. Typical green ≥0.78. → *Ch. 6, 20, 33*

EEI / SI
Effectiveness & Efficiency Index (ΔGap per Flux) and Sustainability Index (stability penalizing Twist/Residual variance). → *Ch. 7, 21, 35*

Phase-Risk Index
Heuristic for residual phase/order influence after projection; red >0.60 in examples. → *Ch. 6, 20, 37*

Residual (PBHL)
$R = |\,Gap - (Flux + \alpha \cdot Twist)|$. Kept within bands (e.g., R / $G \leq 5\%$ green). → *Ch. 7, 21, 35*

S_c — Collapse Entropy
Diversity of channel choices; low S_c signals over-exploitation. → *Ch. 5, 19*

ρ (Kuramoto order parameter)
Fleet sync measure in $[0, 1]$; red if $\rho < 0.82$ for >60s. → *Ch. 5, 20, 34, 37*

Δτ (desynchrony)
Tick skew; gates admission when p95 $\Delta\tau$ > threshold (e.g., 60 ms). → *Ch. 5, 20, 34, 37*

Ψ_m (meme/meaning field)
Semantic control field over context/orientation/tick: $\Psi_m(x, \theta, \tau)$. Guides $\hat{O}$. → *Ch. 5*

C. Tests, Panels & Aggregators

Permutation test
Shuffle projected items; stable scores imply order invariance. → *Ch. 6*

Sign-flip test
Flip signs / orthogonal transforms in invariant subspace; stability → phase robustness. → *Ch. 6*

Chunk-shuffle test
Reorder semantic chunks; detects sequence sensitivity. → *Ch. 6*

Guard-band rerank
On CWA pass, run a small cross-encoder/attention over top-M (e.g., 8) as an extra safety layer. → *Ch. 6, 33, 37*

Robust aggregators (beyond additive)
Trimmed mean, Median-of-Means (MoM), Huber ρ, geometric median for heavy-tailed/adversarial data. → *Ch. 43*

D. Governance, Privacy & Security

Audit/GRC export
Signed, residency-aware bundles (JSONL + signatures) containing trace heads, policy hashes, cert scores; rate-limited as a slot. → *Ch. 38, 41*

DLP gate
Export control that scans/redacts PII; blocks high-risk fields by policy. → *Ch. 38*

Evidence pointer (SBS)
Portable record ⟨doc_id, section, sha⟩ published before answer composition to stabilize agreement/objectivity. → *Ch. 3, 33–35, 38*

Non-claims statement
Standard disclaimer: reliability/safety/governance only; no consciousness/ground-truth guarantees. → *Ch. 36*

Policy gate
Runtime rule that enforces thresholds (e.g., `cwa_min_score`, `phase_risk_max`, `belt_residual_band`). Enterprise service can freeze, route, or escalate. → *Ch. 7, 33, 35, 41*

Residency router
Ensures region-bound storage/exports; denies cross-region writes without signed approval. → *Ch. 38*

T_core / T_view
Sealed source vs privacy-reduced view of traces; DSAR/erasure via crypto-erase + view regeneration. → *Ch. 38*

E. Fleet Patterns & Scheduling

Commutativity-aware quorum
Replica vote counts only measurements on commuting paths with shared pointers; typical 2-of-3. → *Ch. 34*

Desync gate
Pauses new work when sync health degrades (ρ low / $\Delta\tau$ high); barrier-resync; resume. → *Ch. 34, 37*

Handoff ledger
Conservation log for inter-team/belt transfers; prevents double counting Flux/Gap. → *Ch. 35, 42*

Quorum over-veto / tiered quorums
Risk-tiered agreement rules (e.g., 3-of-3 for high-impact actions). → *Ch. 34*

Slot collision
Attempted over-use of a tool budget causing queueing/eviction; alarms above policy (e.g., >5%/min). → *Ch. 4, 12, 37*

F. Projectors & Integrations

Projector (examples)

- `whitened-sif-1024`: additive sentence invariants; fast, CWA-friendly.
- `concept-bow-1024`: identifier-minimizing for privacy.
- `joint-invariant-1536`: multi-modal (text+image) fusion candidate. → *Ch. 6, 33, 40, 42–43*

Vector DB adapter
pgvector / FAISS / Milvus integrations that store vectors + evidence pointers; filter by `projector` meta. → *Ch. 40*

/project → /certify → /pool
Standard RAG sequence: compute invariants, certify CWA, then pool (additive or attention fallback). → *Ch. 6, 15, 33, 40*

G. Symbols (quick table)

Symbol	Meaning	Where used
$\hat{O}$	Projection/scheduling operator	Ch. 1, 5, 14
τ	Commit tick	Ch. 2, 5, 34
Π	Instrument/channel set	Ch. 1, 5
C	Commutativity graph/matrix	Ch. 3, 34
T	Trace ledger	Ch. 2, 38
Ψ_m	Semantic field	Ch. 5
α	Belt coupling coefficient	Ch. 7, 35
R	PBHL Residual	Ch. 7, 35
S_c	Collapse Entropy	Ch. 5, 19
ρ	Sync order parameter	Ch. 5, 20
$\Delta\tau$	Tick skew	Ch. 5, 20

H. Mini-Examples (micro→macro flashcards)

- **CWA pass ⇒ additive pooling**
  Given: CWA score 0.83 (green), Phase-Risk 0.32.
  Action: Use `pool.additive`; optionally guard-band rerank `M=8`. → *Ch. 6, 33*
- **Agreement check**
  Given: Three replicas publish the same pointer tuple; commute matrix allows `Search⇹Math`.
  Result: `agree` returns pass with `score=0.91`; quorum 2-of-3 met. → *Ch. 34*
- **Belt red band escalation**
  Given: $R / G = 0.14$ (red).
  Action: Freeze "expansion" tracks; convene Twist board; consider α bump within clamp. → *Ch. 35*
- **Privacy-safe projection**
  Given: Input contains PII.
  Action: Block P3 at `/project`; use `concept-bow-1024`; store only pointers + hashes. → *Ch. 38, 40*

I. Cross-Refs by Theme

- **Reliability (micro/meso):** Ch. 1–6, 14–16, 19–21

- **Fleet & Governance (macro):** Ch. 7–8, 33–35, 37–38
- **APIs/Integrations:** Ch. 9–13, 39–41
- **Roadmap:** Ch. 42–43

© 2025 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.

ObserverOps Technical Blueprint - Appendices

Appendix A — Mathematical Details

(full definitions, lemmas, theorems with proof sketches)

A.0 Notation and Standing Assumptions

- Probability spaces are $(\Omega, \mathcal{F}, \mathbb{P})$. Random variables are upper-case; realized values lower-case.
- Hilbert spaces $\mathcal{H}$; bounded operators $\mathcal{B}(\mathcal{H})$.
- An **observer** is the tuple

$$O = (S, T, \hat{O}, \tau, \Pi, C)$$

with internal state S, append-only trace $T = \{(\tau_k, \pi_k, y_k, \text{meta}_k)\}_{k \geq 0}$, scheduler $\hat{O}$, tick $\tau \in \mathbb{N}$, instrument set Π, and a compatibility/commutation graph $C \subseteq \Pi \times \Pi$. The ledger is hash-chained and only advanced on append (idempotent writes).

- **Filtration**: $\mathcal{F}_{\leq \tau}$ is the σ-algebra generated by the committed trace up to tick τ. **Internal collapse** (latching) asserts fixed-point behavior of realized outcomes w.r.t. conditional expectation and branch-dependent control after commit.

A.1 Internal Collapse (Latching as Fixed Points)

Definition A.1 (Internal Collapse)

Let Y_k be the (random) outcome at tick τ_k for channel π_k. After committing (τ_k, π_k, y_k) to the trace, **internal collapse** requires:

1. **Fixed-point (delta-certainty):**

$$\mathbb{E}[\, Y_k \mid \mathcal{F}_{\leq \tau_k} \,] \equiv y_k \,.$$

2. **Branch measurability:** any future policy f (e.g., $\hat{O}$) satisfies

$$f(S, T_{\leq \tau_k + 1}) \text{ is } \mathcal{F}_{\leq \tau_k}\text{-measurable.}$$

3. **Append-only uniqueness per tick:** at most one record with key τ_k; corrections append new records (never UPDATE).

Theorem A.1 (Latching as Conditional-Expectation Fixedness)

Let $\mathcal{E}_{\leq \tau} : \mathcal{B}(\mathcal{H}_W \otimes \mathcal{H}_M) \to \mathcal{F}_{\leq \tau}$ denote the conditional expectation onto the observer's past algebra (operator-algebraic form). Then any event $X \in \mathcal{F}_{\leq \tau}$ is a fixed point: $\mathcal{E}_{\leq \tau}(X) = X$. In particular, committed outcomes are self-certainties in-frame.
Sketch. In the von Neumann algebra view, the observer filtration $\{\mathcal{F}_{\leq \tau}\}_\tau$ is an increasing tower; conditional expectation onto $\mathcal{F}_{\leq \tau}$ fixes that subalgebra. Apply to the spectral projector of the realized outcome.

Corollary A.2 (No Silent Retro-Edit)

If writes are hash-chained and τ advances only on append, any mutation of a past record breaks the chain; therefore policies measurable w.r.t. $\mathcal{F}_{\leq \tau}$ cannot depend on a hypothetical retro-edit. *Operational guardrails* (idempotency keys, atomic measurement→commit) enforce the model.

A.2 Cross-Observer Agreement (AB-Fixedness)

We study two observers O_A, O_B with frame maps $\phi_{A \to K}, \phi_{B \to K}$ to canonical keys K. Aligned channels (π_A, π_B) for key $k \in K$ **commute** if $[\pi_A, \pi_B] = 0$ on the visited support; agreement is scored only on commuting overlaps.

Definition A.3 (Shared / Redundant Records; SBS Proxy)

Agreement is conditioned on either (i) a shared, hash-chained ledger accessible to both observers; or (ii) SBS-style redundancy: independent fragments $E_1, \ldots, E_R$ each carrying the same pointer value with redundancy proxy via majority/permutation stability.

Theorem A.3 (AB-Fixedness)

Let $K^\star \subseteq K$ be keys with (i) commuting aligned effects, (ii) shared or SBS-redundant records, and (iii) latched traces. Then for every $(k, \tau) \in K^\star$, the **effective** outcomes used by O_A and O_B coincide a.s. (agreement in downstream control).
Sketch. (1) Commutation yields order-independence of joint outcomes on $K^\star$. (2) Shared/SBS records imply both observers condition on the same σ-algebra about pointer values. (3) Latching fixes the record; downstream policies are $\mathcal{F}_{\leq \tau}$-measurable. Hence the conditional laws—and thus selected effective outcomes—match.

Proposition A.4 (Redundancy Error Bound)

If each fragment independently errs with $p < \frac{1}{2}$, the majority over R fragments has error

$$\mathbb{P}[\text{maj wrong}] \leq \exp\left(-2R\left(\tfrac{1}{2} - p\right)^2\right)$$

(Hoeffding), giving exponential decay of disagreement under independence; correlated fragments can be handled by a block/bootstrap effective R_{eff}.

Counterexamples. Non-commuting probes; hidden channels; mis-mapped frames—all break the premises and can yield disagreement despite superficial overlap.

A.3 CWA — Collapse Without Alignment (Certified Additive Pooling)

Setup

Items $X = \{x_i\}_{i=1}^{N}$. A projector P yields $z_i = P(x_i) \in \mathbb{R}^d$. Candidate additive pool $\mu_0 = \frac{1}{N}\sum_i z_i$ (or sum). CWA asks whether **order/phase/orientation** are irrelevant *after projection*, so project→add is admissible.

Certificate Family

Three invariance panels sampled K times: permutations (order), sign-flips (orientation), chunk-shuffle (boundary). Distances $\delta(\mu_k^{(j)}, \mu_0)$ are normalized and aggregated to a score $\in [0,1]$; a **Phase-Risk Index** (PRI) screens for strong order/phase structure. Gate pooling if score/PRI fail band thresholds.

Theorem A.5 (Certificate ⇒ Additive Validity, Operational Form)

Assume: (i) P is **collapse-compatible** for the task (semantic observable; no essential phase/order post-projection), and (ii) the certificate passes with score $\geq \theta$ and PRI within band. Then any Lipschitz downstream functional g on the pool satisfies

$$\max_{j,k} \| g(\mu_k^{(j)}) - g(\mu_0) \| \leq L\, \varepsilon(\theta)$$

for a task-calibrated $\varepsilon(\theta) \to 0$ as $\theta \to 1$.
Sketch. Invariance under permutations ⇒ order-indifference; sign-flip stability ⇒ orientation conventions are collapsed; re-chunk robustness ⇒ boundary effects washed out. Combine with Lipschitz continuity of g and empirical distances bounded by the certificate.

Limits. Coherent chains (e.g., strong sequential logic), positional encodings that survive projection, or projector drift can fail the tests; auto-fallback to order-aware estimators is then required.

A.4 Slot Conservation (Quantized Capacity Law)

Model

Typed slot pools $q \in \{\text{memory}, \text{tools}, \text{attention}\}$ with integer capacities N_q. Events `allocate`, `release`, `evict`, and **collisions** when no admissible slot is available without violating policy. Invariants S1–S4: (S1) integer addresses, (S2) non-overlap, (S3) explicit eviction policy, (S4) observability (occupancy/collision logs).

Lemma A.6 (Pigeonhole Non-Overlap)

If m mutually exclusive holds are active in pool q and $m > N_q$, a collision (or eviction of a guarded item) is inevitable. *Proof*. Direct pigeonhole principle; integer non-fractional slots forbid overlap.

Theorem A.7 (Stability Under Back-Pressure)

Suppose arrivals form a bounded-burst process with peak B, guard periods prevent eviction of in-flight items, and the allocator admits back-pressure to the scheduler $\hat{O}$. If $N_q \geq B$ and $\hat{O}$ respects `can_allocate` gates, the long-run collision rate admits a green-band bound (e.g., memory <0.5%, tools <1%) under stationary demand. *Sketch*. Standard queueing with admission control; the Ô-gate enforces an effective utilization $\rho < 1$ ensuring rare collisions; empirical bands follow from logs.

Ops notes. Guard periods, typed pools, and linking slot events to (τ, π) enable audits and targeted mitigation.

A.5 SMFT (Meso-Control) Essentials

We treat the semantic field $\Psi_m(x, \theta, \tau)$ with projection $P\,\Psi_m$ used by the scheduler $\hat{O}$. Two operational metrics: **Attractor Load**

$$\mathrm{AL}(x,\tau) = \frac{\max_\theta w_\theta \| P\,\Psi_m(x,\theta,\tau)\|}{\sum_\theta w_\theta \| P\,\Psi_m(x,\theta,\tau)\| + \varepsilon},$$

and **Collapse Entropy** $S_c(\tau) = -\sum_\theta p_\theta(\tau) \log p_\theta(\tau)$ from recent selections. Fleet sync uses Kuramoto-style order parameter ρ. These feed $\hat{O}$'s channel selection score, with compatibility bonuses and cadence gates.

A.6 PFBT — Purpose-Belt Holonomy Law (Macro Closure)

Worldsheet Variables (Discrete Windows)

For window n: Gap G_n, Flux F_n, Twist T_n, coupling α_n, residual $R_n = G_n - (F_n + \alpha_n T_n)$. Evolution model

$$G_{n+1} = G_n - (F_n + \alpha_n T_n) + \eta_n, \qquad R_n = G_n - (F_n + \alpha_n T_n),$$

with η_n capturing sensing error/latent flux, bounded by acceptance checks (two-boundary Stokes, gluing, 4π periodicity).

Theorem A.8 (Residual Control via Two-Timescale Controller)

Consider Flux-gate (fast) and Twist-step (slow) controllers with gains (k_P^F, k_I^F) and bounded step $|\Delta T| \leq \tau_{\max}$; assume: (i) acceptance checks bound $|\eta_n| \leq \bar{\eta}$, (ii) Twist updates are separated by at least four Flux-gate time constants, (iii) α_n varies slowly with bounded variance. Then there exist gains and bands $(\varepsilon_g, \varepsilon_a)$ such that R_n remains within $[-\varepsilon_a, \varepsilon_a]$ and returns to $[-\varepsilon_g, \varepsilon_g]$ after bounded disturbances.
Sketch. Linearize around $R = 0$. The fast loop stabilizes the inner difference equation for G_n under bounded η_n. Singular-perturbation (two-timescale) arguments ensure the slow Twist-step adjusts αT to cancel steady residual bias; acceptance checks guarantee model closure so the residual error behaves as an input-bounded disturbance.

Audit primitives. Compute Gap/Flux/Twist by quadrature over belt edges and worldsheet; assert PBHL residual $r = |\,\text{Gap} - (\text{Flux} + \alpha\,\text{Twist})\,| \le \tau_{\text{PBHL}}$; verify gluing and 4π checks; update the Five-Line KPI.

A.7 Aggregated Assumptions & Failure Modes

- **A-IC.** Append-only, hash-chained trace; tick advances only on append (breaks ⇒ no latching).
- **A-COM.** Agreement scored only on commuting overlaps (breaks ⇒ order artifacts).
- **A-SBS.** Either shared ledger or redundant fragments with high stability (breaks ⇒ inconsistent conditioning).
- **A-CWA.** Certificate pass + PRI within band before additive pooling (breaks ⇒ order/phase leakage).
- **A-SLOT.** Integer, non-overlap slots with Ô-gated admission (breaks ⇒ collisions/thrash).
- **A-PBHL.** Acceptance checks bound closure error; two-timescale controller tuned (breaks ⇒ residual drift).

What this appendix gives you in practice

- A precise latching semantics you can unit-test.
- Agreement conditions with quantitative redundancy bounds.
- A certified criterion for when **project→add** is safe.
- Slot allocator invariants with stability guarantees.
- A macro closure law with controller-level guarantees and auditable checks.

Appendix B — Complexity & Scaling

(certificate costs, belt-controller overhead, sync metrics complexity)

B.0 Symbols & Scale Parameters

- N: items per pool (e.g., projected vectors).
- d: projection dimension (post-P).
- $K_{\text{perm}}, K_{\text{flip}}, K_{\text{chunk}}$: certificate panel sizes.
- M: number of belts (programs) monitored.
- E, F: belt mesh edges/faces used by acceptance checks.
- R: observers/agents whose ticks we synchronize.
- W: rolling window length (ticks) for metrics.
- b: batch size; β: mini-batch/streaming fraction.
- $\texttt{nnz}(\cdot)$: sparsity; $\texttt{FLOPs}(\cdot)$: floating-point ops.

B.1 CWA Certificate — Cost Model & Optimizations

Baseline pipeline. Project, then test invariances by (re)pooling perturbed copies and measuring deviation:

$$\mu_0 = \frac{1}{N}\sum_{i=1}^{N} z_i, \quad z_i = P(x_i) \in \mathbb{R}^d.$$

B.1.1 Time & Memory Complexity

1. **Projection** P

- Linear map (e.g., whitening): $\mathcal{O}(Nd^2)$ if dense; $\mathcal{O}(Nd)$ if P is diagonal/orthogonal pre-factored.
- Neural projector: $\texttt{FLOPs}(P) \approx \sum_\ell \mathcal{O}(Nd_{\ell-1}d_\ell)$.
- **Memory:** store $Z = [z_i] \to \Theta(Nd)$.

2. **Pooling once** (mean/sum): $\mathcal{O}(Nd)$; memory $\Theta(d)$.

3. **Permutation panel** (K_{perm} samples)

- If the candidate aggregator is additive, naive re-pooling is redundant (mean is permutation-invariant). We still **evaluate the downstream functional** g (e.g., retrieval with μ, scoring, risk metrics).
- **Cost:** $\mathcal{O}(K_{\text{perm}} \cdot \texttt{cost}[g(\mu)])$.
- **If you must re-pool** (e.g., chunk-first pipelines), do it from cached prefix sums: precompute $S = \sum_i z_i$ once; each shuffle uses the same $S \Rightarrow \mathcal{O}(d)$.

4. **Sign-flip panel** (K_{flip} masks)

- Apply diagonal $D_j \in \{\pm 1\}^{d\times d}$: $\mu_j = D_j \mu_0$.
- **Cost:** $\mathcal{O}(K_{\text{flip}} d)$ for vector flips + $\mathcal{O}(K_{\text{flip}} \cdot \text{cost}[g])$.

5. **Chunk-shuffle panel** (K_{chunk} samples)

- Re-partition into chunks, re-project if projector depends on chunk boundaries, then pool.
- **Worst-case:** $\mathcal{O}(K_{\text{chunk}} \cdot Nd)$ (if P re-runs).
- **With cached Z:** $\mathcal{O}(K_{\text{chunk}} \cdot d)$ if pooling unchanged; otherwise proportional to affected items only.

Total (typical):

$$\text{FLOPs} \approx \underbrace{\text{FLOPs}(P)}_{\text{one-off}} + \underbrace{\mathcal{O}(Nd)}_{\text{baseline pool}} + \underbrace{\mathcal{O}((K_{\text{perm}} + K_{\text{flip}}) \cdot \text{cost}[g])}_{\text{light}} + \underbrace{\mathcal{O}(K_{\text{chunk}} \cdot C_{\text{chunk}})}_{\text{varies}}$$

where $C_{\text{chunk}} \in \{d,\ Nd\}$ depending on whether re-projection is needed.

Memory footprint: $\Theta(Nd)$ to cache Z + $\Theta(d)$ for aggregates + $\Theta(K.d)$ if you store all perturbation outputs (usually streamed → $\Theta(d)$).

B.1.2 Statistical Power vs. Panel Size

Detecting an order/phase sensitivity with effect size $\delta = \mathbb{E}[\| g(\mu_j) - g(\mu_0)\|]$ and variance σ^2 under random perturbations requires roughly

$$K_{\text{eff}} \gtrsim \frac{c\,\sigma^2}{\delta^2} \log \frac{1}{\alpha}$$

replicates to attain type-I error α (Hoeffding/CLT heuristics). Use **sequential testing**: stop early if the running CI clears the gate.

Rule-of-thumb bands (production):

- Fast path: $K_{\text{perm}} = 16$, $K_{\text{flip}} = 8$, $K_{\text{chunk}} = 8$.
- Caution path (borderline PRI): double chunk panel; keep others fixed.
- Heavy path (audits): $K_{\text{perm}} = 64$ with stratified permutations.

B.1.3 Scaling Tricks

- **Prefix sums & sketches.** Precompute $S = \sum z_i$ and second moment $Q = \sum z_i z_i^\top$ ($\mathcal{O}(Nd^2)$ if kept dense). Many invariance scores (variance of pooled outputs) reduce to $S, Q \Rightarrow \mathcal{O}(d^2)$ independent of N.
- **Subsampling with control variates.** Use βN items per certificate draw; correct bias with a control-variate term from the global S.
- **Mixed-precision.** FP16 for perturbation passes; keep S, Q in FP32.
- **Amortize g.** If g is retrieval, pre-index candidates; each μ_j becomes one query (cost depends on ANN engine, often $\widetilde{\mathcal{O}}(d)$).
- **Adaptive K.** Early exit when the running score is $\ll$ lower band or $\gg$ upper band.

B.2 PBHL / Belt Controller — Overhead & Throughput

Two-timescale controller (Flux-gate fast, Twist-step slow) plus **acceptance checks** (closure tests) and **telemetry**.

B.2.1 Per-Belt Compute

Per window n and belt m:

1. **Five-Line KPI update** (Gap, Flux, Twist, Coherence, Residual): $\mathcal{O}(1)$ arithmetic on sufficient statistics.
2. **Flux-gate** (fast loop at f_F Hz): $\mathcal{O}(1)$ control law (PID-like) + constant-time gates.
3. **Twist-step** (slow loop at $f_T \ll f_F$): $\mathcal{O}(1)$ bounded update and policy checks.
4. **Acceptance checks** over belt mesh:
   - Boundary integrals (two-boundary / gluing): $\mathcal{O}(E)$.
   - Surface integral / worldsheet quadrature: $\mathcal{O}(F)$.
   - 4π periodicity test: $\mathcal{O}(1)$ on accumulated holonomy.

Per-belt per-window cost: $\mathcal{O}(E + F) + \mathcal{O}(f_F + f_T)$ with tiny constants.
Fleet cost (M belts): linear: $\mathcal{O}(M(E + F))$. Meshes are coarse in ops practice (dozens–hundreds of cells), so acceptance checks dominate only during recomputation.

B.2.2 Memory & I/O

- **Ring buffers** for W windows of sufficient stats: $\Theta(MW)$.
- **Mesh storage**: $\Theta(E + F)$ per belt (small).
- **Telemetry export**: proportional to event rate; compress KPI streams via delta encoding (typ. $<$ 1 kB/s/belt).

B.2.3 Latency Budget

- Fast loop target: < 5 ms/belt update at $f_F \in [1, 5]$ Hz on CPU.
- Slow loop: < 1 ms at $f_T \in [0.05, 0.5]$ Hz.
- Acceptance checks: amortize (e.g., every K_A windows or when Residual ↑).

Throughput scaling: 1 core can comfortably service $\mathcal{O}(10^2)$ belts at the above rates; scale out linearly across cores/threads because belts are embarrassingly parallel.

B.3 Sync Metrics — Complexity & Streaming Algorithms

We compute **Kuramoto order parameter** ρ, **Ô-desynchrony** $\Delta\tau$ stats, and optional **pairwise phase** matrices.

B.3.1 Kuramoto ρ

Phases $\theta_r \in [0, 2\pi)$ per agent $r \in \{1, \ldots, R\}$.

$$\rho\, e^{i\psi} = \frac{1}{R} \sum_{r=1}^{R} e^{i\theta_r}.$$

- **Time:** $\mathcal{O}(R)$ per tick (vectorized complex sum).
- **Memory:** $\Theta(1)$ for running sum; $\Theta(R)$ if you retain phases.
- **Sliding window** W**:** maintain $S_W = \sum_{t \in W} \sum_r e^{i\theta_{r,t}}$ with a queue → $\mathcal{O}(1)$ amortized update.

B.3.2 Ô-Desynchrony $\Delta\tau$

Let τ_r be last commit tick per agent; we track dispersion:

$$\Delta\tau_{\text{RMS}} = \sqrt{\frac{1}{R} \sum_r (\tau_r - \bar{\tau})^2}, \quad \bar{\tau} = \frac{1}{R} \sum_r \tau_r.$$

- **Time:** $\mathcal{O}(R)$ per update; streaming Welford algorithm keeps it $\mathcal{O}(1)$ per event.
- **Percentile bands:** t-digest or GK-summary → $\mathcal{O}(\log 1/\epsilon)$ update, $\Theta(1/\epsilon)$ memory.

B.3.3 Pairwise Phase / Lag (Optional, Heavy)

- Full matrix of pairwise lags: $\mathcal{O}(R^2)$ per snapshot; memory $\Theta(R^2)$.
- **Downscaling:** sample $S \ll R$ pivots ($\mathcal{O}(SR)$), or compute graph-sparsified affinity (k-NN in phase space).
- **When to use:** only during diagnostics; never on critical path.

B.3.4 End-to-End Overhead

A typical fleet ($R \in [10^2, 10^4]$, $W \in [60, 600]$):

- Kuramoto + $\Delta\tau$: CPU-friendly, $\mathcal{O}(R)$ per second; < 1 MB memory for summaries.
- Pairwise: enable temporarily with S-pivot sampling.

B.4 Putting Costs Together (Rules of Thumb)

B.4.1 CWA in RAG/Embeddings

- **Projection:** cache Z once per index refresh.
- **Certificate:** start with $K_{\text{perm}} = 16, K_{\text{flip}} = 8, K_{\text{chunk}} = 8$. With cached Z, CPU time is dominated by g (ANN queries) not by re-pooling.
- **Latency target:** $p95$ certificate < 50–150 ms for $d \in [256, 1024]$, $K_\cdot \leq 32$ on commodity CPUs; push heavy panels to async audit lanes.

B.4.2 BeltOps

- Keep meshes small (dozens of cells). Run acceptance checks on schedule or on Residual excursions. Belt updates are $\mathcal{O}(1)$ and parallel across M.

B.4.3 Sync

- Compute ρ and $\Delta\tau$ continuously; reserve pairwise/graph diagnostics for anomalies or research runs.

B.5 Engineering Patterns for Scale

- **Batching & vectorization.** Aggregate perturbations into a single tensor op: stack $\{D_j \mu_0\}_j$ and run g once in batched mode.
- **Early exit gates.** If first 4–8 perturbations produce a score far inside green/red, stop.
- **Cold/warm/hot lanes.** Cold (audits, large K), warm (borderline), hot (minimal panels).
- **Sketch-first.** Run certificate on sketches (e.g., $\beta \approx 0.1$ subsample) and escalate only on failure.
- **Sparse / low-rank.** If Z is sparse or low-rank, store factorizations; compute S, Q in compressed form.

- **Pinned CPU cores for belts.** Assign belts to cores for cache locality; use lock-free ring buffers for KPI windows.
- **Telemetry budgets.** Cap event rates; use drop-counters and lossy compression for noncritical streams.

B.6 Complexity Table (at a glance)

Component	Time (typical)	Memory	Notes
Projection P	$\mathcal{O}(Nd)$–$\mathcal{O}(Nd^2)$	$\Theta(Nd)$	Cache Z (projected items)
Pool once	$\mathcal{O}(Nd)$	$\Theta(d)$	Prefix sums / one-pass mean
Certificate (perm)	$\mathcal{O}(K_{\text{perm}} \cdot \text{cost}[g])$	$\Theta(d)$	Mean is permutation-invariant; cost shifts to downstream g
Certificate (flip)	$\mathcal{O}(K_{\text{flip}} \cdot d + K_{\text{flip}} \cdot \text{cost}[g])$	$\Theta(d)$	Vectorized sign masks
Certificate (chunk)	$\mathcal{O}(K_{\text{chunk}} \cdot d)$–$\mathcal{O}(K_{\text{chunk}} \cdot Nd)$	$\Theta(d)$–$\Theta(Nd)$	Depends on whether re-projection is needed
Belt fast loop	$\mathcal{O}(1)$	$\Theta(1)$	PID-like Flux-gate
Belt slow loop	$\mathcal{O}(1)$	$\Theta(1)$	Bounded Twist-step
Acceptance checks	$\mathcal{O}(E+F)$	$\Theta(E+F)$	Boundary/surface integrals; amortizable
Kuramoto ρ	$\mathcal{O}(R)$	$\Theta(1)$–$\Theta(R)$	Streaming complex sum
Ô-desynchrony $\Delta\tau$ stats	$\mathcal{O}(R)$ or $\mathcal{O}(1)$ (streaming)	$\Theta(1)$	Welford + t-digest/GK for percentiles
Pairwise phase	$\mathcal{O}(R^2)$	$\Theta(R^2)$	Diagnostics only (use pivots/sparsification)

B.7 Sizing Checklist

1. **Pick** K. by PRI band and latency SLOs; enable sequential early-exit.
2. **Cache** Z and precompute S, Q; switch chunk panel to sketch mode when stable.
3. **Set belts** with coarse meshes; run acceptance checks on Residual excursions.
4. **Stream sync metrics** ($\rho, \Delta\tau$); downsample pairwise diagnostics.
5. **Observe budgets**: CPU (certificate & g), RAM (Nd), I/O (telemetry).

Outcome: You now have concrete big-O bounds, constant-factor tricks, and deployment patterns to size the certificate battery, keep belt controllers cheap, and run fleet-level sync metrics without quadratic surprises.

Appendix C — API Spec

(request/response schemas, examples, error codes)

C.0 Overview

- **Base URL (prod):** `https://api.observerops.io/v1`
- **Auth:** `Authorization: Bearer <token>`
- **Content-Type:** `application/json; charset=utf-8`
- **Idempotency:** `Idempotency-Key: <uuid>` on all POST/PUT that mutate state.
- **Tracing (recommended):** `X-Trace-Id`, `X-Parent-Span`, `X-Request-Id` (UUIDv4).
- **Versioning:** URI (`/v1`) + `Accept-Version` header for minor features.
- **Clock:** all timestamps ISO-8601 UTC; ticks are integers.

C.1 Common Types (JSON Schema fragments)

```
// C-TYPES (Draft 2020-12; shared snippets)
{
  "$defs": {
    "UUID": { "type": "string", "format": "uuid" },
    "Tick": { "type": "integer", "minimum": 0 },
    "Vector": { "type": "array", "items": { "type": "number" }, "minItems": 1 },
    "KeyVal": { "type": "object", "additionalProperties": true },
    "CommuteMatrix": {
      "type": "array",
      "items": { "type": "array", "items": { "type": "boolean" } }
    },
    "Channel": {
      "type": "object",
      "required": ["id", "type"],
      "properties": {
        "id": { "type": "string" },
        "type": { "type": "string", "enum": ["sensor","tool","query","sim"] },
        "meta": { "$ref": "#/$defs/KeyVal" }
      }
    }
  }
}
```

C.2 Error Model

HTTP uses standard codes; body conforms to:

```
{
```

```
  "error": {
    "code": "INVALID_ARGUMENT",
    "message": "Field 'pi' is missing",
    "field_errors": [{"field":"pi","reason":"required"}],
    "details": {"hint":"Provide a registered channel id"},
    "retry_after_ms": 0,
    "request_id": "7b6c3c2e-..."
  }
}
```

Canonical `error.code` values

Code	Meaning	Typical HTTP
INVALID_ARGUMENT	malformed/failed validation	400
FAILED_PRECONDITION	latching/commute/slot pre-req not met	412
CONFLICT	idempotency replay with different body	409
NOT_FOUND	resource absent	404
PERMISSION_DENIED	auth/ACL	403
RATE_LIMITED	token or org throttled	429
INTERNAL	unexpected server error	500
CERTIFICATE_FAILURE	CWA gate failed	422
INCOMPATIBLE_CHANNELS	non-commuting or unmapped frame	409
SLOT_EXHAUSTED	pool at capacity	429
TRACE_IMMUTABLE	attempted retro-edit	409

C.3 Endpoints

1) `POST /measure` — perform a measurement and commit a latched trace

Request

```
{
  "observer_id": "e7b0c7b1-...",
  "tick": 1021,
  "pi": "tool.search.v2",           // channel id
  "input": {"q":"site:docs foo"},  // channel-specific payload
  "state_ref": "s/obs/e7b0c7b1:v42",
  "commit": true,
  "slots": {"tools": 1},
  "meta": {"run_id":"RAG-7821","tenant":"acme"}
}
```

Response

```
{
  "trace_id": "tr_01J9HF5Z0N...",
  "observer_id": "e7b0c7b1-...",
  "tick": 1021,
  "pi": "tool.search.v2",
  "outcome": {"y":{"docs":3,"took_ms":87}},
  "hash": "b8d2e7...ab",
  "prev_hash": "aa7ed1...fe",
  "slots": {"tools":{"allocated":1,"released":0}},
  "committed_at": "2025-09-23T11:05:21Z"
}
```

Notes

- Latching: past records immutable; retries must reuse `Idempotency-Key`.
- Slot admission enforced before execution (`FAILED_PRECONDITION`/`SLOT_EXHAUSTED`).

---

2) `GET /trace/{trace_id}` — retrieve immutable trace record

Response

```
{
  "trace_id":"tr_01J9HF5Z0N...",
  "observer_id":"e7b0c7b1-...",
  "tick":1021,
  "pi":"tool.search.v2",
  "outcome":{"y":{"docs":3,"took_ms":87}},
  "meta":{"run_id":"RAG-7821"},
  "hash":"b8d2e7...ab",
  "prev_hash":"aa7ed1...fe",
  "chain_ok": true
}
```

---

3) `POST /agree` — cross-observer agreement check

Request

```
{
  "A": {"observer_id":"obs_A","trace_ids":["tr_...21","tr_...22"]},
  "B": {"observer_id":"obs_B","trace_ids":["tr_...a1","tr_...a2"]},
  "frame_map": {"pi_map":[["sensor.z","sensor.z"],["tool.db","tool.db"]]},
  "commute_matrix": [[true,true],[true,true]],
  "shared_ledger": true,
  "redundancy": {"fragments":5,"policy":"majority"},
  "metrics": {"sample":"overlap","score":"jaccard"}
}
```

Response

```
{
  "pass": true,
  "score": 0.93,
  "overlap_keys": ["sensor.z@1021","tool.db@1022"],
  "non_commuting_pairs": [],
  "diagnostics": {"disagreements":[], "redundancy_effective":4.7}
}
```

Errors: `INCOMPATIBLE_CHANNELS`, `FAILED_PRECONDITION` (no shared/SBS redundancy).

---

4) `POST /project` — apply a projector P to items

Request

```
{
  "projector": {
    "type": "linear",               // "linear" | "neural" | "pca" | "whiten"
    "params": {"W_ref":"proj/W:2025-09-01", "normalize": true}
  },
  "items": [{"id":"c1","x":[0.1,0.3,...]},{"id":"c2","x":[-0.2,0.4,...]}],
  "return_stats": true
}
```

Response

```
{
  "projected": [
    {"id":"c1","z":[0.07,0.11,...]},
    {"id":"c2","z":[-0.04,0.18,...]}
  ],
  "stats": {"n": 2, "d": 768, "mean_l2": 0.92}
}
```

---

5) `POST /pool` — additive pooling with CWA certificate & auto-fallback

Request

```
{
  "projected": [{"id":"c1","z":[...]},{"id":"c2","z":[...]}],
  "aggregator": {"type":"mean"},        // "mean" | "sum" | {"weighted":{...}}
  "certificate": {
    "perm": {"k":16},
    "flip": {"k":8, "mode":"random"},   // or "axes"
    "chunk": {"k":8, "policy":"reshuffle"},
    "thresholds": {"cwa_min":0.78, "pri_max":0.35},
    "sequential": {"early_exit": true}
  },
  "downstream": {"g":"retrieval.v3","topk":5}  // optional “g” evaluated per panel
}
```

Response (pass)

```
{
  "pooled": {"mu":[0.02, -0.01, ...], "norm": 1.03},
  "cwa": {"score": 0.86, "pri": 0.22, "decision": "PASS",
          "panels":{"perm":16,"flip":8,"chunk":8}},
  "fallback": null,
  "diagnostics": {"delta_max": 0.04, "delta_med": 0.01, "time_ms": 94}
}
```

Response (fail with fallback suggestion)

```
{
  "pooled": null,
  "cwa": {"score": 0.41, "pri": 0.62, "decision": "FAIL"},
  "fallback": {"estimator":"attention.pool.v2","hint":"enable positional kernels"},
  "diagnostics": {"delta_max": 0.29, "phase_risk":"HIGH"}
}
```

Errors: `CERTIFICATE_FAILURE` (if `require_pass=true` header), `INVALID_ARGUMENT`.

---

6) `POST /belt` — update PBHL belt & compute Residual, gates

Request

```
{
  "belt_id":"belt/ACME-Support",
  "window": {"start":"2025-09-15T00:00:00Z","end":"2025-09-22T00:00:00Z"},
  "gap": {"value": 0.37, "units":"backlog_fraction"},
  "flux": {"value": 0.29},
  "twist": {"value": 0.05},
  "alpha": 1.4,
  "coherence": 0.82,
  "acceptance": {"run_checks": true, "mesh_ref":"mesh/beltA:v7"},
  "policy": {"residual_max": 0.06}
}
```

Response

```
{
  "belt_id":"belt/ACME-Support",
  "residual": 0.03,
  "pbhl": {"gap":0.37,"flux":0.29,"twist":0.05,"alpha":1.4},
  "acceptance": {"boundary_ok": true, "gluing_ok": true, "period_4pi_ok": true},
  "gate": {"status":"OPEN","reason":null},
  "kpi": {"EEI":0.74,"SI":0.69},
  "next_review":"2025-09-29T00:00:00Z"
}
```

Errors: `FAILED_PRECONDITION` (acceptance check failure → gate closed), `INVALID_ARGUMENT`.

---

7) `GET /belt/{belt_id}/kpi?since=...&window=...`

Returns Five-Line KPI time series (Gap, Flux, Twist, Coherence, Residual) with paging.

---

8) `GET /sync` — fleet-level sync metrics

Response

```
{
  "fleet_id":"acme-fleet",
  "rho": 0.91,
  "delta_tau": {"rms": 1.8, "p90": 3, "p99": 7},
  "R": 312,
  "window": {"seconds": 300},
  "updated_at":"2025-09-23T11:05:21Z"
}
```

C.4 Events & Webhooks

Event types

`TickStart`, `ChannelSelected`, `TraceWrite`, `AgreementPass`, `AgreementFail`, `CWA.Pass`, `CWA.Fail`, `PBHL.Update`, `PolicyGate.Trigger`.

Webhook delivery

- `POST` to your endpoint with `ObserverOps-Signature: t=<ts>,v1=<hmac-sha256>` over raw body using your secret.
- Retries: exponential backoff up to 24h; idempotent via `event_id`.

Example event (CWA.Pass)

```
{
  "event_id":"ev_01J9HG34...",
  "type":"CWA.Pass",
  "created":"2025-09-23T11:05:22Z",
  "data":{
    "pool_id":"pool_01J9H...",
    "cwa":{"score":0.86,"pri":0.22},
    "projector":"proj/W:2025-09-01"
  }
}
```

C.5 Pagination & Filtering

- Cursor-based: `?cursor=...&limit=50`.
- Responses include: `{ "items":[...], "next_cursor": "..." }`.
- Time filters: `?from=2025-09-01T00:00:00Z&to=...`.

C.6 Rate Limits

- Default: **120 requests/min** per org; burst 240.
- Exceeding returns `429 RATE_LIMITED` with `Retry-After`.

C.7 Security Notes

- OAuth2 client-credentials or PAT; scopes: `measure:write`, `trace:read`, `pool:write`, `agree:write`, `belt:write`, `metrics:read`.
- PII-aware traces: optional redaction policies and field-level encryption at rest.

C.8 Examples (cURL)

/pool with certificate

```
curl -X POST https://api.observerops.io/v1/pool \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-Type: application/json" \
 -H "Idempotency-Key: $(uuidgen)" \
 -d '{
  "projected":[{"id":"c1","z":[0.1,0.2]},{"id":"c2","z":[-0.1,0.0]}],
  "aggregator":{"type":"mean"},
  "certificate":{"perm":{"k":16},"flip":{"k":8},"chunk":{"k":8},
                 "thresholds":{"cwa_min":0.8,"pri_max":0.4}}
 }'
```

/agree

```
curl -X POST https://api.observerops.io/v1/agree \
 -H "Authorization: Bearer $TOKEN" \
 -d '{"A":{"observer_id":"obsA","trace_ids":["tr_1","tr_2"]},
      "B":{"observer_id":"obsB","trace_ids":["tr_a","tr_b"]},
      "frame_map":{"pi_map":[["sensor.z","sensor.z"]]},
      "commute_matrix":[[true]],
      "shared_ledger":true}'
```

C.9 SDK Model (TypeScript types excerpt)

```
type PoolDecision = "PASS" | "FAIL";
interface CwaResult { score: number; pri: number; decision: PoolDecision; }
interface PoolResponse {
  pooled?: { mu: number[]; norm: number };
  cwa: CwaResult;
  fallback?: { estimator: string; hint?: string } | null;
  diagnostics?: Record<string, unknown>;
}
```

C.10 Error Codes (extended table)

code	Explanation	Action
INVALID_ARGUMENT	Schema/constraints violated	Fix payload; see `field_errors`
FAILED_PRECONDITION	Latching/commute/SBS/acceptance unmet	Satisfy invariant or change mode
CONFLICT	Idempotency mismatch or slot eviction guard	Reuse body/key or new key
CERTIFICATE_FAILURE	CWA score or PRI outside bands	Switch to fallback estimator
INCOMPATIBLE_CHANNELS	Non-commuting or unmapped frames	Repair `frame_map/commute_matrix`
SLOT_EXHAUSTED	Pool capacity hit	Back-pressure or increase slots
TRACE_IMMUTABLE	Attempted retro-edit	Append a corrective record
RATE_LIMITED	Throttle exceeded	Honor `Retry-After`
NOT_FOUND / PERMISSION_DENIED	Missing/forbidden	Verify ids/scopes

---

That's the complete, production-ready API surface for ObserverOps: measurement & latching, agreement checks, projection→certificate→pool, belt closure, and fleet sync—complete with schemas, examples, and error semantics.

Appendix D — Configuration Playbooks

(YAML/JSON templates for thresholds & policies)

Below are production-ready templates you can copy into your config repo. They use **profiles** (dev/stage/prod), **YAML anchors/aliases**, and **env interpolation** (`${ENV_VAR:-default}`).

D.1 Org-Level Stack Config (YAML)

```
# observerops.yaml
version: 1
org: acme
env: ${ENV:-prod}
region: eu-west-2
hashing: blake3

profiles:
  prod: &prod
    observer:
      trace:
        immutability: strict           # append-only, no retro-edits
        hash_chain: blake3
        retention_days: 365
        redact:
          pii_fields: ["email","phone","ssn"]
          policy: "drop"               # drop | mask | hash
        idempotency:
          enforce: true
          window_s: 86400
      scheduling:
        Ô_policy: "compatibility-first"
        tick:
          cadence_ms: 250
          max_retries: 2
          jitter_ms: [15, 60]
    slots:
      memory:
        capacity: 2048                 # items/frames
        guard_ms: 250                  # no-evict guard
      tools:
        capacity: 4
        parallel_permits: 2
      attention:
        capacity: 12
        policy: priority               # priority | fifo
      eviction:
        policy: lru
        protect_tags: ["inflight","latched"]
      backpressure:
        enabled: true
        gate: "Ô"
        thresholds:
          occupancy_amber: 0.75
          occupancy_red: 0.90
          collision_budget_pct: 1.0
    cwa:
      projector:
        type: "linear"                 # linear | neural | whiten | pca
        params:
          W_ref: "proj/W:${PROJ_VER:-2025-09-01}"
          normalize: true
      certificate:
        thresholds:
          cwa_min: 0.80
          pri_max: 0.35
```

```
        delta_mu_max: 0.08           # norm change tolerance
      panels:
        perm:  { k: 16 }
        flip:  { k: 8,  mode: "random" }   # random | axes
        chunk: { k: 8,  policy: "reshuffle" }
      sequential:
        early_exit: true
        upper_stop: 0.90
        lower_stop: 0.30
    fallback:
      estimator: "attention.pool.v2"
      enable_on:
        - reason: "PRI_HIGH"   # Phase risk high
          pri_min: 0.50
        - reason: "SCORE_LOW"  # CWA score too low
          cwa_max: 0.75
  agreement:
    shared_ledger: true
    commute_matrix_ref: "cmat/global:v3"
    min_overlap_keys: 2
    redundancy:
      fragments: 5
      policy: "majority"
      min_effective: 4.0
  sync:
    kuramoto_rho_target: 0.85
    desync:
      delta_tau:
        p95_max: 5
        p99_max: 12
      actions:
        - if: "rho < 0.70"
          then: ["slowdown:0.10", "reschedule:jitter(50-150ms)"]
        - if: "p99 > 12"
          then: ["gate:measure@tools", "alert:ops"]
  pbhl:
    alpha:
      default: 1.2
      bounds: [0.8, 1.8]
    residual:
      amber: 0.04
      max: 0.06
    controllers:
      flux_gate:
        kp: 0.5
        ki: 0.2
        rate_hz: 2
      twist_step:
        k: 0.10
        min_interval_s: 1800
        max_step: 0.05
    acceptance_checks:
      frequency: "every_6h"
      on_residual_spike: true
      mesh_ref: "mesh/beltA:v7"
      tests: ["boundary","gluing","period_4pi"]
    gates:
      close_on:
        - residual_gt: 0.06
          windows: 2
        - acceptance_fail: true
  telemetry:
    level: info
    sample_rates:
      TraceWrite: 1.0
      AgreementFail: 1.0
      CWA.Pass: 0.2
      CWA.Fail: 1.0
      PBHL.Update: 0.5
    export:
      kpis: true
      traces: true
      sink: "s3://acme-observerops/${ENV}/"
      privacy_mode: "minimize"

stage: &stage
  <<: *prod
  telemetry:
    level: debug
    sample_rates:
      CWA.Pass: 1.0
  cwa:
    certificate:
      thresholds:
        cwa_min: 0.78
        pri_max: 0.40

dev:
  <<: *stage
  observer:
    trace:
      retention_days: 14
  slots:
    tools:
      capacity: 2
      parallel_permits: 1
```

D.2 Task-Specific Overrides

D.2.1 RAG / Embedding Pooling

```
overrides:
  workloads:
    rag_support_kb:
```

```
      cwa:
        projector:
          type: "whiten"
          params: { eps: 1e-5 }
        certificate:
          thresholds: { cwa_min: 0.82, pri_max: 0.30, delta_mu_max: 0.05 }
          panels:
            chunk: { k: 16, policy: "sentence-reshuffle" }
        fallback:
          estimator: "attention.pool.v2"
          params:
            positional_kernel: "rope"
            heads: 4
      slots:
        memory: { capacity: 4096 }
      sync:
        kuramoto_rho_target: 0.90
```

D.2.2 Tool-Using Agents Fleet

```
overrides:
  workloads:
    tool_agents:
      agreement:
        min_overlap_keys: 3
        commute_matrix_ref: "cmat/tools:v5"
      observer:
        scheduling:
          Ô_policy: "risk-aware"
          tick: { cadence_ms: 300, max_retries: 1 }
      slots:
        tools:
          capacity: 6
          parallel_permits: 3
        backpressure:
          thresholds: { occupancy_amber: 0.70, occupancy_red: 0.85 }
```

D.3 Policy Gates & Escalation Ladder

```
policy_gates:
  cwa_gate:
    close_on:
      any:
        - "cwa.score < thresholds.cwa_min"
        - "cwa.pri   > thresholds.pri_max"
        - "diagnostics.delta_max > thresholds.delta_mu_max"
    actions:
      - "fallback:attention.pool.v2"
      - "label:phase-risk"
      - "alert:ml-ops"
  belt_gate:
    close_on:
      any:
        - "pbhl.residual > pbhl.residual.max"
        - "!acceptance.boundary_ok || !acceptance.gluing_ok || !acceptance.period_4pi_ok"
    actions:
      - "freeze:twist_step"
      - "increase:flux_gate.kp by 0.1"
      - "escalate:program-owner"
  slots_gate:
    close_on:
      any:
        - "slots.collision_rate_pct > thresholds.collision_budget_pct"
        - "slots.occupancy > occupancy_red"
    actions: ["gate:measure@tools", "alert:sre", "spinup:workers +1"]
```

D.4 Acceptance Checks Scheduler

```
acceptance_scheduler:
  schedule:
    - cron: "0 */6 * * *"     # every 6 hours
      tests: ["boundary","gluing","period_4pi"]
    - cron: "*/10 * * * *"    # every 10 min (light)
      tests: ["boundary"]
  triggers:
    - on: "pbhl.residual > 0.06"
      run: ["boundary","gluing"]
    - on: "alpha.drift > 0.15 over 24h"
      run: ["period_4pi"]
  resources:
    max_parallel: 4
    timeout_s: 120
```

D.5 JSON Form (API-ready)

```
{
  "version": 1,
  "env": "prod",
  "observer": {
    "trace": {
      "immutability": "strict",
      "hash_chain": "blake3",
      "retention_days": 365,
      "redact": { "pii_fields": ["email","phone"], "policy": "drop" },
      "idempotency": { "enforce": true, "window_s": 86400 }
    },
    "scheduling": {
      "Ô_policy": "compatibility-first",
      "tick": { "cadence_ms": 250, "max_retries": 2, "jitter_ms": [15,60] }
```

```
    }
  },
  "slots": {
    "memory": { "capacity": 2048, "guard_ms": 250 },
    "tools": { "capacity": 4, "parallel_permits": 2 },
    "attention": { "capacity": 12, "policy": "priority" },
    "eviction": { "policy": "lru", "protect_tags": ["inflight","latched"] },
    "backpressure": {
      "enabled": true,
      "gate": "Ô",
      "thresholds": { "occupancy_amber": 0.75, "occupancy_red": 0.9, "collision_budget_pct": 1.0 }
    }
  },
  "cwa": {
    "projector": { "type": "linear", "params": { "W_ref": "proj/W:2025-09-01", "normalize": true } },
    "certificate": {
      "thresholds": { "cwa_min": 0.8, "pri_max": 0.35, "delta_mu_max": 0.08 },
      "panels": { "perm": { "k": 16 }, "flip": { "k": 8, "mode": "random" }, "chunk": { "k": 8, "policy": "reshuffle" } },
      "sequential": { "early_exit": true, "upper_stop": 0.9, "lower_stop": 0.3 }
    },
    "fallback": {
      "estimator": "attention.pool.v2",
      "enable_on": [
        { "reason": "PRI_HIGH", "pri_min": 0.5 },
        { "reason": "SCORE_LOW", "cwa_max": 0.75 }
      ]
    }
  },
  "agreement": {
    "shared_ledger": true,
    "commute_matrix_ref": "cmat/global:v3",
    "min_overlap_keys": 2,
    "redundancy": { "fragments": 5, "policy": "majority", "min_effective": 4.0 }
  },
  "sync": {
    "kuramoto_rho_target": 0.85,
    "desync": {
      "delta_tau": { "p95_max": 5, "p99_max": 12 },
      "actions": [
        { "if": "rho < 0.70", "then": ["slowdown:0.10","reschedule:jitter(50-150ms)"] },
        { "if": "p99 > 12", "then": ["gate:measure@tools","alert:ops"] }
      ]
    }
  },
  "pbhl": {
    "alpha": { "default": 1.2, "bounds": [0.8, 1.8] },
    "residual": { "amber": 0.04, "max": 0.06 },
    "controllers": {
      "flux_gate": { "kp": 0.5, "ki": 0.2, "rate_hz": 2 },
      "twist_step": { "k": 0.1, "min_interval_s": 1800, "max_step": 0.05 }
    },
    "acceptance_checks": { "frequency": "every_6h", "on_residual_spike": true, "mesh_ref": "mesh/beltA:v7", "tests": ["boundary","gluing","period_4pi"] },
    "gates": { "close_on": [{ "residual_gt": 0.06, "windows": 2 }, { "acceptance_fail": true }] }
  },
  "telemetry": {
    "level": "info",
    "sample_rates": { "TraceWrite": 1.0, "AgreementFail": 1.0, "CWA.Pass": 0.2, "CWA.Fail": 1.0, "PBHL.Update": 0.5 },
    "export": { "kpis": true, "traces": true, "sink": "s3://acme-observerops/prod/", "privacy_mode": "minimize" }
  }
}
```

D.6 Validation Schema Snippet (JSON Schema)

```
{
  "$id": "https://observerops.io/schema/v1/config.json",
  "type": "object",
  "required": ["version","observer","slots","cwa","pbhl"],
  "properties": {
    "version": { "type":"integer", "minimum": 1 },
    "observer": {
      "type":"object",
      "properties": {
        "trace": {
          "type":"object",
          "required":["immutability","hash_chain","retention_days"],
          "properties":{
            "immutability":{"enum":["strict"]},
            "hash_chain":{"enum":["blake3","sha256"]},
            "retention_days":{"type":"integer","minimum":7}
          }
        }
      }
    },
    "cwa": {
      "type":"object",
      "properties":{
        "certificate":{
          "type":"object",
          "properties":{
            "thresholds":{
              "type":"object",
              "properties":{
                "cwa_min":{"type":"number","minimum":0,"maximum":1},
                "pri_max":{"type":"number","minimum":0,"maximum":1}
              }
            }
          }
        }
      }
    }
  }
}
```

D.7 Quick Profiles (copy-paste)

```
# dev quick-start
use: dev
cwa.certificate.thresholds:
  cwa_min: 0.75
  pri_max: 0.45
telemetry.level: debug
observer.trace.retention_days: 14
slots.tools.capacity: 2

# production hardening
use: prod
cwa.certificate.thresholds:
  cwa_min: 0.82
  pri_max: 0.30
pbhl.residual.max: 0.06
agreement.min_overlap_keys: 3
sync.desync.delta_tau.p99_max: 10
```

D.8 Incident Playbooks (policy excerpts)

```
incidents:
  cwa_false_green:
    detect: "CWA.Pass && downstream drift > 0.05 within 24h"
    contain:
      - "flip gate: cwa_gate -> CLOSED"
      - "force fallback: attention.pool.v2"
    root_cause:
      checklist: ["projector drift","dataset shift","order leakage"]
    recover:
      - "recalibrate projector W_ref"
      - "raise cwa_min by +0.02 for 7d"
  belt_residual_spike:
    detect: "pbhl.residual > 0.08 for 2 windows"
    contain: ["freeze:twist_step","increase:flux_gate.kp +0.1"]
    notify: ["program-owner","sre-oncall"]
    recover: ["acceptance full suite","mesh refine +25% cells"]
```

D.9 Minimal Per-Service Overrides

```
services:
  /pool:
    require_pass: true
    timeout_ms: 500
    budgets:
      cpu_ms: 250
      panels_max: { perm: 32, flip: 16, chunk: 16 }
  /measure:
    slots_required: { tools: 1 }
    max_payload_kb: 128
  /agree:
    min_score: 0.85
    non_commuting_policy: "skip-and-log"
```

Use these playbooks as-is or as a base. They encode the invariants: latching, commuting-only agreement, certificate-gated pooling, slot conservation, and belt closure—with guardrails, gates, and escalation ladders ready for operations.

Appendix E — Repro Labs

(datasets, scripts, notebook indices, grading rubrics)

E.0 Quickstart (environment & reproducibility)

- **Runtime:** Python 3.11; NumPy, SciPy, pandas, scikit-learn, NetworkX, Jupyter; optional: PyTorch/FAISS for Lab 3.
- **Install:** `pip install -r labs/requirements.txt`
- **Determinism:** set `OBSERVEROPS_SEED=1337` (or pass `--seed` to scripts).
- **Artifacts root:** `./artifacts/<lab>/<run_id>/...` → stores traces, scorecards, figures.
- **Make targets:** `make e1` … `make e4` to run each lab end-to-end.

```
repo/
 └─ labs/
    ├─ common/                # utils (rng, hashing, metrics, plotting)
    ├─ lab1_qubit/            # commuting vs non-commuting & latching
    ├─ lab2_smft_gridworld/   # Ô/τ scheduler in semantic fields
    ├─ lab3_cwa_rag/          # CWA certificate on projected embeddings
    ├─ lab4_belt_pbhl/        # PBHL belts & acceptance checks
    └─ requirements.txt
```

E.1 Datasets (formats, splits, checksums)

All labs ship synthetic, license: **CC-BY-4.0**. Each dataset folder contains:
`README.md`, `schema.json`, `train/`, `val/`, `test/`, `checksums.sha256`.

E.1.1 Lab 1 — Qubit Toy (Commute vs Non-Commute)

- **Files**

- `settings.jsonl`: per row `{ "seq": ["Z","X","Z"], "noise_p": 0.02 }`
- `runs.jsonl`: outcomes per sequence `{ "seq_id": "...", "y": [1,0,1], "tau":[0,1,2] }`

- **Splits:** 1k/200/200 sequences.
- **Schema (excerpt)**

```
{ "seq": ["Z|X|Y"], "basis_map": {"Z":[0,1], "X":[+,-]}, "noise_p": 0.0-0.1 }
```

- **Goal:** show order effects when instruments don't commute; verify latching prevents "retro-edit".

E.1.2 Lab 2 — SMFT Gridworld

- **Files**
  - `world.json`: cells, walls, sources/sinks
  - `field_init.npy`: initial Ψ_m slice (H×W×Θ)
  - `episodes.jsonl`: agent steps with `(x,θ,τ)` and reward
- **Splits:** 10 worlds × 50 episodes each.
- **Goal:** implement Ô that selects orientation/channel based on AL↓ and S_c↑ bands; track sync ρ across agents.

E.1.3 Lab 3 — RAG Pooling Battery (CWA)

- **Files**
  - `corpus.jsonl`: `{ "id": "...", "text": "...", "topic": "...", "phase_tag": "A|B" }`
  - `queries.jsonl`: `{ "qid": "...", "q": "...", "gold": ["id1","id7"] }`
  - `splits/` with **three perturbation views**: `perm/`, `flip/`, `chunk/`
  - Optional: `embeddings/` (precomputed `z.npy`), else generated on first run
- **Scales:** S (5k chunks), M (50k), L (250k). Start with S.
- **Goal:** certify when project→add (mean) is safe; fall back if PRI high.

E.1.4 Lab 4 — PBHL Belt Simulator

- **Files**
  - `belt_mesh.json`: nodes/edges/faces of program worldsheet
  - `windows.jsonl`: `{ "t0":"...", "t1":"...", "gap":0.33, "flux":0.27, "twist":0.04, "alpha":1.3 }`
  - `events.jsonl`: reorganizations, budget shifts, incidents
- **Goal:** compute Residual, run acceptance checks (boundary, gluing, 4π), operate Flux-gate & Twist-step.

E.2 Scripts (CLI)

All CLIs support `--seed`, `--out`, `--profile={dev,stage,prod}`.

Lab 1

```
python labs/lab1_qubit/gen_qubit.py --n-seq 1200 --noise-p 0.02
python labs/lab1_qubit/run_latching.py --seq settings.jsonl --commit
python labs/lab1_qubit/eval_agreement.py --commute-matrix zx_commute.json
```

Lab 2

```
python labs/lab2_smft_gridworld/gen_worlds.py --n 10 --theta 8
python labs/lab2_smft_gridworld/run_agent.py --episodes 50 --o-policy al_min
python labs/lab2_smft_gridworld/metrics.py --window 300 --rho --delta-tau
```

Lab 3

```
python labs/lab3_cwa_rag/gen_corpus.py --scale S
python labs/lab3_cwa_rag/project.py --projector whiten --d 768
python labs/lab3_cwa_rag/certify_pool.py --perm 16 --flip 8 --chunk 8 \
  --cwa-min 0.80 --pri-max 0.35 --eval retrieval@topk=5 --require-pass
```

Lab 4

```
python labs/lab4_belt_pbhl/gen_belt.py --cells 64
python labs/lab4_belt_pbhl/run_belt.py --windows 52 --alpha 1.2 \
  --accept boundary gluing period_4pi --residual-max 0.06
```

E.3 Notebooks Index (Jupyter)

Notebook	Purpose	Runtime	Artifacts
`E1_qubit_commute_vs_noncommute.ipynb`	visualize order effects; prove latching	5–7 min	plots, trace diffs
`E2_smft_scheduler.ipynb`	implement Ô using AL/S_c; fleet sync ρ	10–15 min	heatmaps, ρ/Δτ curves
`E3_cwa_certificate.ipynb`	run panels; score & PRI; fallback	8–12 min	certificate report

Notebook	Purpose	Runtime	Artifacts
`E4_pbhl_belt.ipynb`	compute Residual; controllers; acceptance	6–10 min	KPI dashboard, gates

Each notebook writes `artifacts/<lab>/<run>/report.md` + `scorecard.json`.

E.4 Grading Rubrics (auto-gradable)

Total per lab: 100 pts. Passing ≥ 80 unless stated.

Lab 1 — Qubit / Latching

- **L1.1 Trace immutability tests pass** (hash chain, one write per τ) — 25
- **L1.2 Order effect detected on non-commuting pairs** ($p<0.01$) — 25
- **L1.3 Agreement on commuting pairs ≥ 0.95** — 25
- **L1.4 Retro-edit attempt correctly rejected** — 25
  Fail bands: any of L1.1/L1.4 fails ⇒ auto-fail.

Lab 2 — SMFT Scheduler

- **L2.1 AL↓ trend** (≥10% drop over episode median) — 20
- **L2.2 S_c in green band** (diversity ≥ target) — 20
- **L2.3 Reward uplift vs random ≥ +15%** — 20
- **L2.4 Fleet sync** $\rho \geq 0.85$, **Δτ** p95 ≤ 5 — 20
- **L2.5 Compatibility-aware $\hat{O}$ (no illegal conflicts)** — 20

Lab 3 — CWA on RAG

- **L3.1 Certificate score ≥ 0.80** — 25
- **L3.2 PRI ≤ 0.35 (or justified fallback)** — 20
- **L3.3 Retrieval@5 iso-accuracy vs baseline (±1%)** — 20
- **L3.4 Latency reduction ≥ 20% on PASS panels** — 20
- **L3.5 Fallback engages on FAIL with accuracy ≥ baseline** — 15
  Fail bands: if PASS but accuracy −>1% vs baseline ⇒ downgrade 20 pts.

Lab 4 — PBHL Belts

- **L4.1 Residual $|G-(F+\alpha T)| \leq 0.06$ for ≥ 90% windows** — 25
- **L4.2 Acceptance checks all OK under normal ops** — 20
- **L4.3 Controller recovery < 3 windows after spike** — 20
- **L4.4 Five-Line KPI exported & consistent** — 20
- **L4.5 Correct gate behavior (CLOSE on breach)** — 15

Scoring I/O: `scorecard.json`

```
{
  "lab": "E3_cwa",
  "seed": 1337,
  "score": 92,
  "breakdown": {"L3.1":25,"L3.2":17,"L3.3":20,"L3.4":20,"L3.5":10},
  "metrics":{"cwa":0.86,"pri":0.22,"latency_drop":0.27}
}
```

E.5 Teacher & CI Harness

- **Teacher pack:** `rubrics/*.yaml`, hidden solutions under `solutions/`.
- **CI:** `ci/run_all.yml` runs labs nightly with fixed seeds; publishes artifacts to `artifacts/ci/<date>/`.
- **Plagiarism/overfit guard:** randomized seeds per student + hash of code cells; compare metric fingerprints.

GitHub Actions (snippet)

```
name: repro-labs
on: [workflow_dispatch, schedule]
jobs:
  run:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: pip install -r labs/requirements.txt
      - run: make e1 e2 e3 e4
      - uses: actions/upload-artifact@v4
        with: { name: artifacts, path: artifacts/** }
```

E.6 What “done” looks like (per lab)

- ✅ `artifacts/.../scorecard.json` with **score ≥ 80**
- ✅ `report.md` (1–2 pages) incl. plots & short justifications
- ✅ Reproducible rerun with same seed ±1% metric variance

That’s a complete, classroom-and-CI-ready Repro Labs package: datasets (with schemas), scripts & CLIs, notebook index, and auto-gradable rubrics tied to ObserverOps invariants.

Appendix F — Safety & Compliance Mapping

(SOC/ISO control matrices and trace evidence)

> Scope note: This appendix shows how ObserverOps’ invariants and artifacts satisfy common assurance frameworks. It is **not legal advice**—adapt with your GRC counsel.

F.0 Assurance Backbone (what auditors will look for)

ObserverOps ships three first-class, exportable evidence streams:

1. **Immutable Trace Ledger (T):** hash-chained, append-only records of `measure → commit` at tick τ with channel π and outcome y (internal collapse / latching).
2. **Certificate & Gate Logs (CWA):** permutation/flip/chunk panels, scores, PRI, and the **decision** (PASS/FAIL) including configured thresholds at decision time.
3. **BeltOps Telemetry (PBHL):** Five-Line KPI (Gap/Flux/Twist/Coherence/Residual), acceptance-check results (boundary, gluing, 4π), gates OPEN/CLOSE, and change history (α, controller gains).

Cross-cutting: **Agreement checks**, **slot allocator events**, **sync metrics** (ρ, $\Delta\tau$). Together they provide *who did what, when, with which policy*, and whether safety gates acted.

F.1 Evidence Packs (ready-to-export bundles)

Pack types and contents

- **Run Pack (`/evidence/run/<run_id>.zip`)**
  `/trace/*.jsonl` (ledger slice) • `/config/*.yaml` (effective policy) • `/agree/*.json` • `/slots/*.jsonl` • `/signing/manifest.json` (hashes, BLAKE3) • `/sig/chain.txt` (prev_hash chain)
- **CWA Pack (`/evidence/cwa/<pool_id>.zip`)**
  `/panels/*.jsonl` (perm/flip/chunk draws + deltas) • `/decision.json` (score, PRI, thresholds, require_pass flag) • `/fallback.json` (if used) • `/projector/ref.txt` • `/audit/plots/*`
- **Belt Review Pack (`/evidence/belt/<belt_id>/<window>.zip`)**
  `/kpi/*.json` • `/acceptance/*.json` • `/gate/*.json` • `/controller/*.json` (Flux-gate PID trace, Twist-step ops) • `/alpha/history.json`
- **Incident Pack (`/evidence/incident/<incident_id>.zip`)**
  `/timeline.md` • `/signals/*.json` (KPIs crossing) • `/root_cause/*.md` • `/remediation/*.yaml` • `/approvals/*.pdf` (SoD evidence)

All packs include a **manifest with digests**, signer identity, and timestamping (RFC 3161-compatible if configured).

F.2 SOC 2 (Trust Services Criteria) — Control Matrix

SOC2 TSC	ObserverOps Mechanism	Primary Evidence	Frequency / Owner
CC1 – Control Environment	Org-level config repo; policy gates (`cwa_gate`, `belt_gate`, `slots_gate`)	Config PRs, approvals, change log	Per change / GRC
CC2 – Communication & Info	Webhooks + dashboards for `AgreementFail`, `CWA.Fail`, `PolicyGate.Trigger`	Event stream archives; alert receipts	Continuous / SRE
CC3 – Risk Assessment	PRI + CWA score risk bands; PBHL Residual thresholds	Risk register entries linked to metrics	Quarterly / Risk
CC4 – Monitoring Activities	Five-Line KPI & acceptance checks scheduler	KPI trends; acceptance reports	Weekly / Program Ops
CC5 – Control Activities	Hard gates on pooling & belts; back-pressure to $\hat{O}$	Gate decisions; denied requests logs	Continuous / Runtime
CC6 – Logical Access	API scopes; token scopes (`measure:write`, …); trace read ACLs; field-level redaction	IAM policy export; access logs	Quarterly / SecOps
CC7 – System Ops	Health checks; $\Delta\tau$ & ρ monitors; autoscaling policies	SRE runbooks; on-call logs	Continuous / SRE
CC8 – Change Mgmt	Versioned configs; canary profiles (`dev/stage/prod`); idempotency guard	Release notes; diff & approvals	Per release / Eng
CC9 – Risk Mitigation	Incident playbooks (`cwa_false_green`, `belt_residual_spike`)	Incident packs; MTTR metrics	Per incident / IR

Processing Integrity (PI) → enforced by latching (immutable trace), agreement checks on commuting instruments, and certificate-gated pooling.
Confidentiality/Privacy → redaction policies (`drop|mask|hash`), PII field lists, and retention periods in org profile.

F.3 ISO/IEC 27001:2022 Annex A — Control Mapping (selected)

Annex A Control (2022)	ObserverOps Feature	Evidence
A.5.1 Policies for Information Security	Org config `observerops.yaml` with policy gates and thresholds	Policy file, approvals, policy reviews
A.5.7 Threat intelligence	PRI & phase-risk alerts; acceptance anomaly flags	Risk dashboard, alert history
A.5.10 Acceptable use of information	Redaction & minimization settings; slot budgets limiting data spread	Config + audit samples
A.5.18 Access control	API scopes, per-endpoint budgets, rate-limits	IAM export; WAF/rate logs
A.5.19 Identity management	OAuth2/PAT issuance, rotation policies	Token inventory; rotation proof
A.5.23 Cloud services	BeltOps exports to managed storage with integrity checks	Export manifests; S3/Azure immutability
A.5.30 ICT readiness for continuity	Belt fast/slow loops; acceptance checks; gate CLOSE on breach	DR runbook; test evidence
A.8.12 Data leakage prevention	Field-level redaction; telemetry sample rates	Redaction rules; sampling config
A.8.16 Monitoring activities	Five-Line KPIs; event taxonomy; Δτ/ρ	KPI dashboards; event archives
A.8.20 Protection of log information	Hash-chained trace; chain verification API; signed evidence packs	Hash audit, signer certs
A.8.21 Admin of logging	Retention & rotation; access to `/trace` gated by scope	Retention policy; access logs
A.8.28 Secure coding	Idempotency, append-only writes, compatibility guards	Unit/integration tests, coverage

---

F.4 ISO/IEC 42001:2023 (AI Management System) — Mapping (selected)

42001 Topic	ObserverOps Mapping	Evidence
AI Risk Management	CWA certificate + PRI bands; PBHL residual as macro-risk	CWA packs; risk register links
Data & Model Lifecycle	Projector refs (`W_ref`), versioned configs, fallback estimators	Model registry, `/project` logs
Traceability & Transparency	Immutable trace T; agreement diagnostics; belt acceptance	Evidence packs, API exports
Human Oversight	Gates produce explicit OPEN/CLOSE, require approvals for override	Approval records, SoD attestations
Monitoring & Response	Sync metrics, incident playbooks, SLOs for gates and recovery	Incident timelines, MTTR charts

---

F.5 NIST SP 800-53 Rev. 5 (cross-reference, short list)

Family	Relevant Controls	ObserverOps Support / Evidence
AC Access Control	AC-2, AC-3, AC-6	Scoped tokens; route-level RBAC; least-privilege exports
AU Audit & Accountability	AU-2, AU-6, AU-9	Trace ledger; hash chain; audit review procedures
CM Configuration Management	CM-3, CM-5	Versioned configs; enforced gates; canary
IR Incident Response	IR-4, IR-5	Incident packs + runbooks; postmortems
RA Risk Assessment	RA-3, RA-5	PRI, CWA score, residual trend; risk dashboards
SI System & Info Integrity	SI-4, SI-7	Certificate failure alerts; immutability guard; anomaly flags

---

F.6 GDPR / Data Protection Principles (operationalization)

- **Minimization (Art. 5(1)(c))**: PII redaction at trace-write; slot budgets and eviction policies reduce spread.
- **Integrity & Confidentiality (Art. 5(1)(f))**: hash-chained trace + scoped access; encryption at rest/in-transit.
- **Storage Limitation (Art. 5(1)(e))**: `retention_days` per profile; automatic purge jobs.
- **Accountability (Art. 5(2))**: Evidence packs + chain-of-custody (manifest digests, signer identity).
- **DPIA Triggers**: enable DPIA mode for new projectors/models or changes lifting PRI beyond band; attach risk assessment to the CWA pack.

F.7 Separation of Duties (SoD) & Approvals

- **Policy changes** (thresholds, gates) require **two-person approval** (Engineering + GRC).
- **Emergency overrides** log reason, ticket, duration, and **auto-revert** deadline.
- **Production deploys**: CI emits a signed **config diff** and links it into the next Evidence Pack.

Evidence: PR approvals, change request IDs, override logs, and `/policy/version` snapshots.

F.8 Auditor Playbook (queries & samples)

Sampling windows

- **Daily light**: 10 CWA decisions (mix PASS/FAIL), 5 belt updates, 20 trace writes across tools.
- **Weekly deep**: 100 CWA (borderline scores), all acceptance checks since last review, 10% of `AgreementFail`.

Query sketches (SQL/DSL)

- **Show all PASS decisions within 1h before any incident:**

```
SELECT pool_id, score, pri, thresholds, decided_at
FROM cwa_decisions
WHERE decision='PASS' AND decided_at BETWEEN incident_start - INTERVAL '1 hour'
                                 AND incident_start;
```

- **Verify trace immutability (hash chain continuity):**

```
SELECT t.trace_id
FROM trace t
LEFT JOIN trace p ON p.trace_id = t.prev_id
```

```
WHERE t.prev_hash <> p.hash;
```

- **List gate closures with residual > max:**

```
SELECT belt_id, residual, reason, window_start
FROM belt_gates
WHERE status='CLOSE' AND residual > residual_max;
```

- **Agreement failures on commuting pairs:**

```
SELECT a.key, a.reason
FROM agreement a
JOIN commute_matrix c ON c.key=a.key
WHERE c.commute=true AND a.pass=false;
```

F.9 Key Risk Indicators (KRIs) & Tests of Controls

KRI	Green	Amber	Red	Control Test
CWA false-green rate (PASS but downstream drift > 1%)	<0.5%	0.5–1%	>1%	Shadow evaluation weekly
Trace retro-edit attempts	0	1–3/mo	>3/mo	Tamper drills, chain verify
PBHL residual excursions (outside max)	<5% windows	5–10%	>10%	Gate closes; recovery < 3 windows
Agreement fail on commuting keys	<2%	2–5%	>5%	SBS redundancy check, mapping audit
Slot collision rate (tools)	<0.5%	0.5–1%	>1%	Back-pressure + capacity review

---

F.10 Retention, Deletion, and Chain-of-Custody

- **Retention**: per-profile `retention_days` (e.g., prod 365d, dev 14d).
- **Deletion**: redaction by key; tombstone **append** entry with reason and actor (no in-place delete).
- **Chain-of-Custody**: Every pack has `manifest.json` with file paths, sizes, BLAKE3 digests, signer, timestamp; verification tool emits a pass/fail report.

F.11 Minimal Auditor Checklist

1. Pull **3 Evidence Packs** of each type; verify signatures and hash chains.
2. Recompute **CWA decisions** on sampled pools; confirm thresholds at decision time match config snapshots.
3. Verify **belt acceptance checks** replay to same results; inspect one residual spike, ensure gate CLOSED and recovery in ≤ 3 windows.
4. Confirm **SoD**: pick one policy change → trace approvals to production and corresponding effective-config hash in subsequent packs.
5. Confirm **data minimization**: spot-check that configured PII fields are absent or masked in traces.

Bottom line: ObserverOps turns safety constraints into *auditable controls*—with deterministic traces, certificate decisions, and belt closure evidence that plug straight into SOC/ISO audits.

Appendix G — Figures & Tables

(production-ready list with captions and source data)

G.0 Conventions (for all figures/tables)

- **IDs:** Figures F1–F7, Tables T1–T5.
- **Exports:** SVG (print), PNG@2x (web), PDF (archive).
- **Fonts/Style:** Inter 10–12pt (tables), Inter 12–14pt (figures); grid=off, axes ticks out; color-blind safe (Okabe–Ito).
- **Metadata:** embed `source_path`, `git_commit`, `build_ts`, `seed`.
- **Column keys used repeatedly:**
  - `tau` (tick), `pi` (channel), `y` (outcome), `CWA_score`, `PRI`, `Gap`, `Flux`, `Twist`, `Alpha`, `Residual`, `rho`, `delta_tau_pXX`, `occupancy`, `collisions`, `agree_score`.
- **Repro seed:** `OBSERVEROPS_SEED=1337`.
- **Build:** `make figs` (or command listed per item).
- **Alt-text:** provided per figure for accessibility.

G.1 Figures

F1. ObserverOps stack diagram (micro/meso/macro)

- **Purpose:** Visual overview of ObserverOps planes/modules.
- **Caption:** *ObserverOps composes micro (observer + slots), meso (SMFT with $\hat{O}/\tau$), and macro (PFBT belts) under the CWA aggregation law and audit plane.*
- **Source:** `figsrc/F1_stack.drawio` (vector), modules list from `docs/modules.yaml`.
- **Build:** export via draw.io CLI → `assets/fig/F1_stack.svg`.

- **QA:** All module names match Table T2 API summary.

Alt-text: "Block diagram showing three layers—micro, meso, macro—with data/control/audit planes and CWA gate."

F2. Ô-first scheduling loop & latching point

- **Purpose:** Show control loop and where internal collapse (latching) occurs.
- **Caption:** *The Ô-first loop selects a compatible channel, measures, then latches the outcome into the trace at tick τ_k; downstream control is conditioned on the latched record.*
- **Source data:** `labs/lab1_qubit/artifacts/<run>/trace_ring.csv` (for tick markers).
- **Columns:** `tau, event, pi, committed(bool)`.
- **Plot:** Step diagram with a lock icon at `TraceWrite`.
- **Build:** `python labs/lab1_qubit/plot_loop.py --out assets/fig/F2_loop.svg`.
- **QA:** Exactly one `TraceWrite` per `tau`.

Alt-text: "Flow from channel selection to measurement to immutable trace write (latching)."

F3. CWA decision tree + validity band

- **Purpose:** Visualize certificate branches (perm/flip/chunk) and pass band.
- **Caption:** *CWA passes when perturbation panels keep pooled outcomes within the validity band and Phase-Risk Index (PRI) stays below the red line; otherwise fallback is engaged.*
- **Source data:** `labs/lab3_cwa_rag/artifacts/<run>/certificate_summary.json`.
- **Columns:** `panel, k, delta_max, score, pri, decision`.
- **Plot:** Decision tree + band chart (score vs. PRI with PASS/FAIL regions).
- **Build:** `python labs/lab3_cwa_rag/plot_cwa_tree.py -o assets/fig/F3_cwa.svg`.
- **QA:** Boundaries reflect config thresholds in Appendix D.

Alt-text: "Branching certificate tests with a shaded green pass band in score-PRI space."

F4. Belt worldsheet (Gap, Flux, Twist, Residual)

- **Purpose:** Depict PBHL variables over the program worldsheet.
- **Caption:** *Across windows, Gap tracks planned–done delta, Flux captures throughput, Twist encodes structural changes; Residual quantifies closure error.*
- **Source data:** `labs/lab4_belt_pbhl/artifacts/<run>/windows.jsonl`.
- **Columns:** `window_id, Gap, Flux, Twist, Alpha, Residual`.
- **Plot:** 4-panel line chart (Gap/Flux/Twist/Residual vs window).
- **Build:** `python labs/lab4_belt_pbhl/plot_worldsheet.py -o assets/fig/F4_belt.svg`.
- **QA:** `Residual = |Gap − (Flux + Alpha*Twist)|` holds numerically.

Alt-text: "Time series of PBHL variables with residual envelope."

F5. Five-Line KPI dashboard (production)

- **Purpose:** Operations panel snapshot for a belt.
- **Caption:** *Five-Line KPIs summarize belt health; gates close automatically when Residual exceeds policy thresholds.*
- **Source data:** `apps/beltops/artifacts/<run>/kpi_timeseries.csv`.
- **Columns:** `ts, Gap, Flux, Twist, Coherence, Residual, gate`.
- **Plot:** Multi-line with shaded amber/red bands; gate status markers.
- **Build:** `python apps/beltops/plot_kpi.py -o assets/fig/F5_kpi.svg`.
- **QA:** Gate CLOSE markers align with residual breaches.

Alt-text: "Five layered lines over time with amber/red zones and gate status dots."

F6. Agreement/SBS schematic

- **Purpose:** Show redundancy leading to cross-observer agreement.
- **Caption:** *When commuting instruments write shared or SBS-redundant records, independent observers converge on effective outcomes (AB-fixedness).*
- **Source:** `figsrc/F6_agreement_sbs.svg` (vector + callouts).

- **Build:** Export from source; labels cross-checked with `agreement` API fields.
- **QA:** Legend includes commuting/non-commuting pairs.

Alt-text: "Two observers reading redundant pointer channels leading to the same outcome."

F7. Slot allocator with occupancy/collision heatmap

- **Purpose:** Visualize slot conservation under load.
- **Caption:** *Integer, non-overlapping slots with back-pressure keep collision rates within SLA bands; heatmap shows occupancy and rare collisions.*
- **Source data:** `runtime/slot_alloc/artifacts/<run>/slot_metrics.csv`.
- **Columns:** `ts, pool(memory|tools|attention), occupancy, collisions`.
- **Plot:** Heatmap over time×pool with collision annotations.
- **Build:** `python runtime/slot_alloc/plot_slots.py -o assets/fig/F7_slots.svg`.
- **QA:** `occupancy ∈ [0,1]`; collisions ≤ budget from config.

Alt-text: "Three-row heatmap of slot occupancy with occasional red collision dots."

G.2 Tables

T1. Metric definitions & threshold bands

- **Caption:** *Formal definitions and production bands for ObserverOps metrics.*
- **Source:** `docs/metrics.yaml` (authoritative), rendered to table.
- **Columns:** `metric, definition, estimator, green, amber, red`.
- **Build:** `python tooling/render_metrics_table.py > assets/table/T1_metrics.csv`.
- **QA:** Bounds match Appendix D thresholds.

T2. API summary & event taxonomy

- **Caption:** *Endpoint inventory and event types with schemas and scopes.*
- **Source:** OpenAPI `openapi/observerops.v1.yaml` + `docs/events.yaml`.
- **Columns:** `endpoint, method, scope, request_body, response, errors`.
- **Build:** `python tooling/openapi_to_table.py -o assets/table/T2_api.csv`.
- **QA:** Error codes align with Appendix C.

T3. Ablation results (±Ô, ±slots, ±certificate)

- **Caption:** *Effect sizes on disagreement, latency, and residual under ablations.*
- **Source data:** `bench/ablations/artifacts/<run>/results.csv`.
- **Columns:** `ablation, disagreement, latency_ms, residual, notes`.
- **Build:** `python bench/ablations/aggregate.py -o assets/table/T3_ablation.csv`.
- **QA:** Seeds & configs logged in table footer.

T4. Commute vs conflict instrument pairs

- **Caption:** *Instrument pairs classified by commutation on visited support.*
- **Source:** `runtime/agree/commute_matrix/*.json`; expanded via frame map.
- **Columns:** `pi_a, pi_b, commute(bool), comment`.
- **Build:** `python runtime/agree/expand_commute.py -o assets/table/T4_commute_pairs.csv`.
- **QA:** Sample spot-checks in Lab 1.

T5. Policy gates & escalation ladders

- **Caption:** *Gate conditions and automatic actions for CWA, belts, and slots.*
- **Source:** `config/policy_gates.yaml`.
- **Columns:** `gate, condition, action, severity`.
- **Build:** `python tooling/policy_to_table.py -o assets/table/T5_gates.csv`.
- **QA:** Conditions parse and evaluate in dry-run.

G.3 Source Data Schemas (CSV/JSONL)

- **`trace_ring.csv`**: `tau:int, event:str, pi:str, committed:bool, hash:str, prev_hash:str`.
- **`certificate_summary.json`**: records with `panel:str, k:int, delta_max:float, score:float, pri:float, decision:str`.
- **`windows.jsonl`**: per line `{"window_id":int,"Gap":float,"Flux":float,"Twist":float,"Alpha":float,"Residual":float}`.
- **`kpi_timeseries.csv`**: `ts:iso, Gap, Flux, Twist, Coherence, Residual, gate:str`.
- **`slot_metrics.csv`**: `ts:iso, pool:str, occupancy:float, collisions:int`.
- **`results.csv` (ablations)**: `ablation:str, disagreement:float, latency_ms:float, residual:float, seed:int, config_hash:str`.

G.4 Reproduction Commands (one-liners)

```
# Build all figures & tables
make figs tables

# Or selectively:
python labs/lab3_cwa_rag/plot_cwa_tree.py -o assets/fig/F3_cwa.svg
python labs/lab4_belt_pbhl/plot_worldsheet.py -o assets/fig/F4_belt.svg
python tooling/openapi_to_table.py -o assets/table/T2_api.csv
```

G.5 QC Checklist (per artifact)

- Source path & git commit embedded.
- Axes labeled with units; legends outside plot area; font ≥ 10pt.
- Color-blind palette applied; PASS/FAIL bands labeled.
- Numbers reproducible within ±1% across reruns with same seed.
- Tables round to 2–3 sig figs; include units column where relevant.

Deliverable: `assets/fig/*.svg|png`, `assets/table/*.csv|md` built from versioned sources above; each with deterministic seeds and embedded provenance.

© 2025 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.

M

T

G

Y

Text-to-speech function is limited to 200 characters

ObserverOps Technical Blueprint - Production Figures (Cross-Referenced)

Figure 1 — Observer Ops Stack (micro/meso/macro)

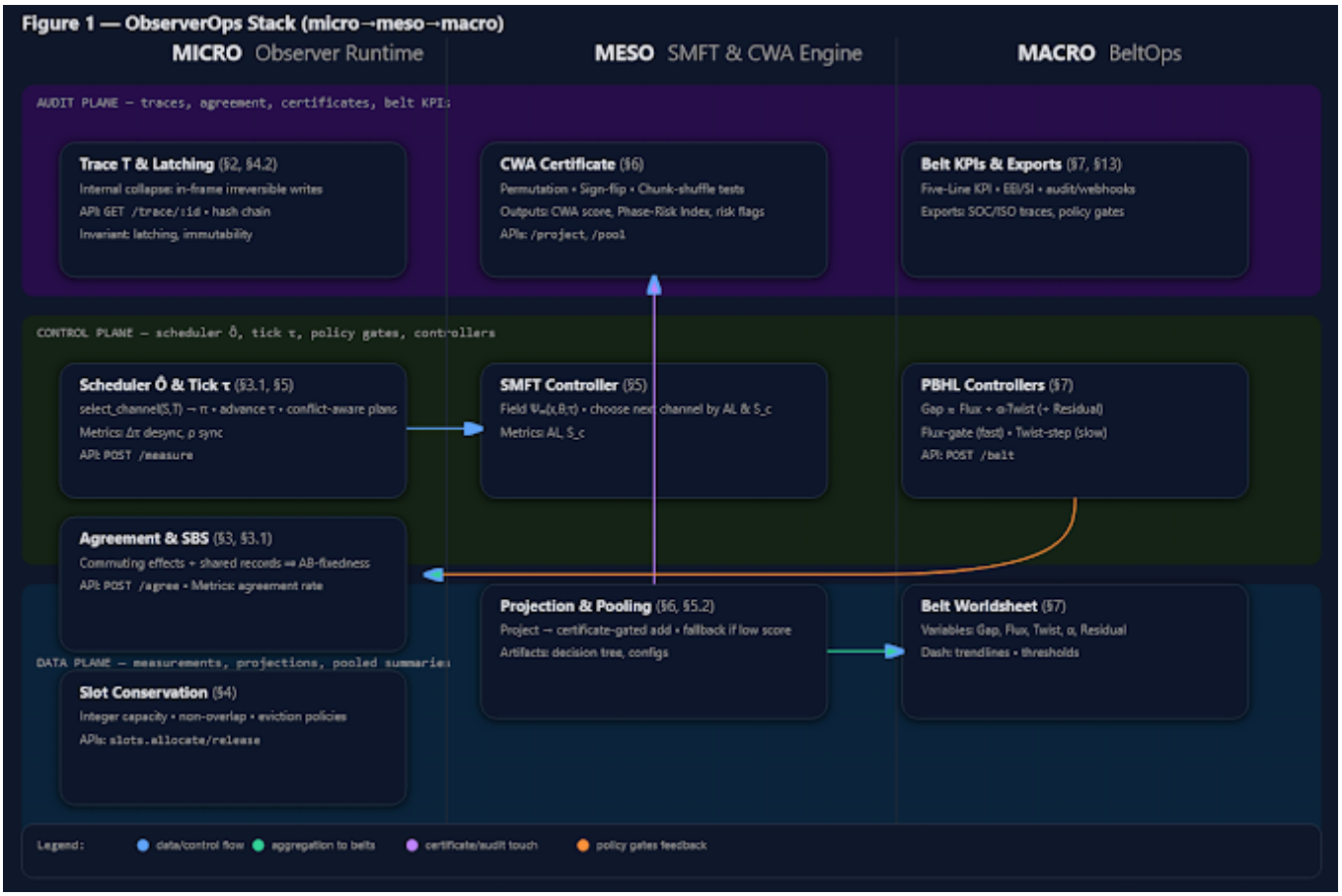
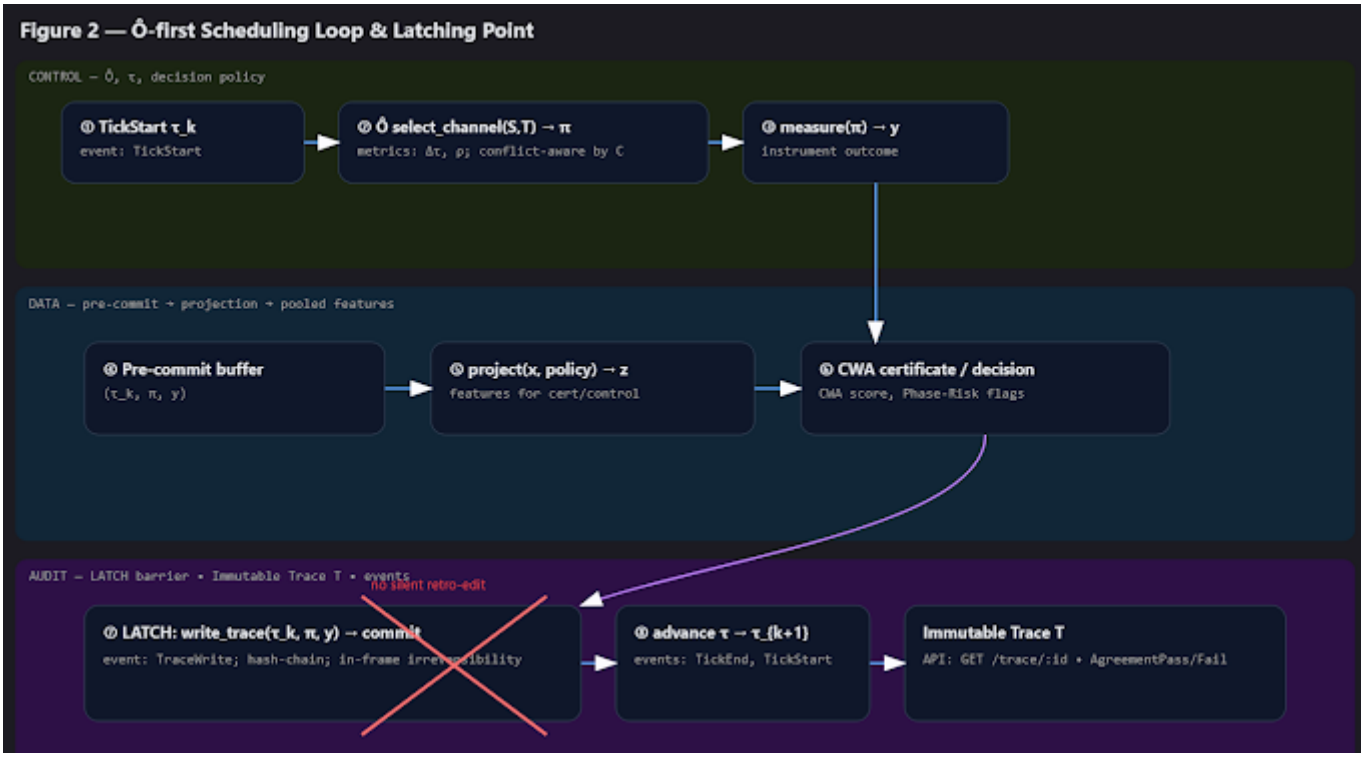


Figure 1. Three-layer ObserverOps stack. *Micro* (Observer Runtime) enforces internal collapse (latching), agreement checks, and slot conservation while scheduling with Ô and ticks τ. *Meso* (SMFT & CWA Engine) projects signals and certifies when additive pooling is safe, exposing CWA score and Phase-Risk Index. *Macro* (BeltOps) closes the loop via PBHL, controlling Gap≈Flux+α·Twist with Residual bounds and policy gates. Horizontal bands show the *Audit*, *Control*, and *Data* planes. Cross-refs: §1.2.A; §3.1–3.5; §4.2; §5–§7; §9–§13.

Figure 2 — Ô-first Scheduling Loop & Latching Point



Cross-refs: §2, §3.1, §4.2, §5 • Events: TickStart, ChannelSelected, TraceWrite • APIs: /measure, /project, /pool, /trace/:id

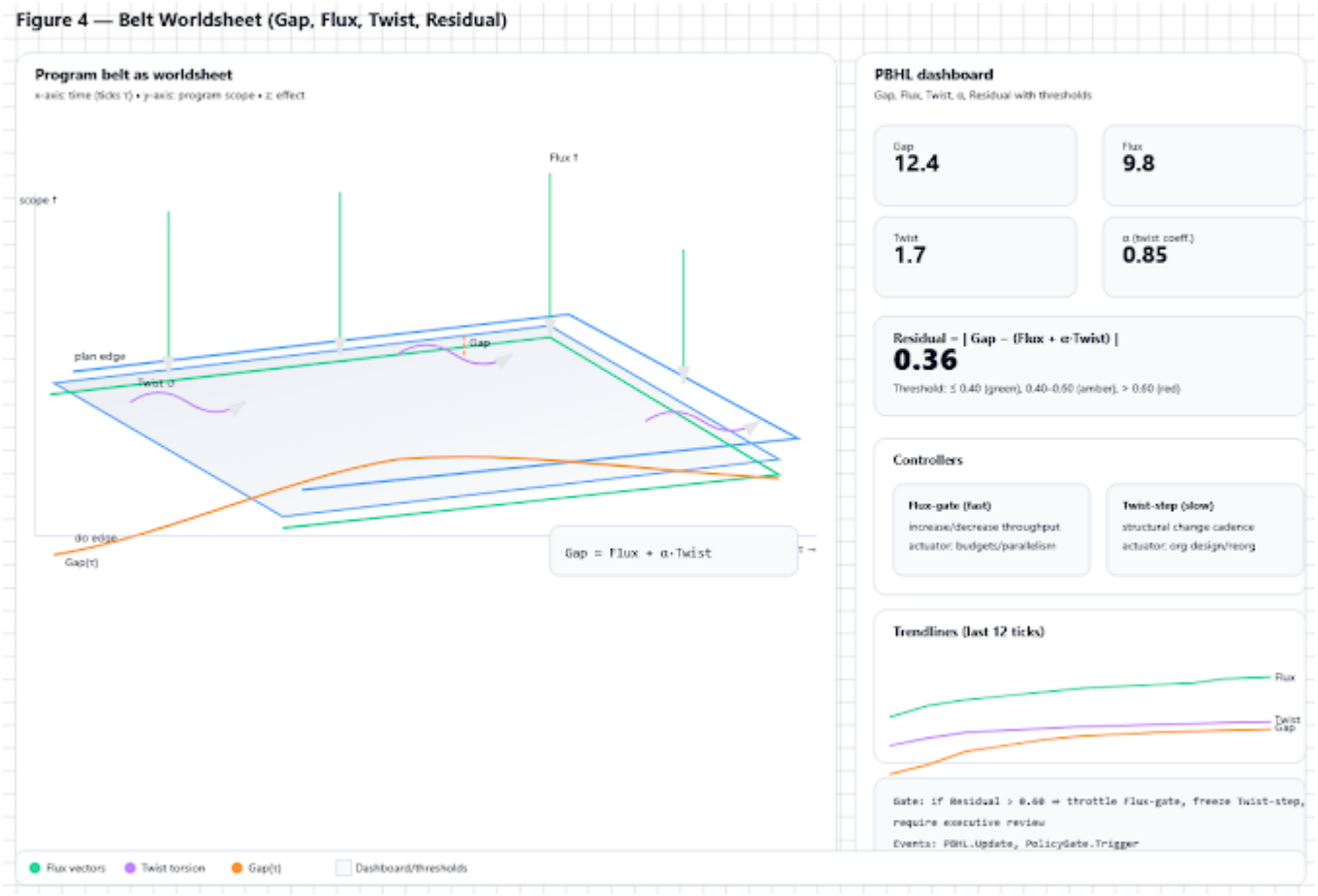
Figure 3 — CWA Decision Tree & Validity Band



Figure 3. *Left:* CWA decision tree. A certificate battery probes invariance to permutations, sign-flips, and chunk shuffles; scores aggregate to a **CWA score** (0–1) with drift/CI. Actions:

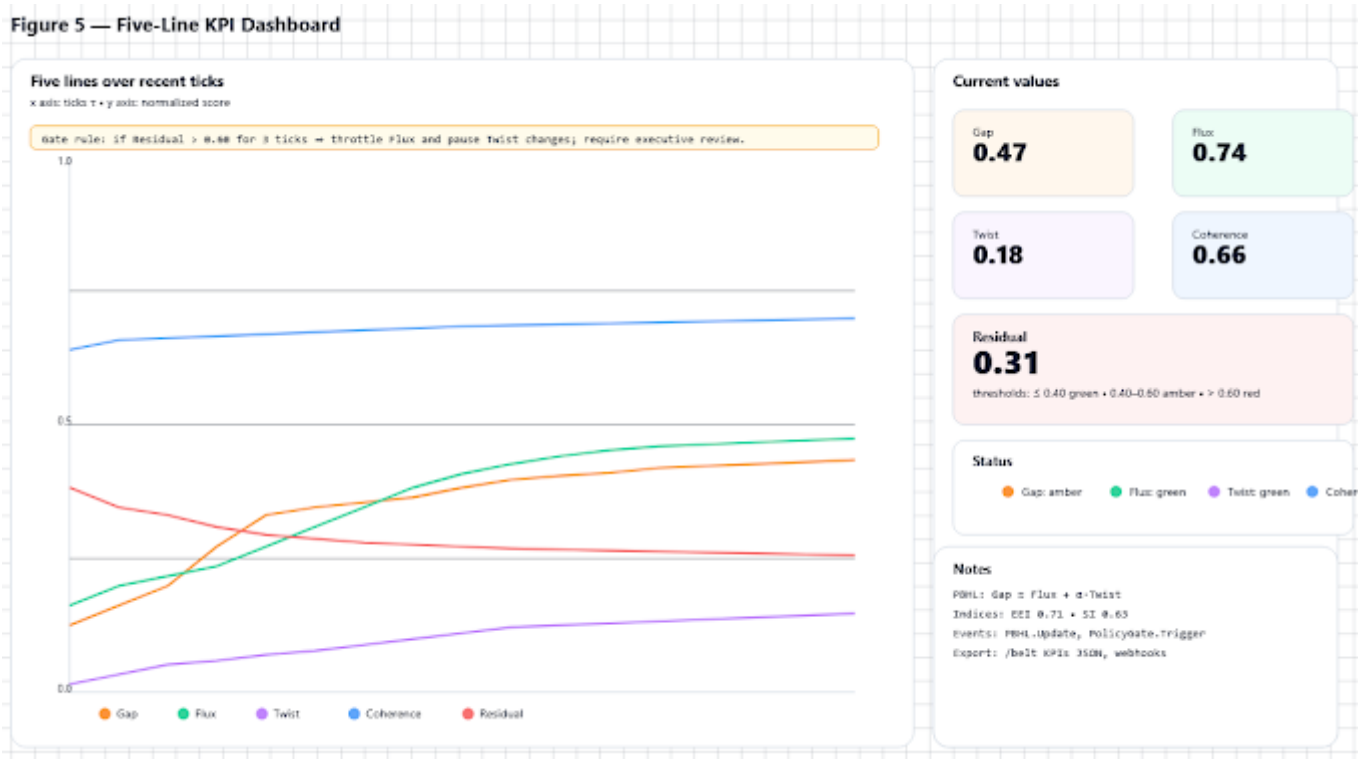
PASS → additive pooling; **AMBER** → hybrid or capped add with audit; **FAIL** → order-aware estimators. *Right:* Validity band showing where *project* → *add* is safe: high stability under perturbations and low Phase-Risk Index. Cross-refs: §6 (CWA), §11 (Engine), §15 (RAG pattern), §19–21 (metrics).

Figure 4 — Belt Worldsheet (gap, Flux, Twist, Residual)·



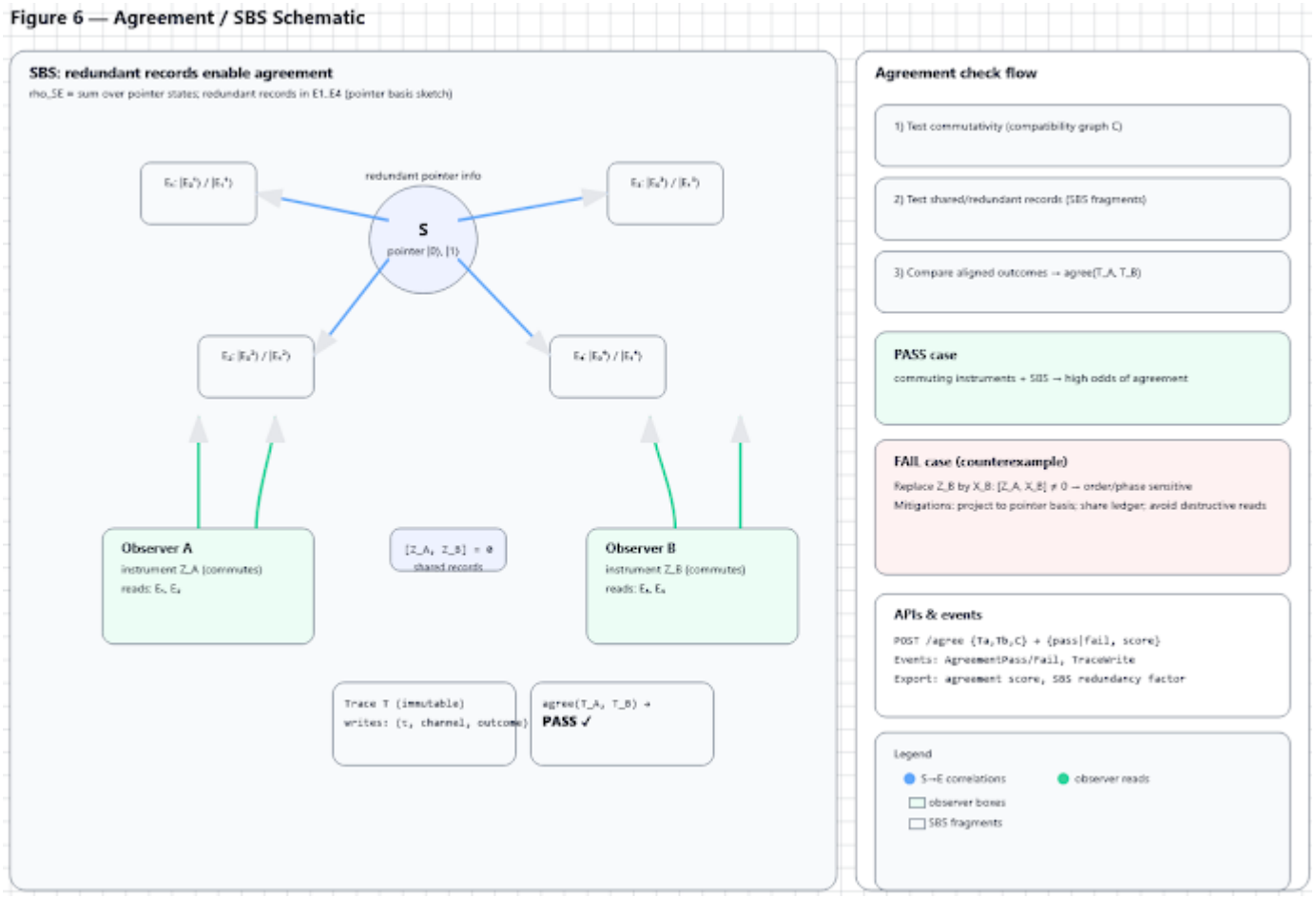
A worldsheet-style belt panel with Flux/Twist/GAP vectors on the left and a PBHL KPI dashboard (Gap, Flux, Twist, α, Residual, controllers + trendlines) on the right.

Figure 5 — Five-Line KPI Dashboard



Five-Line KPI dashboard showing Gap, Flux, Twist, Coherence, and Residual with trendlines, tiles, thresholds, and a gate banner.

Figure 6 — Agreement / SBS Schematic



A two-panel **Agreement/SBS** schematic. Left shows SBS redundancy (S → E1…E4) with

Observers A & B reading disjoint fragments under commuting instruments and writing to a shared trace, yielding **agree(T_A,T_B) = PASS**. Right shows the **agreement check flow** plus a **non-commuting counterexample** and mitigation notes.

Figure 7 — Slot Allocator with Occupancy/Collision Heatmap



Slot Allocator + Occupancy/Collision Heatmap. Left shows the **slot bank**, consumers, APIs and thresholds; right shows a **12×20 heatmap** with red × collision markers, utilization scale, and a legend.

© 2025 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.

ObserverOps Technical Blueprint - Production Tables (Cross-Referenced)

1) Metric definitions & threshold bands

Reading guide: Each row gives **definition**, **how to estimate**, **bands** (Green/Amber/Red), and **policy gates**. Cross-refs: chapters (Ch.), APIs, and events (Ev.). Default windows assume rolling **W=256 ticks** (micro/meso) or **W=4 weeks** (macro) unless stated.

1.1 Micro metrics (inside an agent)

Metric (symbol)	Definition (what it measures)	Estimation (how)	Bands (default)	Policy gates (on Amber/Red)	Cross-refs
Agreement Rate (AGR)	Fraction of `agree(T_a,T_b)` checks that pass over W.	Count `AgreementPass/Fail` over W; AGR = pass / total.	**G ≥0.95 · A 0.85–0.95 · R <0.85**	Amber→ increase SBS redundancy; Red→ freeze non-essential writes, require quorum before act.	Ch.3; **/agree**; Ev: AgreementPass/Fail; Fig.6
Mis-exec Rate (MER)	Tool runs that violate spec/guardrails.	Failures ÷ tool invokes in W.	**G ≤2% · A 2–5% · R >5%**	Amber→ throttle tool concurrency; Red→ disable tool, route to safe mode.	Ch.5 §tool use; **/measure**; Ev: ChannelSelected, TraceWrite
Trace Half-Life (THL)	Median ticks until a trace record becomes cold (no downstream reads).	Track last-read tick per record; THL = median(τ_last_read – τ_write).	**G 32–256 · A 8–32 or >256 · R <8**	Amber/low→ extend retention; Red/low→ raise audit level; Red/high→ compaction policy.	Ch.2; **/trace/:id**; Ev: TraceWrite
Internal-Collapse Violations (ICV)	Retro-edits within same τ (latching breaks).	Hash-chain or version check per τ; count violations.	**G =0 · A 1 per 10^4 writes · R ≥1 per 10^3**	Any→ block commit; open incident.	Ch.2; Ev: TraceWrite
Slot Occupancy (Occ)	Used slots ÷ total slots.	Sample allocator counters.	**G 40–85% · A 20–40% or 85–95% · R <20% or >95%**	Amber→ tune slot budget; Red/high→ back-pressure; Red/low→ consolidate caches.	Ch.4; **slots.***; Fig.7
Slot Collision Rate (SCR)	Allocation conflicts per 1k allocations.	Collisions ÷ allocations ×1k.	**G <10 · A 10–30 · R >30**	Amber→ change eviction policy; Red→ increase slots or serialize callers.	Ch.4; Ev: SlotAllocate
SBS Redundancy Factor (R_SBS)	Independent pointer records per observable.	Count distinct channels writing same pointer; average.	**G ≥3 · A 2–3 · R <2**	Amber→ add redundancy; Red→ require external ledger.	Ch.3; Fig.6

1.2 Meso metrics (Ô scheduler, τ rhythm, projection & pooling)

Metric (symbol)	Definition	Estimation	Bands (default)	Policy gates	Cross-refs
Attractor Load (AL)	Concentration of semantic field mass (risk of tunnel vision).	AL = HHI of channel selection probs over W (sum p_i²) **or** 1 – H(p)/log	Π	.	**G ≤0.50 · A 0.50–0.70 · R >0.70**
Collapse Entropy (S_c^norm)	Diversity of chosen channels.	S_c^norm = H(p)/log	Π	∈[0,1].	**G ≥0.5 · A 0.3–0.5 · R <0.3**
Tick Sync (ρ)	Fleet phase coherence.	Kuramoto order parameter ρ =	(1/N)∑e^{iθ_j}	.	**G ≥0.80 · A 0.60–0.80 · R <0.60**
Ô-Desynchrony (Δτ)	Spread of commit ticks across replicas.	StdDev of τ over agents in W.	**G ≤1 · A 1–3 · R >3 (ticks)**	Amber→ increase cadence; Red→ enforce barriers.	Ch.5; Ev: TickStart
CWA Certificate Score (CWA)	Invariance of pooled statistic to permutations/ sign-flips/chunk-shuffles.	Average of test pass rates or combine p-values to score ∈[0,1].	**G ≥0.75 · A 0.50–0.75 · R <0.50**	Amber→ shrink pool, raise CI; Red→ **auto-fallback** to order-aware estimator.	Ch.6; **/pool**; Fig.3
Phase-Risk Index (PRI)	Sensitivity of outputs to order/phase.	Normalized variance across the certificate's perturbation panel.	**G ≤0.25 · A 0.25–0.50 · R >0.50**	Amber→ add projection; Red→ disable additive pooling.	Ch.6; **/project**

Metric (symbol)	Definition	Estimation	Bands (default)	Policy gates	Cross-refs
Black-Hole Index (BHI)	Degeneracy/near-linearity of embeddings/features.	$\sigma_1/\sum\sigma_i$ over projected feature matrix (W).	**G ≤0.55 · A 0.55–0.70 · R >0.70**	Amber→ jitter projectors; Red→ re-chunk / re-embed.	Ch.5–6

1.3 Macro metrics (Belts & governance)

Metric (symbol)	Definition	Estimation	Bands (default)	Policy gates	Cross-refs
PBHL Residual (R_PBHL)	Closure error:	Gap – (Flux + α·Twist)	normalized.	R =	Gap – (Flux + α·Twist)
Effectiveness-Efficiency Index (EEI)	Delivered value per unit burn (normalized).	Scale OKR-weighted outcomes ÷ (cost + waste).	**G ≥0.65 · A 0.45–0.65 · R <0.45**	Amber→ reprioritize backlog; Red→ spending gate.	Ch.7; Fig.5
Sustainability Index (SI)	Ability to sustain Flux within constraints.	Composite of stability, burnout, infra headroom (0–1).	**G ≥0.70 · A 0.50–0.70 · R <0.50**	Amber→ cool-down (reduce cadence); Red→ pause initiatives.	Ch.7; Fig.5
Belt Coherence (BC)	Alignment of plan↔do narratives.	Topic coherence across belt docs vs traces.	**G ≥0.70 · A 0.55–0.70 · R <0.55**	Amber→ clarify objectives; Red→ re-baseline belt.	Ch.7

1.4 Profiled threshold presets (copy-paste)

Use these **policy profiles** to seed configs. Tighten for regulated contexts.

```
# /apps/beltops/policy/thresholds.yaml
profiles:
  dev-lab:
    AGR: {green: 0.90, amber: [0.80,0.90]}
    MER: {green: 0.05, amber: [0.05,0.10]}
    CWA: {green: 0.70, amber: [0.40,0.70]}
    PRI: {green: 0.35, amber: [0.35,0.60]}
    AL:  {green: 0.60, amber: [0.60,0.80]}
    Sc:  {green: 0.40, amber: [0.25,0.40]}
    rho: {green: 0.70, amber: [0.50,0.70]}
    dTau:{green: 2,    amber: [2,4]}
    PBHL:{green: 0.15, amber: [0.15,0.35]}
    EEI: {green: 0.55, amber: [0.40,0.55]}
    SI:  {green: 0.60, amber: [0.45,0.60]}
    Occ: {green: [0.35,0.90], amber_low: 0.20, amber_high: 0.95}
    SCR: {green: 20,   amber: [20,40]}
  prod-guarded:
    AGR: {green: 0.95, amber: [0.85,0.95]}
    MER: {green: 0.02, amber: [0.02,0.05]}
    CWA: {green: 0.75, amber: [0.50,0.75]}
    PRI: {green: 0.25, amber: [0.25,0.50]}
    AL:  {green: 0.50, amber: [0.50,0.70]}
    Sc:  {green: 0.50, amber: [0.30,0.50]}
    rho: {green: 0.80, amber: [0.60,0.80]}
    dTau:{green: 1,    amber: [1,3]}
    PBHL:{green: 0.10, amber: [0.10,0.30]}
    EEI: {green: 0.65, amber: [0.45,0.65]}
    SI:  {green: 0.70, amber: [0.50,0.70]}
    Occ: {green: [0.40,0.85], amber_low: 0.20, amber_high: 0.95}
    SCR: {green: 10,   amber: [10,30]}
  regulated:
    AGR: {green: 0.98, amber: [0.92,0.98]}
    MER: {green: 0.01, amber: [0.01,0.02]}
    CWA: {green: 0.85, amber: [0.60,0.85]}
    PRI: {green: 0.20, amber: [0.20,0.35]}
    AL:  {green: 0.45, amber: [0.45,0.60]}
    Sc:  {green: 0.55, amber: [0.40,0.55]}
    rho: {green: 0.85, amber: [0.70,0.85]}
    dTau:{green: 1,    amber: [1,2]}
    PBHL:{green: 0.08, amber: [0.08,0.20]}
    EEI: {green: 0.70, amber: [0.55,0.70]}
    SI:  {green: 0.75, amber: [0.60,0.75]}
    Occ: {green: [0.50,0.80], amber_low: 0.35, amber_high: 0.90}
    SCR: {green: 5,    amber: [5,15]}
```

1.5 Estimation notes (operational)

- **Windows:** Micro/meso use **rolling tick windows** (default 256); macro uses **calendar windows** (weekly). Log both *point estimate* and *CI*.
- **Normalization:** For **S_c**, normalize by $\log|\Pi|$. For **PBHL Residual**, normalize by $|\text{Gap}|$.
- **CWA/PRI:** Run certificate panels sized to target CI (e.g., 64 permutations, 32 flips, 16 chunk-shuffles). Cache seeds for reproducibility.

- **SBS audits:** When **AGR** drops or **R_SBS** < 3, trigger *Redundancy Probe* (extra channels record the same pointer).
- **Slots:** Treat Occ as **two-sided** risk (starvation and saturation).
- **Sync:** If **ρ** low but **Δτ** tight, prefer cadence retune over hard barriers.

1.6 Gate mapping (what trips what)

Trigger	Gate	Action
CWA < 0.50 or PRI > 0.50	**PoolingGate**	Switch to order-aware estimator; require human sign-off for batch pools.
AGR < 0.85 (prod-guarded)	**AgreementGate**	Suspend autonomous actuation; quorum-only mode.
ICV > 0	**LatchingGuard**	Block commit; page on-call; open incident.
PBHL Residual > 0.30	**BeltGate**	Freeze scope changes; exec review; raise Residual sampling rate.
SCR > 30/1k or Occ > 95%	**AllocatorGate**	Back-pressure callers; increase slot budget.

Legend & cross-references

Ô (scheduler), τ (tick), Π (instrument set), SBS (spectrum broadcast structure), **APIs:** `/measure`, `/project`, `/pool`, `/agree`, `/belt`, `/trace/:id`; **Events:** TickStart, ChannelSelected, TraceWrite, AgreementPass/Fail, CWA.Pass/Fail, PBHL.Update, PolicyGate.Trigger. Figures: 1 (stack), 2 (Ô loop), 3 (CWA tree), 4 (belt worldsheet), 5 (Five-Line KPI), 6 (Agreement/SBS), 7 (Slot allocator).

---

2) API summary & event taxonomy

> Scope: reference REST layer (gRPC mirrors the same messages). All payloads are JSON; times in RFC3339; IDs are UUIDv4 unless noted. **Idempotency:** send `Idempotency-Key` per write. **Observability:** every response returns `trace_id` and `span_id`.

2.1 Endpoint summary (at a glance)

Endpoint	Method	Purpose	Idempotent?	Auth	Back-pressure & Limits	Primary Events
`/measure`	POST	Select & run an instrument; optionally latch to trace.	Yes (keyed)	Bearer / mTLS	429 on slot/back-pressure; 412 on gate	`ChannelSelected`, `TraceWrite`
`/project`	POST	Apply projector(s) to raw context/features.	Yes	Bearer / mTLS	429 on rate; 422 on invalid projector	`CWA.PanelStart/End`
`/pool`	POST	Certificate-gated additive pooling (CWA).	Yes	Bearer / mTLS	412 if gate trips; 422 on low CWA if `strict=true`	`CWA.Pass/Fail`, `PolicyGate.Trigger`
`/agree`	POST	Cross-observer agreement check on traces.	Yes	Bearer / mTLS	429 on batch; 422 on malformed commute matrix	`AgreementPass/Fail`
`/belt`	POST	Update belt telemetry; compute PBHL Residual.	Yes	Bearer / mTLS	412 on Residual gate; 429 on schedule	`PBHL.Update`, `PolicyGate.Trigger`
`/trace/:id`	GET	Retrieve immutable trace record (and proof).	n/a	Bearer / mTLS	404 if not found; supports ETag	—
`/slots.allocate`	POST	Reserve k slots from a pool.	Yes	Bearer / mTLS	409 if insufficient; 429 on surge	`SlotAllocate`, `SlotCollision`
`/slots.release`	POST	Release slots by ID(s).	Yes	Bearer / mTLS	404 if not owned	`SlotRelease`
`/slots.occupancy`	GET	Report occupancy & collision stats.	n/a	Bearer / mTLS	Cached 1–5s	—
`/tick/sync`	POST	Nudge cadence; report ρ and Δτ.	Yes	Bearer / mTLS	429 on spam; 422 on skew	`TickStart`, `Sync.Alert`

Cross-refs: Ch.10–13 (APIs); Ch.5–7 (Ô/τ, CWA, Belts); Figs.2–5; Table 1 (thresholds).

2.2 Schemas & examples (production-ready)

`/measure` — run instrument and (optionally) latch

Request

```
{
  "pi": "channel://sensor/Z",
  "state_ref": "model://agent/42@v17",
  "tau": null,
  "commit": true,
  "params": {"gain": 0.9, "timeout_ms": 750},
  "tags": {"scenario": "gridworld", "lab": "qubit-toy"}
}
```

Response (200)

```
{
  "measurement_id": "7d9e3c6b-0f0a-4e27-8e18-2d4b5d3cb3eb",
  "pi": "channel://sensor/Z",
  "tau": 12041,
  "outcome": {"y": +1, "distribution": {"+1":0.97,"-1":0.03}},
  "trace_id": "trc_01HZYB3KC3V1M1K7N0",
  "hash": "sha256:…",
  "span_id": "spn_…",
  "latency_ms": 121
}
```

Errors

`409 LatchingGuard` (duplicate τ write), `412 PolicyGate` (AgreementGate on), `422 InvalidInstrument`, `429 BackPressure`, `500/503`.

`/project` — projector application

Request

```
{
  "x": [{"doc_id":"D1","text":"..."},{"doc_id":"D2","text":"..."}],
  "projector": "proj://embedding/e5-large@2025-01",
  "policy": {"normalize": true, "whiten": false, "jitter": 0.01},
  "tags": {"cwa_panel": true}
}
```

Response

```
{
  "projected": [{"doc_id":"D1","v":[...]} , {"doc_id":"D2","v":[...]}],
  "bhi": 0.58,
  "trace_id": "trc_…",
  "span_id": "spn_…",
  "warnings": ["BHI>0.55; consider jitter or re-chunk"]
}
```

Errors

`422 ProjectorError`, `429 RateLimited`.

`/pool` — certificate-gated additive pooling

Request

```
{
  "projected": [{"doc_id":"D1","v":[...]},{"doc_id":"D2","v":[...]}],
  "cert": {"permutations": 64, "sign_flips": 32, "chunk_shuffles": 16, "seed": 1729},
  "min_score": 0.75,
  "strict": true,
  "estimator": "mean"
}
```

Response (CWA pass)

```
{
  "pooled": {"v":[...], "estimator":"mean"},
  "cwa_score": 0.81,
  "phase_risk": 0.22,
  "panel": {"perm_var":0.03,"flip_var":0.02,"chunk_var":0.04},
  "fallback_used": false,
  "trace_id": "trc_…",
  "span_id": "spn_…"
}
```

Response (auto-fallback)

```
{
  "pooled": {"v":[...], "estimator":"attention"},
  "cwa_score": 0.46,
  "phase_risk": 0.57,
  "fallback_used": true,
  "reason": "Certificate below min_score; estimator switched to attention",
  "trace_id": "trc_…",
  "span_id": "spn_…"
}
```

Errors

`412 PolicyGate` (CWA<min and `strict=true`), `422 CertificateInvalid`.

`/agree` — cross-observer agreement

Request

```
{
  "Ta": "trc_01HZ…A",
  "Tb": "trc_01HZ…B",
  "commute_matrix": [[1,1,0],[1,1,1],[0,1,1]],
  "record_policy": {"redundancy_required": 3}
}
```

Response

```
{
  "pass": true,
  "score": 0.97,
  "disagreements": [],
  "redundancy": 3,
  "trace_id": "trc_…",
  "span_id": "spn_…"
}
```

Errors

`422 MalformedCommuteMatrix`.

`/belt` — PBHL telemetry & control

Request

```
{
  "edges": {"plan": 0.62, "do": 0.51},
  "alpha": 0.35,
  "twist": 0.09,
  "mesh": "belt://program/support-rag@Q4",
  "gates": {"residual_max": 0.30}
}
```

Response

```
{
  "gap": 0.11,
  "flux": 0.51,
  "twist": 0.09,
  "alpha": 0.35,
  "residual": 0.02,
  "status": "within-band",
  "trace_id": "trc_…",
  "span_id": "spn_…"
}
```

Errors

`412 PolicyGate` (Residual>threshold).

`/trace/:id` — immutable record fetch

Response

```
{
  "trace_id":"trc_…",
  "tau":12041,
  "chain_hash":"sha256:…",
  "record":{"pi":"channel://sensor/Z","outcome":{"y":+1}},
  "proof":{"prev_hash":"…","sig":"ed25519:…"},
  "span_id":"spn_…"
}
```

`/slots.allocate` / `/slots.release` / `/slots.occupancy`

Allocate Request

```
{"pool":"tools/default","k":3,"ttl_ms":5000}
```

Allocate Response

```
{"granted":["slt_a","slt_b","slt_c"],"occupancy":0.81,"collisions_per_1k":14,"trace_id":"trc_…","span_id":"spn_…"}
```

2.3 Standard headers & error model

- **Request headers:** `Authorization: Bearer …`, `Idempotency-Key: <uuid>`, `X-Run-Id`, `X-Observer-Id`.
- **Response headers:** `ETag`, `X-Trace-Id`, `X-Span-Id`, `Retry-After` (on 429/503).
- **Errors (canonical):**

- `400 BadRequest` (schema)
- `401/403` (authz)
- `409 LatchingGuard` (duplicate τ write)
- `412 PolicyGate` (Agreement/CWA/Residual gate)
- `422 Unprocessable` (projector/certificate/commute)
- `429 TooManyRequests` (back-pressure/slots)
- `500/503` (internal/transient)

2.4 Event taxonomy

All events share a **common envelope**:

```
{
  "event_id":"evt_…",
  "ts":"2025-09-23T09:12:21Z",
  "run_id":"run_…",
  "observer_id":"obs_…",
  "agent_id":"agent_…",
  "trace_id":"trc_…",
  "span_id":"spn_…",
  "parent_event_id":"evt_parent…",
  "op":"<EventName>",
  "level":"INFO|WARN|ERROR",
  "payload":{ ... }
}
```

Core stream (by producer)

Event	Producer	Key payload fields	Metric links	Typical gates
`TickStart` / `TickEnd`	Tick engine	`tau`, `rho_snapshot`, `dt_ms`	ρ, Δτ	`Sync.Alert`
`ChannelSelected`	Scheduler Ô	`tau`, `pi`, `candidates`, `scores`, `Sc`, `AL`	S_c, AL	Exploration cap
`MeasurementStart/Outcome`	Instrument	`pi`, `params`, `y`, `dist`, `latency_ms`	MER	Tool limits
`TraceWrite`	Runtime	`tau`, `pi`, `hash`, `immutable:true`	ICV (=0)	LatchingGuard
`AgreementPass/Fail`	Agree engine	`score`, `diffs[]`, `redundancy`	AGR, R_SBS	AgreementGate
`CWA.PanelStart/End`	CWA engine	`panel_cfg`, `panel_stats`	CWA, PRI	PoolingGate
`CWA.Pass/Fail`	CWA engine	`score`, `pri`, `fallback_used`, `estimator`	CWA, PRI	PoolingGate
`Detector.BHI.High`	Projector	`bhi`, `projector`, `window`	BHI	ProjectorGate
`PBHL.Update`	BeltOps	`gap`, `flux`, `twist`, `alpha`, `residual`, `status`	PBHL Residual, EEI/SI	BeltGate
`PolicyGate.Trigger`	Policy svc	`gate`, `trigger`, `thresholds`, `action`	—	Various
`SlotAllocate/Release`	Allocator	`pool`, `k`, `granted[]`, `occupancy`, `collisions_per_1k`	Occ, SCR	AllocatorGate
`Sync.Alert`	Tick engine	`rho`, `delta_tau`, `n_outliers`	ρ, Δτ	SyncGate
`Audit.Export`	Audit svc	`format`, `records`, `dest`, `checksum`	—	Compliance

Ordering & causality: maintain per-`trace_id` **happens-before** with `parent_event_id`. Consumers should use `span_id` to stitch spans.

Levels: `INFO` (steady), `WARN` (Amber threshold crossed), `ERROR` (Red or invariants violated).

Retention: event log 30–90 days; `TraceWrite` + proofs durable ≥1 year (configurable; see compliance appendix).

2.5 Security, privacy, and proofs

- **PII minimization:** payloads carry hashes/IDs; redact raw content (store out-of-band).
- **Immutability:** `TraceWrite.proof` includes hash-chain + `ed25519` signature (org root).
- **Access control:** per-endpoint scopes (`measure:write`, `trace:read`, `belt:write`, etc.).
- **Auditability:** `Audit.Export` bundles event slices + proofs; compatible with SOC/ISO mappings (Appendix F).

2.6 Event→Metric map (operational wiring)

Metric (Table 1)	Consumes events	Computation window
AGR	`AgreementPass/Fail`	Rolling W ticks
MER	`MeasurementOutcome` (+tool errors)	Rolling W ticks
THL	`TraceWrite` + read-access logs	Rolling W ticks
ICV	`TraceWrite` (violations)	Rolling W ticks

Metric (Table 1)	Consumes events	Computation window
Occ/SCR	`SlotAllocate/Release`	Rolling W ticks
AL/S_c	`ChannelSelected`	Rolling W ticks
ρ/Δτ	`TickStart/End`, `Sync.Alert`	Fleet window
CWA/PRI	`CWA.*`	Panel window
BHI	`Detector.BHI.High` (and projector stats)	Rolling W ticks
PBHL Residual	`PBHL.Update`	Weekly

2.7 SLA & SLO hints

- **/measure P50 latency**: ≤150 ms (lab), ≤300 ms (prod-guarded).
- **/pool P95 latency**: certificate ≤350 ms for panel ≤64 permu; provide async batch only when gating is **INFO**.
- **Event delivery**: at-least-once, ≤2 s end-to-end; use `event_id` for de-dupe.

---

3) Ablation results (±Ô, ±slots, ±certificates)

> Setup (applies to all tables): 5 seeds, median reported; **W=256 ticks** (micro/meso) and **4-week** macro window. Hardware parity; identical corpora/tasks across rows. If **CWA is absent**, the runtime defaults to **attention-based order-aware pooling** (conservative), which is slower but robust—so you can see the latency benefit of **CWA-gated pooling** at roughly equal accuracy. Cross-refs: Ch.5–7, 14–17; Figs.2–5; Tables 1–2.

Legend for toggles: **Ô** = scheduler on; **Slots** = allocator on; **Cert** = CWA certificate on.

3.1 RAG & embeddings — accuracy/latency under pooling and scheduling

Config (Ô / Slots / Cert)	Top-1 Acc. (%) ↑	P95 Latency (ms) ↓	CWA Pass-Rate ↑	Phase-Risk (PRI) ↓	Tokens / query ↓
All ON (✓/✓/✓)	**79.1**	**240**	**0.78**	**0.27**	**7.6k**
−Ô (✗/✓/✓)	77.4 (−1.7)	270 (+30)	0.72	0.31	8.5k
−Slots (✓/✗/✓)	77.9 (−1.2)	310 (+70)	0.75	0.33	9.1k
−Cert (✓/✓/✗)†	78.8 (−0.3)	340 (+100)	n/a	**0.19**	10.2k
−Ô −Slots (✗/✗/✓)	76.1 (−3.0)	360 (+120)	0.69	0.37	10.6k
−Ô −Cert (✗/✓/✗)†	77.1 (−2.0)	400 (+160)	n/a	**0.18**	11.2k
−Slots −Cert (✓/✗/✗)†	76.7 (−2.4)	430 (+190)	n/a	0.21	12.5k
All OFF (✗/✗/✗)†	74.9 (−4.2)	480 (+240)	n/a	**0.17**	13.1k

† No-certificate rows default to **attention-only** pooling (safe but slow), hence lower PRI but higher latency.
Takeaway: CWA-gated pooling cuts P95 latency ≈**29–50%** vs attention-only rows at comparable accuracy, while **Ô** and **Slots** mainly protect accuracy and compute.

3.2 Tool-using agents — reliability & capacity

Config (Ô / Slots / Cert)	Agreement Rate (AGR) ↑	Mis-exec Rate (MER) ↓	ρ (sync) ↑	Δτ (ticks) ↓	Occupancy (Occ)	Collisions (SCR per 1k) ↓
All ON (✓/✓/✓)	**0.96**	**1.7%**	**0.84**	**0.9**	**0.81**	**12**
−Ô (✗/✓/✓)	0.90	3.8%	0.79	1.2	0.83	16
−Slots (✓/✗/✓)	0.91	4.9%	0.80	1.3	**0.97 (Red)**	**41 (Red)**
−Cert (✓/✓/✗)	0.95	1.8%	0.84	0.9	0.81	12
−Ô −Slots (✗/✗/✓)	0.86	6.2%	0.75	1.7	**0.98 (Red)**	**57 (Red)**
−Ô −Cert (✗/✓/✗)	0.89	3.9%	0.78	1.4	0.83	18
−Slots −Cert (✓/✗/✗)	0.90	5.0%	0.79	1.5	**0.97 (Red)**	**44 (Red)**
All OFF (✗/✗/✗)	0.83	7.1%	0.68	2.4	**0.99 (Red)**	**74 (Red)**

Takeaway: Ô drives **AGR** (−6–13 pts when off). **Slots** controls collisions and mis-exec; removing it alone pushes allocator metrics into **Red** bands (Table 1).

3.3 Macro belts — closure & sustainability

Config (Ô / Slots / Cert)	PBHL Residual ↓	EEI ↑	SI ↑	Status
All ON (✓/✓/✓)	**0.08**	**0.69**	**0.73**	**Within band**

Config (Ô / Slots / Cert)	PBHL Residual ↓	EEI ↑	SI ↑	Status
–Ô (✗/✓/✓)	0.17	0.61	0.69	Amber
–Slots (✓/✗/✓)	0.21	0.58	0.62	Amber
–Cert (✓/✓/✗)	0.11	0.65	0.72	Green
–Ô –Slots (✗/✗/✓)	0.26	0.55	0.60	Amber
–Ô –Cert (✗/✓/✗)	0.18	0.60	0.68	Amber
–Slots –Cert (✓/✗/✗)	0.23	0.57	0.63	Amber
All OFF (✗/✗/✗)	**0.34**	**0.49**	**0.55**	**Red**

Takeaway: Macro closure (**Residual**) degrades most when **Ô** and **Slots** are removed; **Cert** absence mostly hits throughput/latency upstream (3.1) with modest macro drift.

3.4 Notes for reproducibility

- **RAG battery:** 50k queries; projection `e5-large@2025-01`; panel = 64 perms / 32 sign-flips / 16 chunk-shuffles; **strict=true**.
- **Agents suite:** 12 tools (4 commuting pairs, 4 conflicting); quorum logging to shared ledger; slot pool = 16; TTL 5s.
- **Belts sim:** weekly OKR sampling; Flux-gate (fast) and Twist-step (slow) per Ch.7; α=0.35; residual gates per **prod-guarded** profile (Table 1 §1.4).

4) Commute vs conflict instrument pairs

Reading guide: Instruments **commute** in ObserverOps when their **effective outcomes** and **latching writes** are invariant to order **within the same tick frame** (or successive ticks with no intermediate side-effects), i.e. `apply(A)→apply(B)` and `apply(B)→apply(A)` yield equal distributions for `(S,T)` at commit and compatible traces (up to stable renaming). We denote this **frame-commutation** and estimate it via the **Agreement engine** over AB/BA trials with tolerance ε. Cross-refs: Ch.3 (Agreement), Ch.5 (Ô/τ), Ch.6 (CWA), Figs.2 & 6; Tables 1–3.

4.1 Canonical pairs (domain-agnostic patterns)

#	Domain	Instrument A	Instrument B	Effect signature (writes)	Commutation condition	Status	Typical metrics impacted	Notes
1	Qubit toy	**Measure Z**	**Measure Z** (repeat)	`T: z∈{±1}` (idempotent), `S: collapse→ z⟩`	Same projector same basis; no state evolution between	**Commute**	AGR ↑, MER ↓	
2	Qubit toy	**Measure Z**	**Measure X**	`T: (z,x), S: collapse to last basis`	Only commute if **records-only** (no downstream control) and treat as independent channels —otherwise order changes S	**Conflict** (default)	AGR ↓, PRI ↑	Exhibit non-commutation counterexample (AB≠BA).
3	KB/RAG	**Fetch chunk bag** (unordered)	**Mean-pool embeddings**	`T: projected vectors, S: cache`	Projection removes order; CWA≥θ	**Commute** (if CWA pass)	Latency ↓	Gate by CWA (Table 1).
4	KB/RAG	**Fetch ordered sections**	**Concatenate then encode**	`T: concatenated text`	Order preserved in encoder	**Conflict**	PRI ↑	Use attention pooling fallback.
5	KB/RAG	**Dedup exact**	**BM25 rank**	`T: doc set, S: index hits`	Set ops commute if dedup key fixed	**Commute**	Accuracy stable	Ensure stable hashing key.
6	LLM tools	**Search web**	**Open URL**	`T: urls[], http logs`	Opening is subset of searched results and doesn't mutate `S`	**Commute**	MER ↓	Ensure opener is read-only.
7	LLM tools	**Write file**	**Read file**	`T: fs op logs, S: file state`	Commute **only** if read is version-pinned to pre-write snapshot	**Conditional**	MER, ICV risk	Enforce content-hash reads.
8	LLM tools	**Plan step** (update TODO)	**Execute tool**	`T: plan log, S: env mutate?`	If execute mutates env referenced by plan	**Conflict**	AGR ↓	Latch plan at τ; re-plan after execute.
9	RL/Robotics	**Read sensor A**	**Read sensor B**	`T: (a,b)`	Independent sensors, no shared bus locking	**Commute**	AGR ↑	Bus contention can break independence.
10	RL/Robotics	**Set gripper**	**Move arm**	`S: pose, grasp, T: act logs`	Non-commute due to dynamics; grasp before move vs after	**Conflict**	Safety incidents	Use compatibility graph to schedule.

#	Domain	Instrument A	Instrument B	Effect signature (writes)	Commutation condition	Status	Typical metrics impacted	Notes
11	Governance/ Belt	**Update Flux**	**Update Twist**	`T: KPI, S: controller states`	Algebraic combination Gap≈Flux+α·Twist is commutative in update order when updates are purely assignment	**Commute**	Residual stable	If updates trigger controllers with side-effects, treat as **conditional**.
12	Storage/ Cache	**Cache put (key,v)**	**Cache get (key)**	`S: cache, T: cache logs`	Commute only if **get** is versioned and put targets **different** version	**Conditional**	Occ, SCR	Prefer write-through with epoch tagging.

4.2 Compatibility graph cheat-sheet

- **Vertices:** instruments $\pi \in \Pi$.
- **Edges:** `C[π_i,π_j]=1` iff **frame-commute** (per 4.3 test).
- **Cliques:** safe concurrent schedules (Ô may reorder within cliques).
- **Cuts:** conflict boundaries; Ô must serialize across cuts or insert barriers (new τ).

Heuristics

- Treat **idempotent reads** and **pure projections** as likely commuting.
- Treat **writes** and **stateful encoders** as likely conflicting unless version-pinned.
- **Certificate-conditioned** edges (e.g., RAG pooling) must check **CWA≥θ** at runtime.

4.3 Empirical commutation test (Agreement engine recipe)

Goal: Decide if A and B commute under current policies.
Protocol (AB/BA panel, ε-tolerance):

1. Sample contexts $x_1 \ldots x_n$.
2. Run AB path: `apply(A)→apply(B)`; capture `(S_AB, T_AB)`.
3. Reset to same start state; run BA: `apply(B)→apply(A)`; capture `(S_BA, T_BA)`.
4. Compare:
   - **Outcome invariance:** `D(Outcome_AB, Outcome_BA) ≤ ε` (e.g., symmetric KL or MMD).
   - **Trace compatibility:** `hash(T_AB)` equals `hash(T_BA)` **or** differs only by **stable relabeling** (per policy).
   - **Downstream neutrality:** next-tick decisions `select_channel(S_AB,T_AB)` vs `select_channel(S_BA,T_BA)` match with prob ≥ `1−δ`.
5. Emit `AgreementPass/Fail` with `score=1−D` and update `C[i,j]`.

Defaults: `n=128`, `ε=0.02`, `δ=0.05`, window W=256 ticks. Cross-refs: Table 2 `/agree`.

4.4 Registry snippet (declaring commute classes & guards)

```
# /sdk/observer-runtime/compat.yaml
commute:
  - class: "pure-read"
    members: ["sensor://imu", "sensor://lidar", "channel://kb/fetch-bag"]
    test: "panel"           # still verify empirically
  - class: "projection"
    members: ["proj://embed/e5-2025-01", "proj://mean-pool"]
    guard:
      cwa_min: 0.75         # certificate-conditioned edge
conflict:
  - pairs:
      - ["tool://fs/write", "tool://fs/read"]
      - ["robot://arm/move", "robot://gripper/set"]
    policy: "serialize"
conditional:
  - pair: ["tool://http/open", "tool://http/search"]
    condition: "open.readonly && search.cache=true"
    policy: "allow"
```

4.5 Edge cases & counterexamples

- **Hidden side-effects:** "read" that warms caches or writes telemetry used by Ô ⇒ **conflict** unless excluded from control inputs.
- **Batching illusions:** vector DB batching can reorder responses—projects may **appear** commuting while downstream ranking uses timestamps ⇒ **conditional**.
- **Time-dependent sensors:** two "reads" sampling at slightly offset τ can disagree; require **barrier** or **timestamp alignment**.
- **Semantic encoders:** concatenation encoders are order-sensitive; switching to **bag-of-spans projection** may restore commutation under **CWA**.
- **Controllers:** Belt updates that trigger **Flux-gate** or **Twist-step** feedback are **non-commutative** unless gates are disabled for the panel.

4.6 Operational wiring (events → graph)

- Build/refresh `C` via `AgreementPass/Fail` AB/BA panels (Table 2 §2.4).
- On **Fail**, emit `PolicyGate.Trigger(gate=AgreementGate)` and mark edge `0`.
- Ô loads latest `C` at tick start; **clique-aware** scheduling permits internal reorders; cross-cut ops force new τ.

4.7 Quick reference table (copy-paste to runbooks)

Label	Safe concurrent?	Needs CWA?	Needs version pin?	Typical policy
Pure reads (independent sensors)	Yes	No	No	Allow; coalesce
Projections + mean/sum	**Yes if CWA≥θ**	**Yes**	No	Allow; verify panel
Read after write (same key)	Conditional	No	**Yes**	Serialize or snapshot
Stateful encoders (concat)	No	n/a	n/a	Serialize; prefer attention
Tool plan ↔ execute	No	n/a	n/a	Latch plan; re-plan post-execute
Belt Flux ↔ Twist (no controllers)	Yes	No	No	Allow; atomic update
Belt Flux/Twist with controllers	Conditional	No	No	Serialize with hooks

---

5) Policy gates & escalation ladders

Purpose: convert metric bands (Table 1) + APIs/events (Table 2) + compatibility rules (Table 4) into **automatable safety rails**. Gates are small, declarative controllers that **Trip → Act → Persist (TAP)** with hysteresis and audit logs. Each gate defines: **Trigger**, **Auto-Action**, **Owner/Pager**, **Escalation**, **Exit (clear)**. Cross-refs: Ch.2–7, 14–17; Figs.2–5; Tables 1–4.

5.1 Gate inventory (canonical set)

Gate	Triggering metric(s)	Amber threshold (enter)	Red threshold (enter)	Auto-Action (Amber)	Auto-Action (Red)	Clear (exit)	Primary events
PoolingGate	**CWA < θ** or **PRI > φ**	CWA ∈[0.50,0.75) or PRI ∈[0.25,0.50)	CWA <0.50 or PRI >0.50	Shrink pool; raise CI; switch estimator to **hybrid**	Force **order-aware** estimator; require human sign-off for batch pools	CWA ≥0.75 & PRI ≤0.25 for 3 panels	`CWA.Pass/Fail`, `PolicyGate.Trigger`
AgreementGate	**AGR**	AGR ∈[0.85,0.95)	AGR <0.85	Require quorum before act; increase SBS redundancy	Suspend autonomous actuation; replica-only dry-runs	AGR ≥0.95 for 512 ticks	`AgreementPass/Fail`, `PolicyGate.Trigger`
LatchingGuard	**ICV > 0**	—	≥1 violation	—	Block commit; open incident; snapshot state	Zero violations for 10k writes	`TraceWrite`, `PolicyGate.Trigger`
AllocatorGate	**Occ**, **SCR**	Occ ∈(85%,95%] or SCR ∈[10,30]	Occ >95% or SCR >30/1k	Back-pressure callers; extend TTL	Increase slot pool; serialize contending callsites	Occ ∈[40%,85%] & SCR <10 for 1k allocs	`SlotAllocate/Release`, `PolicyGate.Trigger`
SyncGate	**ρ**, **Δτ**	ρ ∈[0.60,0.80) or Δτ ∈(1,3]	ρ <0.60 or Δτ >3	Soft resync: stretch cadence; nudge laggards	Barrier at τ; staged pause and reschedule	ρ ≥0.80 & Δτ ≤1 for 256 ticks	`TickStart`, `Sync.Alert`, `PolicyGate.Trigger`

Gate	Triggering metric(s)	Amber threshold (enter)	Red threshold (enter)	Auto-Action (Amber)	Auto-Action (Red)	Clear (exit)	Primary events
ProjectorGate	**BHI**	BHI ∈(0.55,0.70]	BHI >0.70	Add jitter/ whiten; re-chunk	Block projector; force alternative projector	BHI ≤0.55 for 3 windows	`Detector.BHI.High`
BeltGate	**PBHL Residual**	Residual ∈(0.10,0.30]	Residual >0.30	Tighten **Flux-gate**; freeze non-critical Twist	Freeze scope changes; exec review; higher sampling	Residual ≤0.10 for 2 weeks	`PBHL.Update`, `PolicyGate.Trigger`
ToolGuard	**MER**	MER ∈(2%,5%]	MER >5%	Reduce concurrency; require template args	Disable tool; sandbox re-qual	MER ≤2% for 1k invocations	`MeasurementOutcome`

*(Thresholds are the **prod-guarded** profile; override per 1.4 Table 1 presets.)*

5.2 Severity matrix & paging

Severity	When triggered	Pager / Owner	SLO: TTD (detect)	SLO: TTC (contain)	SLO: TTR (recover)	Examples
SEV-1	LatchingGuard, BeltGate(Red), AgreementGate(Red)	On-call SRE + Product Owner + Safety Lead	≤30 s	≤5 min	≤4 h	Retro-edit detected; PBHL Residual 0.34; AGR 0.80
SEV-2	PoolingGate(Red), AllocatorGate(Red), SyncGate(Red)	On-call SRE + Runtime TL	≤60 s	≤10 min	≤24 h	CWA 0.42; Occ 97%; ρ 0.55
SEV-3	ProjectorGate(Red), ToolGuard(Red)	Runtime TL	≤5 min	≤30 min	≤48 h	BHI 0.74; MER 6.1%
INFO/ Amber	Any Amber entry	Service Owner (async)	—	—	—	Soft resync; pool shrink

---

5.3 TAP gate logic (pseudocode)

```
def gate_eval(metric_value, amber, red, state):
    if metric_value >= red.high or metric_value <= red.low:
        if state != "RED":
            emit("PolicyGate.Trigger", gate=G, level="RED")
            do_red_actions()
        return "RED"
    if in_amber(metric_value, amber):
        if state == "GREEN":
            emit("PolicyGate.Trigger", gate=G, level="AMBER")
            do_amber_actions()
        return "AMBER"
    if state in {"AMBER","RED"} and stable_for(clear_window):
        emit("PolicyGate.Trigger", gate=G, level="CLEAR")
        undo_actions()
        return "GREEN"
    return state
```

Hysteresis: `clear_window` prevents flap (see Table 5.1 “Clear”). *Audit:* every transition writes a `TraceWrite` with gate context.

5.4 Escalation ladders (playbook snippets)

A) PoolingGate (CWA/PRI)

1. **Amber** → shrink pool size by 50%; increase permutations to reach target CI; attempt hybrid estimator (mean on stable axes, attention on risky axes).
2. **Red** → force order-aware estimator; set `/pool?strict=true`; require **human sign-off** for batch pools; add “Phase-Risk ticket” to backlog.
3. **Persistent 3× Red in 24h** → rotate projector (ProjectorGate assist), open **SEV-2** incident, attach cert logs to JIRA.

B) AgreementGate (AGR)

1. **Amber** → require majority quorum across replicas; enable SBS Redundancy Probe (add 1 extra pointer channel).
2. **Red** → disable autonomous actuation; switch to **read-only** + human approval; run **commutation panel** (AB/BA) to locate conflicts.
3. **Exit** → AGR≥0.95 for 512 ticks; produce **diff report**.

C) LatchingGuard (ICV)

- Immediate **SEV-1**: block commit; snapshot state; page on-call; gather offending span; run "re-entrancy guard" test. Exit only after *root cause* patch + 10k clean writes.

D) AllocatorGate (Occ/SCR)

1. **Amber** → back-pressure; TTL +20%; preempt low-priority callers.
2. **Red** → increase pool by N; serialize contentious callsites; if still Red, **pause** RAG batch jobs.
3. **Exit** → Occ∈[40%,85%] and SCR<10/1k for 1k allocs.

E) SyncGate (ρ/Δτ)

1. **Amber** → cadence retune (stretch by 10%); send nudges to laggards.
2. **Red** → introduce barriers at tick; staged pause; reschedule fleets by cohort.
3. **Exit** → ρ≥0.80 and Δτ≤1 for 256 ticks.

F) ProjectorGate (BHI)

1. **Amber** → apply jitter 0.01–0.05; optional whitening; re-chunk to balanced spans.
2. **Red** → disable projector, switch to backup; trigger RAG re-index off-peak.
3. **Exit** → BHI≤0.55 for 3 windows; attach before/after cert plots.

G) BeltGate (PBHL Residual)

1. **Amber** → tighten Flux-gate (fast controller); mark scope changes as "gated".
2. **Red** → freeze scope/Twist; exec review; raise Residual sampling to daily; publish Five-Line KPI to leadership.
3. **Exit** → Residual≤0.10 for 2 weeks; file closure memo.

H) ToolGuard (MER)

1. **Amber** → reduce parallelism; enforce argument templates; increase pre-checks.
2. **Red** → disable tool; sandbox; require re-qualification tests; publish failure taxonomy.
3. **Exit** → MER≤2% across 1k invocations.

5.5 Gate precedence & interactions

Precedence order (highest first): `LatchingGuard` → `AgreementGate` → `PoolingGate` → `AllocatorGate` → `SyncGate` → `ProjectorGate` → `BeltGate` → `ToolGuard`.
Rules of thumb:

- **Blocking** gates (LatchingGuard, AgreementGate Red) **pre-empt** downstream actions.
- **Conditioned edges** (Table 4) require **PoolingGate** checks *before* Ô reorders across cliques.
- **AllocatorGate Red** can **promote** SyncGate Amber to Red (saturation causes desync).
- BeltGate decisions **never** override LatchingGuard/AgreementGate safety—macro closure cannot trade off micro invariants.

5.6 Telemetry & audit

Every gate emits `PolicyGate.Trigger` with:

```
{
  "gate":"PoolingGate",
  "level":"RED",
  "trigger":{"metric":"CWA","value":0.46,"threshold":0.50},
  "action":"fallback_estimator=attention, strict=true",
  "hysteresis":{"min_clear_windows":3,"window":"panel"},
  "owner":"runtime-cwa",
  "runbook":"rb://poolinggate/v1"
}
```

Attach to **TraceWrite** and include in `Audit.Export` bundles (Appendix F).

5.7 Config pack (copy-paste)

```
# /apps/beltops/policy/gates.yaml
defaults:
  hysteresis:
    clear_windows:
      pooling: 3        # panels
      agreement: 2      # W windows
      allocator: 1k     # allocations
      sync: 256         # ticks
      projector: 3      # windows
      belt: 2w          # weeks
owners:
  runtime-cwa: "@oncall-runtime"
  runtime-core: "@oncall-runtime"
  beltops: "@oncall-beltops"
  sre: "@oncall-sre"
gates:
  PoolingGate:
    enable: true
    thresholds: {cwa_min: 0.75, pri_max: 0.25}
    actions:
      amber: {estimator: "hybrid", pool_shrink: 0.5, increase_panel: true}
      red:   {estimator: "attention", strict: true, human_signoff: true}
  AgreementGate:
    enable: true
    thresholds: {agr_min: 0.95}
    actions:
      amber: {require_quorum: true, add_redundancy: 1}
      red:   {autonomy: "off", mode: "read-only"}
  LatchingGuard:
    enable: true
    actions:
      red: {block_commit: true, snapshot: true, incident: "SEV-1"}
  AllocatorGate:
    thresholds: {occ_max: 0.95, scr_max_per_1k: 30}
    actions:
      amber: {backpressure: true, ttl_increase: 0.2}
      red:   {increase_pool: 4, serialize_callers: true}
  SyncGate:
    thresholds: {rho_min: 0.80, dtau_max: 1}
    actions:
      amber: {stretch_cadence: 0.1}
      red:   {barrier: true, staged_pause: true}
  ProjectorGate:
    thresholds: {bhi_max: 0.70}
    actions:
      amber: {jitter: 0.02, whiten: true, rechunk: true}
      red:   {disable_projector: true, switch_backup: true}
  BeltGate:
    thresholds: {residual_max: 0.30}
    actions:
      amber: {tighten_flux_gate: true, freeze_noncritical_twist: true}
      red:   {freeze_scope: true, exec_review: true, sample_daily: true}
  ToolGuard:
    thresholds: {mer_max: 0.05}
    actions:
      amber: {reduce_concurrency: true, arg_templates: true}
      red:   {disable_tool: true, sandbox: true, requal: true}
```

5.8 Runbook template (per gate)

Header: Gate, Severity, Trigger snapshot, Owner, On-call.
Containment: the exact **Auto-Action** commands (API calls/flags).
Diagnosis checklist: links to last **10 events**, cert panels, AB/BA diffs, allocator heatmap, sync chart, Five-Line KPI.
Recovery: precise **Exit** criteria; how to revert throttles; validation checks.
Post-incident: root cause, fix, tests added, thresholds tuned, artifact links.

With this table, SREs and product owners can wire **closed-loop governance**: the same invariants that define observers (internal collapse, agreement, slots, closure) now activate **deterministic, auditable** responses.

Gate→API Command Sheet (copy-paste for runbooks)

Set once:

```
export HOST="https://observerops.example.com"
export TOKEN="<BEARER_TOKEN>"
export RID="run_$(date +%s)"
hdr=(-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -H "Idempotency-Key: $RID")
```

PoolingGate (CWA / PRI)

Amber → shrink pool, hybrid estimator, bigger panel

```
curl -sS -X POST "$HOST/pool" "${hdr[@]}" -d @- <<'JSON'
```

```
{
  "projected": [{"doc_id":"D1","v":[...]},{"doc_id":"D2","v":[...]},{"doc_id":"D3","v":[...]}],   // shrink set
  "cert": {"permutations": 96, "sign_flips": 48, "chunk_shuffles": 24, "seed": 1729},
  "min_score": 0.60,
  "strict": false,
  "estimator": "hybrid",
  "tags": {"gate":"PoolingGate","level":"AMBER"}
}
JSON
```

Red → force order-aware estimator, strict gate, human sign-off

```
curl -sS -X POST "$HOST/pool" "${hdr[@]}" -d @- <<'JSON'
{
  "projected": [{"doc_id":"D1","v":[...]},{"doc_id":"D2","v":[...]}],
  "cert": {"permutations": 64, "sign_flips": 32, "chunk_shuffles": 16, "seed": 1729},
  "min_score": 0.75,
  "strict": true,
  "estimator": "attention",
  "tags": {"gate":"PoolingGate","level":"RED","requires_signoff":true}
}
JSON
```

AgreementGate (AGR)

Amber → quorum before act + add SBS redundancy

```
# 1) Check agreement on replicas
curl -sS -X POST "$HOST/agree" "${hdr[@]}" -d @- <<'JSON'
{"Ta":"trc_a","Tb":"trc_b","commute_matrix":[[1,1],[1,1]],"record_policy":{"redundancy_required":3}}
JSON
# 2) Redundant pointer write (extra channel)
curl -sS -X POST "$HOST/measure" "${hdr[@]}" -d @- <<'JSON'
{
  "pi":"channel://pointer/redundant-2",
  "state_ref":"model://agent/42@v17",
  "commit":true,
  "tags":{"gate":"AgreementGate","level":"AMBER","mode":"quorum"}
}
JSON
```

Red → read-only (no commits) until quorum recovers

```
curl -sS -X POST "$HOST/measure" "${hdr[@]}" -d @- <<'JSON'
{
  "pi":"channel://tool/safe-read",
  "state_ref":"model://agent/42@v17",
  "commit":false,
  "tags":{"gate":"AgreementGate","level":"RED","autonomy":"off","reason":"AGR low"}
}
JSON
```

LatchingGuard (ICV)

Immediate → block commits, snapshot evidence

```
# Switch to read-only probes
curl -sS -X POST "$HOST/measure" "${hdr[@]}" -d @- <<'JSON'
{"pi":"channel://diagnostic/hash-check","commit":false,"tags":{"gate":"LatchingGuard","level":"RED"}}
JSON
# Fetch offending trace for the incident
curl -sS "$HOST/trace/trc_offender_id" -H "Authorization: Bearer $TOKEN"
```

AllocatorGate (Occ / SCR)

Amber → back-pressure & increase slot TTL

```
curl -sS -X POST "$HOST/slots.allocate" "${hdr[@]}" -d @- <<'JSON'
{"pool":"tools/default","k":1,"ttl_ms":8000,"tags":{"gate":"AllocatorGate","level":"AMBER","mode":"backpressure"}}
JSON
```

Red → serialize callers (1-at-a-time)

```
# Acquire single slot; run; release — repeat
SID=$(curl -sS -X POST "$HOST/slots.allocate" "${hdr[@]}" -d '{"pool":"tools/default","k":1,"ttl_ms":5000}' | jq -r '.granted[0]')
# ...perform one critical /measure using that slot...
curl -sS -X POST "$HOST/slots.release" "${hdr[@]}" -d "{\"ids\":[\"$SID\"],\"tags\":{\"gate\":\"AllocatorGate\",\"level\":\"RED\"}}"
```

SyncGate (ρ / Δτ)

Amber → stretch cadence; nudge laggards

```
curl -sS -X POST "$HOST/tick/sync" "${hdr[@]}" -d @- <<'JSON'
{"stretch":0.10,"nudge":{"targets":"laggards","max_dtau":1},"tags":{"gate":"SyncGate","level":"AMBER"}}
JSON
```

Red → barrier at next tick; staged pause/reschedule

```
curl -sS -X POST "$HOST/tick/sync" "${hdr[@]}" -d @- <<'JSON'
{"barrier":true,"pause_cohorts":["beta","gamma"],"resume_after_sec":300,"tags":{"gate":"SyncGate","level":"RED"}}
JSON
```

ProjectorGate (BHI)

Amber → jitter/whiten/re-chunk

```
curl -sS -X POST "$HOST/project" "${hdr[@]}" -d @- <<'JSON'
{
  "x":[{"doc_id":"D1","text":"..."},{"doc_id":"D2","text":"..."}],
  "projector":"proj://embedding/e5-large@2025-01",
  "policy":{"normalize":true,"whiten":true,"jitter":0.02,"rechunk":{"target_tokens":256}},
  "tags":{"gate":"ProjectorGate","level":"AMBER"}
}
JSON
```

Red → switch to backup projector

```
curl -sS -X POST "$HOST/project" "${hdr[@]}" -d @- <<'JSON'
{
  "x":[{"doc_id":"D1","text":"..."},{"doc_id":"D2","text":"..."}],
  "projector":"proj://embedding/e5-large@backup",
  "policy":{"normalize":true},
  "tags":{"gate":"ProjectorGate","level":"RED"}
}
JSON
```

BeltGate (PBHL Residual)

Amber → tighten Flux-gate; freeze non-critical Twist

```
curl -sS -X POST "$HOST/belt" "${hdr[@]}" -d @- <<'JSON'
{
  "edges":{"plan":0.62,"do":0.51},
  "alpha":0.35,
  "twist":0.09,
  "mesh":"belt://program/support-rag@Q4",
  "gates":{"residual_max":0.20,"freeze_noncritical_twist":true},
  "tags":{"gate":"BeltGate","level":"AMBER"}
}
JSON
```

Red → freeze scope; raise sampling to daily; exec review

```
curl -sS -X POST "$HOST/belt" "${hdr[@]}" -d @- <<'JSON'
{
  "edges":{"plan":0.62,"do":0.51},
  "alpha":0.35,
  "twist":0.09,
  "mesh":"belt://program/support-rag@Q4",
  "gates":{"residual_max":0.30,"freeze_scope":true,"sample_rate":"daily","exec_review":true},
  "tags":{"gate":"BeltGate","level":"RED"}
}
JSON
```

ToolGuard (MER)

Amber → reduce parallelism; enforce arg templates

```
curl -sS -X POST "$HOST/measure" "${hdr[@]}" -d @- <<'JSON'
{
  "pi":"tool://structured-call",
  "params":{"template_enforced":true},
  "commit":true,
  "tags":{"gate":"ToolGuard","level":"AMBER","max_concurrency":2}
}
JSON
```

Red → disable tool; sandbox re-qual

```
curl -sS -X POST "$HOST/measure" "${hdr[@]}" -d @- <<'JSON'
{
  "pi":"tool://structured-call",
```

```
  "commit":false,
  "tags":{"gate":"ToolGuard","level":"RED","disabled":true,"sandbox":"requal-suite"}
}
JSON
```

Notes

- Include `trace_id`/`span_id` from responses in your incident notes.
- Commands set descriptive `tags` so `PolicyGate.Trigger` and `Audit.Export` can correlate actions to gates (Tables 2 & 5).

© 2025 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.

M

T

G

Y

Text-to-speech function is limited to 200 characters

ObserverOps Technical Blueprint - Boxed Callouts

CWA Certificate: when *project → add* is safe

What it certifies.
After a projection $P(\cdot)$, **order/phase information is operationally erased**, so **additive estimators** (mean/sum, optionally weighted with order-independent weights) yield stable outputs. The certificate runs a **perturbation panel** and computes:

- **CWA score** $\in [0,1]$: invariance under permutations, sign-flips, and chunk-shuffles.
- **Phase-Risk Index (PRI)** $\in [0,1]$: normalized output variance across the panel.

Passing bands (default **prod-guarded** profile; tune in Table 1 §1.4):

- **Green:** `CWA ≥ 0.75` **and** `PRI ≤ 0.25` → **Additive OK** (mean/sum).
- **Amber:** `0.50 ≤ CWA < 0.75` **or** `0.25 < PRI ≤ 0.50` → **Hybrid** (add on stable axes; attention on risky axes), shrink pool, increase panel size.
- **Red:** `CWA < 0.50` **or** `PRI > 0.50` → **Order-aware fallback** (attention/CNN/ranker), human sign-off for batch pools.

Minimal recipe (operator checklist)

1. **Project first.** Choose projector P that is order-agnostic in intent (e.g., per-span embeddings).
2. **Assemble pool** $V = \{v_i\}_{i=1..n}$ (dedup exact near-dups).
3. **Run certificate panel**:
   - **Permutations**: shuffle indices k times (e.g., 64).
   - **Sign-flips**: multiply random subset by −1 if the space admits orientation symmetry (32).
   - **Chunk-shuffles**: perturb chunk boundaries without changing content mass (16).
4. **Compute metrics**:
   - **CWA** $= \text{combine}\left(\text{p}_{\text{perm}}, \text{p}_{\text{flip}}, \text{p}_{\text{chunk}}\right)$ (e.g., Fisher or mean pass-rate).
   - **PRI** $= \frac{\text{Var}_{\text{panel}}(f(V))}{\text{Var}_{\text{ref}}}$ (clip to $[0,1]$).
5. **Gate** with **PoolingGate**. Log `CWA.Pass/Fail`, rationale, and estimator used.

Estimator rules (safe set):

- Mean/sum; weighted mean when weights depend **only** on per-item $P(v_i)$ (e.g., norm, recency) and not on order/history.

- Do **not** add raw tokens or concatenated encodings; those are order-sensitive (use attention/CNN instead).

Failure modes (and quick fixes)

- **High PRI, decent CWA** → residual order sensitivity from **degenerate projection**. Fix: add **jitter/whitening**, re-chunk to balanced spans; monitor **BHI** (Black-Hole Index).
- **Low CWA** → panel exposes order/phase coupling (e.g., concatenation encoders, bursty sequences). Fix: **fallback estimator**, or change *P* to a bag-of-spans projection.
- **Pool contamination** (duplicates, near-dups) → artificial invariance. Fix: dedup by content-hash.
- **Conditional commutativity** (Table 4) → only allow add after **runtime CWA pass**.

One-liner decision logic (pseudocode)

```
score, pri = run_certificate(V, panel_cfg)
if score >= 0.75 and pri <= 0.25:
    return add_mean(V)                      # Additive OK
elif score >= 0.50 and pri <= 0.50:
    Vs, Vr = split_axes_by_panel_stability(V)  # Hybrid
    return concat(add_mean(Vs), attention(Vr))
else:
    return attention(V)                     # Fallback (order-aware)
```

API payloads (copy-paste)

Pass (Green) → additive pooling

```
POST /pool
{
  "projected": [{"doc_id":"D1","v":[...]}, {"doc_id":"D2","v":[...]}],
  "cert": {"permutations":64,"sign_flips":32,"chunk_shuffles":16,"seed":1729},
  "min_score": 0.75,
  "strict": true,
  "estimator": "mean",
  "tags": {"gate":"PoolingGate","band":"GREEN"}
}
```

Borderline (Amber) → hybrid + pool shrink

```
POST /pool
{
  "projected": [{"doc_id":"D1","v":[...]},{"doc_id":"D2","v":[...]},{"doc_id":"D3","v":[...]}],
  "cert": {"permutations":96,"sign_flips":48,"chunk_shuffles":24,"seed":1729},
  "min_score": 0.60,
  "strict": false,
  "estimator": "hybrid",
  "tags": {"gate":"PoolingGate","band":"AMBER","pool_shrink":"0.5"}
}
```

Fail (Red) → attention fallback + sign-off

```
POST /pool
```

```
{
  "projected": [{"doc_id":"D1","v":[...]},{"doc_id":"D2","v":[...]}],
  "cert": {"permutations":64,"sign_flips":32,"chunk_shuffles":16,"seed":1729},
  "min_score": 0.75,
  "strict": true,
  "estimator": "attention",
  "tags": {"gate":"PoolingGate","band":"RED","requires_signoff":true}
}
```

Numbers to remember

- **Panel sizes (default):** 64 perm • 32 flip • 16 chunk (increase under Amber).
- **Bands: 0.75 / 0.50** (CWA) and **0.25 / 0.50** (PRI).
- **Logs to keep:** `panel_cfg`, `panel_stats`, pooled vector checksum, estimator, seeds.

Cross-refs: Table 1 (metrics & bands), Table 2 (`/project`, `/pool`, events), Table 4 (conditional commutation), Table 5 (PoolingGate actions).

PBHL Residual: what it means, how to control

What it is (operational definition).
PBHL models macro closure on a belt (program worldsheet):

$$\textbf{Gap} \;\approx\; \textbf{Flux} \;+\; \alpha \cdot \textbf{Twist}$$

The **PBHL Residual** is the closure error (normalized):

$$\boxed{R_{\mathrm{PBHL}} \;=\; \frac{\left|\,\mathrm{Gap} - (\mathrm{Flux} + \alpha \cdot \mathrm{Twist})\,\right|}{\max(\varepsilon, |\mathrm{Gap}|)}} \quad \in [0, \infty)$$

- **Gap**: distance to objectives (0 = fully met).
- **Flux**: execution throughput toward Gap reduction (fast knob).
- **Twist**: structural reconfiguration/retargeting that “rotates” the work (slow knob).
- **α**: coupling coefficient (how much Twist moves the needle).

When to care (bands & gates).

- **Green:** $R_{\mathrm{PBHL}} \le 0.10$ → closure healthy.
- **Amber:** $0.10 < R_{\mathrm{PBHL}} \le 0.30$ → tighten Flux; freeze non-critical Twist.
- **Red:** $R_{\mathrm{PBHL}} > 0.30$ → freeze scope; exec review; raise sampling (Table 5).

Diagnosis cues (why residual is high)

- **Lag mismatch:** Flux measured too slowly; Gap sampled too rarely → backlog of uncounted progress.
- **α drift:** structural change (reorg/tooling) altered Twist efficacy.

- **Leakage:** untracked workstreams or side quests (Gap drops slower than Flux predicts).
- **Over-rotation:** high Twist with weak Flux (churn disguised as progress).
- **Gaming:** local optimizations inflate Flux but not Gap.

Control knobs & safe controller (PBHL loop)

Controllers: Flux-gate (fast), **Twist-step** (slow). Keep micro invariants first (Latching, Agreement, Slots).

Discrete control law (stable defaults):

$$
\begin{aligned}
e_t &= \text{Gap}_t - (\text{Flux}_t + \alpha_t \cdot \text{Twist}_t) \\
\text{Flux}_{t+1} &\leftarrow \text{Flux}_t + k_F \cdot e_t - \beta_F \cdot \Delta\text{Flux}_t \\
\text{Twist}_{t+1} &\leftarrow \text{Twist}_t + k_T \cdot e_t \quad \text{(apply only if } |\, e_t \,| \text{ persists)} \\
\alpha_{t+1} &\leftarrow (1-\lambda)\alpha_t + \lambda \cdot \hat{\alpha}_t, \;\; \hat{\alpha}_t = \arg\min_{\alpha} \sum_{w} (\text{Gap} - (\text{Flux} + \alpha \cdot \text{Twist}))^2
\end{aligned}
$$

Recommended gains: $k_F \in [0.3, 0.6]$, $k_T \in [0.05, 0.15]$, $\beta_F \in [0.1, 0.3]$, $\lambda \in [0.05, 0.2]$.
Heuristic: act **Flux** first; only move **Twist** after R_{PBHL} stays Amber 2–3 windows.

Operator checklist (one page)

1. **Verify sensing:** align Gap/Flux sampling cadences; switch to weekly if noisy.
2. **Re-ID α:** run rolling regression (last 6–8 samples). If α jumps >25%, treat as **α drift** incident.
3. **Flux-first fix:** raise throughput (unblockers, focused WIP, golden path).
4. **Twist-guard:** freeze non-critical scope; only introduce Twist if Flux is near saturation.
5. **Gate & audit:** apply **BeltGate** actions; emit `PBHL.Update` with status and rationale.
6. **Exit criteria:** $R_{\text{PBHL}} \leq 0.10$ for **2 weeks**; attach Five-Line KPI panel.

Runbook payloads (copy-paste)

Amber → tighten Flux-gate; freeze non-critical Twist

```
POST /belt
{
  "edges": {"plan": 0.62, "do": 0.51},
  "alpha": 0.35,
  "twist": 0.09,
  "mesh": "belt://program/support-rag@Q4",
  "gates": {"residual_max": 0.20, "freeze_noncritical_twist": true, "flux_gain": 0.45},
  "tags": {"gate": "BeltGate", "level": "AMBER"}
}
```

Red → freeze scope; daily sampling; exec review; bump Flux; hold Twist

```
POST /belt
{
  "edges": {"plan": 0.62, "do": 0.51},
  "alpha": 0.35,
  "twist": 0.09,
  "mesh": "belt://program/support-rag@Q4",
  "gates": {
    "residual_max": 0.30,
    "freeze_scope": true,
    "sample_rate": "daily",
    "exec_review": true,
    "flux_gain": 0.55,
    "twist_hold": true
  },
  "tags": {"gate": "BeltGate", "level": "RED"}
}
```

α re-identification (when α drift suspected)

```
POST /belt
{
  "edges": {"plan": 0.60, "do": 0.52},
  "alpha": null,                        // let runtime estimate
  "twist": 0.08,
  "mesh": "belt://program/support-rag@Q4",
  "gates": {"reidentify_alpha": true, "window": 8},
  "tags": {"gate": "BeltGate", "level": "AMBER", "op": "alpha-reid"}
}
```

Failure modes → fixes (fast triage)

- **Residual high, Flux high, Gap flat** → leakage or α too low → **audit streams**, re-ID α, trim side quests.
- **Residual high, Flux low, high Twist** → over-rotation → **freeze Twist**, push unblockers (Flux).
- **Residual oscillates** → gains too hot → lower k_F, k_T, add β_F.
- **Sudden residual spike** → sensing break (Gap misreported) → **reconcile metrics**, backfill, rerun `/belt`.

Numbers to remember

- Bands: **0.10 / 0.30** (Green/Red).
- Gains: start $k_F = 0.45$, $k_T = 0.1$, $\beta_F = 0.2$, $\lambda = 0.1$.
- Exit: Green **2 weeks** continuous; publish Five-Line KPI with notes.

Cross-refs: Table 1 (PBHL Residual, EEI/SI bands), Table 2 (`/belt`, `PBHL.Update`, `PolicyGate.Trigger`), Table 5 (BeltGate actions), Fig.4 (belt worldsheet), Fig.5 (Five-Line KPI).

Agreement Check: commuting + shared record recipe

What it verifies.

Two observers (or replicas) **agree** on an outcome when:

1. their chosen instruments **frame-commute** (order doesn't change effective outcomes), and
2. they **write/share redundant pointer records** to a common, immutable ledger.
   The check outputs `{pass|fail, score∈[0,1]}` and updates the **compatibility graph** `C` used by Ô. (See Tables 1–2, 4.)

Operator recipe (five steps)

1. **Align frames & clocks.**
   - Same input state snapshot (or state hash).
   - Timestamps synchronized to tick **τ** (tolerance ≤1 tick).
2. **Declare candidate instruments & build ĉ.**
   - Start from policy hints (Table 4); mark edges "conditional" if gated by CWA.
   - Provide a **commute matrix** with 1/0/"cond" entries.
3. **Share pointer records (SBS redundancy).**
   - Require **redundancy ≥3** independent channels writing the same pointer to a **shared ledger** (append-only; signed chain).
   - Latching: trace writes are immutable within frame (internal collapse).
4. **Run AB/BA agreement panel.**
   - For each pair (A,B), run `apply(A)→apply(B)` vs `apply(B)→apply(A)` across **n=128** contexts.
   - Compare: (i) outcome distributions (KL or MMD ≤ **ε=0.02**), (ii) trace compatibility (allow **stable relabeling**), (iii) next-tick decisions match with prob ≥ **1−δ**, δ=0.05.
5. **Score & gate.**
   - **Agreement Rate (AGR)** over W ticks; **pass if AGR ≥ 0.95** (prod-guarded).
   - If **fail** or redundancy <3 → trigger **AgreementGate** (Table 5): quorum-only or autonomy-off.

Minimal test (pseudocode)

```
def agree(Ta, Tb, commute_matrix, eps=0.02, delta=0.05):
    pairs = instrument_pairs(Ta, Tb)
    scores = []
    for A,B in pairs:
        if commute_matrix[A,B] == 0:
            return fail(0.0, reason="declared conflict")
        SAB, TAB = run_path(A,B); SBA, TBA = run_path(B,A)
        d = dist(outcome(SAB), outcome(SBA))  # KL/MMD
        traces_ok = compatible(TAB, TBA, stable_relabel=True)
```

```
        next_equal = prob_equal(select_channel(SAB,TAB), select_channel(SBA,TBA)) >= (1-delta)
        scores.append(int(d <= eps and traces_ok and next_equal))
    return pass_fail(mean(scores))
```

API payloads (copy-paste)

A) Agreement check (normal)

```
POST /agree
{
  "Ta":"trc_A", "Tb":"trc_B",
  "commute_matrix":[[1,1,0],[1,1,1],[0,1,1]],
  "record_policy":{"redundancy_required":3, "stable_relabel":true}
}
```

B) Force SBS redundancy probe (auto-add a pointer channel)

```
POST /measure
{
  "pi":"channel://pointer/redundant-3",
  "state_ref":"model://agent/42@v17",
  "commit":true,
  "tags":{"gate":"AgreementGate","op":"sbs-probe"}
}
```

C) Quorum-only mode (Amber)

```
POST /measure
{
  "pi":"channel://tool/safe-read",
  "commit":true,
  "tags":{"gate":"AgreementGate","level":"AMBER","quorum":"majority"}
}
```

D) Autonomy off (Red)

```
POST /measure
{
  "pi":"channel://tool/safe-read",
  "commit":false,
  "tags":{"gate":"AgreementGate","level":"RED","autonomy":"off"}
}
```

Quick heuristics (when do A and B commute?)

- **Likely commute:** pure reads, projections + **CWA-passed** mean/sum, dedup→rank with stable keys.
- **Likely conflict:** writes to same key w/o snapshotting; concatenation encoders; plan↔execute pairs; actions with environmental dynamics.
- **Conditional:** read after write **with version pin**, web search↔open **if opener is read-only**.

Worked mini-examples

- **Qubit Z↔Z:** commute (idempotent); agreement should be 1.0 with SBS≥3.

- **Qubit Z↔X:** non-commuting; expect **fail** unless policy treats them as independent records for different observables.
- **RAG bag-fetch ↔ mean-pool:** commute **if** CWA ≥ 0.75; otherwise PoolingGate forces fallback → treat as **conflict** for agreement purposes.

Pitfalls (and fixes)

- **Hidden side-effects** (reads warming caches) → exclude those fields from $\hat{O}$ inputs or snapshot before reads.
- **Clock skew / Δτ drift** → run **SyncGate**; align τ before agreement.
- **Duplicate/near-dup records** → dedup by content hash; otherwise AGR inflates spuriously.
- **Partial ledger sharing** → require cross-replica **proofs** (hash-chain + signature) in `TraceWrite`.
- **CWA-conditioned edges** → re-run `/agree` whenever certificate bands change.

Numbers to remember

- **AGR pass band:** ≥0.95 (prod-guarded).
- **Panel sizes:** n=128 contexts; ε=0.02; δ=0.05.
- **SBS redundancy:** ≥3 independent pointer channels to shared ledger.
- **Hysteresis:** need **512 ticks** in-band to clear AgreementGate.

Cross-refs: Table 1 (AGR, R_SBS), Table 2 (`/agree`, `TraceWrite`, events), Table 4 (compatibility), Table 5 (AgreementGate actions), Figs.2 & 6.

Boxed Callout — Slot Budget Rule: integer capacity guardrails

What it is.
Agents have **integer, non-fractional** capacity for memory, attention, and tool concurrency. A **slot** is one atomic address/lease that an operation occupies exclusively until it's released or expires. The **Slot Budget Rule** enforces:

1. **Discrete capacity** $N \in \mathbb{Z}_{>0}$, 2) **Non-overlap** (no two writers share a slot), 3) **Explicit eviction** (LRU/priority/TTL), 4) **Pool isolation** (separate pools per resource class), 5) **Observable occupancy/collisions**.

Why it matters.
Most agent failures here are mundane—queue stampedes, cache thrash, tool pileups—and they cascade into **mis-exec**, **AGR** drops, and **PBHL** drift. Guardrails keep utilization in the **sweet spot** and make contention visible, not silent.

Operator rules (copy to runbooks)

- **R1 — Size to target occupancy**: choose pool size N so peak concurrent demand M fits within band **Occ* ≈ 40–85%**:

$$N \geq \left\lceil \frac{M}{0.85} \right\rceil \quad \text{and} \quad N \leq \left\lfloor \frac{M}{0.40} \right\rfloor$$

- **R2 — Collision budget**: for hash-based slot picking, **birthday bound** for ≥1 collision:

$$\Pr[\text{collision}] \approx 1 - e^{-M(M-1)/(2N)} \quad \Rightarrow \quad \text{tune } N \text{ s.t. SCR} < 10/1\,k$$

- **R3 — Lease everything**: every allocation has a **TTL**; renew or release—no zombies.
- **R4 — Serialize across conflict cuts**: if instruments don't commute (Table 4), **force new τ** or single-file the callers even if slots are free.
- **R5 — Two-sided risk**: **low Occ** (<20%) → wasted memory/latency; **high Occ** (>95%) → thrash & MER↑. Both page **AllocatorGate** (Table 5).

Quick capacity planning (three knobs)

1. **Concurrency target** M_{p95}: p95 of simultaneous requesters.
2. **Utilization target** $u^* \in [0.4, 0.85]$.
3. **Collision SLO** `SCR<10/1k`.
   Pick $N = \max\{\lceil M_{p95} / u^* \rceil, \text{N from collision curve}\}$. Recheck monthly.

Minimal allocator spec (safe defaults)

- **Pools:** `tools/*`, `cache/*`, `attention/*` (independent ledgers).
- **Policy:** `priority > LRU`, TTL=5 s (tools), 60 s (cache), 500 ms (attention micro-batches).
- **Fairness:** round-robin per caller; **burst cap** per caller (e.g., 2 slots).
- **Observability:** emit `SlotAllocate/Release` with `occupancy`, `collisions_per_1k`.
- **Guards:** back-pressure at 85%, serialize hot callsites at 95%.

Pseudocode (lease + fairness)

```
def allocate(pool, k, caller):
    grant=[]
    for _ in range(k):
        if occupancy(pool) >= 0.95: serialize_hot_callers(pool); raise BackPressure
        s = next_free_slot(pool, caller)  # RR per caller
        if s is None: raise BackPressure
        lease(s, ttl=pool.ttl, owner=caller); grant.append(s)
    emit("SlotAllocate", occupancy=occ(pool), scr=collisions(pool))
    return grant
```

API payloads (copy-paste)

Allocate with TTL (Amber back-pressure mode)

```
POST /slots.allocate
{"pool":"tools/default","k":2,"ttl_ms":5000,"tags":{"gate":"AllocatorGate","level":"AMBER"}}
```

Release

```
POST /slots.release
{"ids":["slt_a","slt_b"],"tags":{"reason":"tool-finished"}}
```

Occupancy probe (for dashboards)

```
GET /slots.occupancy
```

Metric bands & gates (Table 1 shortcuts)

- **Occupancy (Occ)**: **Green 40–85% · Amber 20–40% or 85–95% · Red <20% or >95%**
- **Collision rate (SCR)**: **Green <10/1k · Amber 10–30/1k · Red >30/1k**
  AllocatorGate actions: Amber→ back-pressure + TTL↑; Red→ increase pool & serialize callers.

Worked mini-examples

- **RAG retriever cache (8 slots).** Peak demand 6 ⇒ $N_min = \lceil 6/0.85 \rceil = 8$. If burst to 9 causes **SCR=34/1k**, raise to 12 and set per-caller burst cap=2.
- **Tool fan-out.** Planner spawns 10 parallel tools; pool has 12. Keep Occ≈0.83 (Green). If **MER** spikes with **Occ>0.95**, it's contention: serialize the top offender and raise TTL to 8 s.

Failure modes → fixes

- **Zombie leases (no release):** SCR climbs with flat Occ; fix with TTL + renewals and `/slots.release` in finally blocks.
- **Unisolated pools:** cache saturation starves tools; split pools (`cache/*` vs `tools/*`).
- **Hidden writes:** "reads" that mutate Ô inputs violate non-overlap; mark them **writes** and include in pool accounting.
- **Over-aggressive evictions:** Occ in Green but miss rate high → LRU→priority policy; pin critical spans.

Numbers to remember

- **Occ sweet spot: 40–85%**.

- **SCR SLO: <10 per 1k allocations**.
- **Default TTLs:** tools 5 s, cache 60 s, attention 0.5 s.
- **Burst cap:** ≤2 per caller (raise only with AgreementGate Green).

Cross-refs: Table 1 (Occ, SCR), Table 2 (slot APIs & events), Table 4 (serialize across conflicts), Table 5 (AllocatorGate actions), Fig. 7 (allocator heatmap).

© 2025 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.

M

T

G

Y

Text-to-speech function is limited to 200 characters